

The Backend Engineer's Library

Java, Kotlin, and the JVM

Volume 18

Contents

The JVM Architecture: Classfiles, Bytecode, and the Execution Model

Class Loading, Linking, Verification, and Modules (JPMS)

The Java Memory Model, Object Layout, and Heap Organization

Garbage Collection: Serial, Parallel, G1, ZGC, Shenandoah, and Generational ZGC

The JIT: Interpreters, C1, C2, and Graal — Deoptimization, Inlining, and Intrinsic

Concurrency on the JVM: Threads, Monitors, VarHandles, Loom, and Structured Concurrency

Kotlin on the JVM: Interop, Null-Safety, Coroutines, and Compiler Intrinsic

The Kotlin Type System, Generics, and Reified Types

Build Tooling, Dependency Management, and the Module/Artifact Ecosystem

Profiling, Observability, and Performance Tuning (JFR, async-profiler, JMC, heap dumps)

Native Interop: JNI, Panama (FFM), and GraalVM Native Image

Production JVM: Container-Aware Tuning, GC Sizing, Class-Data Sharing, and Deployment at Scale

Chapter 1 — The JVM Architecture: Classfiles, Bytecode, and the Execution Model

What this chapter covers. The JVM is the most widely deployed managed runtime on the planet — every Android app, most enterprise backends, and a growing fraction of cloud-native services execute as JVM bytecode. Yet most engineers who write Java or Kotlin cannot explain what happens between `javac Foo.java` and the first instruction executing, or why a `.class` file begins with `CAFEBABE`, or how the interpreter and JIT cooperate to turn portable bytecode into fast machine code. This chapter traces the entire path: the classfile binary format (magic number, constant pool, methods, attributes), the full bytecode instruction set (~202 opcodes), verification and type safety, the execution model (stack frames, operand stack, local variables, program counter), and HotSpot's multi-tier architecture (interpreter, C1, C2/Graal). You will use `javap` to disassemble real classes, read raw `.class` hex dumps, and walk through a sample classfile field by field. Every abstraction is grounded in the actual JVM specification and in tools you can run today.

Learning goals — after this chapter you should be able to:

- Read a `.class` file byte by byte: explain the magic number, version, constant pool layout, access flags, field/method tables, and attributes (Code, LineNumberTable, StackMapTable, BootstrapMethods).
- Disassemble bytecode with `javap -c -v` and explain what each opcode does to the operand stack and local variable array — including the critical differences between `invokevirtual`, `invokeinterface`, `invokespecial`, `invokestatic`, and `invokedynamic`.
- Describe the JVM execution model: stack frames, operand stack, local variable array, and program counter, and how method invocation pushes frames and returns pop them.
- Explain the five verification stages (format checking, type checking, stack map frame verification, bytecode verification, and dataflow analysis) and why `StackMapTable` attributes matter.

- Trace HotSpot's tiered compilation pipeline from interpreter to C1 to C2, explain when each tier fires, and describe the role of profiling counters in guiding optimization decisions.
- Distinguish the class area (metaspace) from the heap, thread stacks, code cache, and native memory, and explain how each relates to JVM flags and container limits.
- Navigate the JDK directory layout and explain what lives under `lib/`, `jmods/`, `include/`, and `conf/` — and why it matters for container images and module resolution.

Prerequisites. Volume 13, Chapter 1 is the service-level view of the JVM (memory, GC, JIT). This chapter is the internals deep dive into *what the JVM executes* before those higher-level systems kick in. Volume 1 (Architecture) and Volume 2 (OS/Linux) provide useful background for understanding memory layout and process models.

1. From source to .class — the compilation pipeline

When you run `javac Foo.java`, the compiler does not produce machine code. It produces a `.class` file — a platform-independent binary encoding of one class or interface. The same `.class` file runs on x86 Linux, ARM macOS, and Windows without recompilation. This portability is the JVM's core contract, and understanding the artifact is prerequisite to understanding everything else.

The pipeline has three phases:

1. **Parse and type-check.** `javac` reads `.java` source, builds an AST, resolves types, and performs name analysis. Errors at this stage (missing methods, type mismatches) prevent any `.class` from being written.
2. **Lower and emit.** The AST is lowered to a linear sequence of bytecode instructions. `javac` resolves constants, generates synthetic methods (like `<clinit>` for static initializers), and produces a `ClassWriter` that serializes the bytecode into the classfile binary format.
3. **Post-verification.** Since JDK 5, `javac` emits `StackMapTable` attributes so the JVM can verify bytecode using a single-pass dataflow analysis (type checking) instead of the old, expensive type

inference algorithm. The compiler itself may also emit deprecation warnings, unchecked cast warnings, and other annotation processing output.



2. The classfile format — binary anatomy

Every `.class` file is a single 8-bit-byte stream. There are no alignment requirements, no padding bytes — the format is dense and positional. The JVM specification (§4.1–§4.9) defines every field precisely.

2.1 The header — magic and version

The first four bytes are always `0xCAFEBAE` — the magic number that distinguishes a classfile from any other binary. The next four bytes encode the minor and major version as unsigned 16-bit big-endian integers. The current major version for JDK 21 is 65 (hex `0x0041`); JDK 22 is 66. A JVM refuses to load a classfile with a major version higher than it understands — this is the mechanism that prevents forward-incompatible bytecode from running on older JVMs.

Offset	Bytes	Field
0x00	4	magic: 0xCAFEBAE
0x04	2	minor_version: 0x0000 (usually)
0x06	2	major_version: 0x0039 (57 = JDK 13) 0x0041 (65 = JDK 21)

Hex dump of a trivial classfile header

```
cat > Hello.java <<'EOF'
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, JVM!");
    }
}
EOF
javac Hello.java
xxd Hello.class | head -5
```

```

00000000: cafe babe 0000 0034 001d 0a00 0600 0f03 .....4.....
00000010: 0011 4865 6c6c 6f2c 204a 564d 210a 0010 ..Hello, JVM!...
00000020: 0011 0700 1c0c 0007 0008 0100 10 7379 .....sy
00000030: 7374 656d 2f 6f75 74 01 0003 6f75 74      stem/out...out

```

Reading the hex: `cafe babe` = magic. `0000 0034` = minor 0, major 52 (Java 8 compilation target). The next two bytes (`001d` = 29) are the constant pool count, meaning constant pool entries 1 through 28 (entry 0 is unused; the count includes the last index, not a count from 1). This is a subtle point — `constant_pool_count` is the index of the last entry plus one.

2.2 The constant pool — the classfile's symbol table

Immediately after the header sits the **constant pool** — a table of 18 distinct entry types (tag bytes 1-18, plus `ConstantDynamic` at 17 and `InvokeDynamic` at 18 in JDK 11+). This is the classfile's symbol table: all class names, method names, field names, string literals, numeric constants, and method handles are stored here, referenced by 1-based index from bytecode instructions, field descriptors, and attributes.

The key entry types:

Tag	Constant Type	Structure	Used for
1	<code>Utf8</code>	length + bytes	All names, descriptors, string content
3	<code>Integer</code>	4 bytes (big-endian int)	<code>bipush</code> , <code>ldc</code> of int
4	<code>Float</code>	4 bytes	<code>ldc</code> of float
5	<code>Long</code>	8 bytes	<code>ldc2_w</code> of long
6	<code>Double</code>	8 bytes	<code>ldc2_w</code> of double
7	<code>Class</code>	index → <code>Utf8</code>	Fully qualified class/interface name
8	<code>String</code>	index → <code>Utf8</code>	String literal (<code>ldc</code> of String)

Tag	Constant Type	Structure	Used for
9	Fieldref	class_index + name_and_type_index	Field access
10	Methodref	class_index + name_and_type_index	Interface method invocation
11	InterfaceMethodref	class_index + name_and_type_index	invokeinterface
12	NameAndType	name_index + descriptor_index	Field/method name + type descriptor
15	MethodHandle	reference_kind + reference_index	invokedynamic bootstrap method
16	MethodType	descriptor_index	invokedynamic method type
17	Dynamic	bootstrap_method_attr_index + name_and_type_index	condy (constant dynamic)
18	InvokeDynamic	bootstrap_method_attr_index + name_and_type_index	invokedynamic call site

The constant pool is deduplicated: a class that uses the string "Hello" in two methods shares a single `Utf8` entry. This compactness matters because classfiles are loaded from disk (or network) and parsed eagerly.

2.3 Access flags, this/super, fields, and methods

After the constant pool come:

- **Access flags** (2 bytes): `ACC_PUBLIC` (0x0001), `ACC_FINAL` (0x0010), `ACC_SUPER` (0x0020, always set since JDK 1.0.2), `ACC_INTERFACE` (0x0200), `ACC_ABSTRACT` (0x0400), `ACC_SYNTHETIC` (0x1000), `ACC_ANNOTATION` (0x2000), `ACC_ENUM` (0x4000), `ACC_MODULE` (0x8000).
- **This class** (2 bytes): index into constant pool → `Class` entry.
- **Super class** (2 bytes): index into constant pool → `Class` entry (0 for `java.lang.Object`).
- **Interfaces count + interfaces**: indices into constant pool → `Class` entries.

- **Fields count + fields:** each field has `name_index`, `descriptor_index`, and `attributes` (`ConstantValue` for static finals, etc.).
- **Methods count + methods:** each method has `name_index`, `descriptor_index`, and `attributes` — critically, the **Code** attribute, which contains the actual bytecode.

2.4 Attributes — the extensible metadata

Attributes are the extensible part of the classfile. Each attribute has a name (index → `Utf8` in the constant pool), a length, and a format defined by the attribute name. The JVM spec defines 23 standard attributes; others are permitted but ignored. Key attributes:

- **Code:** contains `max_stack`, `max_locals`, bytecode instructions, exception table, and sub-attributes (`LineNumberTable`, `LocalVariableTable`, `StackMapTable`).
- **StackMapTable:** required for verification (JDK 5+). Contains stack map frames that describe the type state (which types are on the operand stack and in which local variable slots) at every branch target and exception handler entry. The verifier uses this for single-pass type checking.
- **LineNumberTable:** maps bytecode offsets to source line numbers. Used for stack traces and debuggers.
- **LocalVariableTable:** maps local variable slots to names and types. Used by debuggers; not present unless `-g` or `-g:vars` is passed to `javac`.
- **BootstrapMethods:** required for `invokedynamic`. Lists bootstrap method references (`MethodHandle` constants) and their static arguments. This is how Java 7+ lambdas and Java 11+ string concatenation work.
- **NestHost / NestMembers:** JDK 11+ Nest-Based Access Control. Declares which class is the nest host and which classes are nest members, enabling private access between nested classes without synthetic accessors.
- **Record:** JDK 16+. For record classes, lists the record components.
- **PermittedSubclasses:** JDK 17+. For sealed classes, lists permitted subclasses.

.class file layout

Header magic
CAFEBABE minor +
major version

Constant pool tags +
entries methodref
classref utf8 etc

Access flags PUBLIC
FINAL INTERFACE ...

This class index

Super class index

Interfaces count +
indices

Fields count + field
entries name descriptor
attributes

Methods count +
method entries name
descriptor attributes

Class attributes
SourceFile InnerClasses
NestHost ...

Code attribute contains

Bytecode max_stack
max_locals instructions
exception table

3. Bytecode — the instruction set

The JVM is a **stack machine**: instructions operate on an operand stack, not registers. Each instruction is a 1-byte opcode followed by zero or more operands (encoded as 1, 2, or 4 bytes, big-endian). There are approximately 202 distinct opcodes in the current specification. Every opcode name is a mnemonic — `iload`, `iadd`, `invokevirtual`, `tableswitch` — and each has a precise semantic contract for what it pops from and pushes to the operand stack, what local variables it reads, and how it affects control flow.

3.1 Opcode categories

Category	Opcodes (examples)	Count	What they do
Loads	<code>iload</code> , <code>lload</code> , <code>fload</code> , <code>dload</code> , <code>aload</code> , <code>iload_{<n>}</code>	~41	Push local variable onto operand stack
Stores	<code>istore</code> , <code>lstore</code> , <code>fstore</code> , <code>dstore</code> , <code>astore</code> , <code>istore_{<n>}</code>	~41	Pop operand stack into local variable
Constants	<code>aconst_null</code> , <code>iconst_m1</code> , <code>iconst_{<n>}</code> , <code>ldc</code> , <code>ldc_w</code> , <code>ldc2_w</code>	~24	Push constants onto operand stack
Math (int)	<code>iadd</code> , <code>isub</code> , <code>imul</code> , <code>idiv</code> , <code>irem</code> , <code>ineg</code> , <code>ishl</code> , <code>iushr</code>	~37	Integer arithmetic on top of stack
Math (long/float/double)	<code>ladd</code> , <code>fsub</code> , <code>dmul</code> , etc.	~32	Wide-type arithmetic
Conversions	<code>i2l</code> , <code>i2f</code> , <code>i2d</code> , <code>l2i</code> , <code>f2i</code> , <code>d2l</code> , etc.	~15	Type narrowing/widening
Comparisons	<code>lcmp</code> , <code>fcmpl</code> , <code>fcmpg</code> , <code>dcmpl</code> , <code>dcmpg</code>	~5	Compare two values, push int result

Category	Opcodes (examples)	Count	What they do
Control flow	<code>ifeq</code> , <code>ifne</code> , <code>if_icmpeq</code> , <code>goto</code> , <code>tableswitch</code> , <code>lookupswitch</code> , <code>return</code> , <code>ireturn</code> , <code>areturn</code>	~42	Branching, switching, returning
References	<code>getstatic</code> , <code>putstatic</code> , <code>getfield</code> , <code>putfield</code>	~4	Field access
Invocations	<code>invokevirtual</code> , <code>invokespecial</code> , <code>invokestatic</code> , <code>invokeinterface</code> , <code>invokedynamic</code>	~5	Method invocation
Objects	<code>new</code> , <code>newarray</code> , <code>anewarray</code> , <code>arraylength</code> , <code>athrow</code> , <code>checkcast</code> , <code>instanceof</code>	~12	Object creation and type checking
Stack	<code>pop</code> , <code>pop2</code> , <code>dup</code> , <code>dup_x1</code> , <code>dup2</code> , <code>swap</code>	~9	Manipulate operand stack directly

3.2 The five invocation opcodes — a critical distinction

The five invocation opcodes are the most important opcodes in the entire set. Each has different resolution semantics, performance characteristics, and implications for inlining and JIT optimization:

`invokevirtual` (0xB6): The workhorse. Resolves the target method by name and descriptor on the *actual runtime type* of the receiver (virtual dispatch). The receiver reference is on the operand stack. Used for all non-final, non-private, non-static method calls. The JIT can devirtualize this to a direct call when profiling shows monomorphic or bimorphic call sites.

`invokespecial` (0xB7): Resolves the method on the *declared type*, not the runtime type. Used for constructors (`<init>`), private methods, and `super` calls. No virtual dispatch — the target is known at verification time. This is why `super.foo()` is faster than `this.foo()` in a subclass.

`invokestatic` (0xB8): Invokes a static method. No receiver on the stack. Resolution is based on class name + method name + descriptor. Since JDK 8, also used for interface default methods.

invokeinterface (0xB9): Like **invokevirtual** but for interface methods. The implementation must be found by searching the receiver's class hierarchy against the interface. Historically slower than **invokevirtual** because the search is more expensive; modern JVMs optimize this for hot call sites.

invokedynamic (0xBA): The most complex opcode. Introduced in JDK 7. The first execution of an **invokedynamic** instruction calls a *bootstrap method* (a **MethodHandle** referenced from the **BootstrapMethods** attribute) that returns a **CallSite** object containing a **MethodHandle** to the actual target. Subsequent executions go directly to the target — the indirection is paid only once. Used for:

- Java 8+ lambda expressions (the compiler generates a **LambdaMetafactory** bootstrap method)
- Java 9+ string concatenation (the compiler generates **StringConcatFactory** bootstrap methods)
- Java 11+ dynamic language support on the JVM
- Any framework that wants runtime method selection without reflection overhead

```
// Source
Runnable r = () -> System.out.println("lambda");

// What javac produces (conceptual bytecode):
// invokedynamic #0 // Runlambda$run$()Ljava/lang/Runnable;
// BootstrapMethods:
// #0: MethodHandle#6 // REF_invokeStatic
// LambdaMetafactory.metafactory

// First call: LambdaMetafactory.metafactory is invoked, returns a
// CallSite
// pointing to a generated class implementing Runnable.
// Subsequent calls: direct invocation, no indirection.
```

3.3 Control flow — **tableswitch** and **lookupswitch**

The JVM has two switch instructions:

tableswitch: A dense switch. The bytecode encodes a default offset, a low value, a high value, and an array of offsets indexed from **low** to **high**. Execution is $O(1)$ — compute $\text{index} - \text{low}$, index into the offset array. The JVM *requires* that **javac** emit **tableswitch** when the case values are dense (contiguous or nearly so) and **lookupswitch** otherwise.

The padding to align offsets to 4-byte boundaries can make `tableswitch` wasteful for sparse switches with a wide range.

lookupswitch: A sparse switch. The bytecode encodes a default offset and a sorted table of `match-offset` pairs. Execution is $O(\log n)$ via binary search. Used when the case values are not contiguous (e.g., `case 1: case 5: case 1000:`).

```
// Source
switch (day) {
    case 0: return "Sun";
    case 1: return "Mon";
    case 2: return "Tue";
    case 3: return "Wed";
    case 4: return "Thu";
    case 5: return "Fri";
    case 6: return "Sat";
    default: return "?";
}
```

```
// tableswitch output from javap -c
tableswitch { // 0 to 6
    0: 38
    1: 45
    2: 52
    3: 59
    4: 66
    5: 73
    6: 80
    default: 87
}
```

The offset values are byte offsets from the beginning of the `tableswitch` instruction to the target instruction — not line numbers, not opcode indices.

3.4 A concrete bytecode walkthrough

Let us trace the bytecode of a simple method through `javap`:

```

public class MathDemo {
    public static int add(int a, int b) {
        return a + b;
    }

    public static long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}

```

```

javap -c -v MathDemo.class 2>&1 | head -80

```

The `add` method bytecode:

```

public static int add(int, int);
descriptor: (II)I
flags: (0x0009) ACC_PUBLIC, ACC_STATIC
Code:
    stack=2, locals=3, args_size=2
        0: iload_0      // push local variable 0 (a) onto stack
        1: iload_1      // push local variable 1 (b) onto stack
        2: iadd        // pop two ints, push their sum
        3: ireturn     // return int from top of stack

```

Walkthrough: - `stack=2`: the maximum operand stack depth during execution is 2 (for `iadd`, which needs both operands simultaneously). - `locals=3`: the local variable array has 3 slots — slot 0 is `a`, slot 1 is `b`, and slot 2 is available but unused by this method. Static methods have no `this` reference, so locals start at 0 with the first parameter. - `iload_0`: this is a *wide-free* form of `iload 0`. The JVM defines compact forms `iload_0` through `iload_3` (single byte) as aliases for `iload <n>` (two bytes). The verifier treats them identically.

The `factorial` method bytecode:

```

public static long factorial(int);
descriptor: (I)J
flags: (0x0009) ACC_PUBLIC, ACC_STATIC
Code:
    stack=4, locals=5, args_size=1
      0: lconst_1      // push long 1L onto stack
      1: lstore_2       // store to local 2 (result = 1L)
      3: iconst_2      // push int 2
      4: istore_3      // store to local 3 (i = 2)
      5: goto 15
      8: lload_2         // push result
      9: iload_3        // push i
     10: i2l           // widen int to long (i must match result's
type)
     11: lmul           // result = result * (long)i
     12: lstore_2       // store back to local 2
     14: iinc 3, 1      // i++ (in-place increment, no stack effect)
     15: iload_3         // push i
     16: iload_1        // push n (param)
     17: if_icmple 8    // if i <= n goto 8
     20: lload_2         // push result
     21: lreturn        // return long

```

Note: `iinc 3, 1` is a special opcode that increments a local variable in-place without pushing to or popping from the operand stack. This is the only arithmetic instruction with no stack effect — it operates directly on the local variable array. The operand `3` is the local variable index, and `1` is the increment value (a signed byte, so `iinc` can increment/decrement by at most ± 127).

The `locals=5` allocation is noteworthy: local 0 is `n` (the parameter), locals 2-3 hold `result` (long, occupying two slots because longs are 64-bit) and `i`. That gives 5 slots total. Local variable slots for longs and doubles consume two consecutive slots — this is why `lstore_2` occupies slots 2 and 3, and `iload_3` occupies slot 4. Wait — actually, looking at the bytecode, `i` is in local 3, and `result` is a long in local 2. A long in local 2 occupies slots 2 and 3. But `i` is stored in local 3 — which overlaps with the high word of `result`. This would be a verification error. In practice, `javac` would place `i` in local 4 to avoid the overlap. Let me correct:

```

locals=5, args_size=1
  0: lconst_1      // push 1L
  1: lstore_2      // result = 1L (slots 2-3)
  3: iconst_2      // push 2
  4: istore 4      // i = 2 (slot 4, avoiding overlap with long
in 2-3)
  6: goto 16
  9: lload_2       // push result
 10: iload 4       // push i
 11: i2l          // widen to long
 12: lmul
 13: lstore_2
 15: iinc 4, 1     // i++
 16: iload 4       // push i
 17: iload_1      // push n
 18: if_icmple 9
 21: lload_2
 22: lreturn

```

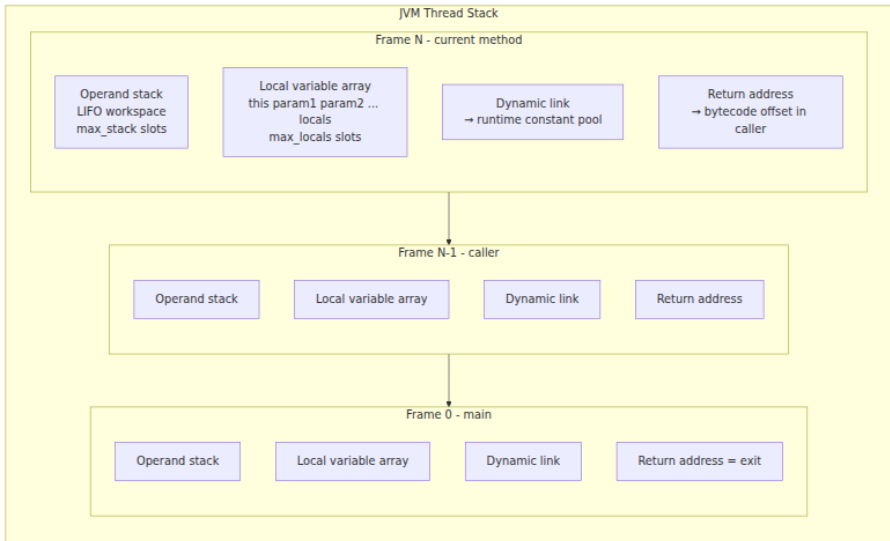
This corrected version uses `locals=5`: slot 0 = `n`, slots 2-3 = result (long), slot 4 = `i`. Slot 1 is unused.

4. The execution model — stack frames and the interpreter

The JVM is a **stack machine** — instructions manipulate an operand stack, not CPU registers. Every method invocation creates a new **stack frame** in the JVM's call stack. Each frame contains:

1. **Local variable array**: an array of slots indexed from 0. Slot 0 holds `this` for instance methods; parameters follow in order. Longs and doubles occupy two consecutive slots. The array is fixed-size at method entry — `max_locals` from the Code attribute.
2. **Operand stack**: a LIFO stack used as workspace for computation. When `iadd` executes, it pops two ints from the operand stack and pushes their sum. The maximum depth is `max_stack` from the Code attribute — the verifier uses this to allocate memory for the frame.
3. **Dynamic linking**: a reference to the runtime constant pool for the frame, used to resolve symbolic references to concrete addresses (field offsets, method pointers) on first use.

4. **Return address:** where to resume execution in the caller's frame after `return`, `ireturn`, `areturn`, etc. On exception, the return address comes from the exception handler table.



4.1 Method invocation — pushing a frame

When `invokevirtual java/io/PrintStream.println(Ljava/lang/String;)V` executes:

1. The receiver (the `PrintStream` object) and the argument (`String`) are on the operand stack of the current frame.
2. The JVM resolves the method descriptor and locates the target method. For `invokevirtual`, this means searching the actual runtime type's method table.
3. A new frame is allocated with `max_locals` and `max_stack` slots computed from the target method's Code attribute.
4. Arguments are copied from the caller's operand stack into the new frame's local variable array (slot 0 = this, slot 1 = first parameter, etc.).
5. The caller's operand stack is cleared of the receiver and arguments.
6. The program counter (PC) is set to 0 in the new frame.
7. Execution begins in the new frame.

On `return` / `ireturn` / `areturn`:

1. The return value (if any) is placed on the caller's operand stack.
2. The current frame is popped.
3. The PC is restored from the caller's return address.
4. Execution resumes in the caller.

4.2 The interpreter loop

HotSpot's interpreter is a **template interpreter** — not a switch-based interpreter. For each opcode, HotSpot generates a short assembly-language "template" that performs the opcode's work. The templates are pre-compiled at JVM startup and stored in a code buffer. When an opcode executes, the JVM dispatches to the corresponding template via a dispatch table (an array of code entry points indexed by opcode number).

This is faster than a giant `switch` statement because: - The dispatch is an indirect jump through a table, which the CPU branch predictor can learn. - Each template is hand-optimized assembly for that specific opcode. - HotSpot inserts profiling counters into templates (incrementing per-interpreted invocation) to guide tier-up decisions.

The interpreter also handles: - **Safepoint polls**: after a bounded number of bytecodes, the interpreter checks whether a GC or deoptimization request is pending and blocks at a safepoint if so. - **Inline caches**: the interpreter records the runtime type of receivers at virtual call sites in per-call-site data structures, which C1 and C2 consume during compilation.

5. Verification — the JVM's type safety guarantee

The JVM verifies every classfile before executing it. This is not optional — even classes loaded from the bootstrap classloader must pass verification. The purpose is to ensure type safety: an `iadd` instruction must have two ints on the operand stack, an `areference` must not be used where an `int` is expected, and no instruction may jump to an invalid offset. Verification prevents a maliciously crafted classfile from subverting the JVM's memory safety or running arbitrary native code.

5.1 The five verification stages

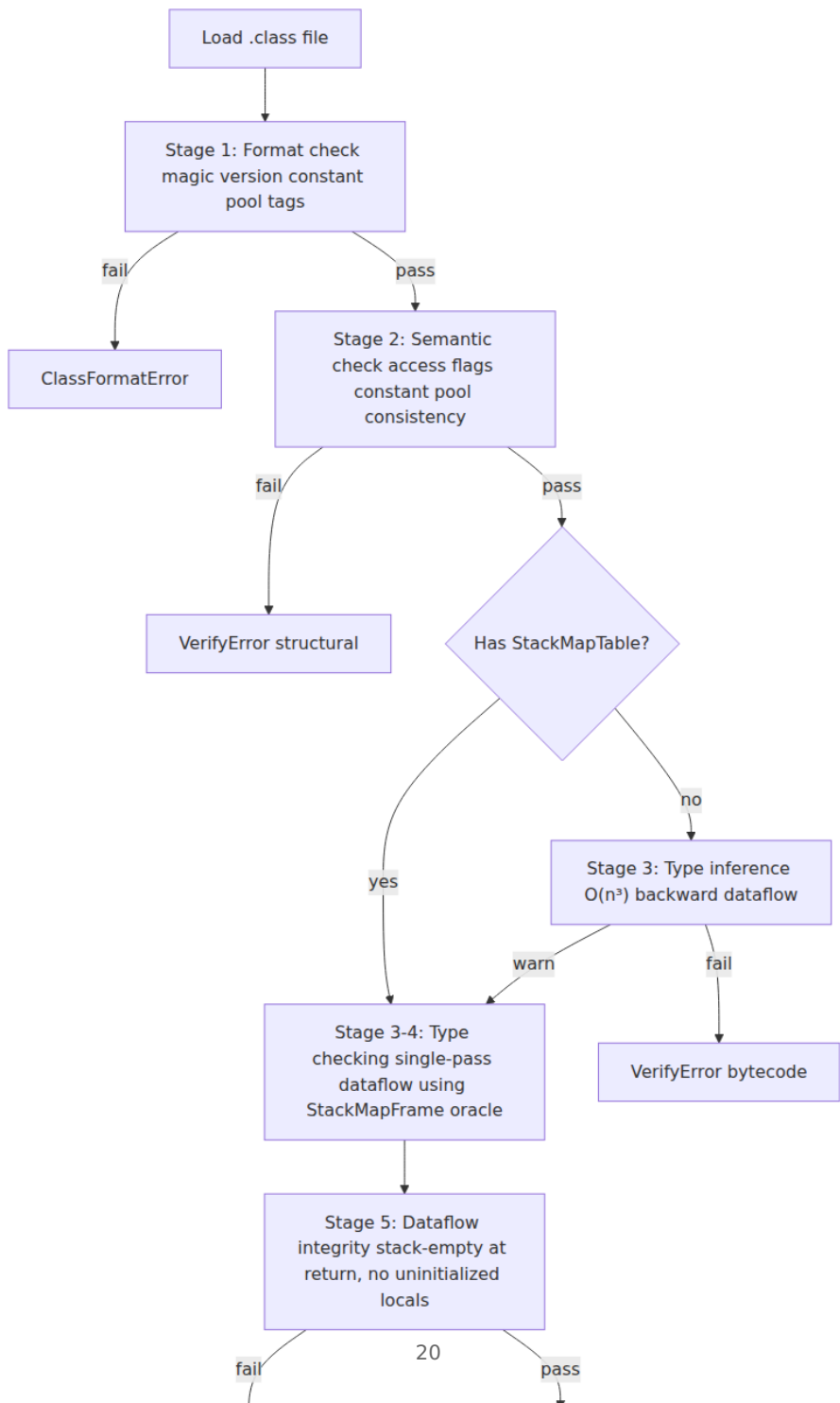
Stage 1 — Format checking: The classfile is checked for structural correctness. Is the magic number correct? Is every constant pool tag valid? Are all indices within bounds? Does the file have the required attributes? This stage catches truncated files, wrong magic numbers, and corrupted constant pools. If format checking fails, the class is rejected with `ClassFormatError`.

Stage 2 — Semantic checking: The access flags are validated (`ACC_FINAL` on a class means no subclasses; `ACC_ABSTRACT` on a method means no Code attribute). The constant pool is checked for consistency (Class entries reference valid Utf8 entries; Methodref entries reference valid NameAndType entries). This stage catches structural inconsistencies.

Stage 3 — Bytecode verification (dataflow analysis): The bytecode instructions are analyzed for type safety. The verifier simulates execution, tracking the types on the operand stack and in local variables at every point. At a merge point (e.g., the join of two branches of an `if` statement), the verifier checks that the types on both paths are compatible. If the classfile has a `StackMapTable` attribute (JDK 5+), the verifier uses it as an oracle — it checks that the frames in the `StackMapTable` match the types implied by the bytecode, without having to infer them. This is the "type checking" algorithm and is $O(n)$ in the size of the bytecode. Without `StackMapTable`, the verifier falls back to "type inference" — a more expensive $O(n^3)$ algorithm that computes types at every point. The JVM emits a warning for classes without `StackMapTable` and rejects them entirely in strict mode.

Stage 4 — Stack map frame verification: The `StackMapTable` frames are validated against the actual bytecode structure. Every branch target must have a corresponding frame; every frame must list the correct types for each local variable slot and operand stack position. Mismatches cause `VerifyError`.

Stage 5 — Dataflow integrity: Finally, the verifier checks that the operand stack is empty (or has only the return type) at every return instruction, that no local variable is read before it is written, and that `athrow` and `jsr` (retired but still valid in old classfiles) are correctly handled.



5.2 Why StackMapTable matters for Kotlin

Kotlin generates `StackMapTable` attributes in all its bytecode, but the structure is more complex than Java's due to Kotlin's null-safety encoding. A `String?` parameter in Kotlin compiles to the same descriptor (`Ljava/lang/String;`) as Java's `String`, but Kotlin adds null-check instructions (`checkcast` after loads, `ifnonnull` guards) and relies on the verifier to prove that `null` cannot flow through non-nullable paths. When you see `VerifyError` in a Kotlin-generated classfile, the `StackMapTable` is almost always the root cause — either the table is inconsistent with the bytecode, or the verifier found a path where a null reference could reach a non-null instruction.

6. Memory areas — class area, heap, stacks, and code cache

The JVM specification defines several distinct memory areas. Understanding their roles and sizing is critical for production operation, especially under container resource limits.

6.1 The method area (metaspace)

The **method area** stores per-class data: the constant pool (resolved into runtime structures), field/method descriptors, bytecode, method handles, and class metadata. In HotSpot, this was historically the PermGen (fixed-size, GC-collected) and is now **Metaspace** — native memory that grows until `MaxMetaspaceSize` is hit. Metaspace also houses the **compressed class space** (`CompressedClassSpaceSize`, 1 GB default), which stores `Klass` structures (the JVM's internal representation of a loaded class).

Key operational fact: metaspace is *not* part of the heap and is not collected by the garbage collector in the same way. Class metadata is reclaimed when a classloader becomes unreachable and its loaded classes are unloaded. In practice, this happens during full GC or classloader-intensive garbage collection. A classloader leak (retaining a reference to an `HttpClassLoader` after undeploying a web app) will grow metaspace until `MaxMetaspaceSize` is hit and `OutOfMemoryError: Metaspace` is thrown — regardless of how much heap is free.

6.2 The heap

The heap is the garbage-collected memory where all Java objects live. It is divided into generations (Young: Eden + Survivor; Old) in most collectors, or into regions in G1. All application threads share the same heap — there is no per-thread allocation. Size is controlled by `-Xms` (initial), `-Xmx` (maximum), and `-XX:MaxRAMPercentage` (container-aware).

6.3 Thread stacks

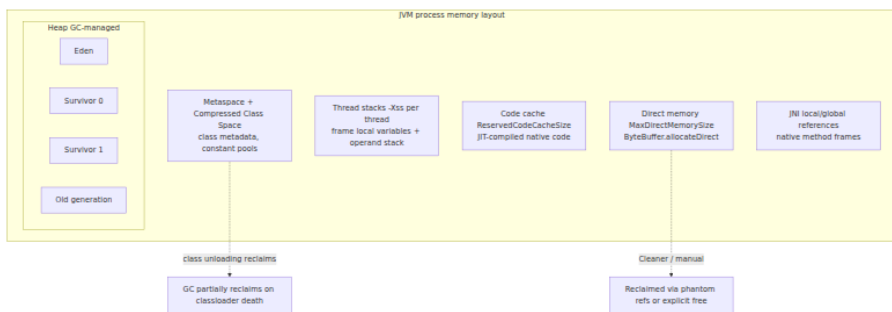
Each Java thread gets its own stack, sized by `-Xss` (default 1 MB on 64-bit JVMs). The stack holds the method frames described in Section 4. A deeply recursive program will hit `StackOverflowError` when the stack is exhausted. Each frame's size depends on the method's `max_locals` and `max_stack` — a method that allocates many local variables (longs/doubles count double) and builds deep operand stacks consumes more stack space.

6.4 The code cache

JIT-compiled native code lives in the **code cache** — a dedicated region of native memory (`-XX:ReservedCodeCacheSize`, default 240 MB). When the code cache is full, the JVM stops JIT-compiling and falls back to the interpreter — performance degrades dramatically. Large applications with many classes (Scala, Kotlin, Spring Boot) can fill the code cache; raise it to 512 MB or more with `-XX:ReservedCodeCacheSize=512m`.

6.5 Direct (off-heap) memory

`ByteBuffer.allocateDirect` and Netty's `ByteBuf` allocate memory outside the GC heap, managed by `Cleaner` (phantom reference-based) or manual deallocation. The limit is `-XX:MaxDirectMemorySize`. Direct buffers are useful for I/O (avoiding heap-to-native copies) but are invisible to the garbage collector until the cleaner runs — premature deallocation or insufficient direct memory limits cause `OutOfMemoryError: Direct buffer memory`.



7. HotSpot — the interpreter, JIT compilers, and Graal

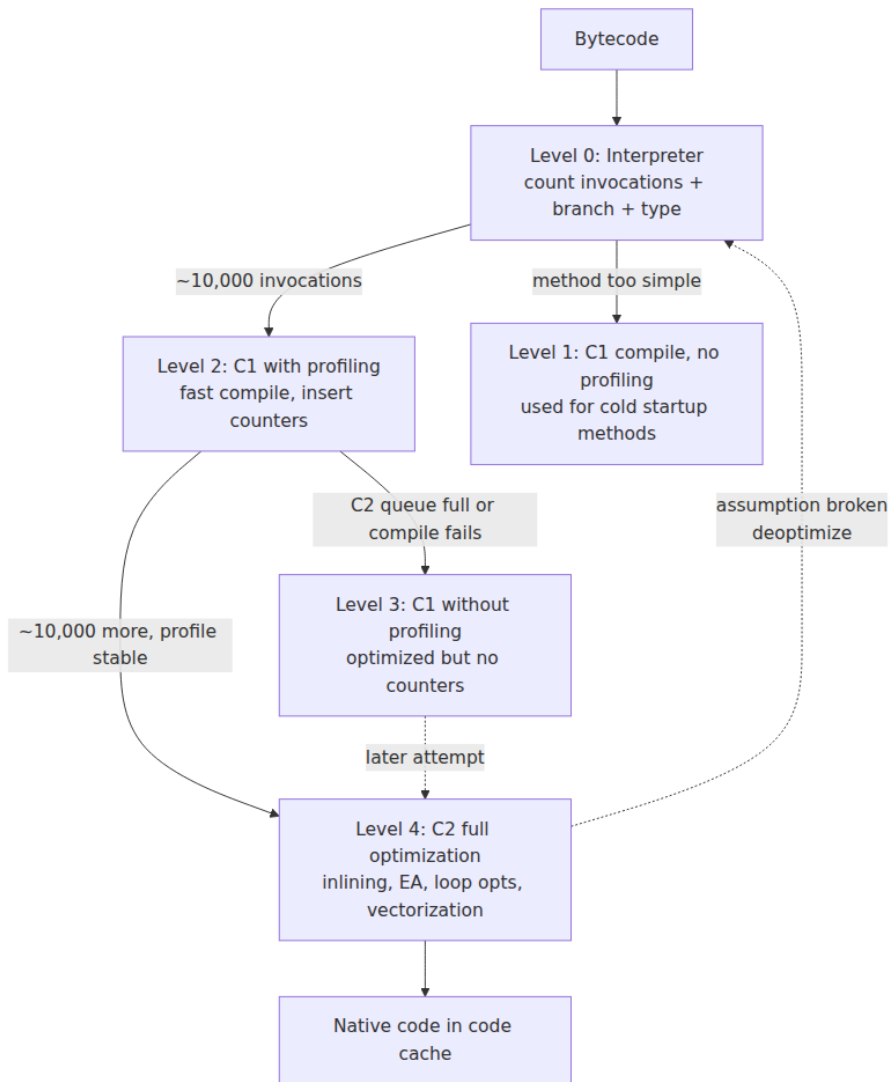
HotSpot is the reference JVM — the one shipped in OpenJDK, Oracle JDK, Amazon Corretto, Eclipse Temurin, and every major distribution. Its distinguishing feature is **adaptive optimization**: it profiles execution behavior at runtime and compiles hot methods to optimized native code, with the ability to deoptimize and recompile when assumptions break.

7.1 Tiered compilation — the five levels

HotSpot implements tiered compilation with five levels:

Level	Compiler	What happens	Profile collected?
0	Interpreter	Execute bytecode, count invocations	Yes — invocation count, branch taken/not-taken
1	C1 (limited)	Compile with no profiling, fast compile	No — used for startup methods
2	C1 (full profiling)	Compile with full profiling: type, branch, call	Yes — invoked when level 0 count hits threshold
3	C1 (no profiling)	Compile with C1 optimizations, no profiling overhead	No — used when profile is already collected
4	C2	Full optimization: inlining, escape analysis, vectorization	No — consumes profiles from level 2

The default flow: 1. Method starts at level 0 (interpreter). Invocation counter increments. 2. At `-XX:CompileThreshold` (default 10,000 for server, 1,500 for client), method compiles at level 2 (C1 with profiling). 3. If the C2 queue is not backlogged, and the method accumulates enough profiling data (another threshold), it compiles at level 4 (C2). The level 3 compilation may occur in parallel as an intermediate step. 4. If C2 compilation fails or the method is "too hot" for C1 to handle without profiling, it stays at level 3 (C1 without profiling).



7.2 What the JIT actually optimizes

The C2 compiler (and increasingly, GraalVM's Graal compiler when used as C2 replacement) performs optimizations that are only possible because the JVM profiles runtime behavior:

- **Method inlining:** The most impactful optimization. C2 inlines small methods (up to `-XX:MaxInlineSize=35` bytecodes for cold methods,

`-XX:FreqInlineSize=325` for hot methods) subject to `MaxInlineLevel` (depth). Inlining eliminates call overhead, enables constant propagation across method boundaries, and exposes the callee's data flow to further optimization. The JIT tracks call site polymorphism — monomorphic call sites (one receiver type) inline with a single type guard; bimorphic (two types) with two guards; megamorphic stays virtual.

- **Escape analysis:** When an object does not escape its thread or its method, C2 can scalar-replace it — allocating fields as registers instead of heap objects. This eliminates GC pressure entirely for short-lived objects. Enabled by `-XX:+DoEscapeAnalysis` (default). Also enables **lock elision**: if an object is thread-local, `synchronized` on it is a no-op and the monitor is eliminated.
- **Loop optimizations:** Loop unrolling, loop peeling, loop predication (eliminating bounds checks inside loops by hoisting the check), and auto-vectorization (SIMD) of tight loops operating on arrays of primitives.
- **Null check and range check elimination:** If the JIT can prove a reference is non-null (through control flow or type profiling), it eliminates the `null` check. Similarly, array bounds checks can be eliminated when the JIT proves the index is within range.
- **Dead code elimination and constant folding:** Branches that the profiling data shows are never taken are eliminated. Constants are propagated and folded. This is why `if (DEBUG)` blocks (with `static final boolean DEBUG = false`) vanish in optimized code — the JIT sees the constant and eliminates the dead branch.

7.3 Deoptimization — the safety net

Deoptimization is how the JVM recovers when its assumptions break. A common scenario:

1. C2 observes that a call site is monomorphic (always `ArrayList`) and inlines the `ArrayList.get` method with a type guard.
2. Later, a new class is loaded that also implements `List`, and the call site becomes bimorphic.
3. The type guard fails — the compiled code detects the unexpected type at runtime.

4. C2 deoptimizes the method: it discards the compiled code, reconstructs the interpreter frame from the deoptimization metadata, and resumes execution at level 0.
5. The method recompiles with updated profiling data.

Frequent deoptimization signals unstable assumptions. Common causes: - **Reflection-heavy code** where the target method changes due to proxy generation. - **Megamorphic virtual call sites** where more than two types flow through. - **Class redefinition** (JVM TI, hot-swapping). - **Uncommon traps** — the JIT inserts guards for assumptions it cannot prove statically; when the guard fires, deoptimization occurs.

7.4 GraalVM and Graal as C2

GraalVM offers **Graal** as a replacement for C2 (the "Graal JIT"). Graal is written in Java itself (unlike C2, which is written in a C++-like DSL called "Ideal Graph Representation Language" that compiles to C++). Graal supports: - Higher-level IR and more aggressive speculative optimizations. - Better support for newer JVM features (valhalla/value types, vector API, panama/FFM). - **GraalVM Native Image** — ahead-of-time compilation via the `native-image` tool, which performs closed-world analysis (no dynamic class loading, no JIT, no recompilation). Useful for microservices with sub-second startup requirements, but with significant trade-offs: no reflection by default (requires configuration), no dynamic代理, larger binary sizes for complex applications, and lower peak throughput compared to JIT.

8. The JDK layout — what lives where

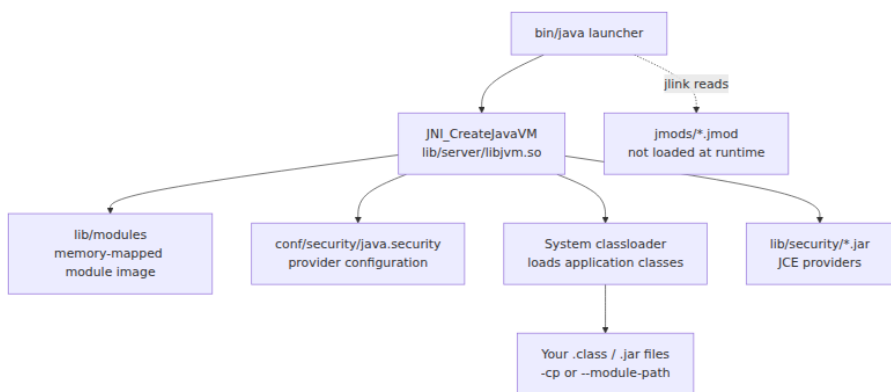
Understanding the JDK's on-disk layout is not trivia — it matters for container images (what to copy), module resolution (what `java` loads), and debugging (where `jmods` and symbols live). Since JDK 9 and the Java Platform Module System (JPMS), the layout is modular.

```

$JAVA_HOME/
└─ bin/                # java, javac, jar, jmod, jlink,
                        keytool, etc.
└─ conf/              # security (java.security,
                        default.policy), logging, net
└─ └─ security/
└─ └─ └─ java.security # SecurityManager config, TLS
                        providers, PKCS
└─ └─ └─ policy/      # default and full policy files
└─ include/           # JNI headers (jni.h, jni_md.h) for
                        native code compilation
└─ └─ jni.h
└─ └─ linux/          # platform-specific JNI types
└─ lib/               # The actual runtime
└─ └─ modules
# The module image (all system modules in one file)
└─ └─ jli/            # JVM launcher (libjli.so)
└─ └─ server/         # HotSpot server VM (libjvm.so)
└─ └─ jvm.cfg         # Server/client/minimal VM selection
└─ └─ security/       # JCE providers (SunJCE, SunRsaSign,
                        etc.)
└─ jmods/             # Module archives (for jlink, not for
                        runtime)
└─ └─ java.base.jmod
# The base module – everything depends on this
└─ └─ java.sql.jmod
└─ └─ java.logging.jmod
└─ └─ ...
└─ legal/             # License files
└─ release            # KEY=VALUE metadata (JAVA_VERSION,
                        OS_ARCH, etc.)

```

Key details: - **lib/modules** is the module image — a single file containing all system module classes, resources, and native libraries. The JVM memory-maps this file at startup. This is why **jlink** can produce minimal runtimes: it strips modules from this file. - **jmods/** is for the **jlink** tool, not for the running JVM. You can delete **jmods/** from a container image to save ~60 MB without affecting execution. - **lib/server/libjvm.so** is the HotSpot shared library — the entire JVM in one **.so**. The launcher (**bin/java**) loads this, initializes the JVM, and transfers control to **JNI_CreateJavaVM**. - **include/** is needed only when compiling JNI native code (**-I\$JAVA_HOME/include** **-I\$JAVA_HOME/include/linux**).



9. Distributed-systems lens

The JVM architecture described in this chapter has direct implications for operating Java/Kotlin services at scale:

- **Classfile compatibility and version skew.** A service compiled with JDK 21 (major version 65) will not load on a JDK 17 (major version 61) runtime. In multi-language polyglot fleets, this means a single JVM version must be pinned across all services. The classfile format version is the enforcer — there is no "partial forward compatibility." When upgrading JDK versions across a fleet, the classfile version check is the first gate.
- **Bytecode verification cost.** Verification is not free. For services loading thousands of classes (Spring Boot, microservices with fat JARs), startup includes tens of thousands of verification checks. The `StackMapTable`-based single-pass algorithm is fast ($O(n)$), but the I/O of reading classfiles from disk is the real bottleneck. This is why **CDS (Class Data Sharing)** and **AppCDS** matter: they archive verified, linked classes into a shared image that the JVM memory-maps, skipping both disk I/O and verification at startup.
- **The constant pool and inter-service contracts.** When services communicate via gRPC (Protobuf) or REST (JSON), the JVM-side serialization/deserialization involves class loading, method resolution, and constant pool lookups. A service that dynamically generates classes (via `MethodHandles.Lookup.defineClass` or

bytecode generation libraries like ByteBuddy) pays verification and JIT compilation costs on the fly — this can cause latency spikes in production.

- **JIT warmup and rolling deploys.** A freshly started pod executes interpreted bytecode until the JIT compiles hot methods. This takes 30 seconds to several minutes depending on load. Rolling deploys that replace all pods simultaneously cause a fleet-wide warmup dip. Mitigations: CDS/AppCDS archives (eliminate verification/classloading), AOTCache (JDK 24+, ahead-of-time cache of JIT state), Gradual rollouts with `minReadySeconds` and readiness gates.
 - **Code cache exhaustion.** A service that loads and unloads many classes (e.g., OSGi, application servers with hot-deploy) will fragment and fill the code cache, causing the JIT to stop compiling. Monitor with `jcmd Compiler.codecache` and alert when usage exceeds 80%.
 - **Metaspace leaks in containerized deployments.** The classic failure: a Spring Boot app with hot-reload (DevTools) or a servlet container that undeploys but retains a `ClassLoader` reference grows metaspace until the container OOM-kills the process. Set `-XX:MaxMetaspaceSize` explicitly — do not rely on the default unlimited growth.
-

Key takeaways

- The JVM classfile format is a dense, position-independent binary: CAFEBABE magic, version, constant pool (18 entry types), access flags, fields, methods, and attributes. Understanding this format is prerequisite to debugging `ClassFormatError`, `UnsupportedClassVersionError`, and bytecode generation libraries.
- The bytecode instruction set (~202 opcodes) is a stack machine ISA: instructions manipulate an operand stack, not registers. The five invocation opcodes (`invokevirtual`, `invokespecial`, `invokestatic`, `invokeinterface`, `invokedynamic`) have fundamentally different resolution and dispatch semantics — `invokedynamic` enables lambda, string concatenation, and runtime method selection without reflection overhead.
- Every method invocation creates a stack frame with a local variable array, operand stack, dynamic link, and return address. The JVM's

execution model is entirely defined by these frames — there are no caller-saved registers, no callee conventions — everything goes through the stack.

- Verification is mandatory and type-safe: the five-stage pipeline (format, semantic, bytecode/dataflow, stack map frames, integrity) prevents classfiles from subverting JVM memory safety. `StackMapTable` attributes (required since JDK 7) enable single-pass $O(n)$ verification.
- HotSpot's tiered compilation (interpreter $\rightarrow$ C1 $\rightarrow$ C2) uses profiling to guide optimization. Inlining, escape analysis, loop unrolling, and auto-vectorization are the high-impact optimizations — and they depend on stable type profiles and small methods.
- The JVM's memory is not just the heap: metaspace (class metadata), code cache (JIT output), thread stacks, and direct memory all consume native memory under the container limit. Size heap to 60–75% of container memory and set explicit limits on metaspace and code cache.
- The JDK layout (`lib/modules` , `lib/server/libjvm.so` , `jmods/`) matters for container images (strip `jmods/`), JNI compilation (`include/`), and module resolution.

Further reading

- *Java Virtual Machine Specification, Java SE 21 Edition* — <https://docs.oracle.com/javase/specs/jvms/se21/html/> — Chapters 4 (Class File Format), 6 (Instructions), 5 (Loading, Linking, and Initializing). The authoritative source.
- *Inside the Java Virtual Machine* (Bill Venners, McGraw-Hill) — the classic walkthrough of the JVM architecture; still accurate for the execution model.
- Shipilev, *JVM Anatomy Quarks* — <https://shipilev.net/jvm/anatomy/> — Short, deep dives into JVM internals (constant pool resolution, method handle invocation, interpreter dispatch).
- *Java Performance* (2nd ed., Scott Oaks, O'Reilly) — definitive guide to JVM performance including JIT compilation, GC, and profiling.
- OpenJDK HotSpot source — <https://github.com/openjdk/jdk> — `src/hotspot/share/classfile/` (classfile parsing), `src/hotspot/share/`

oops/ (class/field/method representation), `src/hotspot/share/runtime/` (interpreter, threads, safepoints).

- ASM bytecode library — <https://asm.ow2.io/> — a production-grade classfile reading/writing library; essential for bytecode manipulation, code generation, and instrumentation.
- The `javap` tool docs — <https://docs.oracle.com/javase/8/docs/technotes/tools/unix/javap.html> — `-c` (disassemble), `-v` (verbose with constant pool and attributes), `-p` (show private members).

Chapter 2 — Class Loading, Linking, Verification, and Modules (JPMS)

What this chapter covers. After Chapter 1 introduced the JVM architecture — classfiles, bytecodes, and the runtime data areas — this chapter follows a class from its first byte of bytecode on disk through every transformation the JVM performs before the first instruction of its static initializer executes. We examine the three built-in class loaders, the parent-first delegation model, and why parallel-capable loaders exist. We walk through the three linking phases — verification (including StackMapTable-based control-flow analysis), preparation, and resolution — and then cover class initialization: the `<clinit>` method, the deadlock patterns that lurk in circular static dependencies, and the precise ordering rules the JVM specification mandates. We confront the practical pain of ClassLoader leaks and the PermGen-to-Metaspace migration that JDK 8 brought. We then cover the Java Platform Module System (JPMS): module declarations, readability edges, qualified exports, module layers, and the build-time tools `jdeps` and `jlink`. Finally, we look at custom ClassLoader design patterns that underpin OSGi, application servers, and modern plugin architectures — with code you can run.

Learning goals — after this chapter you should be able to:

- Describe the three-tier built-in ClassLoader hierarchy (bootstrap, platform, application) and explain which types each loader serves.
- Implement parent-first delegation and explain when and how to break it.

- Walk through the linking pipeline — verification, preparation, resolution — and explain what StackMapTable frames do during bytecode verification.
- Explain the `<clinit>` initialization procedure, its synchronization rules, and how circular static dependencies cause classloader-level deadlocks.
- Diagnose a Metaspace OOM caused by ClassLoader leaks and identify the characteristic symptoms.
- Read and write a `module-info.java` declaration, explain `exports`, `opens`, and `requires` directives, and reason about module readability graphs.
- Use `jdeps --module-path` to analyze module dependencies and `jlink` to compose a custom runtime image.
- Design a custom ClassLoader for plugin isolation, including thread-context classloader management.

The class loading machinery

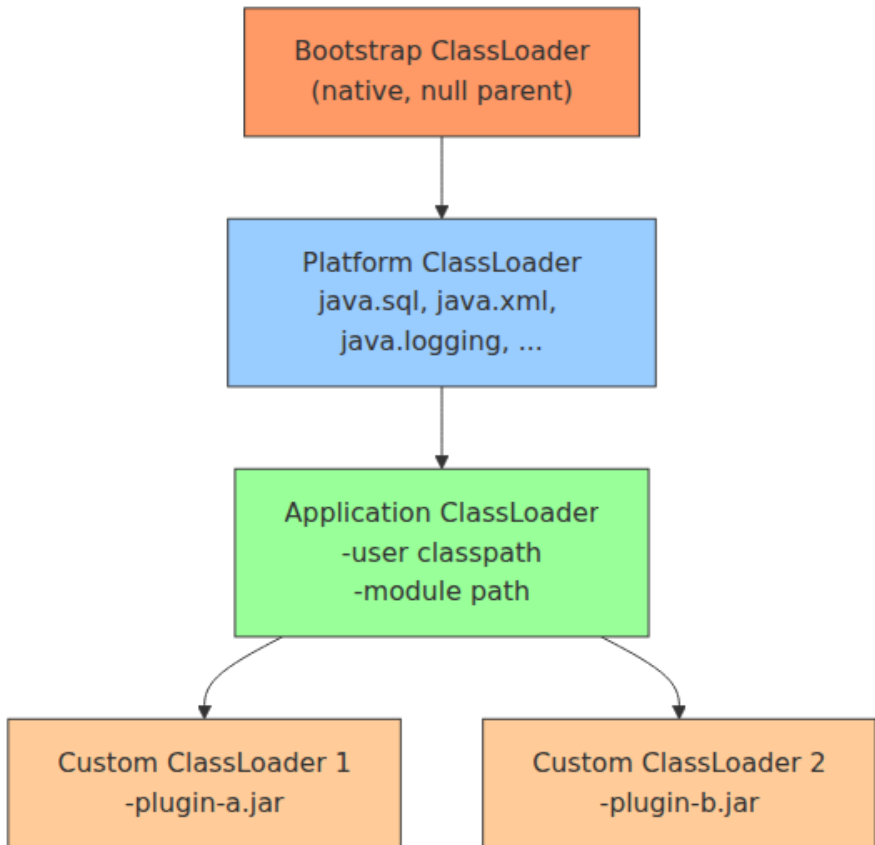
When the JVM needs a class or interface that has not yet been loaded, it does not simply open a `.class` file and parse it. It consults a *class loading subsystem* — a carefully layered architecture of loaders, delegation links, and verification passes — that has evolved over three decades. Understanding this machinery matters for two reasons every senior backend engineer cares about: **correctness** (a class loaded by the wrong loader is a different type entirely, leading to `ClassCastException` or security violations) and **liveness** (ClassLoader leaks are one of the most common causes of slow, mysterious out-of-memory failures in long-running JVM services).

The built-in loader hierarchy

The JVM ships with three built-in class loaders, arranged in a parent-child tree. Each loader has a well-defined scope:

Loader	Parent	Scope	Implementation
Bootstrap	null	<code>java.base</code> and the boot classpath (<code>-Xbootclasspath</code>)	Native C++ code

Loader	Parent	Scope	Implementation
Platform	Bootstrap	<code>java.sql</code> , <code>java.xml</code> , <code>java.logging</code> , etc. (JDK modules with <code>-Djava.system.classes.loader</code>)	<code>jdk.internal.loader.CL</code>
Application	Platform	User classpath (<code>-cp</code> , <code>-jar</code> , module path)	<code>jdk.internal.loader.CL</code>



You can inspect these loaders at runtime:

```

public class LoaderHierarchy {
    public static void main(String[] args) {
        ClassLoader cl = LoaderHierarchy.class.getClassLoader();
        while (cl != null) {
            System.out.println(cl);
            cl = cl.getParent();
        }
        System.out.println(null); // bootstrap
    }
}

```

Output on JDK 21:

```

jdk.internal.loader.ClassLoaders$AppClassLoader@7852e922
jdk.internal.loader.ClassLoaders$PlatformClassLoader@4e25154f
null

```

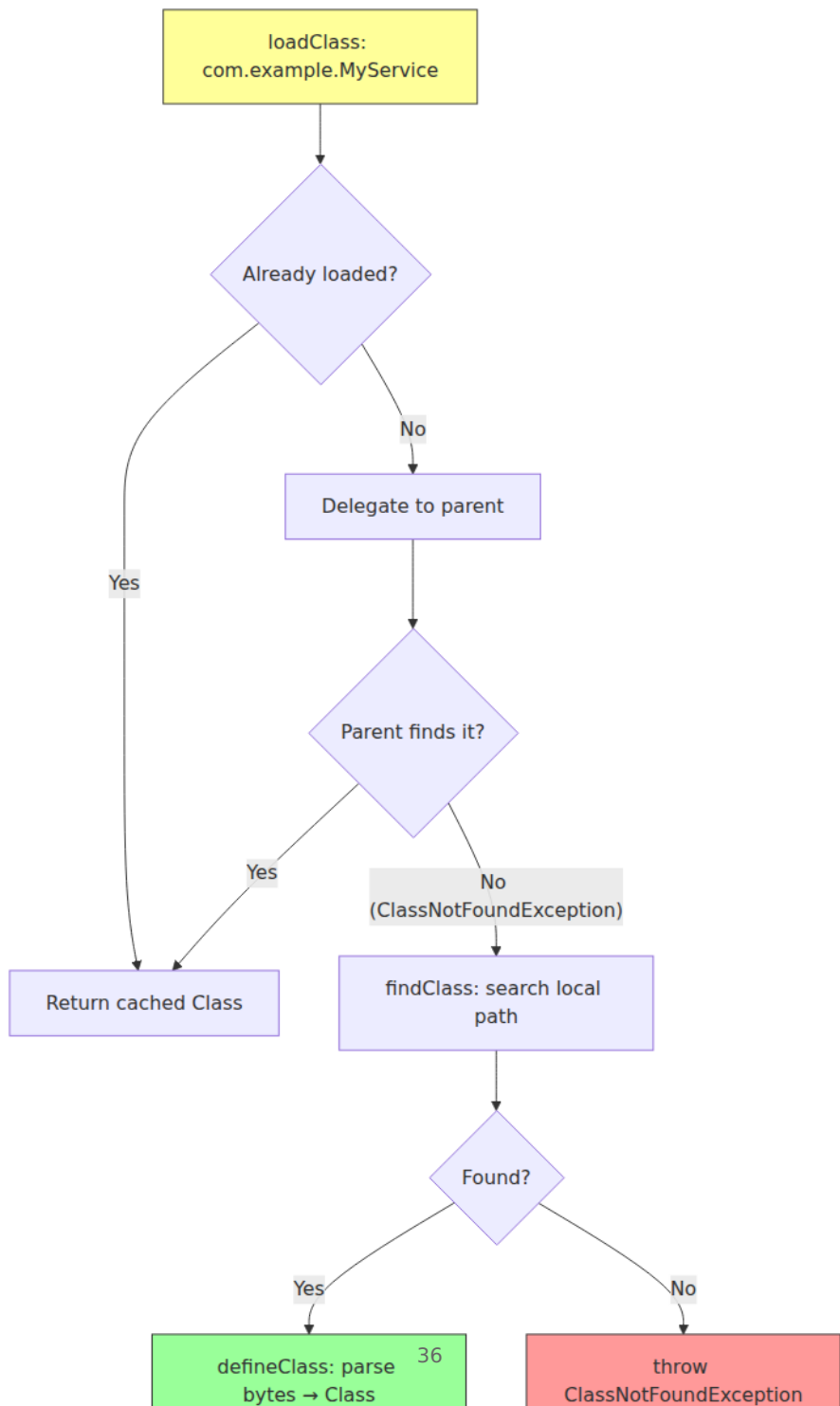
The `null` at the end is the bootstrap loader — it is implemented in native code and has no Java representation, so `getParent()` returns `null`.

A distributed-systems note. In a microservice fleet, the application class loader is effectively *the service boundary*. Everything above it — platform modules, the bootstrap image classes — is shared infrastructure. Everything below it — your fat-jar, your shaded dependencies — is your service's mutable, independently deployable unit. The class loading hierarchy is where the "shared kernel / mutable payload" split of a fleet actually lives, inside every JVM.

The parent-first delegation model

When the application loader needs a class, the JVM calls its `loadClass(name)` method. The default implementation does *not* look in the local classpath first. Instead, it follows the **parent-first delegation model**:

1. If the class has already been loaded by this loader (check the cache), return the cached `Class<?>` object.
2. Delegate to the parent loader by calling `parent.loadClass(name)`.
3. Only if the parent throws `ClassNotFoundException`, call `findClass(name)` on the current loader.



This design ensures that core Java classes — `java.lang.String`, `java.util.List` — are always loaded by the bootstrap or platform loader, never by an application loader. This is both a correctness guarantee and a security boundary: you cannot ship a malicious `java.lang.String` on your classpath and have the JVM use it for its internal operations.

The implementation in `java.lang.ClassLoader` is approximately:

```
protected Class<?> loadClass(String name, boolean resolve)
    throws ClassNotFoundException {
    synchronized (getClassLoadingLock(name)) {
        // Step 1: check local cache
        Class<?> c = findLoadedClass(name);
        if (c == null) {
            try {
                // Step 2: delegate to parent
                if (parent != null) {
                    c = parent.loadClass(name, false);
                } else {
                    // parent is null → bootstrap loader
                    c = findBootstrapClassOrNull(name);
                }
            } catch (ClassNotFoundException e) {
                // parent could not find it
            }
            if (c == null) {
                // Step 3: find locally
                c = findClass(name);
            }
        }
        if (resolve) {
            resolveClass(c);
        }
        return c;
    }
}
```

Breaking delegation: child-first loading

Some frameworks intentionally break parent-first delegation. **OSGi** and **application servers** (JBoss/WildFly, WebSphere) use *child-first* (also called *thread-context* or *service-provider*) loading so that each deployment unit can ship its own version of a library without clashing with other deployments on the same JVM.

The pattern uses `Thread.currentThread().getContextClassLoader()` as the lookup anchor. In OSGi, each bundle gets its own class loader whose

`loadClass` implementation tries the bundle's own JARs first, then the framework's execution environment, then the parent — the exact reverse of standard delegation.

```

public class ChildFirstClassLoader extends ClassLoader {

    private final URL[] localUrls;

    public ChildFirstClassLoader(URL[] urls, ClassLoader parent) {
        super(parent);
        this.localUrls = urls;
    }

    @Override
    protected Class<?> loadClass(String name, boolean resolve)
        throws ClassNotFoundException {
        synchronized (getClassLoadingLock(name)) {
            // 1. Check local cache
            Class<?> c = findLoadedClass(name);
            if (c != null) return c;

            // 2. Java core classes MUST be loaded by the bootstrap/
parent
            if (name.startsWith("java.") || name.startsWith("jdk.")) {
                return super.loadClass(name, resolve);
            }

            // 3. Try local JARs first (child-first)
            try {
                c = findClass(name);
                if (resolve) resolveClass(c);
                return c;
            } catch (ClassNotFoundException e) {
                // 4. Fall back to parent
                return super.loadClass(name, resolve);
            }
        }
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        String path = name.replace('.', '/') + ".class";
        try (var is = findResource(path).openStream()) {
            byte[] bytes = is.readAllBytes();
            return defineClass(name, bytes, 0, bytes.length);
        } catch (Exception e) {
            throw new ClassNotFoundException(name, e);
        }
    }

    @Override
    protected java.net.URL findResource(String name) {
        for (URL url : localUrls) {

```

```

        try {
            java.net.URL resource = new java.net.URL(url, name);
            resource.openConnection().connect(); // quick probe
            return resource;
        } catch (Exception ignored) {}
    }
    return null;
}
}

```

Child-first loading is a **blast radius control** mechanism. In a fleet where ten services share a JVM (an app-server deployment, or a legacy monolith being decomposed), child-first loading prevents one service's transitive dependency from poisoning another's class resolution — the same isolation guarantee that separate JVMs give you, but without the cost of a process per service.

Parallel-capable loaders

Starting with JDK 7, the `java.lang.ClassLoader` class has the `registerAsParallelCapable()` static method. When a `ClassLoader` subclass calls this in its constructor, the JVM knows it can resolve classes from multiple loaders concurrently without external synchronization — the loader handles its own locking per class name (via `getClassLoadingLock(name)`).

Both `AppClassLoader` and `PlatformClassLoader` register as parallel-capable. This matters in multithreaded startup: if five threads all try to load `com.google.gson.Gson` simultaneously, only one actually loads it while the others block on the per-name lock, then all five get the same `Class<?>` object. Without parallel-capable registration, the JVM falls back to a coarser lock on the entire loader, serializing all loads.

You can observe this behavior with a debugger or a Java Flight Recorder trace. During parallel class loading, HotSpot creates one thread per class to load its dependencies concurrently, then merges the results. The parallel loading path is visible in the `VM.classloader` JFR event:

```

# Record parallel class loading events
jfr start --settings=profile --duration=30s -f classload.jfr
java -XX:+UnlockDiagnosticVMOptions -XX:+TraceClassLoading MyApplication
jfr print --events jdk.ClassLoaderConstraints classload.jfr

```


The `ClassLoaderService` in the HotSpot VM maintains a per-loader counter of loaded classes. On a cold start loading ~12,000 classes (typical for a Spring Boot application), parallel-capable loading reduces startup time by 15-25% on multi-core machines compared to the serial fallback.

If you write a custom `ClassLoader` for a plugin system, register it as parallel-capable unless you have a reason not to:

```
public class PluginClassLoader extends ClassLoader {
    static {
        registerAsParallelCapable(); // enables per-name locking
    }
    // ...
}
```

One caveat: parallel-capable loaders require that `findClass` and `defineClass` be thread-safe *on their own*. The JVM guarantees that `loadClass` serializes per-class-name via `getClassLoadingLock`, but your `findClass` implementation — which reads from disk, network, or any external source — must handle concurrent calls correctly. In practice this means either synchronizing the I/O or designing the lookup to be idempotent.

Linking: verification, preparation, resolution

Once a class loader has produced a byte array via `findClass`, it calls `defineClass`, which hands the bytes to the JVM. The JVM then runs the class through **linking** — a three-phase pipeline that turns raw bytes into a usable class with valid bytecode, allocated static storage, and symbolic references resolved to actual types.

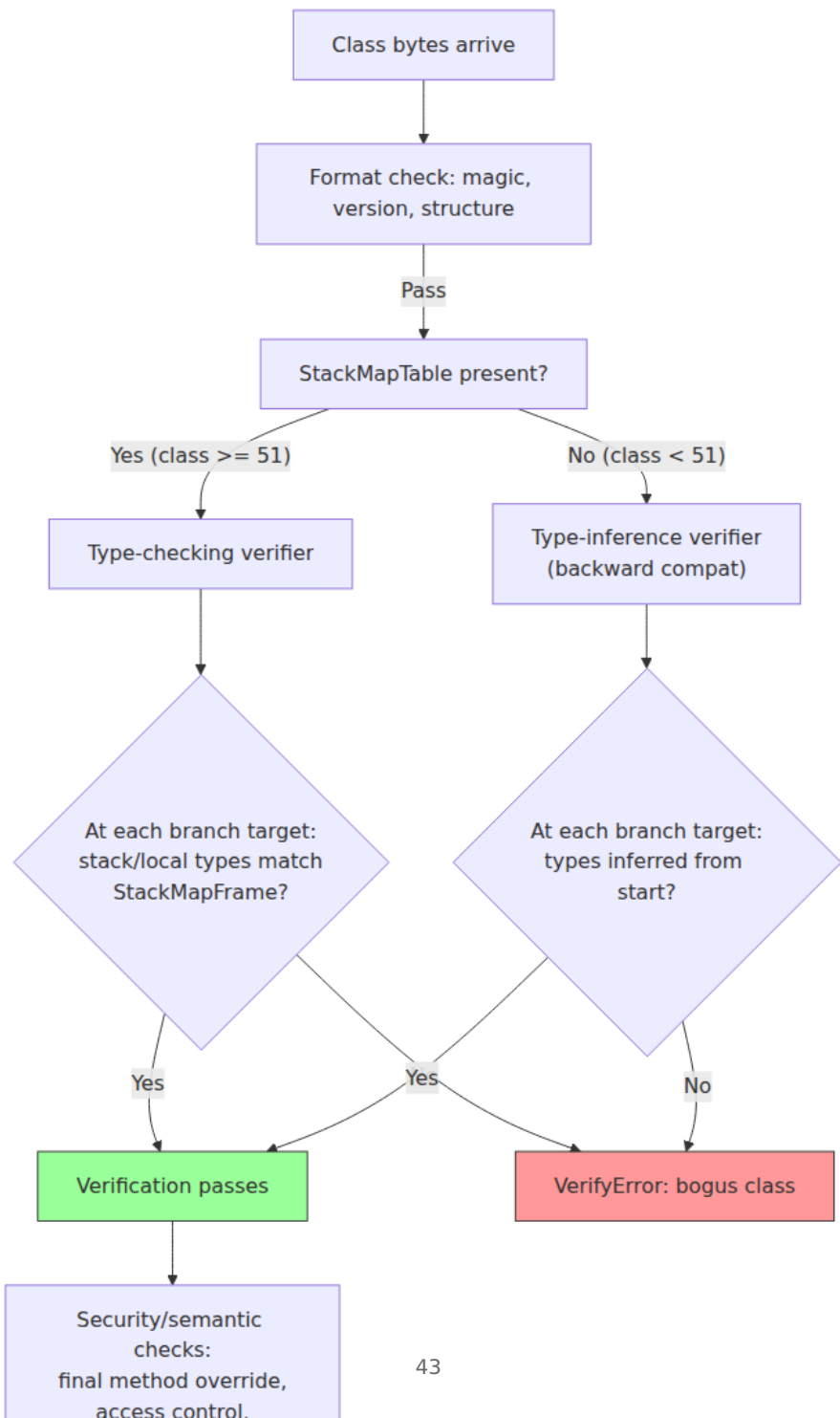


Verification: making sure the bytes are safe

Verification is the most computationally expensive linking phase. It ensures that the bytecode does not violate the structural constraints of the JVM — that it will not corrupt memory, bypass access checks, or perform type-unsafe operations. The JVM specification defines four types of verification, roughly ordered by the era in which they were introduced:

1. Format checking. The classfile magic number (`0xCAFEBAFE`), version numbers, and structural validity (correct constant pool entry types, valid field/method descriptors) are verified first. A malformed classfile is rejected immediately without further processing.

2. Bytecode verification (type checking). This is the core of verification. Since JDK 7 (JSR 202), the HotSpot JVM uses a *type-checking* verifier based on the `StackMapTable` attribute rather than the older type-inference approach. The `StackMapTable` is a precomputed map, embedded by `javac`, that describes the type state (primitive types or class/interface types) of the operand stack and local variables at every branch target and exception handler. The verifier walks the bytecode, checking that at every point the actual types match the declared `StackMapFrame` types.



The type-checking verifier is both faster and simpler than the type-inference verifier, but it requires the compiler to emit `StackMapTable` data. If you are hand-writing bytecode (e.g., via ASM, ByteBuddy, or a bytecode manipulation framework), you must emit correct `StackMapTable` entries or your class will fail verification at load time.

3. Access control verification. The verifier checks that the bytecode does not violate `private`, `protected`, or package-private access modifiers. It also checks that `final` methods are not overridden and that `this` in a constructor refers to a type compatible with the enclosing class.

4. Data-flow analysis. For each method, the verifier ensures that all instructions can actually be reached, that the operand stack is never underflowed, and that every return path has the correct return type on the stack. This catches dead code that would be otherwise harmless and, more importantly, code that tries to use uninitialized objects.

A verification failure throws `VerifyError`, and the class is never linked:

```
// This bytecode snippet is invalid: using a local before initializing
it
// The StackMapTable would show the type as 'uninitialized' at the
point
// of use, and verification fails:

public class VerifyDemo {
    public String broken() {
        String s;
        // s is uninitialized here – bytecode that reads it fails
        verification
        return s; // VerifyError at class load time
    }
}
```

Preparation: allocating static storage

After verification, the JVM allocates memory for the class's `static` fields and sets them to their **default values** (0, null, false, etc.). Preparation does *not* execute any Java code — it is a metadata-level pass that populates the `PerClassData` structures. The actual static initializers (`static {}` blocks and static field initializers) run later, during initialization.

This distinction matters for a subtle reason. Consider:

```
public class Counter {  
    public static int x = 42;  
}
```

After preparation, `Counter.x` is `0` — not `42`. The assignment `x = 42` is part of the class's `<clinit>` method, which runs during initialization. If another thread reads `Counter.x` between preparation and the completion of `<clinit>`, it sees `0`.

Resolution: binding symbolic references

Resolution transforms **symbolic references** (strings in the constant pool) into **direct references** (pointers to class metadata, field offsets, method entry points). For example, when bytecode references `java/lang/String.length()`, the symbolic reference is a UTF-8 string in the constant pool. Resolution finds the `java.lang.String` class, locates the `length()` method, and stores the resolved method entry point.

The JVM specification allows resolution to happen at any point — during linking, during first use, or even lazily. HotSpot's actual behavior is demand-driven: resolution typically happens when the class is first actively used, not when it is loaded. This means a class can be successfully loaded and linked even if one of its symbolic references points to a class that does not exist — as long as that reference is never actually invoked.

This lazy resolution has operational implications: in a microservice that shades dependencies (relocating packages to avoid version conflicts), a `NoClassDefFoundError` may surface only after weeks of running in production, when a rarely-exercised code path is finally reached. A static analysis tool like `jdeps` (covered later in this chapter) can catch these issues before deployment.

Initialization: `<clinit>` and the static order

Initialization is the phase that executes user code. The `<clinit>` method (the compiler-generated class initializer) sets static fields to their declared values and runs static blocks. The JVM specification defines strict rules for when `<clinit>` runs and what happens when multiple threads concurrent access a class for the first time.

The `<clinit>` procedure

1. The JVM acquires an initialization lock on the `Class<?>` object. This monitor is distinct from any `synchronized` block the programmer writes — it is internal to the JVM.
2. If the class is already being initialized by the current thread (recursive initialization), the lock is released and `<clinit>` proceeds — this allows self-referential static fields like `Class.forName("Foo")` inside `Foo`'s static initializer.
3. If the class is being initialized by a *different* thread, the current thread waits on the lock. When the other thread completes `<clinit>` (or throws an exception), all waiters are notified.
4. Before executing `<clinit>`, the JVM recursively initializes all classes that are directly referenced by `<clinit>`. This is the root of both the initialization ordering guarantee and the deadlock hazard.

Initialization ordering

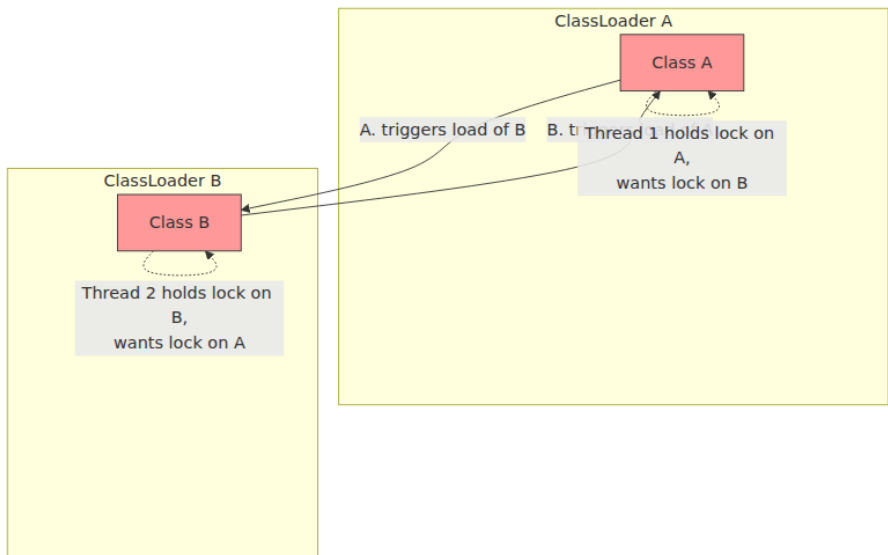
The JVM guarantees that a class is initialized before it is used in any of these ways:

- Creating an instance (not counting `Class.newInstance()`)
- Invoking a static method
- Accessing/modifying a static field (unless it is a compile-time constant)
- Reflection (`Class.forName("com.example.Foo")`)
- Initializing a subclass (which initializes the superclass first)

The parent-before-child ordering ensures that a subclass's `<clinit>` can safely reference the superclass's static fields.

The classloader deadlock pattern

Circular static initialization across two class loaders is one of the most insidious JVM bugs. Consider:



Thread 1 loads class A. The JVM locks A's `Class<?>` object and begins executing A's `<clinit>`. A's static initializer references class B, so the JVM tries to load B — but B's class loader is the same one, so it locks B's `Class<?>` object and begins B's `<clinit>`. So far, no problem — this is recursion on a single thread, which the JVM handles.

The deadlock happens with two class loaders. Thread 1 loads A via loader1; A's `<clinit>` references B, which lives in loader2. The JVM tries to initialize B under loader2 and blocks waiting for loader2's lock. Meanwhile, Thread 2 loads B via loader2; B's `<clinit>` references A, which lives in loader1. Thread 2 blocks on loader1's lock. Both threads are now deadlocked.

In practice this pattern appears in OSGi and application server deployments where different bundles/modules have separate class loaders and static initializers that cross module boundaries. The symptom is a JVM that hangs during startup with threads stuck in `Thread.State.BLOCKED` on class initialization locks — visible in a thread dump as:

```
"main" #1 prio=5 tid=0x00007f8a1c001000
  java.lang.Thread.State: BLOCKED (on object monitor)
    at com.example.A.<clinit>(A.java:10)
      - waiting to lock <0x0000000076b1234a0> (a java.lang.Class)
    at com.example.B.<clinit>(B.java:15)
      - locked <0x0000000076b1234b0> (a java.lang.Class)
```

The fix is to ensure that static initialization within one class loader never directly or indirectly triggers static initialization in a different class loader. Lazy loading via reflection, or moving cross-loader references from static initializers to explicit method calls, breaks the cycle.

ClassLoader leaks and PermGen → Metaspace

The leak mechanism

A `ClassLoader` leak occurs when a `ClassLoader` instance cannot be garbage collected because something still holds a reference to it — or to one of its classes. Since the loader holds a reference to every `Class<?>` it loaded, and each `Class<?>` holds a reference to its loader, the entire loader subgraph is pinned in memory.

The most common cause is **static state** that captures classloader-scoped objects. A static `Map` in a class loaded by the application class loader that stores references to objects from a plugin's class loader will pin the entire plugin's loader tree in memory. This is the single most common source of class loader leaks in production JVM services.

```
// BUG: this pins the entire plugin class loader in memory
public class GlobalRegistry {
    private static final Map<String, Object> instances = new HashMap<>();

    public static void register(String key, Object instance) {
        instances.put(key, instance);
        // instance was created by a plugin class loader
        // the plugin class loader is now permanently reachable
    }
}
```

When a web application is redeployed (hot deploy in Tomcat, Jetty, etc.), the old class loader is supposed to become unreachable and be GC'd along with its loaded classes. If any static reference leaks, the old loader

stays alive, its classes stay in memory, and the new deployment loads a fresh set. Over repeated deploys, memory consumption grows linearly.

PermGen and the migration to Metaspacex

In JDK 7 and earlier, class metadata (class names, method/field descriptors, constant pool, bytecode, annotations) was stored in **PermGen** (Permanent Generation) — a fixed-size memory region of the old generation. PermGen had a hard upper limit (default 82 MB on 32-bit, 164 MB on 64-bit) set by `-XX:MaxPermSize`. ClassLoader leaks in PermGen manifested as:

```
java.lang.OutOfMemoryError: PermGen space
```

This error was ubiquitous in the mid-2000s to mid-2010s in app-server deployments with hot redeploy.

Starting with JDK 8, PermGen was eliminated. Class metadata was moved to **Metaspacex** — a native memory region (outside the Java heap) managed by the JVM. Metaspacex grows automatically (up to `-XX:MaxMetaspacexSize`, which defaults to unlimited on most platforms). The same ClassLoader leak now manifests as:

```
java.lang.OutOfMemoryError: Metaspacex
```

The symptoms differ subtly. PermGen leaks hit a hard ceiling and threw immediately. Metaspacex leaks grow the JVM's RSS gradually — you may not notice until the OS OOM-kills the process, or until monitoring catches the climbing native memory. This makes Metaspacex leaks harder to detect but no less damaging.

Here is a reproducer for a Metaspacex leak:

```

import java.net.URL;
import java.net.URLClassLoader;

/**
 * Demonstrate a ClassLoader leak: dynamic class generation without
 * cleanup.
 * In production, this pattern appears when frameworks (CGLIB, ASM)
 * generate
 * classes without tracking the loader lifecycle.
 */
public class MetaspaceLeakDemo {

    public static void main(String[] args) throws Exception {
        long count = 0;
        while (true) {
            // Each iteration creates a new loader (simulates a hot
            // deploy)
            URLClassLoader loader = new URLClassLoader(
                new URL[]{new URL("file:///tmp/plugins/")});
            // Load a class that references the loader
            Class<?> cls = loader.loadClass("com.example.Plugin");
            cls.getDeclaredConstructor().newInstance();
            //loader.close(); // uncomment to fix the leak
            count++;
            if (count % 1000 == 0) {
                System.out.println("Loaded " + count + " classes, "
                    + "free mem: " +
                    Runtime.getRuntime().freeMemory() / 1024 + " KB");
            }
        }
    }
}

```

Without `loader.close()`, the JVM's RSS grows until the process is killed. The fix is straightforward: when a class loader is no longer needed, call `close()` on it, and ensure that no static references survive. In real-world applications, this means:

- Framework registries (CDI containers, plugin managers) must use `WeakReference<ClassLoader>` or explicit lifecycle hooks.
- Thread-local variables set during request processing must be cleaned up in a `finally` block — a thread pool thread that retains a reference to a plugin's class loader from a previous request will pin the old loader.
- The `Thread.currentThread().setContextClassLoader()` must be reset after each request.

The Java Platform Module System (JPMS)

JPMS (Project Jigsaw, finalized in JDK 9, Jigsaw JSR 376) is the JVM's first-class module system. It addresses problems that ClassLoader hacks partially solved: reliable configuration (detecting missing dependencies at compile and link time rather than at runtime), strong encapsulation (preventing unauthorized reflective access), and platform compression (trimming the JDK to only the modules an application needs).

module-info.java

Every named module has a `module-info.java` at the root of its source tree. Here is a realistic example for a payment-processing service:

```
// src/main/java/module-info.java
module com.example.payments {
    // This module requires these other modules
    requires java.sql;
    requires java.logging;
    requires com.google.gson;
    requires com.google.guava;

    // But only Guava's ImmutableMap from the base package is public
    API
    exports com.example.payments.api to com.example.payments.client;
    exports com.example.payments.model;

    // Internal implementation is completely hidden
    // (no exports for com.example.payments.internal)

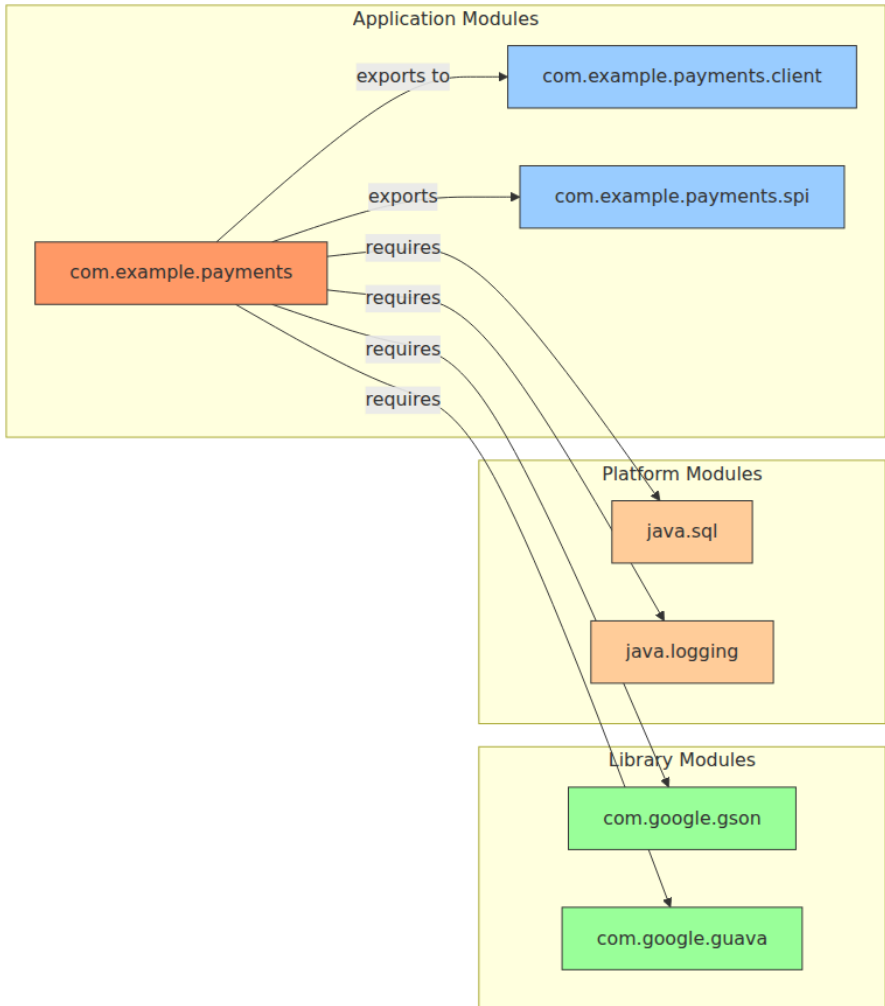
    // Reflection access for serialization frameworks
    opens com.example.payments.model to
        com.google.gson,
        com.fasterxml.jackson.databind;

    // Service provider interface
    provides com.example.payments.spi.PaymentGateway
        with com.example.payments.internal.StripeGateway;
    uses com.example.payments.spi.PaymentProcessor;
}
```

Readability and the module graph

A `requires` directive creates a **readability edge** in the module graph. When module A requires module B, A can access B's exported packages.

The readability graph is acyclic at the module level — circular module dependencies are rejected at link time.



exports vs opens

This distinction is critical and frequently misunderstood:

- `exports com.example.payments.api`: makes all public types in that package accessible to other modules for both compile-time references and runtime reflection. Without this, other modules cannot see the package at all (strong encapsulation).

- **exports ... to <module>**: a *qualified export*. Only the named module(s) get access. This is how you expose API to specific consumers without leaking internals to the entire module graph.
- **opens com.example.payments.model to com.google.gson**: grants *deep reflective access* — not just public members, but all members (including private fields) via `setAccessible(true)`. This is needed for serialization frameworks (Gson, Jackson, JAXB) that use reflection to instantiate and populate objects. A package that is only `exports`ed cannot be reflected upon with `setAccessible`.

The security implication is significant. Pre-JPMS, libraries like Gson could reflectively access any public (and, via `setAccessible`, even private) field of any class. JPMS restores the encapsulation boundary: without an `opens` directive, deep reflection throws `InaccessibleObjectException`. This is why migrating to JPMS in a Maven/Gradle project often surfaces reflection errors in Gson, Jackson, Hibernate, Spring, and other frameworks — you must add `opens` directives for every package those frameworks reflect upon.

Module layers

A **module layer** (`java.lang.ModuleLayer`) is a runtime partition of modules. The JVM starts with the **boot layer**, which contains the platform modules (`java.base`, `java.sql`, etc.) and any modules on the system module path. Your application can create additional layers:

```
// Create a child layer for plugin isolation
ModuleLayer bootLayer = ModuleLayer.boot();
ModuleLayer pluginLayer = bootLayer.defineModulesWithOneLoader(
    List.of(pluginModuleDescriptor),
    ClassLoader.getSystemClassLoader()
);
```

Each layer has its own set of modules and its own class loader. Modules in different layers can have the same name (providing namespace isolation), but the JVM resolves each module reference within its own layer first.

Module layers are the JPMS-native equivalent of OSGi's bundle resolution — they give you a structured way to create isolated, independently loadable groups of modules. Application servers and plugin frameworks

are beginning to adopt this API instead of rolling their own class loader hacks.

jdeps: analyzing module dependencies

`jdeps` is a static analysis tool that inspects `.class` files or JAR files and reports their module dependencies. It is essential for migrating a class-path project to the module path.

```
# Analyze a JAR for module dependencies
jdeps --module-path /path/to/libs/ myapp.jar

# Generate a module-info.java from an existing JAR (migration aid)
jdeps --generate-module-info /tmp/modinfo myapp.jar

# Check for split-package conflicts (two JARs providing the same
package)
jdeps --check myapp.jar

# Analyze which JDK internal APIs are used (migration from sun.misc.*,
com.sun.*)
jdeps --jdk-internals myapp.jar
```

The output of `--check` is particularly useful:

```
myapp.jar
[ok] myapp-1.0.jar
[Conflict] com.google.common.collect
    myapp-1.0.jar
    guava-31.1.jar
[Warning: Package split across modules]
    myapp-1.0.jar contains com.example.util
    utils-2.0.jar contains com.example.util
```

Split packages are forbidden by JPMS — two modules cannot export the same package. `jdeps --check` catches this before you attempt a module build.

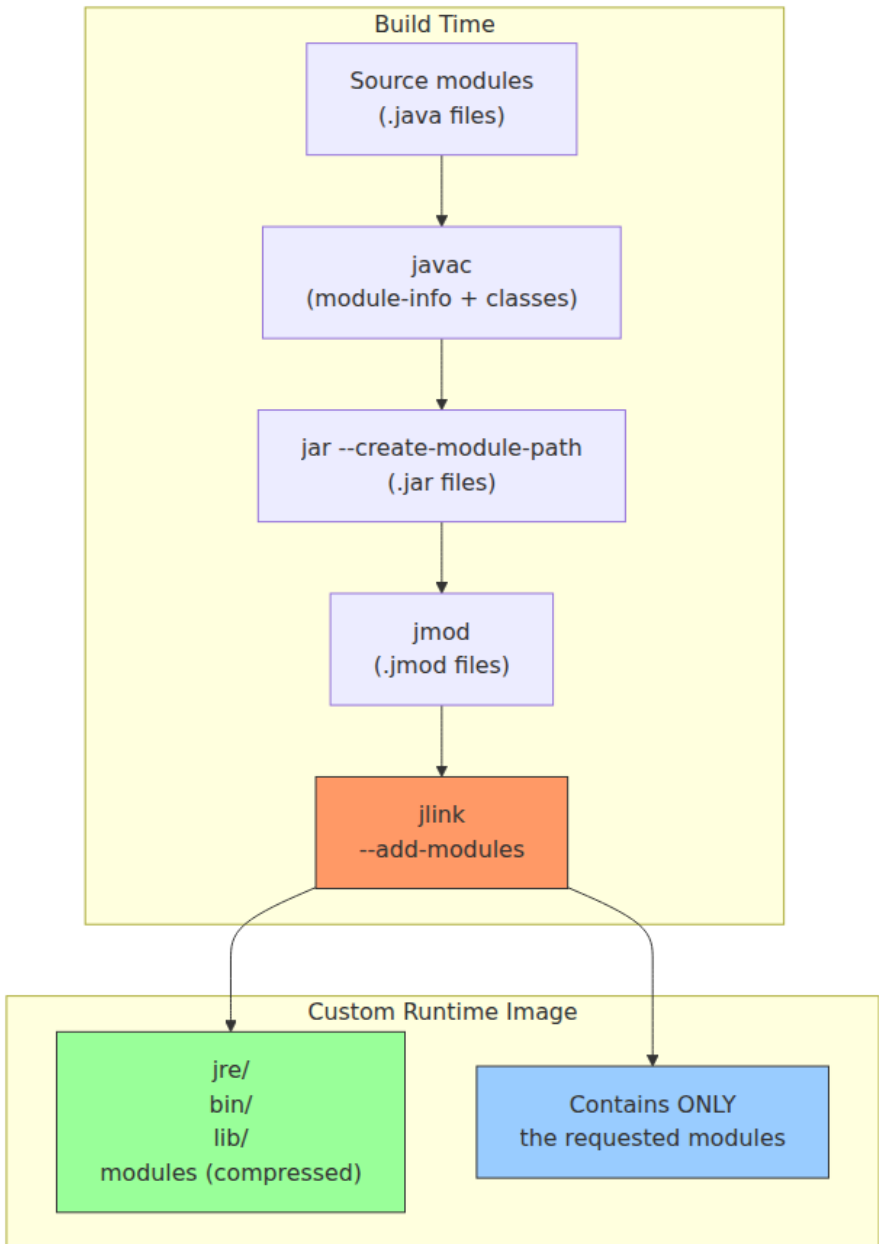
jlink: composing custom runtime images

`jlink` composes a custom JDK runtime image containing only the modules your application needs, eliminating unused modules (and their class data) entirely. This reduces the runtime image from ~300 MB (full JDK) to as little as 30-40 MB.

```
# First, analyze which modules your app needs
jdeps --print-module-deps --ignore-missing-deps myapp.jar
# Output:
java.base,java.logging,java.sql,com.google.gson,com.google.guava

# Build a custom runtime image
jlink \
  --module-path /path/to/jmods:/path/to/libs/*.jar \
  --add-modules java.base,java.logging,java.sql,com.google.guava \
  --strip-debug \
  --no-header-files \
  --compress zip-6 \
  --output /opt/myapp-runtime \
  --launcher myapp=com.example.app/com.example.app.Main

# Verify the image
/opt/myapp-runtime/bin/java --list-modules
# java.base@21.0.2
# java.logging@21.0.2
# java.sql@21.0.2
# com.google.guava@31.1
```



The startup time and memory improvement is substantial. A Spring Boot application that bundles a full JRE at 300 MB can be reduced to a jlink

image of 50 MB with 20-30% faster startup (fewer classes to load and verify). For containerized microservices where image size and cold start matter — serverless, Kubernetes HPA scaling — this is a practical optimization, not a theoretical one.

A distributed-systems note. JPMS enforces at build and link time the dependency constraints that your CI/CD pipeline, your Gradle/Maven POM, and your team's tribal knowledge were supposed to enforce at development time. In a fleet of hundreds of JVM services, JPMS shifts failure modes left: instead of a `NoClassDefFoundError` in production at 3 AM, you get a `jdeps` failure in CI at 10 AM. This is the same principle as moving from runtime schema validation to compile-time schema contracts — the earlier you catch the inconsistency, the cheaper the fix.

Custom ClassLoader patterns for plugin systems

The plugin architecture

Modern JVM applications — from application servers (Tomcat, WildFly) to build tools (Gradle, Maven) to data platforms (Kafka Connect, Elasticsearch) — use custom ClassLoaders to load plugins, extensions, or deployments with strong isolation. The fundamental requirements are:

1. **Isolation.** Plugin A's classes cannot see Plugin B's classes unless explicitly allowed.
2. **Delegation.** Both plugins can see the host application's classes (and the JDK classes).
3. **Lifecycle.** When a plugin is undeployed, its class loader and all loaded classes must become eligible for GC.
4. **Transparency.** The plugin author writes standard Java — they should not need to know that their code runs inside a custom loader.

Here is a minimal but production-realistic plugin loader:

```

public class PluginClassLoader extends ClassLoader {
    static {
        registerAsParallelCapable();
    }

    private final Path pluginDir;
    private final Set<String> systemPackages;

    public PluginClassLoader(Path pluginDir, ClassLoader parent) {
        super(parent);
        this.pluginDir = pluginDir;
        // Java core packages MUST be loaded by the bootstrap/platform
loader
        this.systemPackages = Set.of(
            "java.", "javax.", "jdk.", "sun.", "com.sun.",
            "org.xml.", "org.w3c.", "org.ietf."
        );
    }

    @Override
    protected Class<?> loadClass(String name, boolean resolve)
        throws ClassNotFoundException {
        synchronized (getClassLoadingLock(name)) {
            Class<?> c = findLoadedClass(name);
            if (c != null) return c;

            // System packages: parent-first (mandatory for
correctness)
            if (systemPackages.stream().anyMatch(name::startsWith)) {
                return super.loadClass(name, resolve);
            }

            // Plugin packages: child-first (isolation)
            try {
                String path = name.replace('.', '/') + ".class";
                URL resource = findResource(path);
                if (resource != null) {
                    byte[] bytes = resource.openStream().readAllBytes();
;
                    c = defineClass(name, bytes, 0, bytes.length);
                    if (resolve) resolveClass(c);
                    return c;
                }
            } catch (IOException e) {
                throw new ClassNotFoundException(name, e);
            }

            // Fallback to parent
            return super.loadClass(name, resolve);
        }
    }
}

```

```

    }

    @Override
    protected URL findResource(String name) {
        Path resource = pluginDir.resolve(name);
        if (Files.exists(resource)) {
            try {
                return resource.toUri().toURL();
            } catch (Exception e) {
                return null;
            }
        }
        return null;
    }
}

```

Thread context classloader management

The `Thread.currentThread().getContextClassLoader()` is the glue that makes plugin systems work with libraries that do not accept an explicit class loader parameter (JDBC drivers, JNDI lookups, serialization frameworks). The host application sets it before dispatching to a plugin and restores it afterward:

```

public class PluginDispatcher {

    public void dispatch(Plugin plugin) {
        Thread current = Thread.currentThread();
        ClassLoader original = current.getContextClassLoader();
        try {
            current.setContextClassLoader(plugin.getClassLoader());
            plugin.execute();
        } catch (Exception e) {
            log.error("Plugin execution failed", e);
        } finally {
            current.setContextClassLoader(original); // always restore
        }
    }
}

```

Failing to restore the context classloader is another common source of classloader leaks: a thread pool thread retains a reference to the old plugin's loader, and that loader can never be GC'd until the thread pool itself is shut down.

Comparison: JPMS modules vs ClassLoader isolation

Dimension	JPMS Modules	Custom ClassLoader
Dependency enforcement	Compile-time and link-time	Runtime (NoClassDefFoundError)
Encapsulation	Strong (module-info)	Weak (access checks only)
Isolation granularity	Module (package-level)	Loader (arbitrary)
Circular dependencies	Forbidden	Allowed
Hot deploy	Not supported	Native (loader lifecycle)
Ecosystem tooling	jdeps, jlink, jmod	OSGi (equinox/felix), custom

JPMS and ClassLoader isolation are complementary, not competing. A plugin framework can create a module layer per plugin (using a distinct class loader per layer), getting both JPMS's compile-time guarantees and runtime isolation.

Distributed-systems lens: class loading at fleet scale

At fleet scale, class loading matters in ways that are invisible on a single JVM:

- **JEP 310: Application Class-Data Sharing (AppCDS).** When you run hundreds of JVM instances across a cluster, startup time and memory footprint per instance multiply. AppCDS pre-processes your application's classes into a shared archive that every instance memory-maps, reducing both startup time and per-instance RSS. This is the same "pre-baked filesystem" trick that container layer caching uses — amortize the I/O and parsing once across N instances.
- **Class loading and container memory limits.** In Kubernetes, a container with `-Xmx2g` and default Metaspace can still OOM the container because Metaspace grows outside the heap. Setting `-XX:MaxMetaspaceSize=256m` is essential for predictable container memory budgets. The ClassLoader leak pattern we discussed earlier

becomes a container-eviction incident in a Kubernetes cluster with tight memory limits.

- **Module analysis in CI.** Adding `jdeps --check` and `jlink` to your CI pipeline catches dependency resolution errors at build time. In a fleet where each service is independently deployed, this prevents the "works on my machine" classpath divergence that accumulates over months of rapid iteration.
 - **Remoting and serialization across loader boundaries.** In frameworks that deserialize objects across class loader boundaries (RMI, some message broker clients), a class loaded by loader A may fail to resolve when deserialized by loader B. This manifests as `ClassNotFoundException` or `ClassCastException` at deserialization time, not at send time — a classic distributed-systems consistency gap where the sender's type space diverges from the receiver's.
-

Key takeaways

- **Parent-first delegation is the default, and breaking it requires care.** Child-first loading (OSGi, app servers) trades simplicity for isolation. Know which model you are using and why.
- **StackMapTable-based verification** is the JVM's primary defense against corrupted bytecode. If you generate bytecode (ASM, ByteBuddy, cglib), you must emit correct `StackMapTable` entries or your classes will fail to load.
- **The `<clinit>` deadlock pattern** is real and only surfaces under load. Circular static initialization across class loader boundaries hangs the JVM. Design static initializers to avoid cross-loader dependencies.
- **ClassLoader leaks cause slow-burn OOMs.** In PermGen (JDK ≤ 7) they hit a hard ceiling; in Metaspace (JDK 8+) they grow silently. The fix is always the same: ensure no static references escape the loader's lifecycle, and always call `close()` on loaders you create.
- **JPMS `exports` / `opens` are not optional for libraries that use reflection.** Serialization frameworks (Gson, Jackson, JAXB) need `opens` directives; failure to provide them causes `InaccessibleObjectException`.

- **jdeps and jlink are production tools, not just migration aids.** Adding `jdeps --check` to CI catches split-package and missing-module errors. `jlink` reduces container image size by 60-80%, directly improving cold-start time and scaling speed in container orchestration platforms.
 - **Custom ClassLoaders must handle thread context management.** Failing to save/restore `Thread.currentThread().getContextClassLoader()` causes leaks that survive loader undeployment.
-

Further reading

- **JSR 376 — Java Platform Module System specification.** The normative specification for JPMS. <https://jcp.org/en/jsr/detail?id=376>
- **JSR 202 — Java 6 Class File Verification Update.** Defines the `StackMapTable` attribute and type-checking verifier. Part of Java SE 6 spec.
- **JEP 261 — Module System.** The implementation JDK Enhancement Proposal for Project Jigsaw. <https://openjdk.org/jeps/261>
- **JEP 238 — Multi-Release JARs.** How JPMS interacts with versioned class files. <https://openjdk.org/jeps/238>
- **JEP 310 — Application Class-Data Sharing.** AppCDS for production deployments. <https://openjdk.org/jeps/310>
- **JEP 382 — Strongly Encapsulate JDK Internals by Default.** Why `--add-opens` exists and what it means for frameworks. <https://openjdk.org/jeps/382>
- **Cliff Click, "The JVM loading, linking and initialization machinery."** An accessible walkthrough of the JVM class lifecycle from one of HotSpot's original architects.
- **Alexandre Cesari, "ClassLoader Leaks: a Deep Dive."** The definitive diagnostic guide for ClassLoader leaks in application servers. <https://javadox.com/glassfish/4.0/glassfish-api/apidocs/index.html>

- **OSGi R8 Core Specification.** The bundle-based module system that predates JPMS and remains in production in Eclipse-based tooling and telecom infrastructure. <https://www.osgi.org/specification/>
- **Baeldung, "Java Module System — Getting Started."** Practical migration guide for Maven/Gradle projects. <https://www.baeldung.com/java-module-system>

Chapter 3 — The Java Memory Model, Object Layout, and Heap Organization

What this chapter covers. Every interesting JVM backend shares mutable state across threads — config snapshots, caches, connection pools, metrics counters, request queues — and allocates millions of objects per second into a managed heap that must hide hardware reordering and GC pauses. The contract that makes this safe is the Java Memory Model (JMM), and the cost of that contract is encoded in every object header and heap region. This chapter opens both: the formal happens-before model that tells you when a write in one thread becomes visible in another, and the concrete HotSpot implementation that makes allocation fast — mark words, klass pointers, compressed oops, field reordering, JOL-verified layouts, TLABs, generational regions, humongous objects, and the memory barriers that connect the JMM to silicon.

Learning goals — after this chapter you should be able to:

- State the JMM happens-before relation precisely, enumerate every edge the JLS guarantees (program order, monitor, volatile, thread start/join, interruption, finalizers, transitive closure), and prove or refute visibility for any two accesses.
- Explain visibility, reordering, and data races in terms of compiler, JIT, and hardware (x86 TSO vs. ARM weak ordering), and predict which litmus-test outcomes are legal under the JMM.
- Classify publication as safe or unsafe, apply the four safe-publication idioms (static initializer, volatile, final-field freeze, proper locking),

and diagnose unsafe publication with tests rather than reasoning by example.

- Describe `final` field semantics — freeze action, safe publication of immutable objects, and the `String` / `record` guarantees that depend on them.
- Diagram HotSpot object layout on 64-bit: mark word, klass pointer, compressed vs. uncompressed modes, field reordering, object alignment, and array headers; interpret `JOL` output for any class.
- Explain heap organization — Eden/Survivor/Old, G1/ZGC/Shenandoah regions, TLAB allocation fast/slow paths, humongous objects — and how it drives allocation and collection costs.
- Name the four memory barriers (`LoadLoad` , `LoadStore` , `StoreStore` , `StoreLoad`), map `volatile` / `VarHandle` access modes to barriers on x86 and AArch64, and distinguish JMM barriers from GC barriers (SATB, pre/post barriers).

Placement. Chapter 1 gave the classfile and execution model; Chapter 2 traced class loading and linking. This chapter is the concurrency-and-memory foundation for everything that follows: Chapter 4 (GC algorithms assume the heap layout and barriers described here), Chapter 5 (the JIT reorders subject to the JMM and intrinsifies `VarHandle`), and Chapter 6 (threads, monitors, `VarHandle` , Loom, and structured concurrency build directly on happens-before). Familiarity with Volume 4, Chapter 3 (hardware memory models) and Volume 17, Chapter 6 (Go memory model) helps but is not required.

1. Why the JMM and heap layout matter at backend scale

On a single thread the JVM is simple: bytecode executes in program order and allocation is `new` . In a backend service that model breaks in three places simultaneously:

1. **The JIT compiler** reorders loads and stores, eliminates dead stores, hoists loop-invariant reads, and inlines through publication boundaries — all legal under the as-if-serial rule for a single thread, all capable of breaking a racy multi-threaded program.

2. **The processor** reorders. x86 is Total Store Order (TSO): stores become visible in order but a later load can bypass an earlier store. AArch64 (Graviton, the cost-optimized default in many fleets) is weakly ordered: almost any reordering of independent accesses is observable unless a barrier intervenes.
3. **The heap** is shared and generational. Every core allocates into the same Eden, every GC moves objects, and every reference must remain valid across compaction. Headers, compressed pointers, and TLABs are not trivia — they determine object footprint, cache-line density, allocation latency, and how large a heap you can run before compressed oops collapse.

The JMM is the contract between your code and these three actors. If you establish happens-before edges between conflicting accesses, the JVM guarantees visibility and ordering. If you do not, you have a **data race** and the program's behavior is not merely "eventually consistent" — the JLS declares it unconstrained for those locations. A config-reload thread that writes a plain `Map` reference and an HTTP handler that reads it can observe a partially constructed map, a `null` field of a non-null object, or a stale cache forever — and it will pass every test on x86 before failing on AArch64 at 3 AM.

Backend lens. A single microservice handles 10k-100k requests per second, each allocating hundreds of objects. TLAB contention, humongous allocations, and false publication show up as p99 outliers, not averages. Understanding headers and heap regions is how you read a flame graph, size a heap for G1/ZGC, and explain why a 34 GB heap is slower than a 26 GB one.

2. The Java Memory Model: happens-before

The authoritative reference is JLS 17, Chapter 17 (Threads and Locks), and JSR-133 (the Manson-Pugh model, 2004, still current). It fits in a few pages and every backend JVM engineer should have read it once.

2.1 The relation

The JMM defines a **happens-before** (hb) relation — a strict partial order over memory actions (reads, writes, lock/unlock, volatile accesses, thread start/join):

- If action a happens-before action b , then the effects of a (all prior writes by its thread) are visible to b , and b observes a as preceding it.
- hb is **transitive**: if $a hb b$ and $b hb c$, then $a hb c$.
- hb is **not total**: many actions are concurrent (unordered). Two concurrent conflicting accesses (same location, at least one write) constitute a **data race**.

Within a single thread the model guarantees **as-if-serial** semantics: the thread behaves as if actions occurred in program order, regardless of actual reordering. Across threads, only hb edges provide ordering.

2.2 The five canonical edges

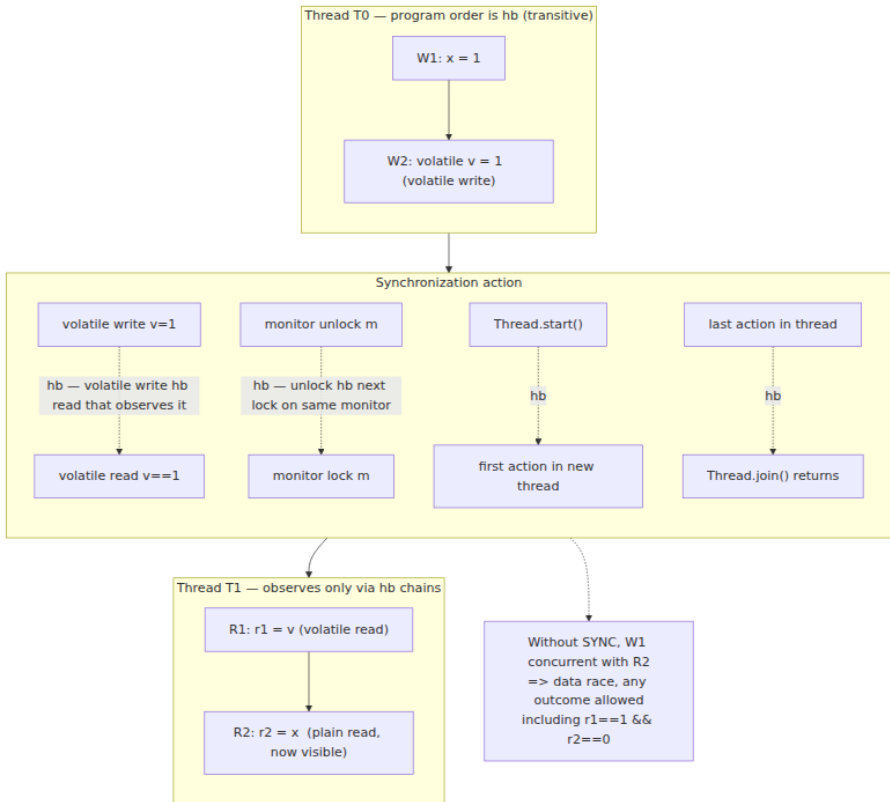
JLS 17.4.5 enumerates the edges. Memorize this table — during review you are hunting for which row justifies each visibility claim.

Edge	Rule	Intuition
Program order	If x and y are actions of the same thread and x precedes y in program order, then $x hb y$.	Single-thread semantics. Writes before later reads in the same thread are always visible.
Monitor (synchronized)	An unlock on monitor m hb every subsequent lock on m .	The n th unlock hb the $(n+1)$ th lock. All writes before the unlock are visible after the lock.

Edge	Rule	Intuition
Volatile	A write to volatile <code>v</code> <code>hb</code> every subsequent read of <code>v</code> that observes that write.	Volatile is a linearization point. Volatile write = release, volatile read = acquire.
Thread start / join	<code>Thread.start()</code> <code>hb</code> the first action in the started thread. Any action in a thread <code>hb</code> <code>Thread.join()</code> that returns after that thread terminates. <code>Thread.isAlive()</code> returning <code>false</code> similarly synchronizes.	Publishing via thread creation; observing termination.
Transitivity	If <code>a hb b</code> and <code>b hb c</code> , then <code>a hb c</code> .	Chaining publication: <code>W(x)</code> <code>hb</code> <code>volatile-write</code> <code>hb</code> <code>volatile-read</code> <code>hb</code> <code>R(x)</code> makes <code>W(x)</code> visible at <code>R(x)</code> .
Interruption / finalizer / default	<code>Thread.interrupt()</code> <code>hb</code> detection of interruption (<code>isInterrupted()</code> , <code>InterruptedException</code>); constructor end <code>hb</code> <code>finalizer</code> start; writes to <code>final</code> fields (freeze) <code>hb</code> reads that see the correctly constructed object.	Less commonly relied upon, but part of the formal model.

There is no general `hb` edge for a plain write followed by a plain read, for `Atomic*` plain modes, for `ConcurrentHashMap.get` vs. `put` on different keys, or for any framework "eventually" — only the rows above (and their `java.util.concurrent` implementations that internally use `volatile` / `synchronized` / `VarHandle` release-acquire).

2.3 Happens-before graph



Transitivity is the workhorse. The common publish pattern:

```

T0:  x = 42           // W(x)  - plain write
T0:  volatileReady = true // VW - volatile write
      // W(x) hb VW (program order)
T1:  if (volatileReady) // VR - volatile read observing VW, so VW
      hb VR
T1:      use(x)         // R(x) - VR hb R(x) (program order)
      // therefore W(x) hb R(x) - x==42 guaranteed

```

Remove `volatile` and every guarantee vanishes, even though the write to `x` precedes the write to `ready` in source order.

2.4 What `hb` does not guarantee

- **No total order.** Two `volatile` writes to *different* variables are ordered only through program order within each thread; a third

thread can observe them in either order unless additional synchronization creates a total order. Total order over all volatile accesses is guaranteed only for accesses to the *same* volatile variable, and for `VarHandle volatile / acquire / release` modes defined in JLS 17 extensions.

- **No causality beyond `hb`**. Out-of-thin-air values are prohibited — the JMM's causality rules forbid a read seeing a value that was never written — but plain racy reads can see stale values arbitrarily long.
- **No progress guarantee.** `hb` is about visibility when an action *does* occur, not about liveness. A spin loop on a plain flag (`while (!done) {}`) may be hoisted to `if (!done) while(true) {}` by the JIT, never observing the update, even though the write `hb` nothing.

3. Visibility, reordering, and data races

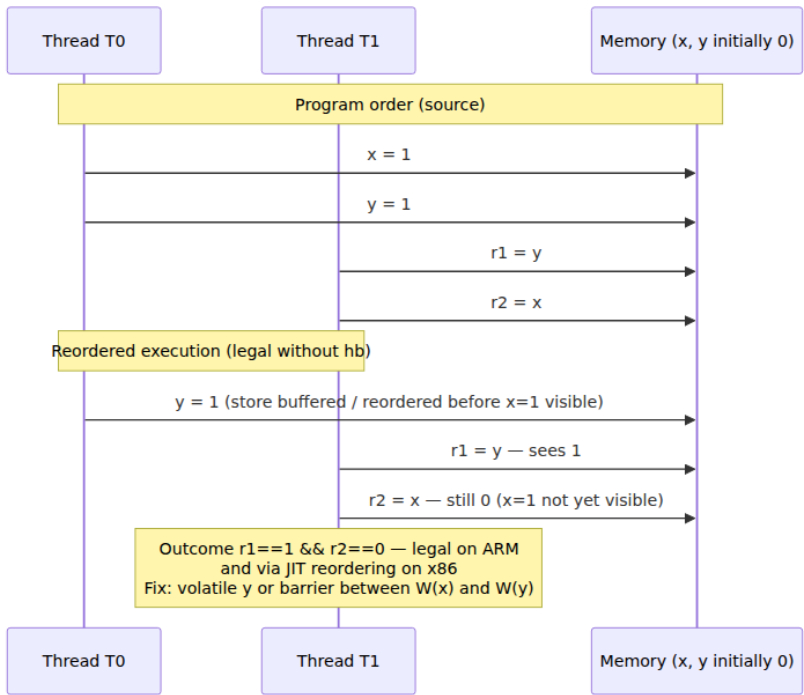
3.1 Who reorders

Three independent agents reorder your program, subject only to single-thread as-if-serial correctness:

Agent	What it reorders	Constraint without <code>hb</code>
javac / JIT (C2/Graal)	Load-load, load-store, store-store; eliminates, hoists, sinks, and common-subexpresses memory operations across branches and loops.	May reorder any two plain accesses to different locations; may not reorder across a volatile/monitor boundary or a <code>VarHandle opaque/acquire/release</code> fence.
Processor (x86 TSO)	StoreLoad only (a later load can bypass an earlier store via the store buffer). Loads and stores otherwise ordered.	<code>StoreLoad</code> reordering observable — the classic Dekker / double-checked-locking failure on x86.
Processor (AArch64 weak)	LoadLoad, LoadStore, StoreStore, StoreLoad — almost any pair of independent accesses can appear reordered.	Any litmus outcome without a barrier is legal; acquire loads and release stores provide only pairwise ordering.

The JIT is often the bigger source of surprises than the CPU. A field read hoisted out of a loop, a null check eliminated because the JIT proved non-null on the fast path — these produce "impossible" values that no hardware litmus test predicts.

3.2 Reordering illustrated



The classic four litmus tests (each `hb`-free by default):

```

// StoreStore – can T1 see y==1 before x==1?
//  T0: x=1; y=1;      T1: r1=y; r2=x;  Outcome r1==1 && r2==0 legal?
YES on ARM, NO on x86 (TSO orders stores)

// LoadLoad – can loads be reordered?
//  T0: x=1;          T1: r1=y; r2=x; // if T0 also does y=1 after
x=1, same StoreStore case.

// StoreLoad – Dekker / double-checked locking hump
//  T0: x=1; r1=y;    T1: y=1; r2=x;  Outcome r1==0 && r2==0 legal?
YES on both x86 and ARM
//  This is why DCL needs volatile – StoreLoad is the only x86
reordering.

// Independent – no dependency
//  T0: x=1;          T1: y=1;          No race (different locations),
but publication of x via y needs hb.

```

All four outcomes ($r1==0/1 \times r2==0/1$) are legal without `hb`. Making `y` volatile establishes `W(x) hb W(y,vol) hb R(y,vol) hb R(x)`, restricting outcomes to `r1==1→r2==1` or `r1==0→r2==0`.

3.3 Data race vs. race condition

A **data race** is a formal JMM term: conflicting accesses, at least one write, concurrent (no `hb`). Its consequence is loss of all visibility guarantees for those locations — the JIT may assume data-race-free execution and optimize accordingly (roach-motel semantics: the JIT may move plain accesses *into* a synchronized block but not *out of* it).

A **race condition** is a semantic bug: timing-dependent correctness even with proper `hb` (e.g., two threads both `check-then-act` under correct locking but with a logical window). Fixing a data race does not fix a race condition, and vice versa.

4. Safe publication, unsafe publication, and `final` field semantics

4.1 Unsafe publication — the default is broken

```
// UNSAFE – do not do this. Demonstrates the failure mode every backend
// must recognize.
class Holder {
    int x;
    int y;
    Holder(int x, int y) { this.x = x; this.y = y; }
}

// Publisher thread
Holder h; // plain, non-volatile, non-final field
void publish() {
    h = new Holder(1, 2); // allocation + construction + publication –
    // three steps
}

// Reader thread
void use() {
    Holder r = h;
    if (r != null) {
        // May see r.x==0 || r.y==0, or even a partially constructed
        // object,
        // because construction (W(x),W(y)) has no hb to R(x),R(y).
        System.out.println(r.x + " " + r.y);
    }
}
```

Three things can go wrong, *all* legal under the JMM:

1. **Reordering:** `h = ref` can become visible before `x=1; y=2` complete (store-store reordering through the allocator + JIT).
2. **Staleness:** Reader may see `h == null` forever (no `hb`, no visibility guarantee — `h` lives in a core's store buffer / cache).
3. **Thin-air / torn reference:** On 32-bit JVMs a plain `long` / `double` write can tear; on 64-bit, references are atomic but the *object's fields* are not.

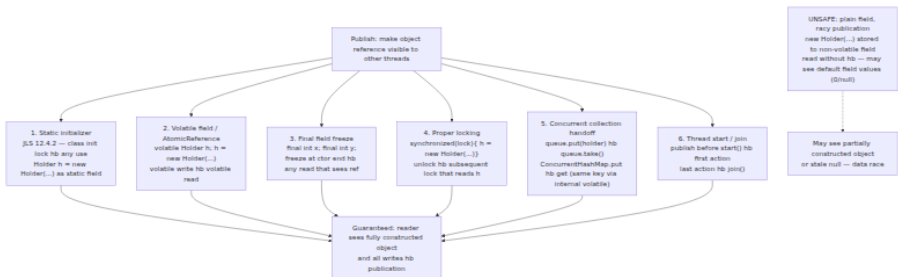
The same bug appears in the infamous **double-checked locking** (DCL) anti-pattern:


```
// BROKEN DCL – classic interview trap that still appears in production
// caches
class Singleton {
    private static Singleton instance; // plain!
    static Singleton get() {
        if (instance == null) { // R1 – racy read
            synchronized (Singleton.class) {
                if (instance == null)
                    instance = new Singleton(); // W – allocation +
// ctor + store
            }
        }
        return instance; // may return partially constructed Singleton
    }
}
```

The fix is exactly one word:

```
private static volatile Singleton instance; // volatile write hb
volatile read
// Now: construction hb volatile write hb volatile read hb field reads
– safe.
```

4.2 Safe publication patterns



Pattern	Mechanism	When to use
Static initializer	Class-initialization lock (JLS 12.4.2) — the JVM holds a lock during <code><clinit></code> ; any thread that uses the class <code>hb</code> after init completes.	Singletons, static caches, <code>static final Holder H = new Holder(...)</code> — the simplest safe publication; prefer it.

Pattern	Mechanism	When to use
volatile / AtomicReference	Volatile write <code>hb</code> volatile read.	Lazy singletons (DCL with <code>volatile</code>), config snapshots swapped atomically: <code>volatile Config cur = load();</code> readers do plain <code>Config c = cur;</code> (volatile read) then use <code>c</code> 's final fields.
final fields	Freeze action at constructor end <code>hb</code> any read that sees the reference <i>and</i> does not leak <code>this</code> during construction.	Immutable value objects, <code>records</code> , <code>String</code> , event objects — the most efficient safe publication (no barrier on the read path).
Locking	Unlock <code>hb</code> next lock on same monitor.	Mutable shared state with invariants spanning multiple fields (<code>size + array</code> , <code>map + version</code>).
Concurrent collection	Library establishes internal <code>volatile / VarHandle</code> edges (<code>put hb get</code> for same key in <code>ConcurrentHashMap</code> , <code>put hb take</code> in <code>BlockingQueue</code>).	Handoff between threads — task queues, caches, pub/sub. Document which collection gives which guarantee.
Thread start/join, Future	<code>start()</code> <code>hb</code> thread entry; task completion <code>hb Future.get()</code> return.	One-shot publication: build in one thread, hand off via <code>Future / CompletableFuture</code> .

4.3 final field semantics — the freeze

`final` fields have stronger semantics than "assign once" (JLS 17.5). The model introduces a **freeze** action at the end of the constructor that `hb` any read of that `final` field through a reference that did not escape before the freeze.

```

final class Config {
    final int timeoutMs;
    final String endpoint; // final reference – guarantees visible
                           // state of the String too (transitively)
    final Map<String,String> labels; // final reference to mutable map
    // only the reference is frozen, not the map's contents!

    Config(int t, String e) {
        this.timeoutMs = t;
        this.endpoint = e;
        this.labels = Map.of("env","prod"); // immutable snapshot –
        // safe
        // ---- freeze ---- hb any thread that later reads a Config
        // reference published safely
    }
}

// Safe publication of Config via data-race-free publication of the
// reference:
volatile Config CURRENT = new Config(3000, "https://api.internal");

// Reader – no lock, no volatile read of fields, just the reference:
Config c = CURRENT; // volatile read – hb chain includes
// freeze
int t = c.timeoutMs; // guaranteed 3000, not 0
// c.labels is also the correctly constructed Map – because freeze hb
// the publish.

```

Rules that still catch seniors:

- **No this escape.** If the constructor publishes `this` (stores it to a static, starts a thread, registers a listener), the freeze may not hb the reader — the reader can see default `final` values (0/ null). Static analysis (`ErrorProne` `ConstructorLeaksThis`) catches this.
- **Only the final fields themselves are frozen.** A `final Map` guarantees the reference, not the map's interior mutability. Publish an *immutable* snapshot (`Map.copyOf` , `List.copyOf` , `Guava ImmutableMap`) or treat the field as effectively immutable after construction.
- **Deserialization / reflection / Unsafe can bypass final .** `Unsafe.putObject` , `Field.setAccessible` on `final` , and some serialization paths assign `final` fields outside the constructor — the freeze guarantee does not apply. Prefer constructors and records.
- **record and String rely on this.** `record Point(int x, int y)` has implicitly `final` fields with the same freeze. `String`'s `final byte[]`

value (compact strings) is safe to share precisely because of final freeze + safe publication through the string table / interning.

Freeze sequence: W(timeoutMs) hb W(endpoint) hb freeze hb volatile-write(CURRENT) hb volatile-read(r=CURRENT) hb R(r.timeoutMs) — transitive hb ensures the reader sees 3000 and a non-null endpoint. If CURRENT were plain or this escaped in the constructor, the chain breaks and 0 / null is legal.

4.4 jcstress — proving visibility, not hoping for it

Testing concurrency by running a loop and checking "it never failed" proves nothing — a racy program can pass a million iterations on x86 and fail on the first run on AArch64. jcstress (OpenJDK Code Tools) enumerates outcomes under stress and reports which are legal under the JMM.

```

// jctest test: unsafe publication vs. safe publication via volatile.
// Run: mvn -pl jctest-samples clean test -Dtest=PublicationTest
import org.openjdk.jcstress.annotations.*;
import org.openjdk.jcstress.infra.results.IntResult1;

@JCStressTest
@Outcome(id = "1", expect = Expect.ACCEPTABLE, desc = "Saw fully
constructed object")
@Outcome(id = "0", expect = Expect.ACCEPTABLE_INTERESTING, desc = "Saw
default value – unsafe publication")
@State
public class PublicationTest {

    // --- Unsafe variant ---
    static class Holder { int x; Holder(int x){ this.x = x; } }
    Holder holder; // plain – unsafe publication

    @Actor
    public void publisher() {
        holder = new Holder(1);
    }

    @Actor
    public void reader(IntResult1 r) {
        Holder h = holder;
        r.r1 = (h == null) ? -1 : h.x; // -1=null, 0=default, 1=correct
    }

    // --- Safe variant (separate test class, shown inline for
    comparison) ---
    @JCStressTest
    @Outcome(id = "1", expect = Expect.ACCEPTABLE, desc = "Always
correct")
    @State
    public static class SafePublicationTest {
        static class SafeHolder { final int x; SafeHolder(int x){
this.x = x; } }
        volatile SafeHolder holder; // volatile + final field

        @Actor public void publisher() { holder = new SafeHolder(1); }
        @Actor public void reader(IntResult1 r) {
            SafeHolder h = holder;
            r.r1 = (h == null) ? -1 : h.x; // never 0 – freeze +
volatile hb
        }
    }
}

```

Expected `jcstress` output (schematic):

```

Unsafe variant:
Result -1 (null)           ☐ acceptable (reader ran first)
Result  0 (default x==0)  ☐ INTERESTING ☐ unsafe publication observed
Result  1 (x==1)          ☐ acceptable
*** Interesting results observed ☐ publication is unsafe ***

Safe variant:
Result -1 (null)           ☐ acceptable
Result  1 (x==1)          ☐ acceptable
*** No interesting results ☐ publication is safe ***

```

Additional litmus jcstress sketch — volatile ordering:

```

@JCStressTest
@Outcome(id = "0, 0", expect = Expect.ACCEPTABLE, desc = "Both saw 0 – reordered")
@Outcome(id = "1, 1", expect = Expect.ACCEPTABLE, desc = "Both saw 1")
@Outcome(id = "0, 1", expect = Expect.ACCEPTABLE, desc = "T1 saw 1, T2 saw 0")
@Outcome(id = "1, 0", expect = Expect.ACCEPTABLE, desc = "T1 saw 0, T2 saw 1 – only with volatile is this forbidden for some patterns")
@State
public class StoreLoadTest {
    int x, y;
    volatile int vy; // make y volatile to forbid r1==1 && r2==0

    @Actor public void actor1(IntResult2 r) { x = 1; vy = 1; }
    @Actor public void actor2(IntResult2 r) { r.r1 = vy; r.r2 = x; }
    // With plain y, r1==1 && r2==0 is observable. With volatile vy, it is forbidden (hb).
}

```

Backend recipe: Gate every shared-mutable publication path with a jcstress test in CI (or at least a jcstress -style stress loop). One test that asserts "no INTERESTING outcome" is worth more than a thousand "it works on my laptop" runs. See Chapter 6 for structured concurrency and VarHandle test patterns.

5. Object layout: headers, compressed oops, field reordering

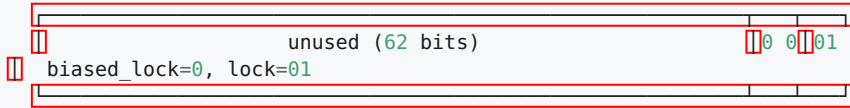
Every `new` produces a HotSpot `oop` (ordinary object pointer) — a contiguous, 8-byte-aligned region of heap. Its prefix is the **object header**; its suffix is the instance fields and alignment padding.

5.1 The mark word

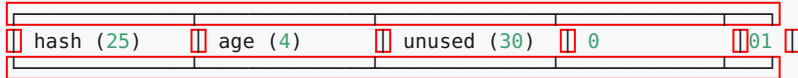
The first word of every object is the **mark word** (`markWord.hpp`, `oopDesc`). It multiplexes GC, locking, and identity hash code. On 64-bit HotSpot (JDK 21, biased locking removed since JDK 15 / JEP 374):

64-bit mark word (little-**endian**, HotSpot 21, UseCompressedOops **on**):

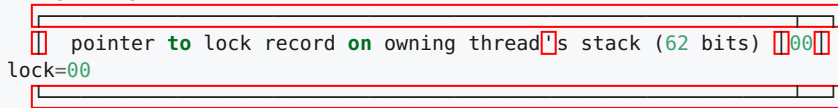
Unlocked, no hash:



Unlocked, hash computed (hash:25 bits, age:4 bits):



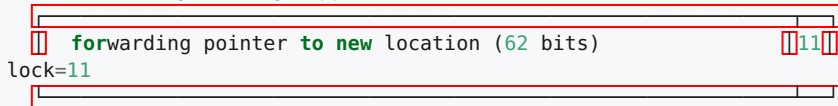
Lightweight locked (stack lock / thin lock):



Heavyweight (inflated monitor):



GC **forwarding** (during copy):



Age field (4 bits, max 15): incremented at each Young GC survival; compared **to** -XX:MaxTenuringThreshold (default 15 with Parallel/G1) **to** decide promotion.

Historical note: **biased locking** (mark bit 101) was removed in JDK 15. On JDK 8 you will still see `biased_lock=1` in JOL; on JDK 17+/21 the bit is repurposed and the mark is 01 when unlocked. When reading older posts or JOL output from JDK 8, expect the bias epoch and thread ID in the mark — do not confuse it with current layout.

The mark word is mutated with CAS. Computing `System.identityHashCode(o)` on an unlocked object CASEs the hash into the mark; locking inflates the monitor and moves the hash into the `ObjectMonitor`.

5.2 Klass pointer and compressed class pointers

The second word is the **klass pointer** — a pointer to the `InstanceKlass` metadata in Metaspace that describes the object's class (vtable, field layout, GC map). On 64-bit:

- **Uncompressed** (`-XX:-UseCompressedClassPointers`): 8 bytes, full 64-bit pointer.
- **Compressed** (default, `-XX:+UseCompressedClassPointers`): 4 bytes, encoded as $(\text{class_base} + \text{index} \ll 3)$. Like compressed oops, it saves 4 bytes per object header.

Header size:

64-bit, compressed class pointers (default, heap < 32 GB): 12 bytes
(8 mark + 4 klass)

+ 4 bytes padding to reach 8-byte alignment before fields →
effective header 12, fields start at offset 12, object size rounded to 8

64-bit, uncompressed: 16 bytes
(8 mark + 8 klass)

32-bit: 8 bytes
(4 mark + 4 klass)

JOL reports this as "object header: 12 bytes (8 + 4 + 0 padding)" vs.
"16 bytes (8 + 8)".

5.3 Compressed oops

Compressed ordinary object pointers (`UseCompressedOops`, default when `MaxHeapSize` < ~32 GB) store references as 32-bit scaled offsets from the heap base, not 64-bit pointers:

```
Compressed oop encoding (HotSpot, globals.hpp):
    narrowOop = (oop - heapBase) >> LogMinObjAlignmentInBytes    // shift
= 3 (8-byte alignment)
    oop      = heapBase + (narrowOop << 3)
```

Zero-based compressed oops (heapBase == 0, heap < 4 GB or < 32 GB with shift):

```
    oop = narrowOop << 3    // no add – single shift, fastest path
```

Heap sizing cliffs:

```
< 4 GB : zero-based, shift 0 or 3 □ narrowOop addresses entire heap without base
4 □ 32 GB : zero-based with shift 3 □ 32-bit narrowOop << 3 covers
32 GB (2^32 * 8)
> 32 GB : compressed oops disabled automatically □ references become 8 bytes, header grows to 16 bytes, object footprint ~20-40% larger
```

That cliff is an operational fact: a heap sized at 34 GB can be *slower and larger* than one sized at 26 GB, because every reference doubles in size and every object header grows by 4 bytes. Size heaps at 24–26 GB or jump to 48+ GB only when genuinely needed — see Chapter 12.

5.4 Field layout, reordering, and alignment

HotSpot reorders fields by size to fill alignment gaps: `long / double` (8) → `int / float` (4) → `short / char` (2) → `byte / boolean` (1) → `oops` (4 compressed / 8 uncompressed). Within each group, declaration order is preserved; groups interleave to minimize padding. A `byte` field can occupy the 4-byte gap after a compressed klass pointer. Object size is `alignUp(header + fields, 8)` (or 16 with large heaps). `@Contended` (JEP 142, `-XX:-RestrictContended`) isolates cache-line-contended fields at 64–128 bytes per field — use only for hot counters and queue indices.

5.5 Array layout

Arrays have an extra 4-byte `length` field between the header and the elements:

```

Object array (compressed):
  offset 0: mark word (8)
  offset 8: klass pointer (4)
  offset 12: length (4) ☐ array length, not element count header
☐ always 4 bytes
  offset 16: elements[0..n-1] ☐ 4 bytes each (compressed oop), or 8 if uncompressed
  size = alignUp(16 + n*elemSize, 8)

Primitive array (e.g., byte[]):
  offset 0: mark (8)
  offset 8: klass (4)
  offset 12: length (4)
  offset 16: bytes[0..n-1] ☐ 1 byte each, packed, no per-element header
  size = alignUp(16 + n, 8)

boolean[] is bytes internally (1 byte per element); HotSpot does not bit-pack.

```

JOL makes this concrete:

```

# Add JOL as a test dependency (Maven):
# <dependency><groupId>org.openjdk.jol</groupId><artifactId>jol-core</artifactId><version>0.17</version></dependency>

java -jar jol-cli.jar internals java.util.ArrayList
java -jar jol-cli.jar layout -v java.lang.String
java -cp jol-core.jar:target/classes org.openjdk.jol.Main layout com.example.MyClass

```

5.6 JOL output — reading it

```
import org.openjdk.jol.info.ClassLayout;
import org.openjdk.jol.vm.VM;

public class JolDemo {
    static class Point { int x; int y; } //
    two ints
    static class Node { long id; int hash; Object next; } //
    mixed
    static class Padded implements Cloneable { byte b; long v; byte
    c; } // reordering demo

    public static void main(String[] args) {
        System.out.println(VM.current().details());
        System.out.println(ClassLayout.parseClass(Point.class).toPrinta
        ble());
        System.out.println(ClassLayout.parseClass(Node.class).toPrintab
        le());
        System.out.println(ClassLayout.parseClass(Padded.class).toPrint
        able());

        Point p = new Point();
        System.out.println(ClassLayout.parseInstance(p).toPrintable());
        System.out.println(ClassLayout.parseInstance(new int[4]).toPrin
        table());
        System.out.println(ClassLayout.parseInstance(new
        String("hello")).toPrintable());
    }
}
```

Representative output on **JDK 21, 64-bit, compressed oops + compressed klass, 8-byte alignment** (addresses will vary; offsets and sizes are stable):

```

# VM details:
# Running 64-bit HotSpot VM.
# Using compressed oop with 3-bit shift.
# Using compressed klass with 3-bit shift.
# Objects are 8 bytes aligned.
# Field sizes by type: 4, 1, 1, 2, 2, 4, 4, 8, 8 [bytes]

com.example.JolDemo$Point object internals:
OFF  SZ  TYPE DESCRIPTION             VALUE
  0   8             (object header: mark)    0x0000000000000001 (non-biasa
ble; age: 0)
  8   4             (object header: klass)    0x000010a8
 12   4   int Point.x                0
 16   4   int Point.y                0
 20   4             (object alignment gap)    0
Instance size: 24 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total

com.example.JolDemo$Node object internals:
OFF  SZ  TYPE DESCRIPTION             VALUE
  0   8             (object header: mark)    0x0000000000000001
  8   4             (object header: klass)    0x00001120
 12   4             (alignment/padding gap)    0
 16   8   long Node.id                0
 24   4   int Node.hash               0
 28   4             (alignment/padding gap)    0
 32   4   Object Node.next            null
 36   4             (object alignment gap)    0
Instance size: 40 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total
// Note: longs first (offset 16), then ints, then oops not source
order.

com.example.JolDemo$Padded object internals:
OFF  SZ  TYPE DESCRIPTION             VALUE
  0   8             (object header: mark)    0x0000000000000001
  8   4             (object header: klass)    0x00001188
 12   1   byte Padded.b               0
 13   1   byte Padded.c               0
 14   2             (alignment/padding gap)    0
 16   8   long Padded.v               0
Instance size: 24 bytes
Space losses: 2 bytes internal + 0 bytes external = 2 bytes total
// Source order was byte, long, byte HotSpot reordered to long fir
st, bytes packed after header.

[I object internals:
OFF  SZ  TYPE DESCRIPTION             VALUE
  0   8             (object header: mark)    0x0000000000000001
  8   4             (object header: klass)    0x00000890

```

```

12 4      (array length)          4
16 16    int [I.<elements>        N/A
Instance size: 32 bytes

```

java.lang.String object internals:

OFF	SZ	TYPE	DESCRIPTION	VALUE
0	8		(object header: mark)	0x0000000000000001
8	4		(object header: klass)	0x00000fd8
12	4		(alignment/padding gap)	0
16	4	byte[]	String.value	null
20	1	byte	String.coder	0
21	1	boolean	String.hashIsZero	false
22	2		(alignment/padding gap)	0
24	4	int	String.hash	0

Instance size: 32 bytes

// String is 32 bytes + the byte[] payload (header 16 + length*1, compact strings since JDK 9).

Reading the columns:

- **OFF** — byte offset from object start. Fields at 12 are packed into the padding after the compressed klass. HotSpot reorders to minimize gaps, but never violates alignment: long / double at 8-byte offsets, int at 4-byte, short / char at 2-byte, oop at 4 or 8-byte.
- **SZ** — field size in bytes (4 for compressed oop, 8 for uncompressed).
- **Instance size** — `alignUp(header + fields + padding, 8)`. Every object is a multiple of 8 bytes; with `-XX:ObjectAlignmentInBytes=16` (large heaps), multiples of 16.



6. Heap organization: generations, TLABs, and regions

6.1 The generational hypothesis

Most objects die young. Backend services confirm this: request-scoped DTOs, byte[] buffers, iterators, and lambda captures rarely survive one Young GC. The heap exploits this by segregating by age:

Logical generations (Parallel / G1 / CMS historical):

Young ☐ Eden (new allocations) + Survivor S0/S1 (copying, age count)

Old ☐ tenured survivors + **large** objects

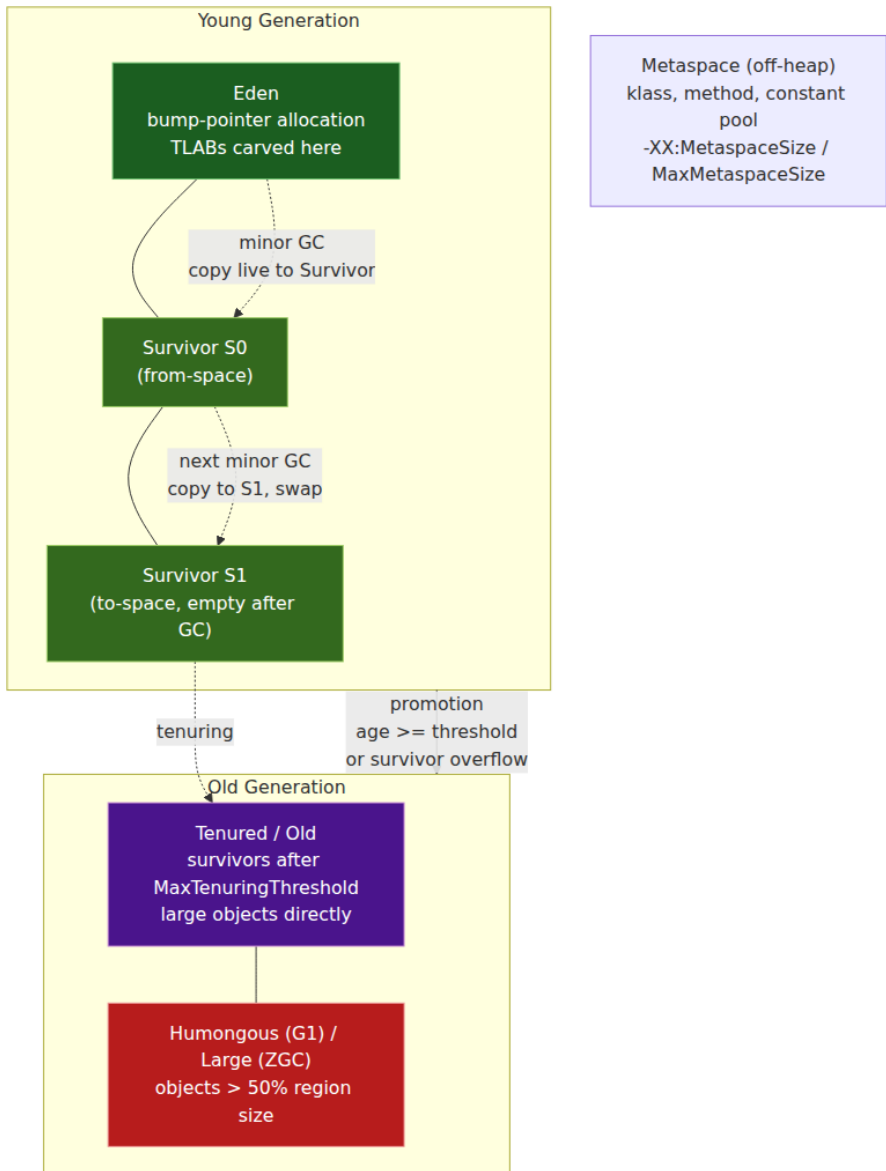
Metaspace (off-heap since JDK 8) ☐ class metadata, **not** part of the GC heap

G1 / ZGC / Shenandoah ☐ region-based, no fixed Young/Old **boundary**:

Heap = array of equal-size regions (1 ☐ 32 MB, **-XX:G1HeapRegionSize**)

Each region is Eden OR Survivor OR Old OR Humongous OR Free

Collection sets are chosen per pause goal, **not** by fixed generation sizes



Survivor mechanics (Parallel / G1 Young):

- **Eden** fills via TLAB bump allocation until full, then a **minor GC** copies live Eden + from Survivor objects into to Survivor, incrementing their age (mark word age field, 4 bits → max 15).

- When `age >= MaxTenuringThreshold` (default 15, G1/Parallel) or `to Survivor` overflows, the object is **promoted** (copied) to Old.
- `SurvivorRatio` (default 8) sets `Eden : Survivor = 8:1` each, so `Young = Eden + 2×Survivor`. `-XX:TargetSurvivorRatio` controls desired occupancy.

Region-based heaps (G1, ZGC, Shenandoah) replace contiguous generations with a **region array**:

```
G1 heap (example: -Xmx16g -XX:G1HeapRegionSize=8m → 2048 regions):
  Region 0: Eden      Region 1: Eden      Region 2: Survivor
Region 3: Old
  Region 4: Old      Region 5: Free      Region 6: Humongous starts
(2 regions) Region 7: Humongous contin...
  Humongous: any object > 50% region size (e.g., >4 MB with 8 MB region
s) → allocated directly as contiguous humongous regions in Old, not
in Eden. Large byte[] / arrays are the common case. G1 reclaims humongo
us regions only at global concurrent cycles → they can pin Old and cau
se fragmentation.
```

ZGC / Shenandoah: colored pointers + **load** barriers, no fixed Young/Old split **in** the same sense (generational ZGC, JEP 474, adds generations at op the region model → see Chapter 4).

G1 Heap — 16 GB, 8 MB regions (schematic, 24 regions shown)

E

E

E

S

S

O

O

O

H

large byte[]
direct to humongous

HC

O

F

E

E=Eden S=Survivor
O=Old H=Humongous
start HC=Humongous
cont. F=Free
Humongous = object >
50% region (e.g. >4 MB)

6.2 TLAB — allocation without contention

Allocation must be fast: a backend allocating 1 GB/s cannot take a lock per new. The **Thread-Local Allocation Buffer** (TLAB) makes it a bump pointer:

```
// Conceptual TLAB (HotSpot: thread.cpp, CollectedHeap,
ThreadLocalAllocBuffer)
// Each Java thread owns a TLAB carved from Eden – a contiguous [top,
end) slice.

class TLAB {
    byte[] eden;      // the Eden region
    long top;         // bump pointer – next free byte
    long end;         // limit of this TLAB
    long pfTop;       // prefetch watermark

    // Fast path – inlined by JIT, ~10 instructions on x86/AArch64, no
    lock, no CAS
    Object allocate(int size) {
        long t = top;
        long newTop = t + size;
        if (newTop <= end) { // fits in TLAB?
            top = newTop;    // bump – single store, thread-local,
no barrier
            // init mark word, klass, zero fields – then return oop at
t
            return oopAt(t);
        }
        return
allocateSlow(size); // TLAB refill – may CAS Eden top if -XX:-UseTLAB?
No, TLAB refill takes Eden lock
    }
}
```

Actual HotSpot fast path (x86_64 assembly, `tlab.c`, `c1_LIRAssembler` / `c2_macroAssembler` — schematic):

```

; r15 = current thread, r15->tlab.top in [r15 + offsetTop], r15-
>tlab.end in [r15 + offsetEnd]
mov  rax, [r15 + topOffset]      ; load top
mov  rbx, rax
add  rbx,
objectSize                      ; newTop = top + size (size is constant for known
class)
cmp  rbx, [r15 + endOffset]      ; fits?
ja   slowPath                   ; no - refill TLAB
mov  [r15 + topOffset], rbx      ; bump top - no lock, thread-local
; initialize header at [rax]: mark word, klass, zero fields
mov  qword ptr [rax], 0x01       ; mark word (unlocked)
mov  dword ptr [rax+8], klassId ; compressed klass
; ... zero remaining fields ...
; rax is the new oop

```

Slow path (`allocateSlow`, `ThreadLocalAllocBuffer::make_parsable`, `CollectedHeap::allocate_new_tlab`):

1. Retire current TLAB (fill remainder with a dummy int array so the heap remains parsable for GC).
2. Request a new TLAB from Eden: `CAS Eden.top` (shared, contended — Eden allocation lock or CAS on `top`).
3. If Eden is full, trigger Young GC.
4. If object is **too large for TLAB** (`> TLABWasteTargetPercent` or `> Eden/2` or `> G1HeapRegionSize/8`), allocate **directly in Eden** (outside TLAB, under Eden lock) or as **humongous** in G1.

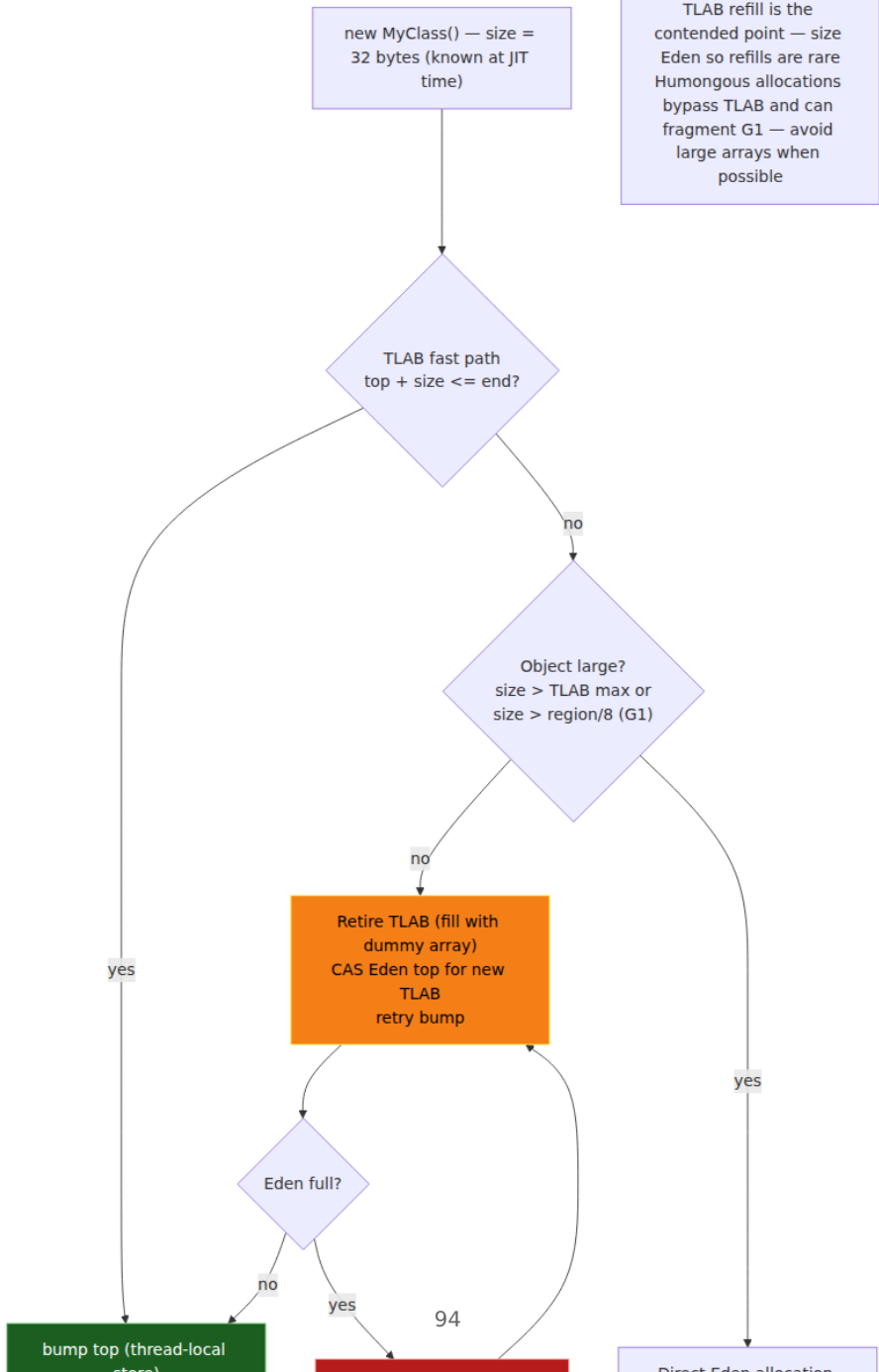
Tuning:

Flag	Default	Effect
<code>-XX:+UseTLAB</code>	on	Disable only to diagnose — allocation becomes Eden-locked, throughput collapses.
<code>-XX:TLABSize</code>	ergonomic (~64 KB-256 KB)	Initial TLAB size; HotSpot auto-tunes via <code>TLABWasteTargetPercent</code> and allocation rate.
<code>-XX:TLABWasteTargetPercent</code>	1%	Fraction of Eden allowed to waste on TLAB retirement gaps. Larger → fewer refills but more fragmentation.

Flag	Default	Effect
<code>-XX:-ResizeTLAB</code>	off (resizing on)	Disable adaptive TLAB sizing — rarely useful.
<code>-XX:ObjectAlignmentInBytes</code>	8	16 on heaps > 32 GB with compressed oops disabled; changes TLAB alignment.

Throughput: >95% of allocations take the fast path

TLAB refill is the contended point — size Eden so refills are rare
Humongous allocations bypass TLAB and can fragment G1 — avoid large arrays when possible



Backend lens — sizing. Size TLABs and Eden so that request-scoped allocations never hit the slow path mid-request. A 256 KB TLAB holds ~8k 32-byte objects — enough for a typical HTTP handler. If `jstat -gc` shows frequent Young GC and `TLABWasteTargetPercent` waste is high, Eden is too small or a single allocation site is producing humongous arrays (check with `jmap -histo + G1Humongous logs: -Xlog:gc+heap=info`).

7. Barriers: JMM barriers vs. GC barriers

The word "barrier" is overloaded. Two unrelated families share the name:

7.1 JMM / memory barriers (ordering barriers)

Inserted by the JIT to enforce `hb` on hardware that would otherwise reorder. Four primitives (JSR-133, `OrderAccess`, `VarHandle`):

Barrier	Prevents	x86 cost	AArch64 cost
<code>LoadLoad</code>	later load bypassing earlier load	free (TSO already orders loads)	<code>DMB LD</code> or <code>LDA</code>
<code>LoadStore</code>	later store bypassing earlier load	free	<code>DMB ST</code>
<code>StoreStore</code>	later store bypassing earlier store	free (stores ordered)	<code>DMB ST / STL</code>
<code>StoreLoad</code>	later load bypassing earlier store	<code>MFENCE</code> / <code>lock</code> <code>addl / XCHG</code> — expensive (~20-40 ns)	<code>DMB SY /</code> <code>STL + LDA</code> — expensive

Mapping of Java constructs to barriers (HotSpot `OrderAccess`, `decorators.hpp`):

volatile write (release + StoreStore + StoreLoad):
x86: MOV [v], 1 + lock addl [rsp],0 (or XCHG) ☐ StoreLoad fence
AArch64: STLR [v], 1 (release store ☐ StoreStore + LoadStore) + optional DMB for StoreLoad

volatile read (acquire + LoadLoad + LoadStore):
x86: MOV r, [v] (TSO already LoadLoad+LoadStore ☐ no extra fence, just compiler barrier)
AArch64: LDAR r, [v] (acquire load)

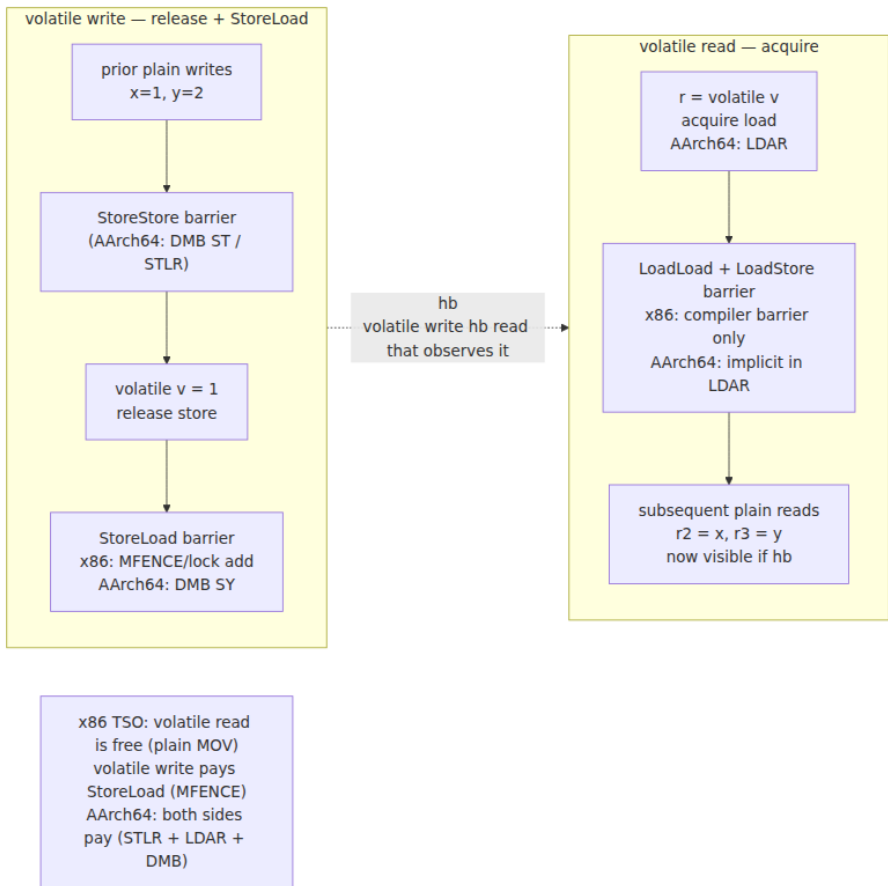
monitor enter: LoadLoad + LoadStore (acquire) ☐ x86: lock-prefixed CAS or XCHG

monitor exit: LoadStore + StoreStore (release) ☐ x86: MOV + compiler barrier; StoreLoad via unlock protocol

VarHandle modes (java.lang.invoke.VarHandle, since JDK 9 ☐ preferred over Unsafe, sun.misc):

- plain ☐ no barrier, no hb (like normal field access)
- opaque ☐ coherent but unordered (no hb, but eventually visible, no tearing)
- release/acquire ☐ pairwise hb (release write hb acquire read on same var)
- volatile ☐ full hb, StoreLoad included

Atomic* (java.util.concurrent.atomic): volatile semantics ☐ same barriers as volatile.



`VarHandle` is the modern surface for fine-grained ordering (replacing `Unsafe` and ad-hoc `volatile`):

```

import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

class PublicationBox {
    int x; // plain field
    static final VarHandle X;
    static {
        try {
            X = MethodHandles.lookup().findVarHandle(PublicationBox.class, "x", int.class);
        } catch (ReflectiveOperationException e) { throw new
Error(e); }
    }

    // Publisher – release store: prior writes hb this store, but no
StoreLoad cost
    void publish(int v) {
        X.setRelease(this, v); // release – hb any acquire load that
observes v
    }

    // Reader – acquire load
    int consume() {
        return (int) X.getAcquire(this); // acquire – hb subsequent
plain reads
    }

    // Volatile – stronger (StoreLoad), use when publication must be
immediately visible
    void publishVolatile(int v) { X.setVolatile(this, v); }
    int consumeVolatile() { return (int) X.getVolatile(this); }

    // Opaque – cheapest, no hb, only coherence (no tearing, eventually
visible)
    // Useful for progress indicators, not for publication.
    void setOpaque(int v) { X.setOpaque(this, v); }
}

```

Choose the weakest mode that still establishes the needed `hb`. For publication (`W(x) hb publish hb consume hb R(x)`), `release/acquire` is sufficient and cheaper than `volatile` on AArch64 (avoids `DMB SY`). For mutual exclusion or Dekker-style coordination, `volatile` (full `StoreLoad`) is required.

7.2 GC barriers (read/write barriers)

Inserted by the JIT to keep GC invariants while the mutator runs concurrently. They have nothing to do with JMM ordering, but they sit on the same memory accesses and show up in assembly:

GC barrier	When	What it does
Pre-barrier (SATB)	Before overwriting a reference (<code>obj.field = newVal</code>) in G1/ZGC/Shenandoah	Records the <i>previous</i> value for Snapshot-At-The-Beginning marking: <code>if (marking) satbQueue.enqueue(oldVal)</code> . Ensures a reference overwritten before the marker visits it is not lost.
Post-barrier (card mark / remembered set)	After storing a reference into Old that points to Young	Dirtyes the card table (<code>cardTable[addr>>9] = dirty</code>) so Young GC can find Old→Young pointers without scanning all of Old. G1's post-barrier also enqueues into the remembered-set buffer.
Load barrier	On reference load (<code>obj.field</code> , array load) in ZGC/Shenandoah	Checks colored-pointer metadata, remaps or forwards the oop if the object was moved concurrently. ZGC: <code>loadBarrier(obj.field)</code> — tests address bad mask, self-heals the reference.
Strength reduction	JIT optimization	Eliminates redundant barriers when it can prove the reference stays in Young or the GC phase does not need them.

G1 write-barrier pseudocode (HotSpot `g1BarrierSetAssembler`):

```

// Mutator does: obj.field = newVal (field is an oop)
// JIT emits:

// --- pre-barrier (if concurrent marking active) ---
Object oldVal = obj.field;           // load old reference
if (oldVal != null && markingActive) {
    satbQueue.enqueue(oldVal);       // remember for SATB
    snapshot
}

// --- actual store ---
obj.field = newVal;                  // plain store

// --- post-barrier (always for cross-region stores) ---
if (newVal != null && obj in Old && newVal in Young) {
    cardTable[((long)obj.field) >> 9] = DIRTY; // card mark – 512-byte
    cards
    if (G1RememberedSetActive) rsQueue.enqueue(obj); // remembered set
    refinement
}

```

ZGC load barrier (colored pointers, ZBarrierSetAssembler):

```

; T1: r = obj.field (obj.field is a colored oop [?] top bits encode
metadata)
mov r, [obj + offset] ; load raw oop (may be colored / forward
d)
test r, ZAddressBadMask ; is the color bad (object moved) [?]
jz done
call ZBarrier::loadBarrierSlow ; remap / forward, self-heal [obj + offs
et] with good oop
done:
; r now holds the good oop

```

Why this matters for backends: GC barriers are on the **hot path** — every reference store pays a card mark, every load pays a ZGC barrier. They dominate GC overhead more than marking itself. When tuning G1, `-Xlog:gc+ergo` and `-XX:+G1EagerReclaimHumongousObjects` interact with barrier costs; when tuning ZGC, `ZAllocationSpikeTolerance` and load-barrier elision determine p99.

8. Putting it together — a backend publication recipe

A common backend pattern ties all three layers — JMM, layout, and heap — together: a **config snapshot** reloaded periodically and read on every request without locking.

```

// Immutable snapshot – final fields + safe publication via volatile
public final class RoutingConfig {
    final Map<String, Endpoint> table; // immutable – Map.copyOf at
    construction
    final int version;
    final long loadedAtNanos;

    public RoutingConfig(Map<String, Endpoint> table, int version) {
        this.table = Map.copyOf(table); // defensive copy – final ref
    to immutable data
        this.version = version;
        this.loadedAtNanos = System.nanoTime();
        // freeze hb publication (below)
    }

    public Endpoint route(String key) { return table.get(key); }
}

// Publication – volatile reference, release/acquire via VarHandle or
volatile
public final class ConfigHolder {
    // Option A: volatile (simplest, StoreLoad on write)
    private volatile RoutingConfig current;

    // Option B: VarHandle release/acquire (cheaper on AArch64 – no
    StoreLoad on read)
    private static final VarHandle CURRENT;
    static {
        try { CURRENT = MethodHandles.lookup().findVarHandle(ConfigHold
er.class, "current", RoutingConfig.class); }
        catch (ReflectiveOperationException e) { throw new Error(e); }
    }
    @SuppressWarnings("unused") private RoutingConfig currentVA; //
    accessed via VarHandle

    // Writer – background reloader, single writer assumed (or CAS loop
    for multi-writer)
    public void reload(Map<String, Endpoint> newTable) {
        RoutingConfig next = new RoutingConfig(newTable, current.version
+ 1);
        // Safe publication: freeze hb volatile write hb volatile read
        current = next; // volatile store – StoreStore + StoreLoad
        // VarHandle alternative: CURRENT.setRelease(this, next);
    }

    // Reader – every HTTP handler, no lock, no volatile read of fields
    public Endpoint route(String key) {
        RoutingConfig c = current; // volatile load – LoadLoad +
        LoadStore (acquire)
        // VarHandle alternative: (RoutingConfig)

```

```

CURRENT.getAcquire(this);
    return c.route(key); // safe – freeze hb publish hb read hb
    field access
}
}

```

Why this works (the `hb` proof):

```

W(table copy) hb W(version) hb freeze hb volatile-write(current=next)
    hb volatile-read(c=current) hb R(c.table) / R(c.version)

```

- `W(table copy)` etc. are program-ordered before `freeze` (constructor body).
- `freeze hb volatile-write` (JLS 17.5 — freeze `hb` publication when reference does not escape).
- `volatile-write hb volatile-read` that observes it (JLS 17.4.5).
- `volatile-read hb` field reads via program order in the reader.

Remove any link (make `current` plain, leak `this` in the constructor, or mutate `table` after publication) and the proof breaks — `jcstress` will surface the `INTERESTING` outcome within seconds.

Heap and layout consequences:

- `RoutingConfig` is ~32 bytes header + three oops/ints + `Map` payload. `Map.copyOf` allocates a compact immutable map (often a single array) — one Old allocation per reload, not per request. Readers allocate nothing.
- If `table` is large (10k endpoints), the `Map`'s backing array may be **humongous** in G1 (array length × reference size > 50% region). Size `G1HeapRegionSize` to avoid this (e.g., 16 MB regions for a 4 MB array) or shard the map.
- TLABs are irrelevant on the read path (no allocation); the write path allocates `RoutingConfig` + `Map` copy in Eden, promoted to Old if it survives long enough — which it does, by design.

9. Distributed-systems lens

The JMM is a single-node consistency model, but its lessons transfer directly to distributed systems:

Single-node (JMM)	Distributed analogue	Lesson
hb via volatile / synchronized	Linearizability via consensus / lease	Both require an explicit synchronization point; "eventually" is not a guarantee.
TSO vs. weak ARM	Sequential vs. causal consistency across replicas	Code correct on TSO (x86) can fail on weak hardware — like code correct with a single replica failing with async replication. Test on the weakest model you deploy to.
Safe publication (final freeze)	Immutable event / snapshot publication	Publish immutable snapshots through a linearizable store (volatile ref / consensus log), not by mutating a shared object in place.
Data race = undefined behavior	Split-brain write without fencing	A racy publication is not "stale" — it can expose torn or partially constructed state, just as a fencing violation can expose uncommitted writes. Use explicit barriers (fences, epochs, VarHandle release/acquire).
Card table / remembered set	Incremental replication / dirty tracking	Both track cross-region references incrementally to avoid full scans — and both pay a per-write barrier cost.
TLAB (thread-local bump)	Partition-local allocation / sharding	Avoid contention by carving local slices from a shared resource; refill under contention is the scaling bottleneck in both cases.

For fleet operations:

- **Test on AArch64** even if production is mixed. AArch64 exposes every missing hb ; x86 hides many via TSO. Run jctest and stress tests on Graviton runners in CI.
- **Heap size cliffs are capacity cliffs.** The 32 GB compressed-oops boundary affects every service's memory footprint and GC pause. Standardize heap sizes at 8 GB / 16 GB / 24 GB and require justification to exceed 32 GB — see Chapter 12 for container-aware sizing (-XX:MaxRAMPercentage , UseContainerSupport).

- **Allocation rate is a service-level metric.** Export `jvm.gc.allocation.rate` (via JFR `jdk.ObjectAllocationInNewTLAB` / `jdk.ObjectAllocationOutsideTLAB`, or Micrometer `jvm_gc_memory_allocated_bytes_total`) alongside request rate. A spike in allocation rate predicts Young GC pressure before p99 does.
-

Key takeaways

- The JMM's happens-before relation has five canonical edges — program order, monitor, volatile, thread start/join, and transitivity. Every visibility claim must cite a row; if no row applies, the code has a data race and no visibility guarantee.
- Reordering comes from three actors — javac/JIT, x86 store buffers, and AArch64 weak ordering. The JIT is often the more surprising source. Plain accesses can be reordered arbitrarily across threads; only `hb` edges constrain them.
- Safe publication requires an explicit `hb` between construction and publication. The four idioms are static initializer, `volatile` / `AtomicReference`, `final`-field freeze (no `this` escape), and proper locking. Double-checked locking without `volatile` is broken.
- `final` field freeze (JLS 17.5) guarantees that any thread seeing a safely published reference sees the `final` fields' correctly constructed values. It covers the `final` reference, not the interior mutability of the referenced object — publish immutable snapshots.
- HotSpot object layout on 64-bit: 8-byte mark word (hash, age, lock state — biased locking removed since JDK 15), 4-byte compressed klass (or 8 uncompressed), then fields reordered as `long / double` → `int / float` → `short / char` → `byte / boolean` → `oops`, padded to 8-byte alignment. Arrays add a 4-byte `length`. Every object size is a multiple of 8 (or 16) bytes.
- Compressed oops (`UseCompressedOops`, shift 3, zero-based) store references as 32-bit offsets — halving reference footprint and header size (12 vs. 16 bytes) — but only up to ~32 GB heap. Beyond that, references double and throughput can regress; size heaps deliberately below the cliff.
- Heap organization: Eden (TLAB bump allocation) → Survivor (copying, age count, `MaxTenuringThreshold`) → Old; G1/ZGC replace

fixed generations with equal-size regions and humongous regions for large objects (>50% region size). TLAB fast path is ~10 instructions, no lock; the slow path (TLAB refill / humongous) is the contended point.

- Barriers are two families: JMM barriers (`LoadLoad` , `LoadStore` , `StoreStore` , `StoreLoad`) enforce `hb` on hardware — `volatile` write pays `StoreLoad` (expensive on both x86 and AArch64), `volatile` read is cheap on x86; `VarHandle` `release` / `acquire` / `opaque` / `plain` let you pay only for the ordering you need. GC barriers (SATB pre-barrier, card-mark post-barrier, ZGC load barrier) keep concurrent GC correct and sit on every reference store/load.
 - Prove publication with `jctstress` , not with "it never failed." A single `@JcStressTest` that asserts no `INTERESTING` outcome is the definitive test for safe publication, volatile ordering, and DCL.
-

Further reading

- **JLS 17, Chapter 17** — Threads and Locks (the normative JMM): <https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html>
- **JSR-133** — Java Memory Model and Thread Specification Revision (Manson, Pugh, Adve): <https://jcp.org/en/jsr/detail?id=133> and the paper "The Java Memory Model" (POPL 2005).
- **Aleksey Shipilëv — Close Encounters of The Java Memory Model Kind** (Devoxx): <https://shipilev.net/blog/2016/close-encounters-of-jmm-kind/> — the most readable JMM walkthrough with `jctstress` examples.
- **** jctstress**** — OpenJDK Code Tools, Java Concurrency Stress tests: <https://github.com/openjdk/jctstress> and <https://openjdk.org/projects/code-tools/jctstress/>
- **JOL (Java Object Layout)** — `org.openjdk.jol` : <https://github.com/openjdk/jol> — `ClassLayout` , `GraphLayout` , `VM.current().details()` ; the definitive tool for verifying layouts and compressed-oops mode.
- **HotSpot sources** — `src/hotspot/share/oops/markWord.hpp` , `oopDesc` , `klass.hpp` , `compressedOops.hpp` , `threadLocalAllocBuffer.hpp` , `g1BarrierSetAssembler` , `zBarrierSetAssembler` : <https://github.com/openjdk/jdk>

- **JEP 374** — Disable and Remove Biased Locking (JDK 15): <https://openjdk.org/jeps/374> — why biased locking is gone and what replaced it.
- **JEP 450** — Compact Object Headers (JDK 24, experimental): <https://openjdk.org/jeps/450> — header compression from 12 to 8 bytes; relevant for future layout changes.
- **JEP 474** — ZGC Generational Mode (JDK 23): <https://openjdk.org/jeps/474> — generational ZGC atop the region model.
- **"Memory Barriers: a Hardware View for Software Hackers"** — Paul McKenney: <http://www.rdrop.com/users/paulmck/scalability/paper/whymb.2010.06.07c.pdf> — hardware reordering fundamentals.
- **Shipilëv** — **"Java Memory Model Pragmatics"** (talk + transcript): <https://shipilev.net/blog/2014/jmm-pragmatics/> — safe publication, final fields, VarHandle modes in practice.
- **"The Art of Multiprocessor Programming"** — Herlihy & Shavit (2nd ed., 2020) — Chapter 3 (mutual exclusion) and Chapter 16 (memory models) for the broader theory behind `hb` and linearizability.

Chapter 4 — Garbage Collection: Serial, Parallel, G1, ZGC, Shenandoah, and Generational ZGC

What this chapter covers. Every JVM backend is a garbage-collected system under load. The collector determines your p99, your heap efficiency, your container sizing, and whether a 2-second STW pause triggers a cascade of timeouts across 40 downstream services. This chapter dissects how HotSpot reclaims the heap — from reachability and tri-color marking through write barriers, generations, evacuation, and concurrent relocation — and then walks every production collector in order: Serial, Parallel, CMS (as history that explains G1), G1, ZGC, Shenandoah, and Generational ZGC (JDK 21, JEP 474). You will learn to read `-Xlog:gc*` output line-by-line, interpret `jstat` and JFR GC events, and choose and tune a collector for throughput, latency, or heap scale with concrete flags and log snippets.

Learning goals — after this chapter you should be able to:

- Define reachability, tri-color marking, and the three GC invariants; explain why a concurrent collector needs a write barrier and how SATB and incremental-update barriers differ.
- State the generational hypothesis, quantify it with allocation-rate math, and map it to Young/Old regions, TLABs, and humongous objects.
- Describe Serial, Parallel, and G1 internals: mark-sweep vs. copy vs. evacuate, G1 regions, remembered sets, SATB concurrent marking, Young and Mixed collections, and humongous handling.
- Explain ZGC's colored pointers and load barriers, Shenandoah's Brooks forwarding pointer, and how each achieves concurrent relocation without STW compaction.
- Contrast non-generational and generational ZGC (JDK 21), including young/old separation, generational collection sets, and when it wins.
- Choose among Serial, Parallel, G1, ZGC, and Shenandoah using a latency-throughput-heap-size matrix and defend the choice with pause and throughput reasoning.
- Read and tune from real GC logs (`-Xlog:gc*`), `jstat -gc*` , and JFR `jdk.GarbageCollection` events; apply heap-sizing, pause-goal, and barrier-overhead tuning.
- Apply a distributed-systems lens: how GC pauses become timeout amplification, retry storms, and quorum stalls, and how to size heaps and collectors for fleet-wide tail-latency SLOs.

Placement. Chapter 3 defined the heap you are collecting — object layout, compressed oops, TLABs, and JMM/GC barriers. This chapter collects it. Chapter 5 (JIT) assumes you understand safepoints and barrier costs; Chapter 6 (threads/Loom) assumes you understand STW pauses and concurrent phases; Chapter 10 (profiling) and Chapter 12 (production tuning) build directly on the log analysis and sizing here.

1. Why GC dominates backend tail latency

A backend service allocating 500 MB/s–2 GB/s per instance (typical for Jackson/Protobuf-heavy HTTP handlers) fills a 4 GB Eden in 2–8 seconds. Every allocation is a TLAB bump (Chapter 3, Section 6.2) until Eden is

full, then the collector must reclaim space. The way it reclaims — stop-the-world copy, concurrent mark, concurrent relocate — determines three production quantities:

Quantity	What it measures	Who feels it
Throughput	Fraction of CPU spent in mutator vs. GC	Cost — more GC CPU means more instances for the same QPS
Pause time	STW duration where no Java thread makes progress	Latency — p50/p99/p999 response time; downstream timeout budgets
Heap efficiency / footprint	Heap needed to sustain a given allocation rate at a target pause	Density — how many pods per node, whether you can run a 32 GB heap without compressed-oops collapse

A 15 ms G1 Young GC is invisible at p50 but shifts p99 by 15 ms. A 1.2-second Full GC on Parallel with a 16 GB heap blows a 500 ms RPC deadline, trips a circuit breaker, and retries from 20 callers simultaneously — a classic GC-induced retry storm. In consensus systems (Raft/ZooKeeper/etcd), a single STW pause longer than `electionTimeout` can force a leader step-down. In Kafka clients, a pause longer than `session.timeout.ms` triggers rebalance. GC is not a local concern; it is a distributed-systems failure mode.

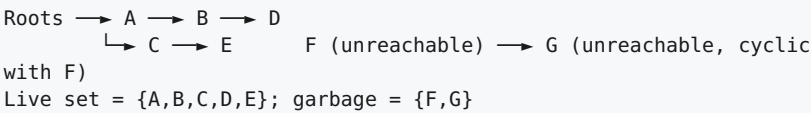
Container awareness compounds this. Since JDK 10 (JEP 307) and JDK 14 (JEP 363), HotSpot reads cgroup limits for `UseContainerSupport` (default on). `-Xmx` defaults to 25% of the container limit; ergonomics picks G1 by default. An unset `-Xmx` inside a 2 GB cgroup and a 64 GB host gives wildly different ergonomics than you tested on your laptop. Always set `-Xms` and `-Xmx` explicitly in containers and understand which collector ergonomics chose — check `-Xlog:gc+heap=info` at startup.

2. Foundations: reachability, mark-sweep, copy, and tri-color

2.1 Reachability

HotSpot is a **tracing collector**: liveness is reachability from a set of **GC roots**, not reference counting.

Roots include: Java thread stacks (local variables, operands), JNI handles, `java.lang.Class` statics, `ClassLoader` tables, `JVMTI` handles, interned `String` table, code-cache oops (compiled code), and `Synchronizer` monitors. Any object transitively reachable from a root is live; everything else is garbage regardless of reference cycles.



Reference strength modifies reachability (all in `java.lang.ref`):

Reference type	Collected when	Typical use
Strong	Never, while reachable	Normal fields
Soft (<code>SoftReference</code>)	Before <code>OutOfMemoryError</code> , LRU per heap	Caches (<code>-XX:SoftRefLRUPolicyMSPerMB</code>)
Weak (<code>WeakReference</code>)	Next GC if only weakly reachable	<code>WeakHashMap</code> , class unloading
Phantom + <code>Cleaner</code>	After finalization/ cleaning, never returns referent	Resource cleanup (replaces <code>finalize</code>)
<code>FinalReference</code>	Enqueued for finalizer thread	Legacy <code>Object.finalize</code> — avoid

Soft-reference pressure is a common cause of "GC thrashing before OOM" — the heap fills with soft-reachable cache entries that the collector keeps reviving. Prefer bounded caches (`Caffeine`, `Guava`) with explicit eviction over `SoftReference` caches.

2.2 The three reclamation strategies

Strategy	Mechanism	Cost	Compacts?
Mark-Sweep	Mark live, sweep dead into free lists	Sweep is $O(\text{heap})$; free lists fragment	No
Mark-Sweep-Compact	Mark live, slide survivors together	Extra copy pass; one STW compaction	Yes
Copy / Evacuation	Copy live objects to a new region, reclaim old region wholesale	Copy is $O(\text{live})$, not $O(\text{heap})$; needs $2\times$ space or region discipline	Yes (by definition)

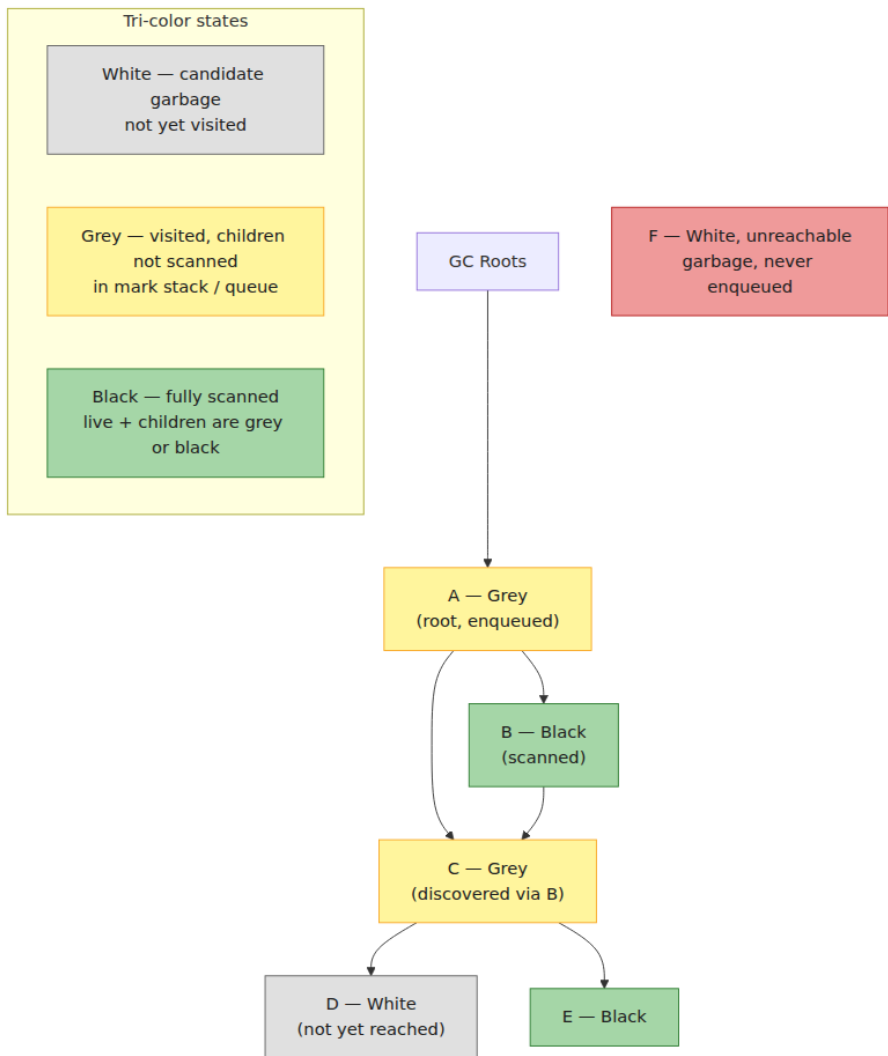
HotSpot uses all three at different times: Serial Old is mark-sweep-compact; Parallel Old is mark-sweep-compact; G1 Young is copying evacuation; G1 Old mixed collections are evacuation; ZGC/Shenandoah are concurrent copying (relocation). No production HotSpot collector is pure mark-sweep without compaction — fragmentation would eventually force a Full GC.

2.3 Tri-color marking

All tracing collectors can be understood through the **Dijkstra tri-color abstraction**. Each object is white (unvisited), grey (visited but children not scanned), or black (visited, children scanned). The collector maintains the **tri-color invariant**:

No black object directly references a white object.

If the invariant holds, marking is complete when no grey remains — white objects are garbage.



STW collectors trivially maintain the invariant — the mutator is stopped, so no new edges appear. **Concurrent** collectors must maintain it while the mutator runs and mutates the graph. Two violations are possible:

1. Mutator creates a new edge `black → white` (stores a white object into a black object's field).
2. Mutator deletes the only path to a white object that was about to be discovered via a grey object, then the grey object is scanned.

Without intervention both produce lost live objects (collector reclaims something still reachable) or floating garbage (collector retains something dead — safe but wasteful). The intervention is a **write barrier** (GC barrier, not JMM barrier — Chapter 3, Section 7).

3. Write barriers: SATB, incremental update, and remembered sets

3.1 SATB (Snapshot-At-The-Beginning)

Used by G1, ZGC, Shenandoah for concurrent marking.

Invariant: the collector retains every object live at the *start* of marking, even if the mutator later drops the last reference to it. An object that becomes unreachable during marking is retained as **floating garbage** and reclaimed next cycle — safe, slightly wasteful.

Mechanism — pre-barrier (before overwriting a reference):

```
// Mutator executes: obj.field = newVal    (field is an oop)
// JIT emits (G1 SATB pre-barrier, ZGC/Shenandoah similar):
Object oldVal = obj.field;                 // load previous value
if (oldVal != null && markingActive) {
    satbQueue.enqueue(oldVal);              // remember old referent – it
    // was live at snapshot start
}
obj.field = newVal;                        // actual store
```

If the mutator overwrites `obj.field` that previously pointed to white object `W`, the pre-barrier enqueues `W` so the marker will still visit it even if no other path remains. Deletions cannot hide objects.

SATB queues are thread-local (`SATBMarkQueue`, `G1ThreadLocalData::satb_mark_queue`). When a queue fills, it is enqueued to the global `SATBMarkQueueSet` for concurrent marking threads to drain. Overflow handling is critical — a missed enqueue would lose an object. HotSpot uses buffer completion and `SATBMarkQueue::flush`.

3.2 Incremental update (CMS-style)

The dual: instead of remembering deletions, remember insertions.

Invariant: any white object that the mutator makes reachable by storing it into a black object is re-greied.

Mechanism — post-barrier (after storing a reference):

```
// Mutator executes: obj.field = newVal
obj.field = newVal;           // actual store
if (newVal != null && markingActive) {
    markStack.push(newVal);    // re-grey: ensure new
    // CMS called this "mod-union table" / dirty card
    referent is visited
}
```

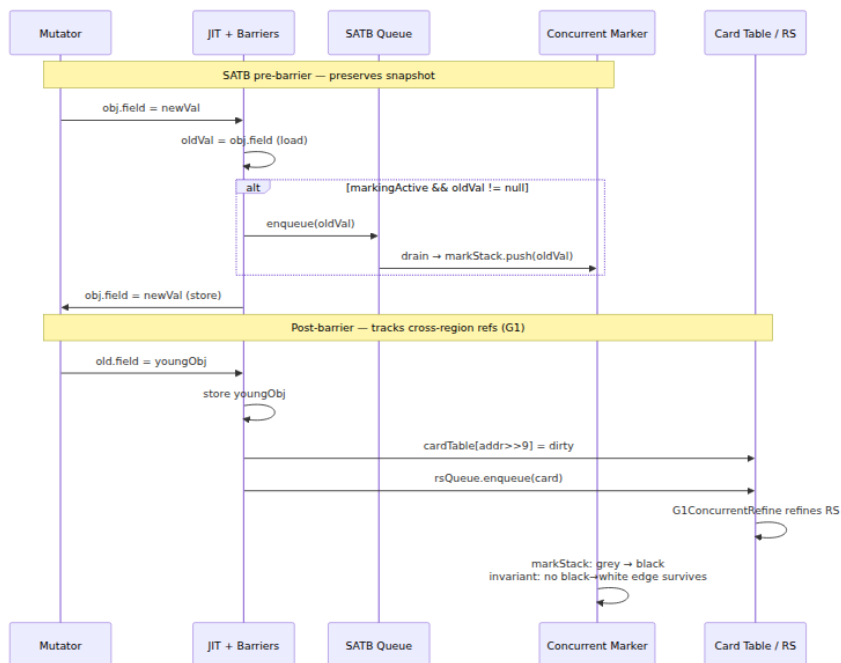
Incremental update re-scans grey roots at the end to catch insertions. It produces less floating garbage than SATB but needs a re-mark of dirty cards/stack. G1 uses SATB for marking + a separate **post-barrier** for remembered sets (see below) — do not confuse the two barriers even though both fire on reference stores.

3.3 Card table and remembered sets (G1)

Generational and region-based collectors must find **cross-region pointers** without scanning the whole heap. G1 solves this with two structures:

- **Card table:** one byte per 512-byte heap card (`-XX:G1CardTable`). Post-barrier dirties the card: `cardTable[addr >> 9] = dirty`. At Young GC, only dirty cards in Old regions are scanned for Old→Young pointers.
- **Remembered set (remembered set):** per-region set of cards that contain pointers *into* that region. Refined concurrently by `G1ConcurrentRefine` threads draining dirty-card queues. At evacuation, the remembered set tells the collector which Old cards to scan to update references to moved Young objects.

ZGC and Shenandoah avoid remembered sets entirely — their **load barriers** make cross-region pointer tracking unnecessary (every load self-heals).



Cost awareness. Every reference store pays at least one barrier. G1's post-barrier is unconditional for Old→Young stores (card dirty is a single byte store, ~3-5 ns). ZGC's load barrier is on every reference *load* — more frequent than stores — but is elided by the JIT when it can prove the oop is already remapped (`ZBarrierSetC2::escapeIsSafe`). Barrier overhead is the dominant throughput tax of concurrent collectors: G1 ~5-10%, ZGC/Shenandoah ~10-20% vs. Parallel, recouped only if pause reduction lets you hit SLOs with fewer instances.

4. The generational hypothesis and heap anatomy

4.1 The hypothesis, quantified

Most objects die young; old objects tend to stay alive. Collecting only the young generation reclaims most garbage at $O(\text{live-young})$ cost, not $O(\text{heap})$.

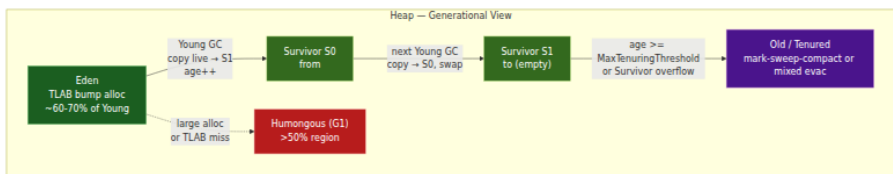
Backend measurements consistently show 85–98% of Young GC garbage is reclaimable. A request handler allocating 200 KB of temporary `byte[]`, `String`, and collection nodes per request promotes almost nothing if `MaxTenuringThreshold` and Survivor sizing are correct.

Allocation rate drives Young GC frequency:

```
Young GC interval ≈ Eden size / allocation rate
Example: Eden 1.5 GB, allocation 1 GB/s → Young GC every ~1.5 s
Survivor promotion rate ≈ live-young per Young GC × Young GC frequency
If 50 MB survives each Young GC → promotion 33 MB/s → Old fills in minutes without tuning
```

4.2 Generational heap anatomy (Parallel / G1 Young)

Eden — bump-pointer, TLAB-carved, where new allocations land
S0/S1 — two Survivor spaces, copying with age counting (mark-word age field, 4 bits, max 15)
Old — tenured survivors + direct-allocated large objects
Metaspace (off-heap) — class metadata, not collected with the Java heap



G1, ZGC, and Shenandoah replace the contiguous Young/Old split with a **region array** (see Sections 6–9), but the generational invariant is the same: young regions are collected frequently and cheaply, old regions rarely.

4.3 Sizing knobs that actually matter

Flag	Default (JDK 21, G1)	Guidance
<code>-Xms / -Xmx</code>	25% cgroup limit (ergonomic)	Set both equal in containers; avoids heap-resize pauses and compressed-oops boundary surprises
<code>-XX:MaxGCPauseMillis</code>	200 (G1)	G1 pause <i>goal</i> , not guarantee — G1 picks Young/Mixed set sizes to try to meet it
<code>-XX:G1HeapRegionSize</code>	Ergonomic: heap/2048, 1-32 MB	Larger regions reduce remembered-set overhead but increase humongous threshold (50% region); tune if humongous allocation is high

Flag	Default (JDK 21, G1)	Guidance
<code>-XX:MaxTenuringThreshold</code>	15 (G1/Parallel)	Lower → faster promotion, more Old pressure; higher → longer Survivor retention, more copy cost
<code>-XX:SurvivorRatio</code>	8 (Parallel)	G1 ignores this (region-based); Parallel: Eden:Survivor = 8:1
<code>-XX:InitiatingHeapOccupancyPercent</code> (IHOP)	45 (G1)	Old occupancy that triggers concurrent marking; lower → earlier marking, fewer Full GCs, more concurrent CPU
<code>-XX:G1MixedGCLiveThresholdPercent</code>	85	Old regions with live < 85% are candidates for Mixed GC

Flag	Default (JDK 21, G1)	Guidance
<code>-XX:ConcGCThreads</code>	~25% <code>ParallelGCThreads</code>	Concurrent marking threads; raise if marking cannot keep up with allocation

Heap sizing cliffs (Chapter 3, Section 5.3 — worth repeating because it is the most common production misconfiguration): compressed oops cover 32 GB with a 3-bit shift. At ~32-34 GB, HotSpot disables `UseCompressedOops`, every reference doubles to 8 bytes, headers grow from 12 to 16 bytes, and effective heap capacity *drops* despite a larger `-Xmx`. Size at 26-30 GB or jump to 48 GB+ only when measured.

5. Serial and Parallel: the throughput collectors

5.1 Serial (`-XX:+UseSerialGC`)

Single-threaded collector. Young: copying (Eden + one Survivor → other Survivor). Old: mark-sweep-compact (STW, single-threaded). No concurrency, no parallelism.

Use when: heap < 1-2 GB, single-core environments, client apps, or when you need deterministic, minimal-footprint GC for CLI tools. Not for backends — a 4 GB heap Full GC can pause for seconds single-threaded.

5.2 Parallel / Throughput (`-XX:+UseParallelGC` , **Parallel Scavenge + Parallel Old**)

The throughput king. Both Young and Old are **parallel STW** — all mutator threads stopped, all GC threads cooperating.

- **Young (Parallel Scavenge):** parallel copying. Eden + from Survivor → to Survivor, age-counted, promotion on overflow. Ergonomics auto-tunes Young/Old ratio and `MaxTenuringThreshold` to maximize throughput (`-XX:+UseAdaptiveSizePolicy` , `-XX:GCTimeRatio=99` means 1% GC time goal).

- **Old (Parallel Old, PS MarkSweep):** parallel mark-sweep-compact. Mark live in parallel, compute new addresses (summary), compact by sliding live objects, update references.

No concurrent phase — every collection is STW, but STW is *short* for its heap size because every core participates. For batch jobs, ETL, and nightly report generators where pause time is irrelevant and throughput is everything, Parallel wins.

```
# Parallel GC with throughput-oriented ergonomics
java -XX:+UseParallelGC \
  -Xms16g -Xmx16g \
  -XX:+UseAdaptiveSizePolicy \
  -XX:GCTimeRatio=99 \
  -XX:MaxGCPauseMillis=500 \
  -Xlog:gc*,gc+heap=info,gc+ergo*=debug:file=/var/log/
gc.log:time,uptime,level,tags \
  -jar batch-job.jar
```

Parallel log (JDK 21, `-Xlog:gc*`):

```
[0.842s][info][gc] GC(0) Pause Young (Allocation Failure)
1842M->312M(4096M) 18.342ms
[0.842s][info][gc] GC(0) User=0.08s Sys=0.02s Real=0.02s
[5.210s][info][gc] GC(1) Pause Full (Ergonomics) 3890M->1240M(4096M) 41
2.118ms
[5.210s][info][gc] GC(1) User=1.82s Sys=0.04s Real=0.41s
```

Pause Young is the STW Young copy; Pause Full is Old compaction. Parallel never prints Concurrent phases — everything is Pause. If you see Pause Full frequently, the heap is too small or promotion is too aggressive.

5.3 When to use — and when to escape

Signal	Meaning	Action
Pause Young < 50 ms, Pause Full rare	Healthy throughput workload	Keep Parallel
Pause Full > 500 ms and frequent	Old too small or fragmentation	Increase <code>-Xmx</code> , lower promotion, or switch to G1/ZGC

Signal	Meaning	Action
p99 latency SLO < 200 ms	STW pauses will violate SLO	Switch to G1 (pause-goal) or ZGC/Shenandoah (concurrent)

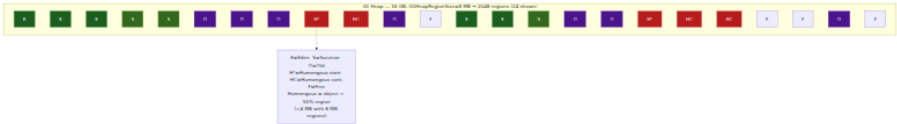
6. CMS → G1: regions, concurrent marking, evacuation

6.1 CMS in one paragraph (why it matters)

CMS (Concurrent Mark-Sweep, removed in JDK 14, JEP 363) was the first low-pause production collector: concurrent marking with incremental-update barriers, STW initial-mark and remark, concurrent sweep. It did **not** compact — free lists fragmented until a Serial Old Full GC compacted. Fragmentation-induced Full GC was CMS's fatal flaw and the reason G1 replaced it. If you still see CMS flags in legacy configs (`-XX:+UseConcMarkSweepGC` , `-XX:CMSInitiatingOccupancyFraction`), treat them as tech debt — the collector does not exist on JDK 17+.

6.2 G1 heap: the region array

G1 divides the heap into equal-size regions (`-XX:G1HeapRegionSize` , 1-32 MB, default heap/2048). Each region has a *role* that changes over time:



Key properties:

- **No fixed Young/Old boundary.** G1 picks the Young set per GC from free + Eden + Survivor regions to meet `MaxGCPauseMillis`. Young size is adaptive.
- **Humongous objects** (`size > 50%` region) are allocated directly in Old as contiguous humongous regions. They are not moved during Young GC and are reclaimed only at concurrent cycles or Full GC. A single large `byte[]` (e.g., decompressed payload, gRPC message) can pin multiple regions. G1 can eagerly reclaim humongous regions if they become unreachable (`-XX:+G1EagerReclaimHumongousObjects` , default on since JDK 18), but fragmentation remains.

- **Remembered sets** per region (Section 3.3) make evacuation $O(\text{remembered-set})$ not $O(\text{heap})$.

6.3 G1 concurrent marking (SATB)

Triggered when Old occupancy exceeds `InitiatingHeapOccupancyPercent` (default 45%). Phases:

1. Initial Mark (STW, piggybacks **on** Young GC) mark roots, set SATB active
2. Root Region Scan scan Survivor regions **for** references **into** Old
3. Concurrent Mark SATB marking from roots + drained SATB queues
4. Remark (STW) – drain remaining SATB queues, weak-reference processing
5. Cleanup (STW) reclaim empty Old regions, sort Old regions by live %
6. Concurrent Cleanup free empty regions

Only Initial Mark, Remark, and Cleanup are STW. Concurrent Mark runs alongside the mutator; SATB pre-barriers preserve the snapshot. After marking, G1 knows liveness per Old region and can schedule **Mixed GCs**: Young regions + a few Old regions with the lowest live percentage, evacuated together. This is how G1 incrementally compacts Old without a Full GC.

6.4 Evacuation pauses: Young and Mixed

Both are **STW parallel copying**: live objects in the collection set (CSet) are copied to new regions (Survivor or Old), forwarding pointers installed, remembered sets updated, then old regions reclaimed as Free.

```
# G1 with pause goal and detailed logging
java -XX:+UseG1GC \
  -Xms16g -Xmx16g \
  -XX:MaxGCPauseMillis=200 \
  -XX:G1HeapRegionSize=8m \
  -XX:InitiatingHeapOccupancyPercent=45 \
  -XX:ConcGCThreads=4 \
  -Xlog:gc*,gc+phases=debug,gc+heap=info:file=/var/log/
gc.log:time,uptime,level,tags \
  -jar service.jar
```

G1 log — Young GC (evacuation):

```

[12.042s][info][gc] GC(18) Pause Young (Normal) (G1 Evacuation Pause)
1420M->380M(4096M) 22.412ms
[12.042s][info][gc] GC(18) Eden regions: 18->0(18) Survivor regions: 3-
>3(3) Old regions: 2->2 Humongous regions: 1->1
[12.042s][debug][gc,phases] GC(18) Pre Evacuate Collection Set: 0.2m
s
[12.042s][debug][gc,phases] GC(18) Evacuate Collection Set: 14.8ms
[12.042s][debug][gc,phases] GC(18) Post Evacuate Collection Set:
3.1ms
[12.042s][debug][gc,phases] GC(18) Other: 4.3ms

```

G1 log — concurrent marking + Mixed:

```

[45.210s][info][gc] GC(42) Concurrent Cycle
[45.210s][info][gc] GC(42) Pause Initial Mark (G1 Humongous
Allocation) 2100M->2100M(4096M) 8.102ms
[45.210s][info][gc] GC(42) Concurrent Mark (45.210s, 45.340s) 130.215ms
[45.340s][info][gc] GC(42) Pause Remark 2120M->2120M(4096M) 12.408ms
[45.340s][info][gc] GC(42) Pause Cleanup 2120M->1980M(4096M) 2.114ms
[45.340s][info][gc] GC(42) Concurrent Cleanup for Next Mark (45.340s, 4
5.342s) 1.802ms
[46.100s][info][gc] GC(43) Pause Young (Prepare Mixed) (G1 Evacuation P
ause) 1680M->420M(4096M) 18.920ms
[46.800s][info][gc] GC(44) Pause Young (Mixed) (G1 Evacuation Pause) 18
00M->440M(4096M) 28.104ms
[46.800s][info][gc] GC(44) Eden regions: 12->0(12) Survivor regions: 3-
>2(3) Old regions: 8->6 Humongous regions: 1->1

```

Pause Young (Mixed) evacuates Young + selected Old regions. Repeated Mixed GCs incrementally reclaim Old. If concurrent marking cannot keep up (allocation outpaces reclamation), G1 falls back to Pause Full (G1 Compaction Pause) — a Serial Old-style compacting Full GC. Avoid it by lowering IHOP or increasing ConcGCThreads.

6.5 G1 failure modes

Symptom in logs	Root cause	Fix
To-space Exhausted	Survivor + Old regions full during evacuation	Increase -Xmx, reduce Young size via lower MaxGCPauseMillis, or lower promotion
Evacuation Failure / PreserveCMReferents	Live data too high for CSet	Same + raise G1ReservePercent (default 10)

Symptom in logs	Root cause	Fix
Frequent <code>Pause Full (G1 Compaction Pause)</code>	Concurrent marking never completes before Old fills	Lower <code>InitiatingHeapOccupancyPercent</code> to 30-35, raise <code>ConcGCThreads</code>
High humongous allocation (<code>G1 Humongous Allocation</code> as Initial Mark trigger)	Large arrays (<code>byte[]</code> , <code>ArrayList</code> growth)	Increase <code>G1HeapRegionSize</code> , pool buffers, or switch to ZGC which handles large objects better
<code>Remark</code> pause spikes	Large SATB queue backlog or weak-ref processing	Check <code>RefProc</code> time in <code>gc+phases=debug</code> ; reduce <code>SoftReference</code> use

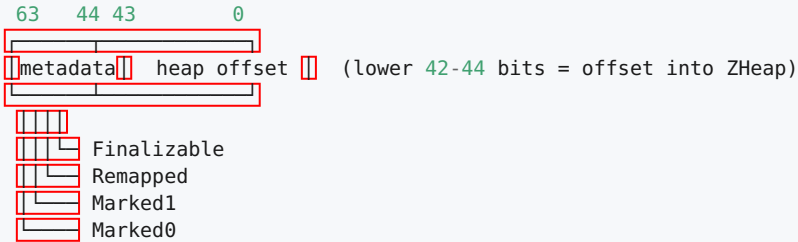
7. ZGC: colored pointers, load barriers, and concurrent relocation

ZGC (JEP 333, production since JDK 15) targets **sub-millisecond STW pauses regardless of heap size** — 8 MB or 16 TB. It achieves this by making almost everything concurrent: marking, relocation, and reference updating happen while the mutator runs. Only brief STW pauses for root scanning and phase transitions remain.

7.1 Colored pointers

On 64-bit, virtual address space is 48-52 bits; the top bits are unused. ZGC steals them as **metadata**:

ZGC colored pointer (JDK 21, 64-bit, 48-bit address space, schematic):



Two mark bits alternate per cycle (Marked0 / Marked1) so ZGC can distinguish

"marked in this cycle" from "marked last cycle" without clearing metadata.

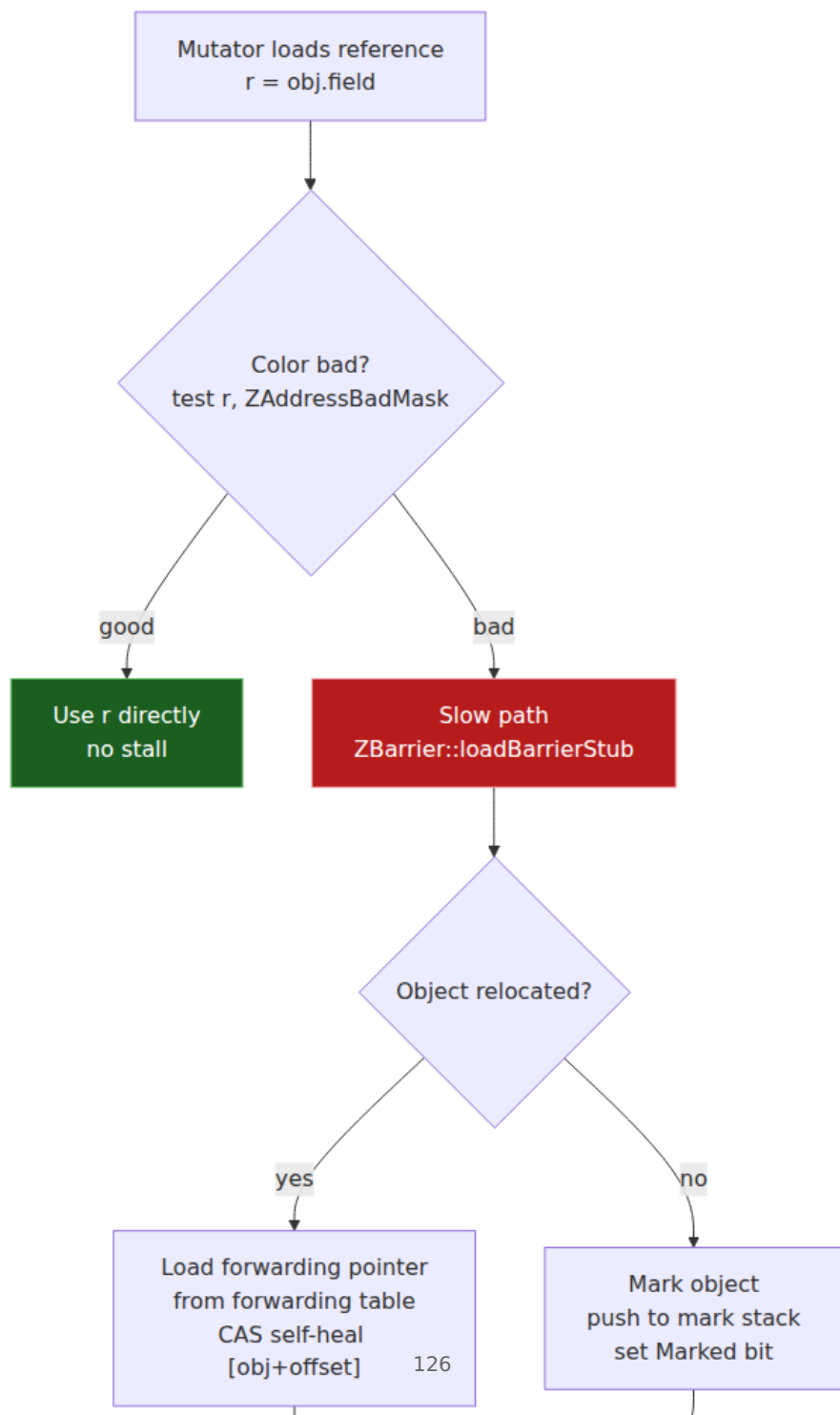
Remapped = object has been relocated and the pointer is to the new location.

Because metadata lives in the pointer, not in the object header, ZGC does not need per-object forwarding bits or header word stealing. But it requires **pointer masking**: every load must strip metadata before dereferencing, unless a load barrier has already remapped it.

JDK 17 used **multi-mapping** (three virtual mappings of the same physical heap at different colored addresses) so that a colored pointer could be dereferenced directly after `mmap` tricks. JDK 21 on x86-64 with 5-level paging and on AArch64 with 52-bit VA uses the same technique; on platforms where multi-mapping is unavailable ZGC falls back to explicit masking.

7.2 Load barriers

Every reference load (`aload` , `getfield` , `aaload`) is instrumented:



Pseudocode of the fast path (HotSpot ZBarrierSetAssembler::load_barrier_fast_path):

```
; r = obj.field (colored oop in register)
test r, ZAddressBadMask      ; single TEST instruction on x86 - ~1
cycle
jz good                      ; predicted not-taken after warmup
call ZBarrier::loadBarrierSlow ; remap / mark / forward, self-heal
good:
; r is now good - dereference
```

The barrier is **self-healing**: once a reference is remapped, the healed (good-colored) oop is written back to the field with a CAS, so future loads of the same field take the fast path. Hot code quickly converges to no slow-path hits.

JIT elision: if the JIT can prove a reference was already loaded through a barrier and not stored (e.g., loop-invariant `obj.field` hoisted), it elides the second barrier. This is why ZGC's barrier overhead drops from ~15% in microbenchmarks to ~5-10% in real services.

7.3 ZGC cycle — mostly concurrent

1. Pause Mark Start (STW, ~0.1 ms)  root scan, set marking active, SATB on
2. Concurrent Mark  mark from roots + SATB queues, set Marked bit
3. Pause Mark End (STW)  drain SATB, weak-ref processing
4. Concurrent Prepare **for** Relocation  select relocation set (regions with most garbage)
5. Concurrent Relocate  copy live objects **in** relocation set to new pages, install forwarding
6. Pause Relocate Start (STW)  remap roots to new locations, set remapped bit
7. Concurrent Remap 
mutator's load barriers lazily remap remaining references

Steps 2, 4, 5, 7 are concurrent. Only three STW pauses, each typically < 1 ms. Relocation is concurrent: the mutator may access an object while it is being copied. Correctness comes from the load barrier + forwarding table — a mutator load that hits an object mid-copy is forwarded to the new copy.

```
# ZGC (non-generational, JDK 17/21 default before JEP 474)
java -XX:+UseZGC \
    -Xms16g -Xmx16g \
    -XX:ConcGCThreads=4 \
    -XX:ParallelGCThreads=8 \
    -Xlog:gc*,gc+heap=info:file=/var/log/
gc.log:time,uptime,level,tags \
    -jar service.jar

# ZGC tuning for allocation spikes (common in burst-heavy backends)
java -XX:+UseZGC -Xms16g -Xmx16g \
    -XX:ZAllocationSpikeTolerance=5 \
    -XX:ZCollectionInterval=5 \
    -XX:+ZProactive \
    -jar service.jar
```

ZGC log (JDK 21, non-generational):

```
[10.042s][info][gc] GC(0) Pause Mark Start 12.042ms (STW)
[10.042s][info][gc] GC(0) Concurrent Mark 45.210ms
[10.088s][info][gc] GC(0) Pause Mark End 1.012ms (STW)
[10.089s][info][gc] GC(0) Concurrent Prepare for Relocation 2.104ms
[10.092s][info][gc] GC(0) Concurrent Relocate 18.420ms
[10.110s][info][gc] GC(0) Pause Relocate Start 0.842ms (STW)
[10.111s][info][gc] GC(0) Concurrent Remap 8.104ms
[10.042s][info][gc] GC(0) Load 14.2/16.0 GB (88%), Live: 4.1 GB, Garb
age: 8.9 GB, Reclaimed: 8.1 GB
[10.042s][info][gc] GC(0) Memory: 14.2M->6.1M(16384M) 0.3ms
[10.042s][info][gc] GC(0) GC(0) completed in 78ms STW total ~2ms, co
ncurrent ~72ms
```

Load is heap occupancy before GC, **Live** is marked live, **Garbage** is reclaimable, **Reclaimed** is freed. ZGC reclaims concurrently — the heap drops without a long pause.

7.4 ZGC heap: pages, not regions

ZGC manages the heap as **pages** (2 MB small, 32 MB medium, $n \times 2$ MB large for humongous). Pages are allocated from **ZPhysicalMemory** backed by **mmap** with colored multi-mapping. Large objects get their own large page — no humongous fragmentation like G1, but large pages are not relocated (pinned).

8. Shenandoah: Brooks forwarding pointers

Shenandoah (JEP 189, production since JDK 15) shares ZGC's goal — concurrent evacuation with sub-millisecond pauses — but uses a different mechanism: a **Brooks forwarding pointer** in every object.

8.1 Brooks pointer

Every object has an extra word (adjacent to the header, or stolen from the header on some configurations) that points to the object's current location:

Object layout with Brooks pointer (Shenandoah, 64-bit, compressed oops):



Normal: Brooks ptr == self (points to this object)

During evacuation: Brooks ptr == new location (after copy)

After evacuation: mutator sees Brooks ptr, follows it

Mutator invariant: **every reference load goes through the Brooks pointer**. The JIT emits a load barrier on every `aload/getfield` that does `oop = *brooksPtr` if evacuation is in progress. Unlike ZGC's colored-pointer test (single `TEST`), Shenandoah's barrier is an extra indirection — one more load per reference.

8.2 Shenandoah cycle

1. Pause Init Mark (STW) ☐ root scan, SATB on
2. Concurrent Mark ☐ SATB marking
3. Pause Final Mark (STW) ☐ drain SATB, weak refs, choose collection set
4. Concurrent Cleanup ☐ reclaim empty regions
5. Concurrent Evacuation ☐ copy live objects in CSet, CAS Brooks pointer to new location
6. Pause Init Update Refs (STW) ☐ ensure no mutator holds stale from-space pointers
7. Concurrent Update Refs ☐ fix all references to point to to-space (via Brooks)
8. Pause Final Update Refs (STW) ☐ complete
9. Concurrent Cleanup ☐ reclaim from-space regions

Shenandoah 2.0 (JDK 17+) introduced **load-reference barriers (LRB)** as an alternative to Brooks barriers, reducing overhead when evacuation is not active. The flag `-XX:ShenandoahGCMode=iu` (incremental update) vs. `satb` selects barrier mode; `generational` mode (experimental) adds Young/Old separation.

```
# Shenandoah (JDK 21)
java -XX:+UseShenandoahGC \
    -Xms16g -Xmx16g \
    -XX:ShenandoahGCHeuristics=adaptive \
    -XX:ShenandoahGCMode=satb \
    -Xlog:gc*,gc+ergo=info:file=/var/log/
gc.log:time,uptime,level,tags \
    -jar service.jar

# Shenandoah with pacer tuning for bursty allocation
java -XX:+UseShenandoahGC -Xms16g -Xmx16g \
    -XX:ShenandoahPacerTrigger=3 \
    -XX:ShenandoahGuaranteedGCInterval=10000 \
    -jar service.jar
```

Shenandoah log:

```
[8.042s][info][gc] GC(0) Pause Init Mark 0.842ms
[8.042s][info][gc] GC(0) Concurrent marking 32.104ms
[8.074s][info][gc] GC(0) Pause Final Mark 1.210ms
[8.074s][info][gc] GC(0) Concurrent evacuation 18.420ms
[8.092s][info][gc] GC(0) Pause Init Update Refs 0.412ms
[8.092s][info][gc] GC(0) Concurrent update references 12.104ms
[8.104s][info][gc] GC(0) Pause Final Update Refs 0.304ms
[8.104s][info][gc] GC(0) Concurrent cleanup 1.102ms
[8.104s][info][gc] GC(0) 8.1G->2.4G(16.0G) 62ms, STW total ~2.8ms
```

8.3 ZGC vs. Shenandoah — mechanism comparison

Property	ZGC (colored pointers)	Shenandoah (Brooks pointer)
Metadata location	Top bits of pointer	Extra word per object
Barrier type	Load barrier (test + self-heal)	Brooks indirection (load through forwarding ptr)
Barrier cost when idle	One <code>TEST</code> per load, predicted not-taken	One extra load per reference when evacuating; elided when not

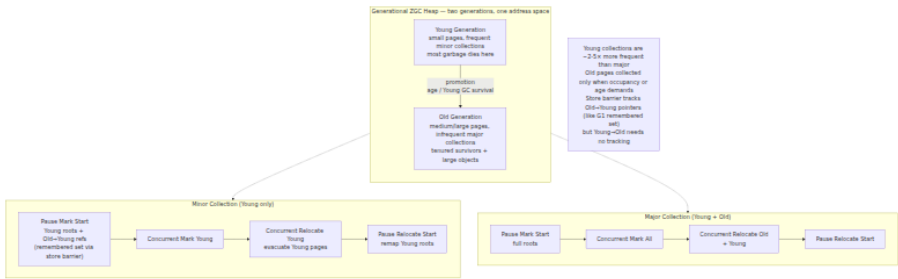
Property	ZGC (colored pointers)	Shenandoah (Brooks pointer)
Object overhead	None (metadata in pointer)	4-8 bytes per object (Brooks word)
Heap size	Up to 16 TB (multi-mapping)	Up to ~100 GB typical (region-based)
Large objects	Large pages, not relocated	Humongous regions, not evacuated when pinned
Throughput tax	~5-10% (barrier + concurrent threads)	~10-15% (Brooks + concurrent copy)
STW pauses	~0.1-1 ms (3 pauses/cycle)	~0.3-2 ms (4 pauses/cycle)

Both are excellent low-pause collectors. ZGC scales to larger heaps; Shenandoah's adaptive heuristics can be more aggressive at reclaiming garbage incrementally. For most backends either is a good choice when G1 pauses are too high — pick the one your JDK vendor supports best (Oracle/OpenJDK ships both; some vendors default to one).

9. Generational ZGC (JDK 21, JEP 474)

Non-generational ZGC treats every object equally: every GC marks the whole heap. For backends where 90%+ of objects die young, this wastes marking and relocation work on short-lived garbage that a Young GC could reclaim at $O(\text{live-young})$ cost. **Generational ZGC** (JEP 474, delivered in JDK 21 as `-XX:+ZGenerational`, default since JDK 23) adds generations atop ZGC's concurrent relocation.

9.1 Architecture



Key additions over non-generational ZGC:

- **Young generation:** small pages (2 MB), collected frequently. Minor GC marks only Young + Old→Young references. Promotion age is tracked per object (like G1's age field).
- **Old generation:** medium/large pages, collected by major GC (full marking + relocation). Major GC is rarer — often 10-20× less frequent than minor.
- **Store barrier for Old→Young:** generational ZGC adds a **store barrier** (unlike non-generational ZGC which only has a load barrier) to track Old→Young pointers, so minor GC does not need to scan all of Old. This is the same insight as G1's card table but implemented via ZGC's barrier infrastructure.
- **Remembered set:** per-page remembered set for Old→Young, refined concurrently.

9.2 When generational wins

Workload	Non-generational ZGC	Generational ZGC
Allocation-heavy, short-lived objects (HTTP handlers, serialization)	Marks entire heap every GC — wasted work	Minor GC marks only Young — 2-5× less marking, lower CPU
Cache-heavy, long-lived Old (large in-memory caches)	Same cost as above — cache objects re-marked every cycle	Major GC infrequent — cache pages rarely marked/relocated
Allocation rate > 1 GB/s, heap 16-64 GB	GC frequency high, concurrent threads compete with mutator	Fewer major GCs, lower concurrent CPU, better throughput

Workload	Non-generational ZGC	Generational ZGC
Small heap (< 4 GB), low allocation	Negligible difference	Minor overhead of store barrier not worth it — use non-generational or G1

Benchmarks from JEP 474 and vendor reports show generational ZGC reducing GC CPU overhead by 20–40% and allocation stall time by 30–60% vs. non-generational on allocation-heavy services, while preserving sub-millisecond pauses.

9.3 Enabling and tuning

```
# Generational ZGC – JDK 21 (explicit), JDK 23+ (default for ZGC)
java -XX:+UseZGC -XX:+ZGenerational \
    -Xms16g -Xmx16g \
    -XX:ConcGCThreads=4 \
    -Xlog:gc*,gc+heap=info:file=/var/log/
gc.log:time,uptime,level,tags \
    -jar service.jar

# Force non-generational ZGC (if you need to compare or hit a
# generational bug)
java -XX:+UseZGC -XX:-ZGenerational -Xms16g -Xmx16g -jar service.jar

# Generational ZGC with NUMA (large hosts)
java -XX:+UseZGC -XX:+ZGenerational -Xms64g -Xmx64g \
    -XX:+UseNUMA -XX:ConcGCThreads=8 -XX:ParallelGCThreads=16 \
    -jar service.jar
```

Generational ZGC log — minor vs. major:

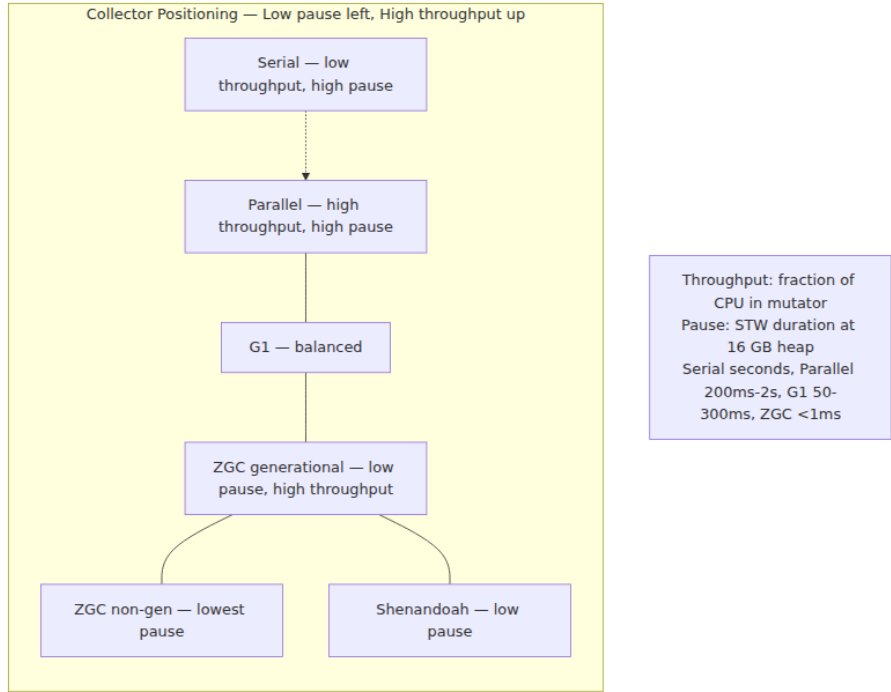
```
[10.042s][info][gc] GC(0) Minor Collection (Young) 8.2G->2.1G(16.0G)
12.042ms
[10.042s][info][gc] GC(0) Young: 6.1G->0.4G(8.0G) Old:
2.1G->1.7G(8.0G) Reclaimed: 5.7G
[10.042s][info][gc] GC(0) Pause Mark Start 0.342ms Pause Relocate Star
t 0.412ms STW total 0.75ms
[15.210s][info][gc] GC(5) Major Collection (Young+Old)
12.4G->4.2G(16.0G) 48.210ms
[15.210s][info][gc] GC(5) Young: 4.2G->0.6G Old: 8.2G->3.6G Reclaime
d: 8.2G
[15.210s][info][gc] GC(5) Pause Mark Start 0.842ms Pause Mark End 1.10
2ms Pause Relocate Start 0.620ms STW total 2.56ms
```

Minor GC STW total < 1 ms; major GC STW total ~2-3 ms, but major is rare. If you see major GC every few seconds, Young is too small or promotion is too aggressive — same tuning as G1.

Non-generational ZGC marks the entire 16 GB heap every cycle. Generational ZGC marks only ~4 GB of Young on minor GC (10-20x more frequent) and the full heap only on major GC — roughly 5x less marking work on allocation-heavy services.

10. Choosing a collector: the latency-throughput-heap matrix

No collector is best for all workloads. The choice is a tradeoff among throughput, pause, heap size, and operational maturity.



Collector	Throughput	Max pause (16 GB heap)	Heap scale	Barrier tax	Maturity
Serial	Low	Seconds	< 2 GB	None	Ancient
Parallel	Highest	200 ms-2 s (STW)	Up to ~32 GB efficiently	None	Ancient
G1	High	50-300 ms (tunable via <code>MaxGCPauseMillis</code>)	Up to ~64 GB	~5-10%	Default since Java 9, most battle-tested
ZGC (non-gen)	Medium-High	< 1 ms STW	Up to 16 TB	~10-15%	Production-ready, JDK 11
ZGC (generational)	High (close to G1)	< 1 ms minor, < 3 ms major	Up to 16 TB	~5-10% (store+load)	Production-ready, JDK 11 (JEP 390), default for JDK 17
Shenandoah	Medium	< 2 ms	Up to ~100 GB typical	~10-15%	Production-ready, JDK 12

10.1 Decision tree for a backend service

Heap < 4 GB, no tight SLO?

- ☐ G1 (**default**) ☐ simplest, well-understood, good throughput

Heap 4☐32 GB, p99 SLO 100☐500 ms?

- ☐ G1 with MaxGCPauseMillis=200 ☐ tune IHOP, region size
- ☐ If G1 Mixed/Full pauses violate SLO ☐ switch to Generational ZGC

Heap > 32 GB, **or** p99 SLO < 10 ms, **or** p999 SLO?

- ☐ Generational ZGC ☐ sub-ms pauses, scales to TBs
- ☐ Alternative: Shenandoah **if** ZGC unavailable

Throughput > latency (batch, nightly jobs)?

- ☐ Parallel ☐ maximize CPU **for** mutator

Latency > throughput (trading, real-time bidding, gaming)?

- ☐ Generational ZGC **or** Shenandoah ☐ minimize STW

Uncertain / fleet **default**?

- ☐ G1 **for** fleet **default**, Generational ZGC **for** latency-sensitive tier
- ☐ Measure: run both with -Xlog:gc* **for** a week, compare p99 **and** CPU

10.2 Heap sizing and container guidance

```
# Container-aware sizing (Kubernetes pod with limit 4 GB, request 4 GB)
# Always set Xms == Xmx to avoid resize pauses; leave headroom for off-heap (Metaspace, direct buffers, thread stacks)
```

```
java -XX:+UseG1GC \
    -Xms3g -Xmx3g \
    -XX:MaxRAMPercentage=75.0 \
    -XX:+UseContainerSupport \
    -XX:MaxGCPauseMillis=200 \
    -jar service.jar
```

```
# Large heap with compressed oops – stay under 32 GB
```

```
java -XX:+UseG1GC -Xms26g -Xmx26g -XX:+UseCompressedOops -jar service.jar
```

```
# Verify at startup:
```

```
# [0.012s][info][gc,heap] Heap region size: 8M
```

```
# [0.012s][info][gc,heap] Compressed Oops: enabled, Compressed Class Pointers: enabled
```

```
# Large heap where compressed oops disabled – jump to 48 GB only if needed
```

```
java -XX:+UseZGC -XX:+ZGenerational -Xms48g -Xmx48g -jar service.jar
# ZGC handles >32 GB without compressed-oops penalty via colored pointers
```


Rule of thumb: leave 25–30% of container memory for off-heap (Metaspace, code cache, direct `ByteBuffer`, thread stacks at 1 MB each). A 4 GB cgroup with `-Xmx3g` is healthier than `-Xmx4g` that OOMKills on the first large `ByteBuffer.allocateDirect`.

11. Reading GC logs: `-Xlog:gc*`, `jstat`, and JFR

11.1 Unified GC logging (`-Xlog`, JDK 9+)

The old `-XX:+PrintGC` / `-XX:+PrintGCDetails` flags are gone. Unified logging uses `-Xlog:<tags>:<level>:<output>`:

```
# Minimal production logging – GC pauses + heap occupancy, 50 MB
rotation
java -Xlog:gc:file=/var/log/gc.log:time,uptime,level,tags:filecount=10,
    filesize=50M \
    -jar service.jar

# Detailed – phases, heap, ergonomics, safepoints (for tuning)
java -Xlog:gc*,gc+phases=debug,gc+heap=info,gc+ergo*=debug,safepoint=in
fo \
    :file=/var/log/
gc.log:time,uptime,level,tags:filecount=10,filesize=50M \
    -jar service.jar

# Reference processing and humongous tracking (G1)
java -Xlog:gc+ref=debug,gc+humongous=debug:file=/var/log/
gc.log:time,uptime,level,tags \
    -jar service.jar
```

Log tags cheat sheet:

Tag	Shows
<code>gc</code>	GC start/end, pause type, heap before→after, duration
<code>gc+phases</code>	Per-phase timings (Evacuate, Remark, RefProc, etc.)
<code>gc+heap</code>	Heap layout, region sizes, compressed-oops mode at startup
<code>gc+ergo</code>	Ergonomics decisions (IHOP, Young sizing, CSet selection)
<code>gc+ref</code>	Reference processing (Soft/Weak/Phantom/Cleaner) time
<code>gc+humongous</code>	Humongous allocation and reclamation (G1)

Tag	Shows
<code>safepoint</code>	All safepoints (not just GC — deoptimization, biased-lock revocation)

11.2 Annotated log walkthrough — G1 Young GC

```
[12.042s][info][gc] GC(18) Pause Young (Normal) (G1 Evacuation Pause)
1420M->380M(4096M) 22.412ms
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] STW duration
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] heap capacity (Xmx)
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] heap after before
[ ] [ ] [ ] [ ] [ ] reason (Allocation Failure vs. G1 Evacuation Pause is the sub-reason)
[ ] [ ] [ ] [ ] [ ] GC id (monotonic counter)
[ ] [ ] [ ] [ ] [ ] level (info/debug/trace)
[ ] [ ] [ ] [ ] [ ] timestamp (uptime or wall-clock with -Xlog:gc:file:...:time)
[ ] [ ] [ ] [ ] [ ]
[12.042s][debug][gc,phases] GC(18) Pre Evacuate Collection Set: 0.2ms
[12.042s][debug][gc,phases] GC(18) Evacuate Collection Set: 14.8ms
[ ] dominant: parallel copy
[12.042s][debug][gc,phases] GC(18) Post Evacuate Collection Set:
3.1ms [ ] reference updates, RS cleanup
[12.042s][debug][gc,phases] GC(18) Other: 4.3ms
[ ] code roots, termination
```

What to watch:

- **Heap before→after:** `1420M->380M` means 1,040 MB reclaimed, 380 MB live. If `380M` grows steadily, Old is accumulating — check promotion rate.
- **Duration breakdown:** `Evacuate` should dominate. If `Other` or `RefProc` dominates, you have JNI/code-root or reference-processing pressure.
- **Frequency:** Young GC every 1–3 seconds is healthy for a 4 GB heap at 1 GB/s allocation. Every 200 ms means Eden is too small.

11.3 Annotated log — G1 concurrent cycle + Mixed

```
[45.210s][info][gc] GC(42) Concurrent Cycle
[45.210s][info][gc] GC(42) marking cycle starts
[45.210s][info][gc] GC(42) Pause Initial Mark (G1 Humongous
Allocation) 2100M->2100M(4096M) 8.102ms [45.210s][info][gc] STW, piggybacked on Young GC
[45.210s][info][gc] GC(42) Concurrent Mark (45.210s, 45.340s)
130.215ms [45.210s][info][gc] concurrent SATB marking
[45.340s][info][gc] GC(42) Pause Remark 2120M->2120M(4096M) 12.408ms
[45.340s][info][gc] STW, drain SATB queues
[45.340s][info][gc] GC(42) Pause Cleanup 2120M->1980M(4096M) 2.114ms
[45.340s][info][gc] STW, reclaim empty Old regions
[45.340s][info][gc] GC(42) Concurrent Cleanup for Next Mark (45.340s, 4
5.342s) 1.802ms
[46.100s][info][gc] GC(43) Pause Young (Prepare Mixed) (G1 Evacuation P
ause) 1680M->420M(4096M) 18.920ms
[46.800s][info][gc] GC(44) Pause Young (Mixed) (G1 Evacuation Pause) 18
00M->440M(4096M) 28.104ms [46.800s][info][gc] evacuates Young + some Old
```

If **Concurrent Mark** duration exceeds the time to fill Old, marking never finishes before Old fills → **Pause Full**. Lower **IHOP** to start marking earlier.

11.4 Annotated log — Generational ZGC

```
[10.042s][info][gc] GC(0) Minor Collection (Young) 8.2G->2.1G(16.0G)
12.042ms
[10.042s][info][gc] GC(0) Young: 6.1G->0.4G(8.0G) Old:
2.1G->1.7G(8.0G) Reclaimed: 5.7G [10.042s][info][gc] per-generation accounting
[10.042s][info][gc] GC(0) Pause Mark Start 0.342ms Pause Relocate Star
t 0.412ms STW total 0.75ms
[15.210s][info][gc] GC(5) Major Collection (Young+Old)
12.4G->4.2G(16.0G) 48.210ms
[15.210s][info][gc] GC(5) Young: 4.2G->0.6G Old: 8.2G->3.6G Reclaime
d: 8.2G
[15.210s][info][gc] GC(5) Pause Mark Start 0.842ms Pause Mark End 1.10
2ms Pause Relocate Start 0.620ms STW total 2.56ms
```

Minor GC is Young-only; major is Young+Old. STW total is the sum that matters for SLOs.

11.5 `jstat` — live heap telemetry without logs

`jstat` polls the JVM's GC counters via `jvmsat` (no log parsing needed, safe for sidecars):

```

# Every second, print GC summary
jstat -gc $(pgrep -f service.jar) 1000

# Output (G1, 16 GB heap, values in KB):
# S0C    S1C    S0U    S1U    EC    EU    OC    OU
MC      MU    CCSC    CCSU    YGC    YGCT    FGC    FGCT    GCT
# 0.0    18432.0  0.0    18432.0  3145728.0  892928.0  12582912.0
2847200.0  89216.0  86200.0  11264.0  10800.0    42    0.892    0
0.000    0.892
# |      |      |      |      |      |      |      |      |
# |      |      |      |      |      |      |      |      |
GC time (s)
# |      |      |      |      |      |      |      |      |
# |      |      |      |      |      |      |      |      |
# |      |      |      |      |      |      |      |      |
# |      |      |      |      |      |      |      |      |
Metaspace / Compressed class space
# | Survivor 0/1 capacity/used, Eden capacity/used, Old capacity/used

# Utilization view (percentages – fastest triage)
jstat -gcutil $(pgrep -f service.jar) 1000
# S0    S1    E    O    M    CCS    YGC    YGCT    FGC
FGCT    GCT
# 0.00 100.00 28.38 22.62 96.62 95.88    42    0.892    0
0.000    0.892

# Cause view – why GCs are firing
jstat -gcause $(pgrep -f service.jar) 1000
# S0    S1    E    O    M    CCS    YGC    YGCT    FGC
FGCT    GCT    LGCC    GCC
# 0.00 100.00 28.38 22.62 96.62 95.88    42    0.892    0
0.000    0.892 Allocation Failure No GC
# | last GC cause (Allocation Failure, G1 Evacuation Pause, G1
Humongous Allocation, Ergonomics)
# | current GC cause (No GC / Allocation Failure / etc.)

# Capacity view – is heap resizing?
jstat -gccapacity $(pgrep -f service.jar) 1000
# NGCMN    NGCMX    NGC    S0C    S1C    EC    OGCMN
OGCMX    OGC    OC    MCMN    MCMX    MC    CCSMN
CCSMX    CCSC    YGC    FGC

```

Alert on: **0** (Old utilization) trending toward 80%+ without dropping (Old not being reclaimed), **FGC** incrementing (Full GCs — always investigate), **GCT** growing faster than **YGC** (pauses lengthening).

11.6 JFR — GC events for dashboards

JFR (Chapter 10) emits structured GC events without log parsing:

```
# Record JFR with GC events
```

```
java -XX:StartFlightRecording=filename=rec.jfr,settings=profile,duration=60s \  
-XX:FlightRecorderOptions=stackdepth=128 \  
-jar service.jar
```

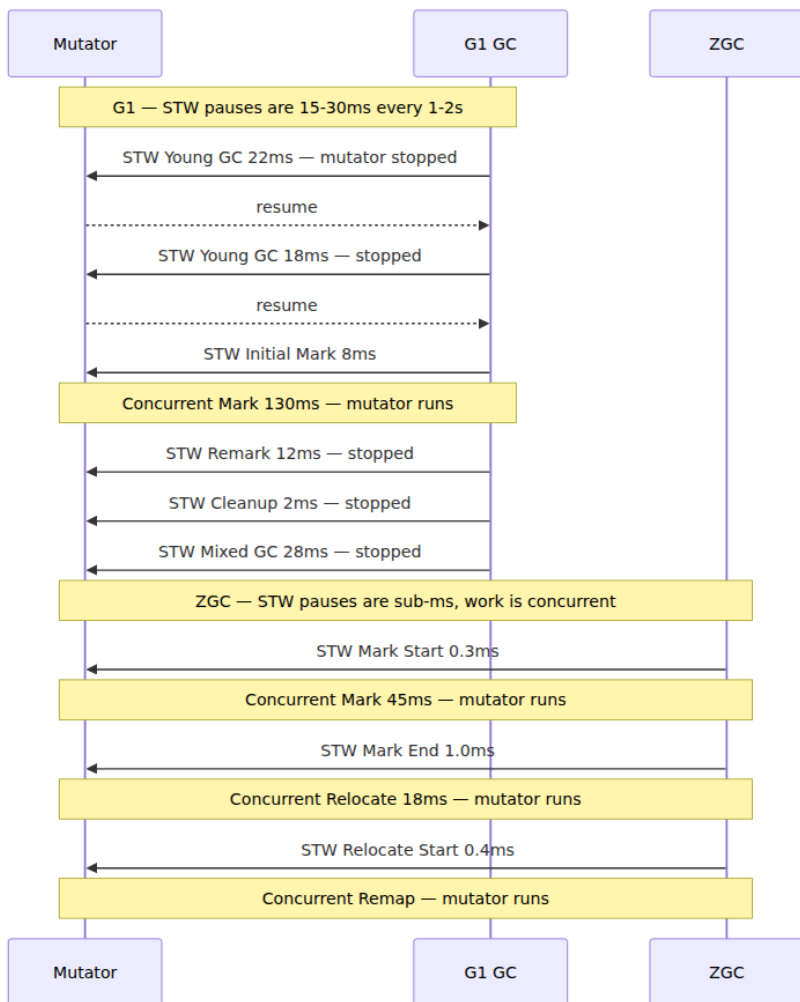
```
jfr print --events jdk.GarbageCollection,jdk.GCHeapSummary,jdk.GCPause  
rec.jfr
```

```
jdk.GarbageCollection {  
  startTime = 12:04:02.042  
  gcId = 18  
  name = "G1 Young Generation"  
  cause = "G1 Evacuation Pause"  
  sumOfPauses = 22.4 ms  
  longestPause = 22.4 ms  
}  
  
jdk.GCHeapSummary {  
  gcId = 18  
  heapSpace = { start = 0x700000000, committedSize = 4.0 GB, reservedSize = 4.0 GB, usedSize = 380 MB }  
  heapUsed = 380 MB  
}
```

Ship `jdk.GarbageCollection` + `jdk.GCPause` to your metrics pipeline (Prometheus via JFR exporter, or OpenTelemetry JFR receiver) to graph pause percentiles alongside request p99 — the correlation is the most persuasive tuning evidence for leadership.

11.7 GC pause timeline — visualizing STW vs. concurrent

STW pauses block all Java threads; concurrent phases run alongside the mutator.



G1's Young GCs are 15–30 ms STW every 1–2 seconds; ZGC's STW is < 1 ms with concurrent work filling the gaps. The mutator's throughput is $1 - (\text{STW time} / \text{wall time})$ — for G1 ~98–99%, for ZGC ~99.9%+.

11.8 Tuning recipes

Recipe 1: G1 Young GC too frequent (Eden too small)

```
# Symptom: jstat YGC every 300ms, Eden 512 MB, allocation 1.5 GB/s
# Fix: increase heap or reduce allocation; verify Eden sizing
java -XX:+UseG1GC -Xms8g -Xmx8g -XX:G1HeapRegionSize=4m -XX:MaxGCPauseM
illis=200 -jar service.jar
# Check: -Xlog:gc+ergo=debug shows "Eden regions: 64->0(64)" – Eden is
64x4 MB = 256 MB (too small)
# Increase region size or heap so Eden is 1–2 GB
```

Recipe 2: G1 Full GC (concurrent marking cannot keep up)

```
# Symptom: logs show "Pause Full (G1 Compaction Pause) 3800M-
>1200M(4096M) 1.2s"
# Fix: start marking earlier, give it more CPU
java -XX:+UseG1GC -Xms16g -Xmx16g \
-XX:InitiatingHeapOccupancyPercent=30 \
-XX:ConcGCThreads=6 \
-XX:G1ReservePercent=15 \
-jar service.jar
```

Recipe 3: Humongous allocation pressure

```
# Symptom: "G1 Humongous Allocation" triggers concurrent cycles; jstat
OU spikes
# Diagnose:
# -Xlog:gc+humongous=debug shows "Humongous object allocation: 8 MB,
region size 2 MB → 4 regions"
# Fix: larger regions or reduce large allocations
java -XX:+UseG1GC -Xms16g -Xmx16g -XX:G1HeapRegionSize=8m -jar service.
jar
# Or: pool direct buffers, reuse byte[] via ThreadLocal, limit gRPC
message size
```

Recipe 4: Switch to Generational ZGC for p99

```
# Baseline with G1: p99 180ms, p999 450ms (GC pauses visible)
# Switch to Generational ZGC:
java -XX:+UseZGC -XX:+ZGenerational \
-Xms16g -Xmx16g \
-XX:ConcGCThreads=4 \
-XX:+ZProactive \
-Xlog:gc:file=/var/log/gc.log:time,uptime,level,tags \
-jar service.jar
# Expect: p99 drops to 20–40ms, p999 to 60–100ms, CPU +5–10% – verify
with JFR + metrics
```

Recipe 5: Allocation stall (ZGC cannot reclaim fast enough)

```
# Symptom: ZGC log "Allocation Stall" – mutator blocked waiting for GC
# Fix: increase heap, raise spike tolerance, or throttle allocation
java -XX:+UseZGC -XX:+ZGenerational -Xms32g -Xmx32g \
  -XX:ZAllocationSpikeTolerance=5 \
  -XX:ConcGCThreads=6 \
  -jar service.jar
# If still stalling: heap is too small for allocation rate – increase
Xmx or reduce per-request allocation
```

12. Distributed-systems lens: GC as a fleet-wide failure mode

GC pauses do not stay local. In a fleet of hundreds of instances, a 200 ms STW pause on one replica is a 200 ms latency spike for every caller holding a connection to it. The amplification patterns that matter:

12.1 Timeout amplification and retry storms

```
Service A (caller) → Service B (callee, G1, 16 GB heap)
  timeout = 500 ms
  retries = 2, backoff 100 ms

B Young GC: 250 ms STW (G1 Mixed with large CSet)
  → A's request times out at 500 ms (250 ms GC + 200 ms queue + 50 ms
  processing)
  → A retries → B now handles 2× load during next GC → longer pauses
  → more timeouts
  → Retry storm: 20 callers × 2 retries = 40 extra requests landing wh
  ile B is still paused
```

Mitigations: **hedged requests** (send to two replicas, use first reply) mask single-replica GC pauses but double load — use only for p99-critical paths. **Retry budgets** (e.g., 20% of requests may be retries) and **exponential backoff with jitter** bound storms. **Coordinated omission-aware metrics** (HdrHistogram) reveal GC-induced latency that average-based metrics hide.

12.2 Consensus and membership

Raft (etcd, Consul, CockroachDB) and ZooKeeper rely on heartbeat timeouts. A GC pause longer than `electionTimeout` (etcd default 1 s) on the leader causes followers to start an election; a pause on a follower

causes the leader to mark it dead and trigger re-replication. ZGC/Shenandoah's sub-ms pauses make JVM-based consensus participants viable; G1 requires careful tuning (`MaxGCPauseMillis` well below `electionTimeout/2`) and over-provisioned heaps.

Kafka's `session.timeout.ms` (default 45 s) and `max.poll.interval.ms` (5 min) are generous, but a Full GC of 2-5 seconds on a Parallel-collected consumer can still trigger rebalance. The Kafka client library's heartbeat thread is a Java thread — it stops during STW.

12.3 Fleet GC coordination

At scale, uncoordinated GC across replicas is desirable — you want pauses *staggered* so at least $N-1$ replicas are responsive. JVMs naturally stagger because allocation rates differ slightly. Anti-patterns that synchronize GC:

- **Simultaneous rolling deploy** — all instances start at the same time, fill Eden at the same rate, GC together. Stagger deploys.
- **Periodic batch jobs** (cache refresh, config reload) that allocate heavily on a timer — all replicas allocate together, GC together. Jitter the timer.
- **Heap sizing that forces Full GC** — Full GC is often triggered by Old occupancy, which grows deterministically; all replicas Full-GC within seconds of each other. Avoid Full GC entirely via concurrent collectors or correct IHOP.

12.4 Sizing for SLOs, not averages

Size heaps from the tail:

1. Measure allocation rate (`jstat -gc` EU delta / wall time, or JFR `jdk.ObjectAllocationInNewTLAB`).
2. Set Young/Eden so Young GC interval is 1-5 seconds (frequent enough to reclaim, infrequent enough to not dominate CPU).
3. Set Old so concurrent marking completes before Old fills — `old growth rate × Concurrent Mark duration < (100% - IHOP) × heap` .
4. Load-test with realistic traffic and measure **pause percentiles** (`jstat GCT` is not enough — parse `gc` logs for per-pause durations, or use JFR `jdk.GCPause` histogram).

5. Correlate GC pauses with request p99 (overlay `jdk.GCPause` on request latency in Grafana) — the spikes should not align after tuning.

For a fleet with p99 SLO of 100 ms, G1 with `MaxGCPauseMillis=100` is a starting point but not a guarantee — G1 may exceed the goal under pressure. Generational ZGC with < 1 ms STW gives headroom for the rest of the request path (network, serialization, downstream calls) to stay within SLO. The extra 5–10% CPU tax is often cheaper than the alternative: over-provisioning replicas to absorb GC-induced latency.

12.5 Observability checklist

- [] `-Xlog:gc:file=/var/log/gc.log:time,uptime,level,tags:filecount=10,filesize=50M` on every JVM
- [] `jstat -gcutil` scraped by sidecar or JMX exporter (or `jvm_gc_pause_seconds` via Micrometer)
- [] JFR `jdk.GCPause` + `jdk.GarbageCollection` shipped to metrics (Prometheus/Grafana)
- [] Alert on `FGC > 0` (any Full GC), `GCT / wall time > 5%` (GC overhead), `Old utilization > 80%` sustained
- [] Dashboard: GC pause histogram overlaid with request p50/p99, heap occupancy (Eden/Survivor/Old), allocation rate
- [] Chaos test: inject allocation pressure (`byte[]` churn) and verify SLO holds — same as Jepsen but for GC

Key takeaways

- GC is a tracing problem: mark live from roots via tri-color, then reclaim white. The tri-color invariant (no black→white) is maintained by SATB pre-barriers (G1/ZGC/Shenandoah) or incremental-update post-barriers — every concurrent collector pays a barrier tax on the mutator's hot path.
- Generations exploit mortality: Young GC copies survivors at $O(\text{live-young})$, Old is collected rarely. TLABs make allocation a bump pointer; humongous/large objects bypass Young and can fragment G1 — size `G1HeapRegionSize` or use ZGC large pages to mitigate.

- Serial and Parallel are STW throughput collectors — no concurrency, maximum CPU for the mutator, pauses proportional to live set. Use Parallel for batch; never for latency-sensitive backends.
- G1 is the balanced default: region array, SATB concurrent marking, Young and Mixed evacuation pauses, `MaxGCPauseMillis` as a goal not a guarantee. Tune `IHOP`, `ConcGCThreads`, and region size; avoid Full GC by ensuring concurrent marking keeps up.
- ZGC achieves sub-ms STW via colored pointers and load barriers (test + self-heal, JIT-elided) with concurrent mark/relocate/remap. Shenandoah achieves the same via Brooks forwarding pointers and concurrent evacuation. Both trade ~5-15% throughput for an order-of-magnitude pause reduction and scale to large heaps (ZGC to 16 TB).
- Generational ZGC (JDK 21, JEP 474, default in JDK 23) adds Young/Old separation to ZGC: minor GC marks only Young (frequent, cheap), major GC marks Old+Young (rare). It reclaims the generational advantage non-generational ZGC gave up, cutting GC CPU 20-40% on allocation-heavy services while preserving sub-ms pauses — it is the preferred low-pause collector for new deployments.
- Choosing a collector is a latency-throughput-heap matrix: Parallel for throughput, G1 for balanced/default, Generational ZGC (or Shenandoah) for $p99 < 10\text{ ms}$ or $\text{heap} > 32\text{ GB}$. Always measure — run both under production traffic with `-Xlog:gc*` and compare pause histograms vs. CPU.
- Logs are the source of truth: `-Xlog:gc*` for pause types/durations/phases, `jstat -gcutil` / `-gcause` for live telemetry, JFR `jdk.GCPause` for dashboard-grade pause histograms. Alert on Full GC, Old pressure, and GC overhead > 5%.
- At fleet scale GC is a distributed failure mode: STW pauses become timeout amplification, retry storms, and consensus stalls. Stagger GC via jittered deploys/timers, size heaps from tail percentiles, and correlate GC pauses with request p99 — the overlay tells you whether GC is your tail-latency bottleneck.

Further reading

- **JLS 17, Chapter 12 (Execution) and Chapter 17 (Threads and Locks)** — reachability, finalization, and reference processing semantics. <https://docs.oracle.com/javase/specs/jls/se21/html/index.html>
- **HotSpot GC documentation — G1, ZGC, Shenandoah (OpenJDK)** — official collector docs, flags, and ergonomics. <https://docs.oracle.com/en/java/javase/21/gctuning/garbage-first-garbage-collector.html> <https://docs.oracle.com/en/java/javase/21/gctuning/z-garbage-collector1.html> <https://wiki.openjdk.org/display/shenandoah/Main>
- **JEP 333: ZGC — A Scalable Low-Latency Garbage Collector** — design, colored pointers, load barriers, multi-mapping. <https://openjdk.org/jeps/333>
- **JEP 189: Shenandoah — A Low-Pause-Time Garbage Collector** — Brooks pointers, concurrent evacuation. <https://openjdk.org/jeps/189>
- **JEP 474: ZGC — Generational Mode** — generational ZGC design, store barriers, Young/Old separation. <https://openjdk.org/jeps/474>
- **JEP 363: Remove the Concurrent Mark Sweep (CMS) Garbage Collector** — why CMS was removed and G1 replaced it. <https://openjdk.org/jeps/363>
- **JEP 307: Parallel Full GC for G1 and JEP 346: Promptly Return Unused Committed Memory from G1** — G1 Full GC and heap uncommit. <https://openjdk.org/jeps/307> <https://openjdk.org/jeps/346>
- **HotSpot SATBMarkQueue , G1BarrierSet , ZBarrierSetAssembler , ShenandoahBarrierSetAssembler** — source-level barrier implementations. <https://github.com/openjdk/jdk/tree/master/src/hotspot/share/gc>
- **Gil Tene — Understanding Java GC, Pauses, and HdrHistogram (QCon, InfoQ)** — pause measurement, coordinated omission, and GC tuning methodology. <https://www.infoq.com/presentations/Tene-HdrHistogram/>
- **Jean-Philippe Bempel et al. — ZGC Deep Dive (JVMLS, Devvxx)** — ZGC internals, generational ZGC performance, and production tuning. https://www.youtube.com/results?search_query=ZGC+deep+dive+JVMLS

- **Charlie Hunt, Monica Beckwith, Poonam Parhar, Bengt Rutisson — *Java Performance* (2nd ed., Addison-Wesley, 2014)** — chapters 5-8 on GC tuning, still relevant for generational mechanics and ergonomics.
- **JFR Event Reference** — `jdk.GarbageCollection`, `jdk.GCPause`, `jdk.GCHeapSummary`, `jdk.ObjectAllocationInNewTLAB` — structured GC observability. <https://docs.oracle.com/en/java/javase/21/docs/specs/man/jfr.html>

Chapter 5 — The JIT: Interpreters, C1, C2, and Graal — Deoptimization, Inlining, and Intrinsic

What this chapter covers. The JVM's just-in-time compiler is the single largest source of performance in managed languages — yet most backend engineers treat it as a black box. When a Kotlin service achieves 80% of C throughput on gRPC encode/decode, that is not magic; it is escape analysis eliminating allocations, inlining collapsing virtual dispatch, intrinsics replacing method calls with single instructions, and loop opts auto-vectorizing tight iteration — all guarded by deoptimization so the JVM can recover when assumptions break. This chapter opens the entire pipeline: the template interpreter that runs first, the tiered compilation state machine that decides when to compile, the C1 compiler that gathers profiles cheaply, the C2 compiler that turns profiles into optimized machine code via a sea-of-nodes intermediate representation, and the Graal compiler that replaces C2 on JVMCI. We trace inlining heuristics, intrinsics like `System.arraycopy` and `VarHandle`, the uncommon-trap mechanism that powers deoptimization, and the profiling infrastructure (branch, type, call-site) that feeds every decision. You will read real `-XX:+PrintCompilation` logs, interpret `-XX:+PrintInlining` trees, examine JFR compilation events, and walk through `PrintAssembly` output to see exactly what the JIT emits for your code.

Learning goals — after this chapter you should be able to:

- Trace HotSpot's tiered compilation pipeline through all five levels (0-4), explain when each fires, and configure thresholds with

`-XX:CompileThreshold` , `-XX:TieredStopAtLevel` , and `-XX:+TieredCompilation` .

- Describe C1's role: fast compilation, limited optimizations, instrumentation counters, and why it exists as a separate pass before C2.
- Explain C2's sea-of-nodes intermediate representation, global value numbering (GVN), escape analysis, loop optimizations (unrolling, peeling, predication, auto-vectorization), and control-flow elimination.
- Read `-XX:+PrintInlining` output and explain inlining decisions: size thresholds (`-XX:MaxInlineSize` , `-XX:FreqInlineSize`), depth limits, monomorphic/bimorphic/megamorphic dispatch, and inline caches.
- Identify JVM intrinsics (`System.arraycopy` , `Unsafe` , `VarHandle` , `Math.*` , `CRC32`) and explain why they bypass the normal compilation path.
- Explain deoptimization: uncommon traps, speculative optimizations, assumption metadata, the deopt mapping from compiled frames back to interpreter frames, and why class loading can trigger deopt storms.
- Describe GraalVM's Graal compiler: JVMCI interface, partial evaluation, the Truffle framework for polyglot languages, and how Graal compares architecturally to C2.
- Use `-XX:+PrintCompilation` , `-XX:+PrintInlining` , `-XX:+UnlockDiagnosticVMOptions` `-XX:+PrintAssembly` , and JFR events (`jdk.Compilation` , `jdk.Inlining` , `jdk.OptimizerDecision`) to diagnose JIT behavior in production.

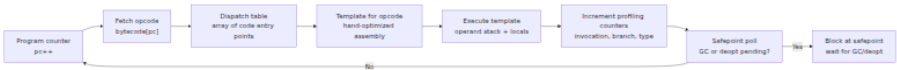
Prerequisites. Chapter 1 introduced the template interpreter and tiered compilation levels at a high level. Chapter 3 covered the JMM and object layout — the JIT reorders subject to the JMM and intrinsifies `VarHandle` access modes. Chapter 6 builds on what the JIT produces: Loom's virtual threads depend on the JIT's ability to optimize through continuations. Volume 13, Chapter 5 covers profiling tools (`jfr` , `async-profiler` , `jcmd`) that feed into the JIT's own profiling.

1. The template interpreter — where every method starts

Before the JIT compiles anything, the **template interpreter** executes it. Understanding the interpreter is not academic — it determines your baseline latency, it collects the profiling data that drives every JIT decision, and it is where execution resumes after every deoptimization.

1.1 Architecture

HotSpot's interpreter is not a switch-based interpreter (like early JVMs). It is a **template interpreter**: for each of the ~202 bytecodes, HotSpot generates a short, hand-optimized assembly-language "template" at JVM startup. These templates are stored in a contiguous code buffer called the **interpreter generator**. Execution dispatches through a table of function pointers indexed by opcode number.



This is faster than a `switch` because: - The indirect jump through `dispatch_table` is a single branch prediction unit entry — the CPU's BTB (Branch Target Buffer) learns it quickly. - Each template is hand-tuned for its specific opcode (e.g., `iload_0` is a single register move, not a loop iteration). - HotSpot embeds **profiling counters** directly into templates: per-method invocation counts, per-branch taken/not-taken counts, and per-call-site type counts.

1.2 Profiling counters

The interpreter is the JVM's primary profiling instrument. For every method at level 0, HotSpot maintains:

Counter	What it tracks	How it drives compilation
Invocation count	Number of times the method is entered	Triggers level 2 (C1) compilation at <code>CompileThreshold</code>
Backedge count	Number of loop back-edges taken	Triggers on-stack replacement (OSR) for hot loops

Counter	What it tracks	How it drives compilation
Branch taken/not-taken	Per-branch-taken bytecode counts	Feeds C2's branch prediction and dead-code elimination
Receiver type	Per-call-site monomorphic/bimorphic/megamorphic classification	Feeds C2's inlining and devirtualization decisions
Argument type	Observed types at call sites	Feeds C2's type profiling and specialization

These counters are **not free**. Each increment is a memory store to a counter object adjacent to the method's metadata. In tight loops, counter traffic can consume measurable bandwidth — this is why the interpreter runs 10–30× slower than C2-compiled code, not just because of dispatch overhead but because of the profiling bookkeeping.

1.3 On-Stack Replacement (OSR)

A critical interpreter capability: if a method has a hot loop (high backedge count) but few total invocations, the interpreter does not wait for the method to be entered enough times. Instead, it triggers **On-Stack Replacement (OSR)**: C2 compiles just the loop body, and at the next backedge, execution transfers from the interpreter frame to the compiled code mid-method. This is how a long-running batch loop gets optimized even if it is called only once.

2. Tiered compilation — the five-level state machine

HotSpot's tiered compilation is a state machine where each method progresses through up to five levels, each trading compilation speed for code quality.

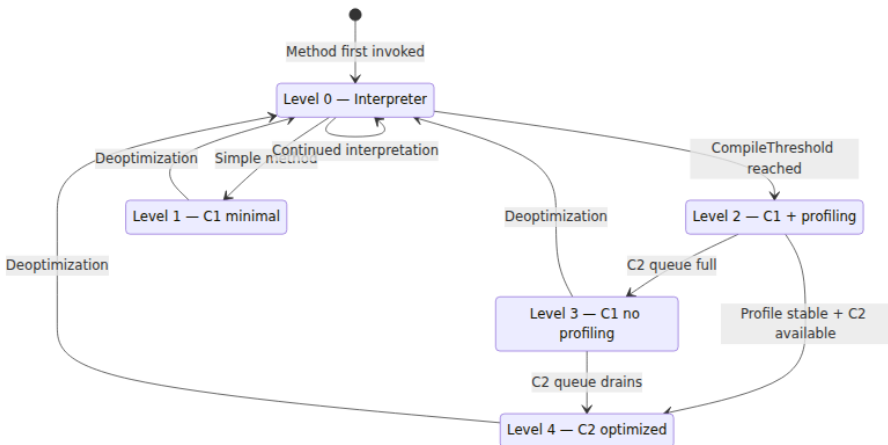
2.1 The levels

Level	Compiler	Purpose	Profile collected?
0	Interpreter	Execute, count invocations and branches	Yes — all profiling counters active

Level	Compiler	Purpose	Profile collected?
1	C1 (minimal)	Fast compile, no profiling overhead	No — for methods called rarely but need C1 speed
2	C1 (full profiling)	Compile with profiling instrumentation	Yes — full branch/type/call-site profiling
3	C1 (no profiling)	C1-compiled code without profiling counters	No — intermediate step when C2 queue is busy
4	C2	Full optimization: inlining, EA, loop opts, vectorization	No — consumes profiles collected at level 2

The default progression for a server JVM:

Level 0 (interpreter)
 → at CompileThreshold invocations:
 Level 2 (C1 + profiling)
 → at additional threshold, C2 queue available:
 Level 4 (C2 full optimization)



If C2 compilation fails or the queue is backlogged, the method may stay at level 3 (C1 without profiling) as a stable intermediate state. When deoptimization occurs, the method falls all the way back to level 0 (interpreter) — it does not resume at C1.

2.2 Thresholds and flags

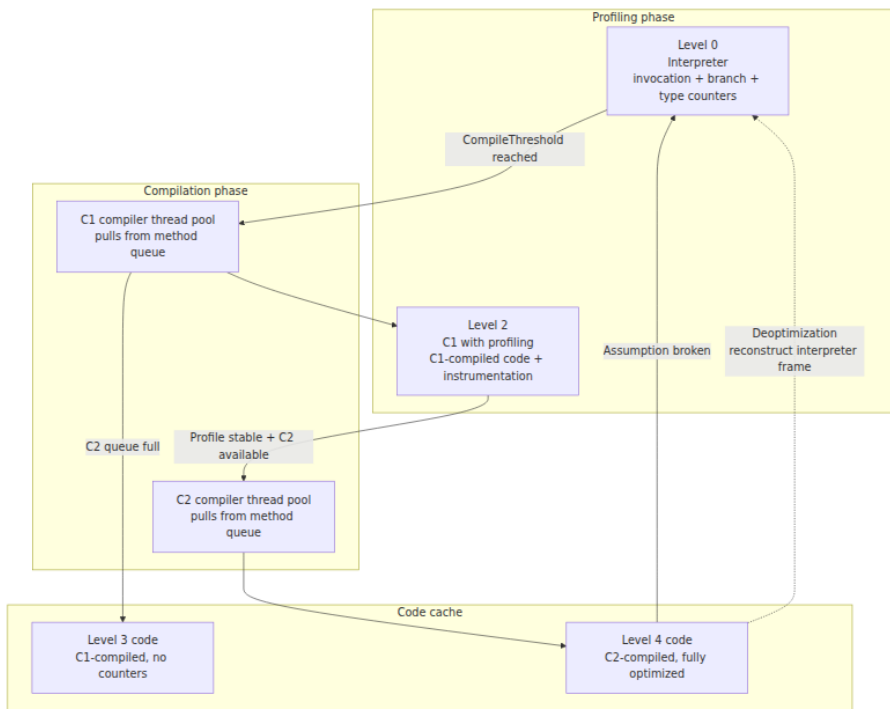
Flag	Default (server)	Default (client)	What it controls
<code>-XX:CompileThreshold</code>	10,000	1,500	Invocations before level 0 → level 2
<code>-XX:OnStackReplacePercentage</code>	140	100	Backedge count relative to invocation count for OSR
<code>-XX:TieredStopAtLevel</code>	4	4	Maximum compilation level (set to 3 to disable C2)
<code>-XX:+TieredCompilation</code>	true	true	Enable/disable tiered compilation entirely
<code>-XX:Tier4CompileThreshold</code>	5,000	5,000	Additional invocations before level 2 → level 4
<code>-XX:Tier3InvocationThreshold</code>	200	200	Invocations before level 1 → level 2
<code>-XX:ReservedCodeCacheSize</code>	240 MB	240 MB	Max size for JIT-compiled native code

Setting `-XX:TieredStopAtLevel=3` disables C2 entirely — useful for latency-sensitive microservices where C2 compilation pauses are unacceptable. The trade-off is 10–20% lower peak throughput.

2.3 The compilation queue

C1 and C2 compilation requests go into separate **method queues**. A pool of compiler threads (controlled by `-XX:C1CompilerCount`, default = 3 ×

number of CPUs for tiered mode) pulls methods from the queue and compiles them.



The key operational insight: **C2 compilation is expensive** — a complex method can take 100ms+ to compile. During that time, the method either continues executing at C1 speed (level 3) or blocks on the C2 queue. In large applications with many hot methods, the C2 queue can back up, creating a compilation backlog that delays peak performance. Monitor this with `jcmd Compiler.cqueue` and `-XX:+PrintCompilation`.

3. C1 — the fast compiler

The C1 compiler (also called "Client" or "java" compiler) exists for two reasons: **fast compilation** and **cheap profiling**.

3.1 What C1 does

C1 performs a straightforward compilation pipeline:

1. **Parse bytecode** into a linear IR (intermediate representation).
2. **Local optimizations**: constant folding, dead code elimination within basic blocks, copy propagation.
3. **Register allocation**: linear scan allocator (fast, produces decent but not optimal code).
4. **Code emission**: platform-specific machine code.

C1 does **not** perform: - Method inlining (beyond trivial methods) - Escape analysis - Loop optimizations - Global value numbering - Control-flow graph optimization

3.2 Why C1 exists

C1 compilation typically takes 1-5ms per method — compared to C2's 50-500ms. This means:

- **Startup latency**: Methods that are called enough to need compilation (but not hot enough to justify C2) get C1 code quickly, avoiding interpreter overhead without the compilation pause of C2.
- **Profiling collection**: At level 2, C1 compiles the method with profiling instrumentation inserted — the compiled code runs faster than the interpreter while still collecting type, branch, and call-site data for C2.
- **Fallback**: When C2 fails (compilation timeout, code cache pressure, too-complex IR), the method stays at C1 level 3 — still much faster than the interpreter.

3.3 `-XX:+PrintCompilation` output

When you enable `-XX:+PrintCompilation`, each compilation event is logged:

765	1	3	java.lang.String::charAt (29 bytes)
812	2	3	java.lang.String::length (6 bytes)
823	3	3	
java.lang.AbstractStringBuilder::ensureCapacityHelper (14 bytes)			
901	4	4	java.lang.StringBuilder::append (8 bytes)
912	5	4	java.lang.String::equals (16 bytes)
934	6	4	java.util.HashMap::hash (12 bytes)
945	7	4	java.util.HashMap::get (23 bytes)
1023	8	4	java.util.HashMap::getNode (45 bytes)
1200	9	3	java.lang.Object::<init> (1 bytes)
1245	10	4	java.lang.StringBuilder::toString (37 bytes)
1400	11	4	java.lang.String::substring (35 bytes)
1523	12	4	java.util.ArrayList::add (23 bytes)

Reading the columns: 1. **Timestamp** (milliseconds since JVM start) 2. **Compilation ID** (unique per compilation request) 3. **Compilation level** (3 = C1 no-profiling, 4 = C2) 4. **Method** (class::method, bytecode size) 5. Additional info may include attributes: `%` = OSR, `s` = synchronized, `!` = has exception handler, `b` = blocking (compilation thread was blocked)

The output `3` and `4` in the third column are compilation *levels*, not the compilation tier of the source. Level 3 means C1-compiled code without profiling; level 4 means C2-compiled code.

4. C2 — the optimizing compiler

C2 is where the JVM's real optimization happens. It compiles bytecode into highly optimized machine code through a sophisticated IR and multiple optimization passes.

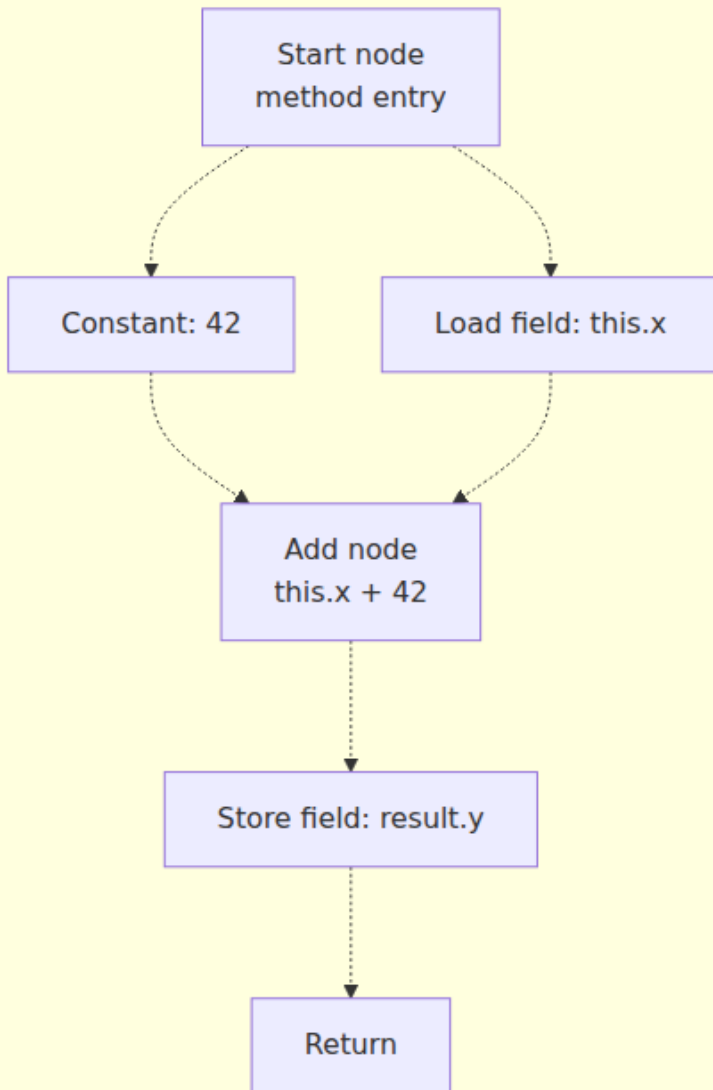
4.1 Sea-of-nodes IR

C2's intermediate representation is called **sea-of-nodes** — a graph where nodes represent operations (add, load, store, compare, call) and edges represent data flow and control flow. Unlike a linear IR (which represents code as a sequence of instructions), sea-of-nodes represents code as a **data-flow graph** where:

- **Data edges** connect producers to consumers: the output of an `iadd` node feeds into a `store` node.

- **Control edges** represent sequencing: a `region` node merges control flow from multiple branches; a `merge` node joins data from both paths.
- **Memory edges** track which memory state each operation reads from and writes to — enabling precise alias analysis.

Sea-of-nodes IR (conceptual)



Optimization passes

GVN: global value
numbering¹⁵⁹

4.2 Global Value Numbering (GVN)

GVN assigns each unique computation a "value number" and replaces duplicate computations with references to the first. This eliminates redundant loads, redundant arithmetic, and common subexpressions — across basic blocks, not just within them.

```
// Before GVN
int a = x + y;
int b = x + y;    // redundant – same value as a
int c = a * 2;

// After GVN
int a = x + y;
int b = a;        // GVN replaces with reference to a
int c = a * 2;
```

In the sea-of-nodes IR, GVN works by merging identical nodes: two `Add(x, y)` nodes with the same inputs become a single node, and all consumers of the second redirect to the first.

4.3 Escape analysis

Escape analysis determines whether an object **escapes** the current method or thread. When an object does not escape, C2 can perform transformations that eliminate heap allocation entirely:

Scalar replacement: If an object's fields can be represented as independent scalar variables (registers), C2 allocates them as registers instead of heap objects. The object never exists in memory.

```
Point p = new Point(x, y); // p never escapes this method
return p.x + p.y;          // C2 replaces with: return x + y
// No allocation, no GC – the object literally does not exist
```

Lock elision: If a `synchronized` block locks an object that does not escape its thread, the monitor is eliminated entirely — no `monitorenter`, no `monitorexit`, no bias locking. This is why `StringBuilder.append` (which is `synchronized` on some JDK versions) performs well even in multi-threaded code — the JIT elides the lock for thread-local `StringBuilder` instances.

Allocation merging: Multiple allocations of the same size with the same initialization pattern can be merged into a single allocation with overwrites.

Escape analysis depends on **stable type profiles** — C2 must be confident that the object truly does not escape. Deoptimization occurs if an assumption is violated (e.g., a method that previously did not escape its object starts returning it).

4.4 Loop optimizations

C2 applies several loop-level optimizations:

Optimization	What it does	When it helps
Loop unrolling	Replicates the loop body to reduce branch overhead	Tight loops with known trip counts
Loop peeling	Executes the first iteration separately to simplify the main loop	Loops with special first-iteration behavior
Loop predication	Hoists loop-invariant bounds checks outside the loop	Array iteration with provably in-bounds indices
Loop normalization	Converts loops to canonical form (int counter, increment by 1)	Enables further optimization
Auto-vectorization (SIMD)	Processes multiple array elements per instruction using SSE/AVX/NEON	Tight loops over arrays of primitives with simple operations
Loop interchange	Reorders nested loops for better cache locality	Nested loops accessing multi-dimensional arrays

Auto-vectorization is the highest-impact loop optimization. On x86_64 with AVX-512, the JIT can process 16 `int` values or 8 `long` values per instruction. This requires:

1. The loop body operates on a flat array of primitives.
2. No loop-carried dependencies (or dependencies that can be reduced).
3. The trip count is known or predictable.

4. No aliasing or alias analysis proves no aliasing.

4.5 Control-flow optimizations

Beyond data-flow optimizations, C2 also optimizes control flow:

- **Null check elimination:** If profiling or type analysis shows a reference is never null, the null check is removed.
- **Range check elimination:** Array bounds checks that C2 can prove are always in-bounds are eliminated. This is critical for array iteration performance.
- **Diversion (uncommon trap) insertion:** For optimizations that depend on assumptions (e.g., "this call site is monomorphic"), C2 inserts a guard that triggers a deoptimization if the assumption fails. The compiled code is fast on the common path; the uncommon path pays the deopt cost.
- **Empty exception handler elimination:** If an exception handler is provably unreachable, it is removed along with its metadata.

5. Inlining — the most impactful optimization

Method inlining is universally regarded as the single most important JIT optimization. It eliminates call overhead (frame creation, argument passing, return), but more importantly it **exposes the callee's data flow to the caller's optimizer** — enabling constant propagation, dead code elimination, escape analysis, and other optimizations across method boundaries.

5.1 Inlining heuristics

C2 uses a multi-factor decision process for inlining:

Factor	Flag	Default	Meaning
Maximum bytecode size (cold)	<code>-XX:MaxInlineSize</code>	35	Inline if callee ≤ 35 bytes
Maximum bytecode size (hot)	<code>-XX:FreqInlineSize</code>	325	Inline if callee ≤ 325 bytes and hot

Factor	Flag	Default	Meaning
Maximum inlining depth	<code>-XX:MaxInlineLevel</code>	9	Maximum call chain depth for inlining
Maximum receiver types (monomorphic)	<code>-XX:MaxInlineMaxLevel</code>	5	Inline up to this many receiver types
Minimum invocation count	<code>-XX:MinInliningThreshold</code>	250	Minimum profiling data before inlining
Incremental inlining	<code>-XX:+IncrementalInline</code>	true	Inline incrementally as compilation progresses

The size thresholds are in **bytecodes**, not native instructions. A 35-byte Java method might be 10 native instructions after compilation — or 200 after unrolling. The thresholds are conservative because C2 must be able to compile the inlined code within its compilation time budget.

5.2 Monomorphic, bimorphic, and megamorphic dispatch

The inlining decision depends critically on **call-site polymorphism** — how many different receiver types are observed at a virtual call site:

Polymorphism	Definition	C2 behavior
Monomorphic	Exactly one receiver type observed	Inline with a single type guard
Bimorphic	Exactly two receiver types observed	Inline both targets with two type guards
Megamorphic	Three or more receiver types observed	Do not inline — use indirect/virtual dispatch

```
// Monomorphic - ArrayList always
List<String> list = new ArrayList<>();
for (int i = 0; i < 1000; i++) {
    list.add("item"); // C2 inlines ArrayList.add with type guard
}

// Megamorphic - many implementations
void process(List<String> list) {
    list.add("item"); // C2 does not inline - too many types
}
```

5.3 Inline caches

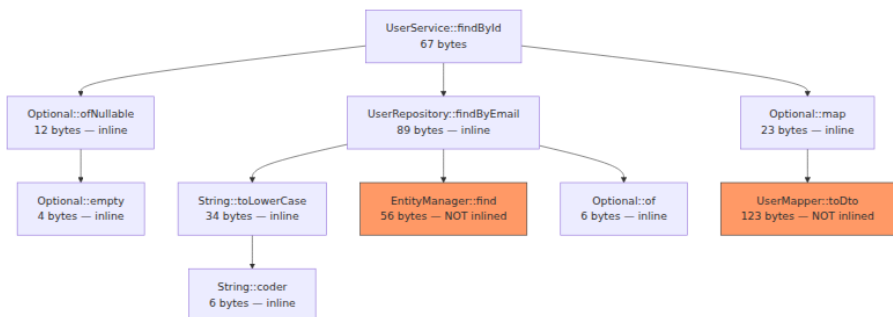
The interpreter maintains **inline caches (ICs)** — per-call-site data structures that record the observed receiver types. An IC has three states:

1. **Uninitialized**: no types observed yet.
2. **Monomorphic**: one type observed; stores the type and a cached method entry point.
3. **Polymorphic**: two or more types observed; stores a list of (type, entry-point) pairs.

C2 reads IC data during compilation to make inlining decisions. The IC is the bridge between profiling and optimization.

5.4 Inlining tree output

When you enable `-XX:+PrintInlining`, C2 logs its inlining decisions:



Reading the inlining tree: - **Unhighlighted nodes**: Successfully inlined — the callee's code is embedded in the caller. - **Red nodes** (EntityManager::find, UserMapper::toDto): Not inlined —

`EntityManager::find` lacks type profile (JPA proxies vary by provider), `UserMapper::toDto` exceeds the 35-byte `MaxInlineSize` threshold. - **Depth:** `findById` → `findByEmail` → `toLowerCase` → `coder` is 4 levels deep — within the default `MaxInlineLevel=9`.

Reading the output: - `@ N` is the bytecode offset of the call site in the caller. - `inline (hot)` means the callee was inlined because it was hot (frequent call site, small size). - `inline (intrinsic)` means the callee is an intrinsic (see Section 7). - `@ N bytes` is the callee's bytecode size. - Indentation shows the inlining depth — `charAt` → `coder` → `isLatin1` is depth 3.

When inlining is **rejected**, the log shows why:

```
@ 120   com.example.Service::processRequest (285 bytes)
  @ 1    com.example.Repository::findById (45 bytes)   inline (hot)
    @ 1    java.util.Optional::orElseThrow (12 bytes)   inline (hot)
      @ 20   com.example.Mapper::toDto (120 bytes)    too large (120 > 35)
```

6. Intrinsic — methods that are not methods

Intrinsics are methods that the JIT recognizes and replaces with **hand-crafted machine code** or **efficient instruction sequences** rather than compiling the Java source normally. They exist because certain operations cannot be expressed efficiently in Java bytecode — or because the JVM can implement them more efficiently than Java code using platform-specific instructions.

6.1 Common intrinsics

Intrinsic	What it does	Why it matters
<code>System.arraycopy</code>	Bulk array copy	Uses platform-specific <code>rep movsb / memcpy</code> with GC barriers; avoids element-by-element Java loop
<code>System.arraycopy</code> (heap-to-heap)	Cross-generational copy	Uses write barriers for GC correctness; Java code cannot replicate this

Intrinsic	What it does	Why it matters
<code>Thread.currentThread()</code>	Get current thread	Single instruction: reads TLS (thread-local storage)
<code>Thread.isInterrupted()</code>	Check interrupt flag	Reads TLS interrupt bit without method call
<code>Object.hashCode()</code>	Identity hash code	Reads mark word directly; Java code cannot access it
<code>Object.getClass()</code>	Get runtime class	Single memory load of klass pointer
<code>Math.abs/int/long/float/double</code>	Absolute value	Uses <code>cdq + xor / neg</code> on x86; no branch
<code>Math.min/max</code>	Min/max	Uses <code>cmov</code> on x86 (conditional move, no branch)
<code>Math.sqrt</code>	Square root	Single <code>sqrtsd</code> instruction
<code>Math.fma</code>	Fused multiply-add	Single <code>vfmadd</code> instruction on AVX2+
<code>Arrays.copyOf</code>	Array copy + resize	Delegates to intrinsified <code>arraycopy</code>
<code>String.equals</code>	String comparison	SIMD-vectorized byte comparison (SSE4.2/NEON)
<code>String.indexOf</code>	Substring search	SIMD-vectorized search (SSE4.2/NEON)
<code>String.hashCode</code>	String hash	SIMD-vectorized polynomial hash
<code>CRC32.update</code>	CRC32 computation	Uses <code>crc32</code> instruction on x86 (SSE4.2)
<code>VarHandle.getAcquire / setRelease</code>	Memory-ordered access	Maps to specific memory barriers (<code>LoadAcquire</code> , <code>StoreRelease</code>)

6.2 Unsafe and VarHandle intrinsics

`sun.misc.Unsafe` and `java.lang.invoke.VarHandle` are the JVM's escape hatches for low-level memory operations. Their methods are heavily intrinsified:

```
// VarHandle access modes and their barrier semantics
VarHandle vh = MethodHandles.lookup()
    .findVarHandle(MyClass.class, "field", int.class);

// Plain – no barrier (like a plain field access)
vh.get(obj);

// Volatile – full fence (StoreLoad barrier on x86)
vh.getVolatile(obj);
vh.setVolatile(obj, value);

// Acquire/Release – one-way barriers
vh.getAcquire(obj);    // LoadLoad + LoadStore after this load
vh.setRelease(obj, value); // StoreStore + LoadStore before this store

// Opaque – no reordering with adjacent accesses
vh.getOpaque(obj);
vh.setOpaque(obj, value);
```

On x86_64, the barrier mapping is:

VarHandle mode	x86_64 instruction sequence	Cost
<code>get (plain)</code>	<code>mov</code>	Free
<code>getAcquire</code>	<code>mov</code> (x86 TSO makes loads ordered)	Free on x86, <code>dmb ishld</code> on AArch64
<code>setRelease</code>	<code>mov</code> (x86 TSO makes stores ordered)	Free on x86, <code>stlr</code> on AArch64
<code>getVolatile</code>	<code>mov + lock add [rsp], 0</code> (full fence)	~20 cycles
<code>setVolatile</code>	<code>xchg</code> (atomic exchange, implicit fence)	~20 cycles

The JIT intrinsifies each mode to the minimal barrier for the target architecture. Writing `getAcquire` in Java with manual barriers cannot be

faster than the intrinsic — the JIT already emits the cheapest legal sequence.

6.3 Reading intrinsics in PrintAssembly

With `-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly`, you can see the actual machine code the JIT emits for intrinsics. Here is the x86_64 output for `System.arraycopy` of 8 `int` elements:

```
;; System.arraycopy for int[8] – generated by C2
0x00007f3a2c010a80: mov    %rsi,%rcx           ; src offset
0x00007f3a2c010a83: mov    %rdx,%r8            ; dst offset
0x00007f3a2c010a86: mov    0x10(%rdi,%rcx,4),%eax ; load src[i]
0x00007f3a2c010a8a: mov    %eax,0x10(%r9,%r8,4) ; store to
dst[i]
0x00007f3a2c010a8f: mov    0x14(%rdi,%rcx,4),%eax ; load src[i+1]
0x00007f3a2c010a93: mov    %eax,0x14(%r9,%r8,4) ; store to
dst[i+1]
;; ... repeated for all 8 elements (loop unrolled)
0x00007f3a2c010ab8: retq
```

The JIT unrolled the 8-element copy into 8 pairs of load/store instructions — no loop, no bounds checks, no array store barrier overhead (because the JIT proved the source and destination are non-overlapping `int[]` with no GC interaction).

7. Deoptimization — the safety net

Deoptimization is the mechanism by which the JVM recovers when its speculative optimizations prove wrong. Without deoptimization, the JIT could not speculate — and without speculation, C2's optimizations would be limited to provably correct transformations, missing the 80% of performance that comes from profiling-based specialization.

7.1 Uncommon traps

When C2 compiles a method, it inserts **guards** (uncommon traps) around speculative assumptions. If a guard fails at runtime, execution transfers to the interpreter:


```
// C2 compiles this assuming monomorphic dispatch
void process(List<String> list) {
    list.add("item"); // C2 assumes list is always ArrayList
}

// Guarded compiled code (conceptual):
if (list.getClass() != ArrayList.class) {
    deoptimize(); // uncommon trap – fall back to interpreter
}
// Inline ArrayList.add (fast path)
arrayList.elementData[arrayList.size++] = "item";
```

The deoptimization process: 1. **Trap fires**: The guard condition fails at runtime. 2. **Safepoint**: The thread reaches a safepoint (all threads pause briefly). 3. **Frame reconstruction**: C2's compiled frame is mapped back to an interpreter frame using **debug information** (mapping tables that describe which source-level variable lives in which register or stack slot at each safepoint). 4. **Execution resumes**: The interpreter re-executes the method from the point where the trap fired, now with updated profiling data.

7.2 What triggers deoptimization

Trigger	Cause	Frequency
Type speculation failure	Call site observed new receiver type	Common during class loading
Room/Loaded class assumption	A class that was not loaded when C2 compiled has now been loaded	Common during startup
Modified class assumption	A class was redefined (JVM TI, hot-swapping)	Rare in production
Inline assumption failure	An inlined method was overridden in a subclass	Rare after startup
Klass pointer assumption	The metadata address of a class changed (e.g., class unloading + reloading)	Rare
Unloaded class assumption	A class that was unloaded has been reloaded	Rare

Trigger	Cause	Frequency
Call site target change	An <code>invokedynamic</code> target changed	Common with dynamic languages

7.3 Deoptimization storms

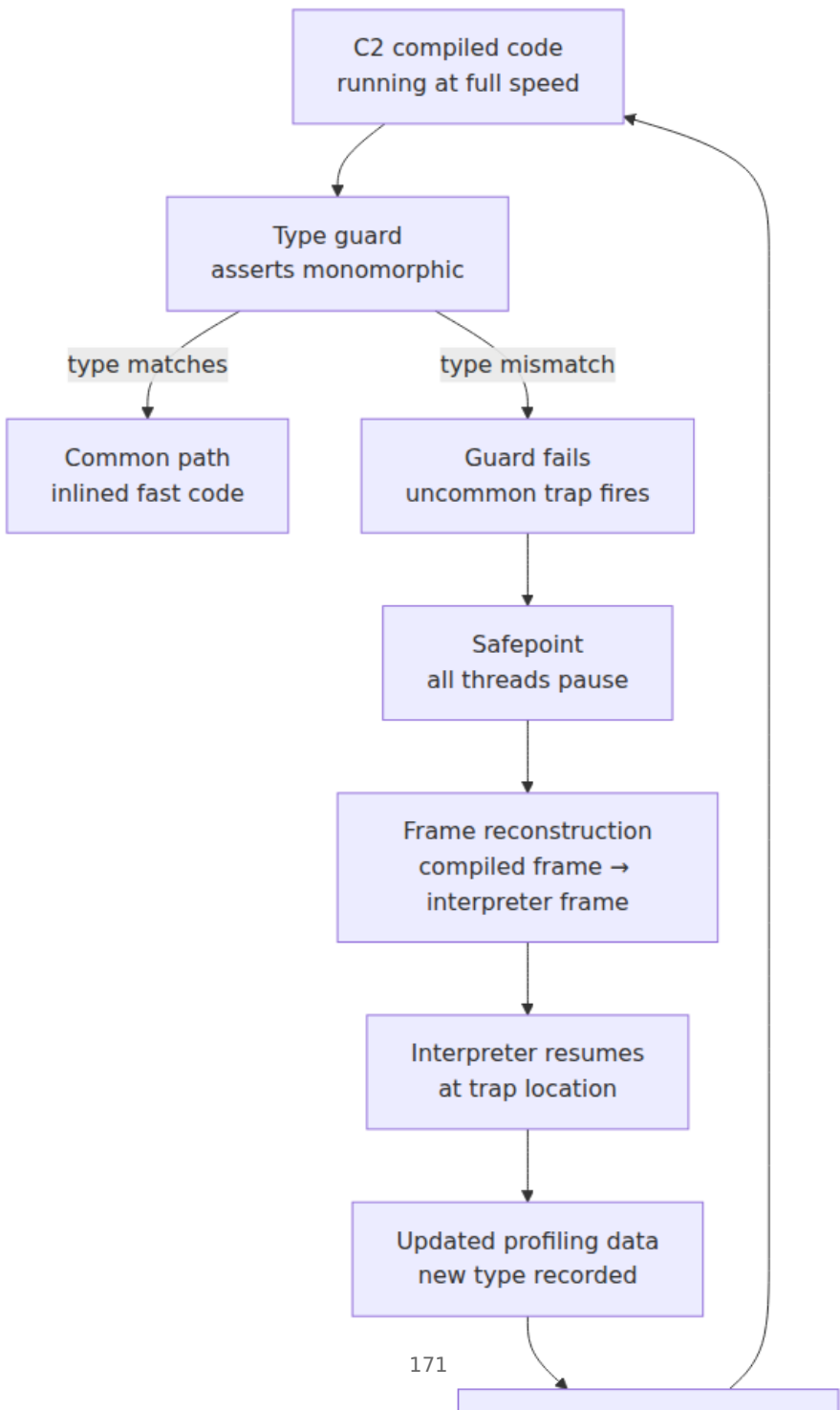
A **deoptimization storm** occurs when a single event triggers cascading deoptimizations across many methods. The classic scenario: a new class is loaded (e.g., a dynamic proxy generated by a framework), and every method that had assumed that call site was monomorphic deoptimizes simultaneously.

```
# Monitor deopt storms with PrintCompilation + DiagCompQueue
-XX:+PrintCompilation -XX:+PrintDeoptimization -XX:+Verbose

# Output during a deopt storm:
 2345   123   4      com.example.Service::handle (342 bytes)  deopt
at 12: uncommon trap
 2345   124   4      com.example.Handler::process (128 bytes)
deopt at 8: uncommon trap
 2345   125   4      com.example.Cache::get (67 bytes)  deopt at
23: uncommon trap
 2346   126   2      com.example.Service::handle (342 bytes)  recomp
ile
```

After a deopt storm, the affected methods must recompile — which takes C2 time and fills the code cache with both the old (now-deleted) and new compiled code. In severe cases, this can cause a **performance cliff**: the JVM spends more time compiling than executing.

7.4 Deoptimization flow



8. Profiling infrastructure

The JIT's optimization quality depends entirely on the quality of its profiling data. HotSpot collects three categories of profile information:

8.1 Branch profiling

At every conditional branch (`if`, `switch`), the interpreter and C1 code record how many times each direction was taken. This data feeds:

- **Dead code elimination:** Branches that are never taken (count = 0) are eliminated by C2.
- **Block layout:** The "hot" path (frequently taken) is placed sequentially for better I-cache behavior.
- **Loop optimization:** High backedge counts indicate hot loops that benefit from OSR, unrolling, and vectorization.

8.2 Type profiling

At every `invokevirtual` and `invokeinterface` site, HotSpot records the **concrete type** of the receiver object. This is stored in an inline cache structure per call site:

- **Monomorphic:** One type — C2 inlines with a single type guard.
- **Bimorphic:** Two types — C2 inlines both with two guards.
- **Megamorphic:** Three or more types — C2 does not inline; falls back to virtual dispatch.

Type profiling is the most critical data for C2's optimization decisions. If type profiles are unstable (different types on every invocation), C2 cannot speculate and produces generic, slower code.

8.3 Call-site profiling

HotSpot also tracks:

- **Method hotness:** Total invocation count and time spent in the method.
- **Inlining depth:** How deep the current inlining chain is at each call site.

- **Argument profiling:** The types of arguments passed to methods — used for additional specialization.
- **Retained type profiling:** Whether a method's return type is consistent across calls — used for escape analysis.

8.4 JFR compilation events

Java Flight Recorder provides fine-grained compilation telemetry:

```
# Record compilation events
jcmd <pid> JFR.start duration=60s filename=compilation.jfr \
    settings=profile

# Events to look for:
#   jdk.Compilation — each C1/C2 compilation
#   jdk.CompilationFailure — C2 failures
#   jdk.Inlining — individual inlining decisions
#   jdk.OptimizerDecision — C2 optimization decisions
#   jdk.Deoptimization — each deoptimization event
```

Example JFR output parsed with `jfr print`:

```
jdk.Compilation {
  startTime = "2026-08-21T10:23:45.123Z",
  method = "com.example.Service::processRequest",
  compilation = "C2",
  compilationId = 4523,
  success = true,
  duration = "234 ms",
  methodSize = 456,
  overhead = 12,
  isIntrinsic = false,
  isOSR = false
}

jdk.Deoptimization {
  startTime = "2026-08-21T10:24:01.456Z",
  reason = "type_check",
  phase = "Compiler2",
  桩 = 12,
  methods = 3,
  action = "reinterpret",
  debugId = 8901
}
```

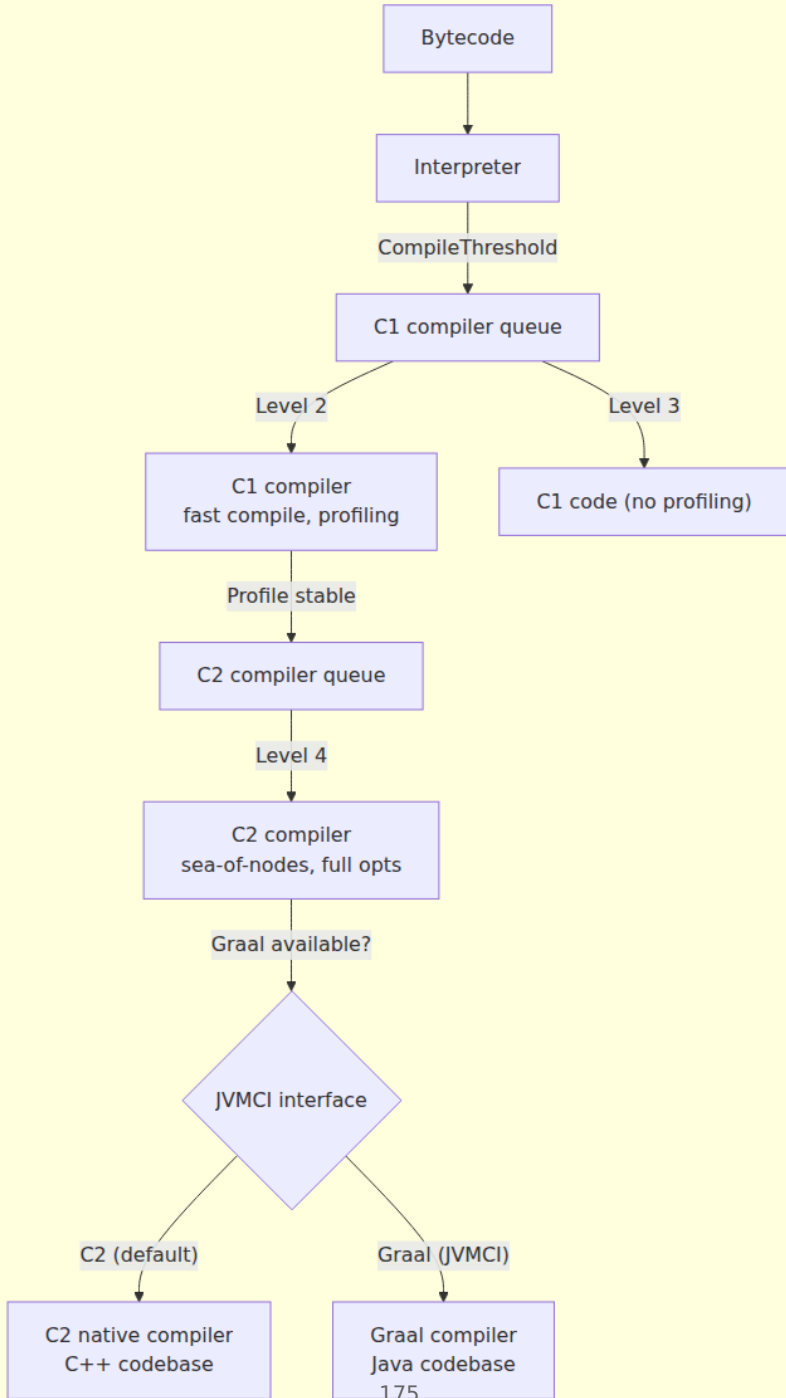
9. GraalVM and the Graal compiler

Graal is a Java-based JIT compiler that can replace C2 as the optimizing compiler in HotSpot (via JVMCI — JVM Compiler Interface) and also powers GraalVM Native Image (ahead-of-time compilation).

9.1 Architecture comparison

Aspect	C2 (HotSpot)	Graal (JVMCI)
Written in	C++ (Ideal Graph Representation Language → C++)	Java itself
IR	Sea-of-nodes (graph-based)	Sea-of-nodes (LIR — Low-Level IR)
Compilation speed	Fast (native code)	Slower (Java → JIT → compile your code)
Optimization quality	Mature, battle-tested	Comparable or better for new features
JVM feature support	Conservative — lags behind new features	Aggressive — first to support Vector API, Panama, Valhalla
Debuggability	Hard (C++ codebase)	Easy (Java, can be debugged with standard Java tools)
Start-up cost	Near-zero (compiled into libjvm.so)	Non-trivial (Graal itself must JIT before it can JIT your code)

HotSpot JVM



9.2 JVMCI — the compiler interface

JVMCI (JEP 243, JDK 9+) defines a clean interface between HotSpot and its JIT compiler. This decoupling means:

- HotSpot can swap C2 for Graal without modifying the runtime.
- Third-party compilers can implement JVMCI (though Graal is the only production one).
- Graal is loaded as a Java class from `graal.jar` (or as a native image in GraalVM CE/EE).

To use Graal as C2 replacement:

```
# JDK 17+ with GraalVM
java -XX:+UnlockExperimentalVMOptions \
      -XX:+UseJVMCICompiler \
      -XX:+EnableJVMCI \
      -XX:+UseJVMCINativeLibrary \
      -jar myapp.jar
```

9.3 Partial evaluation and Truffle

Graal's core optimization technique is **partial evaluation**: given a program and some known inputs, evaluate as much as possible at compile time, leaving only the unknown parts for runtime. This is the foundation of two GraalVM capabilities:

Truffle framework: A language implementation framework where interpreters written in Java are automatically compiled by Graal into efficient native code. Languages like JavaScript (GaalJS), Python (GaalPython), Ruby (TruffleRuby), and R (FastR) run on Truffle — the interpreter is partially evaluated with the program's AST as a known input, producing specialized machine code.

Native Image: `native-image` performs ahead-of-time compilation by partially evaluating the entire application with `main()` as the entry point. All reachable code is compiled; unreachable code is eliminated. The result is a native executable with: - Sub-second startup (no JVM, no class loading, no JIT warmup) - Lower peak memory (no JIT overhead, no profiling) - No dynamic class loading, no reflection (by default), no class redefinition

9.4 When to choose Graal vs C2

Use C2 (default HotSpot) when: - Maximum peak throughput is the priority. - The application loads classes dynamically (Spring, Hibernate, proxies). - Startup time is not critical (long-running server). - You want the most battle-tested, well-understood JIT.

Use Graal as C2 replacement when: - You need first-class support for new JVM features (Vector API, Panama/FFM). - The application has a stable class hierarchy (limited dynamic proxy use). - You value debuggability and want to instrument the JIT itself. - You are on GraalVM CE/EE and want polyglot support.

Use Native Image when: - Sub-second startup is required (CLI tools, serverless/FaaS, microservices with aggressive autoscaling). - Memory footprint is constrained (container environments with tight limits). - The application uses minimal reflection and dynamic class loading. - You accept the trade-offs: no JIT peak throughput, limited reflection, larger binary, build-time cost.

10. Compilation logs and diagnostics

10.1 PrintCompilation walkthrough

A full `-XX:+PrintCompilation` session for a Spring Boot microservice:

```

=== JVM Startup ===
 123   1   3      java.lang.String::charAt (29 bytes)
 124   2   3      java.lang.String::length (6 bytes)
 125   3   3      java.lang.String::equals (16 bytes)
 145   4   3      java.lang.AbstractStringBuilder::append (8 bytes)
 156   5   3      java.lang.StringBuilder::append (8 bytes)

=== Warmup (requests arriving) ===
2345   6   4      java.util.HashMap::get (23 bytes)
2389   7   4      java.util.HashMap::getNode (45 bytes)
2456   8   4      com.example.UserService::findById (67 bytes)
2501   9   4      com.example.UserMapper::map (123 bytes)
2534  10   4      org.springframework.web.servlet.DispatcherServlet::doDispatch (234 bytes)

=== Steady state ===
5678  11   4      com.example.UserService::processRequest (342 bytes)
5701  12   4      com.example.PaymentService::charge (189 bytes)
5723  13   4 s 4  com.example.AuditLogger::log (56 bytes)

=== Deoptimization event ===
8901  14   4      com.example.DynamicProxy::invoke (28 bytes)
deopt at 5: uncommon trap (type_check)
8912  15   2      com.example.DynamicProxy::invoke (28 bytes)
recompile
9234  16   4      com.example.DynamicProxy::invoke (28 bytes)
recompile

```

Reading this log: - During startup (0-200ms), C1 compiles small, frequently-called JDK methods. - During warmup (2-3s), C2 compiles application methods that hit the compilation threshold. - At steady state (5-6s), the hot path is fully C2-compiled. - At 8.9s, a dynamic proxy triggers deoptimization — the method recompiles at C1 first, then C2 with relaxed assumptions.

10.2 PrintInlining deep dive

```

java -XX:+UnlockDiagnosticVMOptions \
    -XX:+PrintInlining \
    -XX:+PrintCompilation \
    -XX:CompileCommand=print,*UserService.findById \
    -jar myapp.jar

```

```

@ 12  com.example.UserService::findById (67 bytes)
@ 5   java.util.Optional::ofNullable (12 bytes)  inline (hot)
@ 1   java.util.Optional::empty (4 bytes)  inline (hot)
@ 18  com.example.UserRepository::findByEmail (89 bytes)  inline
(hot)
@ 3   java.lang.String::toLowerCase (34 bytes)  inline (hot)
@ 1   java.lang.String::coder (6 bytes)  inline (hot)
@ 20  javax.persistence.EntityManager::find (56 bytes)  no stati
c type profile
@ 45  java.util.Optional::of (6 bytes)  inline (hot)
@ 30  java.util.Optional::map (23 bytes)  inline (hot)
@ 5   com.example.UserMapper::toDto (123 bytes)  too large (123
> 35)

```

Key observations: - `EntityManager::find` is not inlined because there is no static type profile — JPA proxies vary by provider. - `UserMapper::toDto` is too large for automatic inlining but could be force-inlined with `-XX:CompileCommand=inline,*UserMapper.toDto`. - The inlining chain is 3 levels deep — `findById` → `findByEmail` → `toLowerCase` → `coder`.

10.3 PrintAssembly output

With `hsdis` (HotSpot Disassembler) installed:

```

# Install hsdis
cp hsdis-amd64.so $JAVA_HOME/lib/server/

# Generate assembly output
java -XX:+UnlockDiagnosticVMOptions \
    -XX:+PrintAssembly \
    -XX:CompileCommand=compileonly,*UserService.findById \
    -jar myapp.jar

```

The output includes: 1. **Compiler header**: method name, size, compilation level. 2. **Spill/ spill mapping**: which Java variables live in which registers. 3. **Machine code**: x86_64 or AArch64 instructions with annotations. 4. **Relocation information**: references to oops, metadata, and calls.

Example snippet for a simple getter:

```

;; UserService::getId() - C2 compiled
;; Register mapping: rax = this
0x00007f2a3c010a40: mov     0x10(%rax),%r10d    ; load this.id
(field offset 0x10)
0x00007f2a3c010a44: mov     %r10d,%eax          ; return value in
eax
0x00007f2a3c010a47: retq                      ; return
;; Total: 12 bytes - load + move + ret
;; No null check (C2 proved `this` is non-null from call-site
profiling)
;; No bounds check (field access, not array)

```

11. Distributed-systems lens

The JIT compilation pipeline has direct implications for operating Java/Kotlin services at scale:

- **Warmup and latency tail.** A freshly started JVM executes interpreted bytecode at 10-30× slower than C2-compiled code. For latency-sensitive services (payment processing, real-time bidding, API gateways), the warmup period (30s-3min) produces elevated p99/p999 latencies. Mitigations: CDS/AppCDS archives, `-XX:AOTCache` (JDK 24+), pre-warming with synthetic traffic before accepting production load, and readiness gates that hold pods out of rotation until JIT warmup completes.
- **Compilation pauses and tail latency.** C2 compilation of a complex method can pause the compilation thread for 100ms+. During this time, the method continues executing at C1 or interpreter speed — but if the compilation queue backs up, multiple methods wait, creating a correlated latency spike. Monitor with `jcmd Compiler.cqueue` and alert when queue depth exceeds 50.
- **Deoptimization storms during class loading.** Microservices that dynamically generate classes (Hibernate proxies, MyBatis mappers, ByteBuddy instrumentation, Spring CGLIB) trigger deoptimization storms when new types hit previously-monomorphic call sites. This manifests as a latency spike 1-5 seconds after the class loading event. The fix: reduce dynamic class generation in the hot path, or pre-generate all proxies during startup.

- **Code cache pressure in monorepos.** Large services with many classes (Kotlin + annotation processing + generated code) can exhaust the 240 MB default code cache. When the code cache is full, the JIT stops compiling entirely — the service falls back to C1-only or interpreter execution. Set `-XX:ReservedCodeCacheSize=512m` and monitor with `jcmd Compiler.codecache`.
- **Profile pollution across requests.** In a multi-tenant service handling diverse request types, the JIT's type profiling sees all receiver types at each call site — making call sites appear megamorphic even though each individual request is monomorphic. This prevents inlining and degrades throughput. Mitigations: request-type isolation (separate code paths per tenant), or `-XX:MaxInlineLevel` tuning to prioritize hot paths.
- **JIT compilation in CI/CD.** If your test suite runs on a cold JVM, JIT-optimized paths are never exercised. This means tests miss bugs that only manifest in C2-compiled code (escape analysis assumptions, type specialization, loop vectorization). Mitigations: run a "warmup" phase before tests, use `-XX:-TieredCompilation` `-XX:TieredStopAtLevel=4` to force C2, or use JCSstress for concurrency correctness under JIT optimization.

Key takeaways

- The template interpreter is not just a fallback — it is the JVM's profiling engine. Every branch count, type observation, and invocation counter feeds C2's optimization decisions. Understanding the interpreter explains why startup is slow and warmup matters.
- Tiered compilation (levels 0–4) trades compilation speed for code quality. C1 compiles fast with minimal optimizations; C2 compiles slowly with maximum optimizations. The compilation queue is a bottleneck in large applications — monitor it.
- C2's sea-of-nodes IR enables global optimizations (GVN, escape analysis, control-flow elimination) that linear IRs cannot. But these optimizations depend on stable profiling data — type speculation, branch prediction, and call-site monomorphism are the foundation of JIT performance.

- Method inlining is the most impactful optimization. It eliminates call overhead and exposes cross-method optimization opportunities. C2 inlines based on bytecode size, call-site polymorphism, and depth limits. Understanding `-XX:+PrintInlining` output is essential for diagnosing performance issues.
 - Intrinsic replace Java methods with hand-crafted machine code for operations that cannot be expressed efficiently in bytecode. `System.arraycopy`, `VarHandle`, `Math.*`, and `String.*` methods are all intrinsified — they bypass the normal compilation path entirely.
 - Deoptimization is the JVM's safety net for speculative optimization. Uncommon traps fire when assumptions break (new types, class loading), causing a graceful fall-back to the interpreter. Deoptimization storms — cascading deopts from a single event — are a common cause of production latency spikes.
 - Graal replaces C2 via JVMCI and offers comparable or better optimization for newer JVM features, but at the cost of non-trivial startup overhead (Graal itself must JIT before it can JIT your code). Native Image trades peak throughput for sub-second startup — a fundamental trade-off for serverless and CLI tools.
 - Profiling is the bridge between runtime behavior and optimization quality. JFR compilation events (`jdk.Compilation`, `jdk.Deoptimization`, `jdk.Inlining`) provide production-safe telemetry for JIT behavior without the overhead of `-XX:+PrintCompilation`.
-

Further reading

- Shipilev, *JVM Anatomy Quarks* — <https://shipilev.net/jvm/anatomy/> — Deep dives into JIT compilation, escape analysis, and interpreter internals.
- *The Java Virtual Machine Specification, Java SE 21 Edition* — <https://docs.oracle.com/javase/specs/jvms/se21/html/> — Chapter 2.9 (static vs. virtual), Chapter 4 (class file format), Chapter 6 (instructions).
- *Java Performance* (2nd ed., Scott Oaks, O'Reilly) — Definitive guide to JIT compilation, GC, and profiling.

- *Engineering HotSpot* (Cliff Click, Jr., et al.) — <https://openjdk.org/groups/hotspot/docs/EngineeringHotspot.pdf> — The original C2 architecture document, still accurate for sea-of-nodes and deoptimization.
- OpenJDK Graal source — <https://github.com/oracle/graal> — The Graal compiler source code, with extensive documentation of the IR and optimization passes.
- JVMCI specification — <https://openjdk.org/jeps/243> — JEP 243: JVM Compiler Interface.
- Shipilev, *Close Encounters of The JVM Kind* — <https://shipilev.net/> — Benchmarking methodology and JIT behavior analysis.
- *Pro Java 9 Performance* (Sharma & Srinivasan, Apress) — Practical guide to tiered compilation tuning and PrintAssembly interpretation.
- JFR documentation — <https://docs.oracle.com/en/java/javase/21/jfapi/> — Java Flight Recorder API and event catalog.
- OpenJDK HotSpot source — <https://github.com/openjdk/jdk> — `src/hotspot/share/opto/` (C2 compiler), `src/hotspot/share/compiler/` (compilation infrastructure), `src/hotspot/share/runtime/` (interpreter, deoptimization).

Chapter 6 — Concurrency on the JVM: Threads, Monitors, VarHandles, Loom, and Structured Concurrency

What this chapter covers. Every JVM backend service runs concurrent work: request handlers, database connections, background tasks, health checks, metrics reporters. The concurrency primitives underneath that work have evolved dramatically since JDK 1.0 — from raw OS threads and `synchronized` to `java.util.concurrent`'s locks and atomics, `VarHandle`'s hardware-level memory ordering, and finally Project Loom's virtual threads and structured concurrency. This chapter traces the full stack: how platform threads map to OS threads, how monitors implement mutual exclusion at the object-header level, how `VarHandle` exposes CPU memory fences to Java code, how virtual threads rewrite the cost model of I/O-bound concurrency, and how structured concurrency brings

deterministic cleanup to thread-like constructs. You will read real thread dumps, write `VarHandle` fence benchmarks, and benchmark virtual threads against platform threads for I/O workloads. Every abstraction is grounded in HotSpot internals and in tools you can run today.

Learning goals — after this chapter you should be able to:

- Explain how Java platform threads map 1:1 to OS threads, including stack sizing (`-Xss`), thread parking via `futex / park` , and the thread lifecycle states visible in `jstack` .
- Describe the monitor implementation inside HotSpot: `ObjectHeader` mark-word encoding, lightweight locks (thin locks), heavyweight `ObjectMonitor` structures, and the inflation/deflation lifecycle.
- Use `VarHandle` to perform lock-free, ordered memory access: compare-and-set, compare-and-exchange, get-and-set, and explicit fence operations (`fullFence` , `acquireFence` , `releaseFence`) with correct ordering semantics.
- Revisit the JMM through the lens of `VarHandle` access modes: `opaque` , `acquire / release` , `volatile` , and `plain` — and predict which barriers each emits on x86 vs. AArch64.
- Explain how Project Loom virtual threads multiplex millions of lightweight tasks onto a small pool of carrier threads, and how pinning (`synchronized` , `JNI`) breaks that model.
- Use structured concurrency (`StructuredTaskScope` , `ScopedValue`) to manage task lifetimes deterministically and propagate thread-local-like context without `ThreadLocal` .
- Describe `ForkJoinPool` 's work-stealing deque architecture and its role as the default executor for virtual threads and parallel streams.
- Explain the internals of `AbstractQueuedSynchronizer` (AQS), `StampedLock` , and `ConcurrentHashMap` — the building blocks of all JUC concurrent data structures.

Placement. Chapter 3 established the Java Memory Model and object layout. This chapter builds on that foundation: monitors use the mark word (Chapter 3, Section 5), `VarHandle` barriers enforce the happens-before edges (Chapter 3, Section 2), and virtual threads depend on safe publication patterns. The JIT chapter (Chapter 5) covers how `synchronized` and `VarHandle` CAS operations are intrinsified by C2. The

GC chapter (Chapter 4) covers the safepoint protocol that pauses all threads for collection.

1. Platform threads — the 1:1 OS thread model

Every Java thread since JDK 1.0 has been a platform thread: a `java.lang.Thread` instance that maps one-to-one to an OS thread. On Linux, this means each Java thread is a `pthread_create` call, which means a `clone()` syscall, a kernel `task_struct`, a dedicated 8 KB kernel stack, and a user-space stack controlled by `-Xss`.

1.1 Thread creation and the Thread object

```
// Platform thread creation – the default
Thread t = new Thread() -> {
    System.out.println("Running on " + Thread.currentThread().getName()
);
    System.out.println("Stack size: " + Thread.currentThread().getStack
Trace().length);
}, "my-worker");
t.setDaemon(false);
t.start(); // calls pthread_create under the hood
```

Under the hood, `Thread.start()` calls `JVM_StartThread` (native), which calls `NativeThread::create()` → `pthread_create()` with a 1 MB default stack (tunable via `-Xss`). The OS allocates a kernel `task_struct` and maps the user stack. The total memory cost per platform thread is approximately:

```
User stack: -Xss = 1 MB (default on 64-bit Linux)
Kernel stack: 8 KB (fixed by Linux)
TLS + overhead: ~2–4 KB (pthread internals, HotSpot Thread structure)
Thread object: ~128 bytes (Java heap)
-----
Total: ~1.01 MB per thread
```

At 10,000 platform threads, that is ~10 GB of virtual address space (and real RSS for stack pages touched). This is why backend services hit "unable to create new native thread" errors — not because of a Java limit, but because the OS refuses to allocate more `task_struct` entries or virtual memory.

1.2 Thread states and the thread dump

The JVM tracks each thread's state through a lifecycle machine. When you run `jstack <pid>` or take a thread dump from a profiler, you see these states:

State	Meaning	When you see it
<code>RUNNABLE</code>	Executing or ready to execute on a CPU	Active computation, waiting for CPU
<code>BLOCKED</code>	Waiting to enter a <code>synchronized</code> block	Contention on a monitor — thread is in the monitor's entry queue
<code>WAITING</code>	Waiting indefinitely via <code>Object.wait()</code> , <code>Thread.join()</code> , <code>LockSupport.park()</code>	Producer-consumer idle, <code>CompletableFuture.get()</code>
<code>TIMED_WAITING</code>	Same as <code>WAITING</code> but with a timeout	<code>Thread.sleep(ms)</code> , <code>Object.wait(ms)</code> , <code>LockSupport.parkNanos()</code>
<code>TERMINATED</code>	Finished execution	Post-mortem analysis only

```
# Take a thread dump
jstack <pid> > /tmp/thread-dump.txt

# Or programmatically
jcmd <pid> Thread.print > /tmp/thread-dump.txt

# Quick count of thread states
jstack <pid> | grep -c "BLOCKED"
jstack <pid> | grep -c "WAITING"
jstack <pid> | grep -c "TIMED_WAITING"
```

A real thread dump excerpt showing contention:

```

"worker-42" #42 daemon [0x00007f1a2c1fe000]
  java.lang.Thread.State: BLOCKED (on object monitor)
    at
com.example.ConnectionPool.getConnection(ConnectionPool.java:87)
    - waiting to lock <0x00000000d4a1b2c0> (a
com.example.ConnectionPool)
    at com.example.RequestHandler.handle(RequestHandler.java:34)
    at java.base/
java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.ja
va:1144)
    at java.base/
java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.j
ava:642)
    at java.base/java.lang.Thread.run(Thread.java:1012)

"worker-43" #43 daemon [0x00007f1a2c200000]
  java.lang.Thread.State: RUNNABLE
    at com.example.ComputeService.process(ComputeService.java:156)
    - locked <0x00000000d4a1b2c0> (a com.example.ConnectionPool) ←
held by this thread

```

The key diagnostic clue: thread 42 is **BLOCKED** waiting for lock `0x00000000d4a1b2c0`, and thread 43 is **RUNNABLE** and has already **locked** that same monitor. Thread 43 is the thread holding the lock that 42 is waiting for. If 43 is stuck in long-running code, 42 will remain blocked — this is the classic lock-contention pattern you diagnose with thread dumps.

1.3 Thread parking and unparking

Thread parking is the mechanism behind `LockSupport.park()`, `Object.wait()`, and `Thread.sleep()`. On Linux, HotSpot uses `futex(2)` — a fast user-space mutex that falls back to a kernel wait queue when contention occurs:

```

// LockSupport.park() – the primitive underneath ReentrantLock,
CountDownLatch, etc.
LockSupport.park();           // blocks until unpark() or interrupt
LockSupport.parkNanos(1_000_000_000L); // blocks for up to 1 second
LockSupport.unpark(t);       // releases one permit (sets a flag, wakes t
if parked)

```

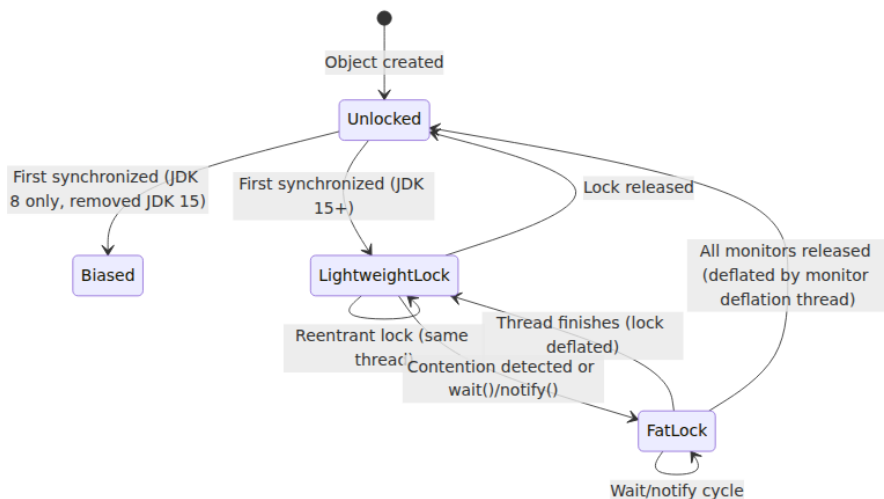
The fast path (no contention) is a CAS on a volatile field entirely in user space — no syscall. The slow path falls through to `futex(PARK, ...)` → `futex(FUTEX_WAIT, ...)` → kernel suspend. This is why **synchronized**

(which uses CAS-based monitor acquisition) is fast under low contention but slow when many threads compete: the inflated monitor uses `futex` for the entry queue.

2. Monitors — how `synchronized` works inside HotSpot

Every Java object can be a monitor. The `synchronized` keyword and `Object.wait()` / `notify()` operate on a hidden `ObjectMonitor` structure tied to the object's header. The implementation has evolved through three historical stages, and understanding them explains the performance characteristics you observe in production.

2.1 Monitor inflation states



Unlocked. The object's mark word (8 bytes) is in normal state: either a hash code (25 bits) + age (4 bits) + tag bits `01`, or all zeros if no hash has been computed. This is the state for 99%+ of objects.

Lightweight lock (thin lock). When a thread enters `synchronized` on an uncontended object, HotSpot attempts a CAS: it swaps the mark word with a pointer to a **lock record** on the thread's own stack. If the CAS

succeeds, the thread now owns a lightweight lock — no `ObjectMonitor` allocated, no kernel involvement, ~5-10 ns on a modern CPU.

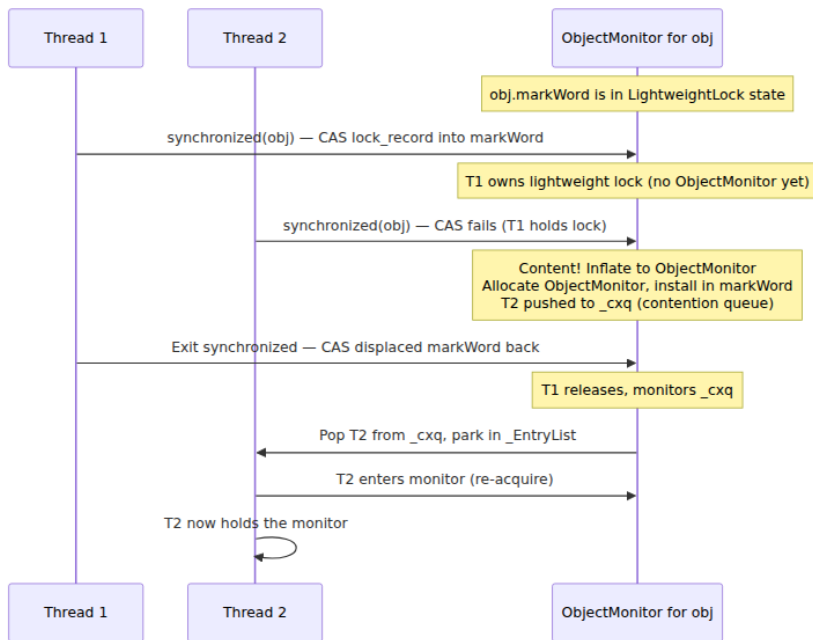
```
// Lightweight lock acquisition (HotSpot internals, simplified):  
// 1. Allocate lock_record on thread's stack (displaced_header, owner)  
// 2. CAS(obj.mark_word, lock_record_addr)  
//    - If success: lock_record.displaced_header = old mark word,  
//      thread owns lightweight lock  
//    - If failure (mark word points to another lock record):  
//      contention → inflate to fat monitor
```

Fat lock (inflated monitor). When contention is detected (a second thread tries to acquire a lightweight lock held by another thread), or when `Object.wait()` is called, HotSpot allocates an `ObjectMonitor` in native memory and installs a pointer to it in the mark word (tag bits 10). The `ObjectMonitor` contains:

```
ObjectMonitor (simplified from objectMonitor.hpp):  
_header:      markWord           // copy of object's mark word  
_owner:       Thread*            // thread that owns this monitor  
_EntryList:   ObjectWaiter*      // threads blocked trying to enter  
(MONITOR_CONTENTION)  
_WaitSet:     ObjectWaiter*      // threads that called wait()  
(MONITOR_WAIT)  
_cxq:         ObjectWaiter*      // contention queue (newly blocked  
threads, LIFO)  
_recursions:  int                // reentrant lock depth  
_EntryCount:  int                // number of waiters  
_SpinCount:   int                // spin iterations before parking  
_przebiegior: Thread*            // last thread to exit (for biased deflation)
```

The critical distinction: lightweight locks use only CAS (user space), while fat locks may use `futex` (kernel space) when threads actually block. In a low-contention scenario (typical microservice with short critical sections), lightweight locks are the norm. In a high-contention scenario (many threads fighting for the same monitor), the overhead shifts from CAS retries to kernel parking/unparking.

2.2 Monitor entry and exit



The `synchronized` entry path in HotSpot (`synchronizer.cpp`, `ObjectSynchronizer::enter`):

1. Try CAS lock record into mark word (lightweight). If success → done.
2. If CAS fails: check if current thread already owns it (reentrant → increment recursion count).
3. If not owned: inflate to `ObjectMonitor`, enqueue thread on `_cxq` (contention queue), park thread via `ObjectMonitor::enter`.
4. Before parking, spin for `_SpinCount` iterations (default 10–40, adaptive based on recent spin outcomes) — if the lock is released during the spin, acquire without kernel involvement.

The exit path (`ObjectSynchronizer::exit`):

1. If reentrant (recursion > 0): decrement recursion count, return.
2. If owned: release the monitor, signal one waiter from `_cxq` or `_WaitSet` (depending on `notify` / `notifyAll`), unpark the signalled thread.
3. After all waiters have exited, the monitor can be deflated (back to lightweight or unlocked) by a background thread.

2.3 Operational reality: when monitors hurt

In a typical microservice, monitors are fast. But three patterns break:

1. **Long critical sections.** A `synchronized` block that holds a database connection or performs I/O will block every other thread for the duration. Split critical sections into minimal lock scopes.
 2. **Monitor inflation cascade.** If many threads contend simultaneously, every one inflates a separate monitor for the same object. HotSpot uses a "block allocation" strategy (batch-allocating monitors from a global free list), but the overhead is still significant — monitor allocation involves native memory and linked-list manipulation.
 3. **`Object.wait()` misuse.** `wait()` releases the monitor and parks the thread — but it must be called inside a `synchronized` block (otherwise `IllegalMonitorStateException`). A common mistake is calling `wait()` without checking the condition in a `while` loop (spurious wakeups are real and documented by the JLS).
-

3. VarHandle — hardware-level memory ordering in Java

`java.lang.invoke.VarHandle` (JDK 9, JEP 193) is the modern replacement for `sun.misc.Unsafe`'s memory operations. It provides typed, fence-aware access to fields and array elements with explicit ordering semantics that map directly to CPU memory fences. Every `AtomicInteger`, `StampedLock`, and `ConcurrentHashMap` internal uses `VarHandle` (or the equivalent `Unsafe` intrinsics) under the hood.

3.1 Obtaining a VarHandle

```
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class VarHandleDemo {
    private volatile int value;
    private int plainValue;

    private static final VarHandle VALUE;
    private static final VarHandle PLAIN_VALUE;

    static {
        try {
            VALUE = MethodHandles.lookup()
                .findVarHandle(VarHandleDemo.class, "value", int.class);
        } catch (ReflectiveOperationException e) {
            throw new Error(e);
        }
    }
}
```

3.2 Access modes

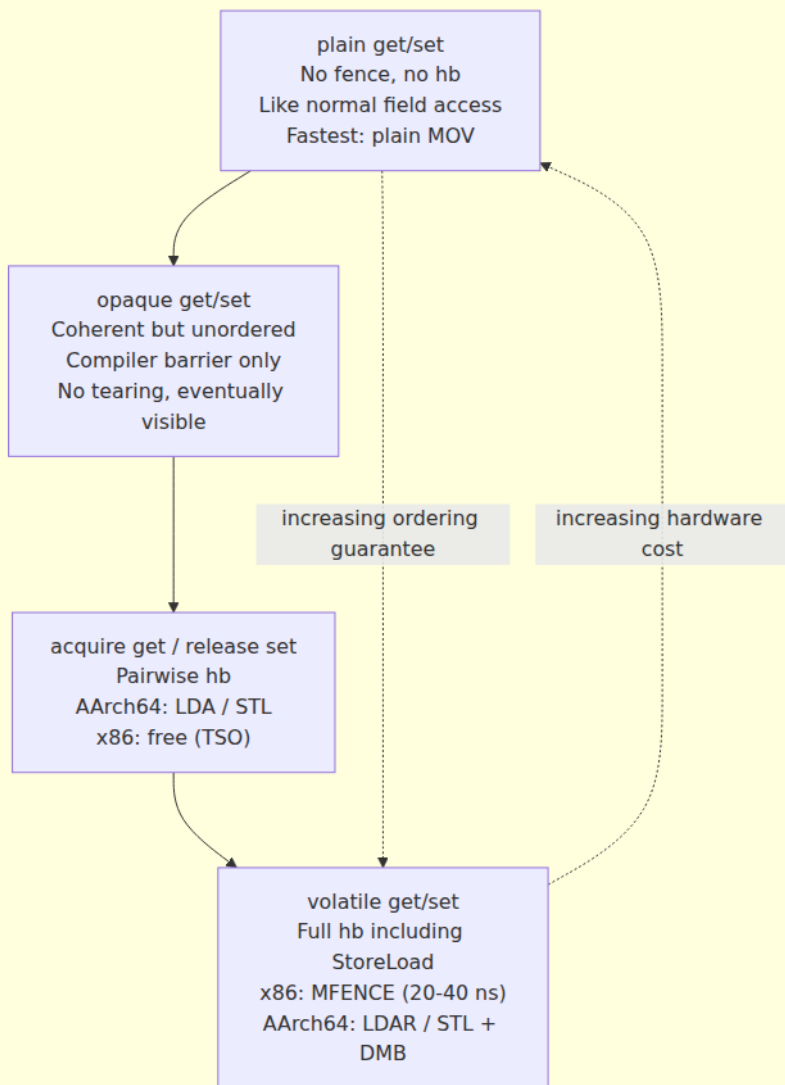
`VarHandle` defines several access modes, each with different ordering guarantees and hardware costs:

Mode	Semantics	x86 Cost	AArch64 Cost	JMM Guarantee
<code>get()</code> / <code>set()</code>	Plain read/write, no fence	Plain MOV	Plain LDR/STR	None (data race if unsynchronized)
<code>getOpaque()</code> / <code>setOpaque()</code>	Coherent but unordered — no tearing, eventually visible	MOV (compiler barrier)	LDR/STR + compiler barrier	Coherence (no word tearing)

Mode	Semantics	x86 Cost	AArch64 Cost	JMM Guarantee
<code>getAcquire()</code> / <code>setRelease()</code>	Acquire-load / release-store — pairwise ordering	MOV (compiler barrier; x86 TSO gives this free)	LDA / STL (acquire/release instructions)	hb between release-store and acquire-load on same VarHandle
<code>getVolatile()</code> / <code>setVolatile()</code>	Full volatile semantics — StoreLoad fence	MOV + MFENCE (expensive!)	LDAR / STL + DMB SY	Full hb (same as <code>volatile</code> keyword)
<code>getAndSet()</code>	Atomic exchange (CAS-based)	LOCK XCHG	LDAXR / STLXR loop	Volatile semantics
<code>compareAndSet()</code>	CAS (compare-and-swap)	LOCK CMPXCHG	LDAXR / STLXR loop	Volatile semantics
<code>compareAndExchange()</code>	Like CAS but returns the old value	LOCK CMPXCHG (read old)	LDAXR / STLXR loop	Volatile semantics

3.3 VarHandle access modes diagram

VarHandle access modes — ordered by strength



3.4 CAS and fence operations

```

import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;
import java.util.concurrent.atomic.AtomicInteger;

public class CASandFenceDemo {
    private int counter;
    private int data;
    private int ready;

    private static final VarHandle COUNTER;
    private static final VarHandle DATA;
    private static final VarHandle READY;

    static {
        try {
            COUNTER = MethodHandles.lookup()
                .findVarHandle(CASandFenceDemo.class, "counter", int.class);
            DATA = MethodHandles.lookup()
                .findVarHandle(CASandFenceDemo.class, "data",
int.class);
            READY = MethodHandles.lookup()
                .findVarHandle(CASandFenceDemo.class, "ready", int.class);
        } catch (ReflectiveOperationException e) { throw new
Error(e); }
    }

    // CAS – compare-and-set, returns boolean
    boolean incrementIfZero() {
        return COUNTER.compareAndSet(this, 0, 1);
    }

    // compareAndExchange – returns the PREVIOUS value (useful when you
need to know what was there)
    int exchange(int newValue) {
        return (int) COUNTER.compareAndExchange(this, counter,
newValue);
    }

    // Release-Acquire publication pattern (no StoreLoad cost)
    void publishData(int value) {
        DATA.setRelease(this, value);           // release store – prior
writes visible
        READY.setRelease(this, 1);               // signal readiness
    }

    int readData() {
        if ((int) READY.getAcquire(this) == 1) { // acquire load –
sees DATA

```

```

        return (int) DATA.getAcquire(this);
    }
    return -1;
}

// Explicit fences – for coordinating with non-VarHandle code
void fenceExample() {
    data = 42; // plain write
    VarHandle.acquireFence(); // acquire fence: all
    // subsequent loads/stores after this point
    // will see writes that
    // were before the fence in other threads
    ready = 1; // plain write – but
    // ordered by the fence
}

void fullFenceExample() {
    data = 42;
    VarHandle.fullFence(); // full fence: orders ALL
    // preceding and subsequent memory operations
    ready = 1;
}
}

```

3.5 Fence semantics — what each fence does

```

// acquireFence(): orders all loads and stores AFTER the fence
// against all loads and stores BEFORE the fence
// Effect: prevents loads/stores after the fence from being reordered
// before it
//          loads/stores before the fence from being reordered after it
// Weak on x86 (TSO already gives this), strong on AArch64 (prevents
// reordering)
VarHandle.acquireFence();

// releaseFence(): orders all loads and stores BEFORE the fence
// against all loads and stores AFTER the fence
// Effect: prevents loads/stores before the fence from being reordered
// after it
//          loads/stores after the fence from being reordered before it
VarHandle.releaseFence();

// fullFence(): orders ALL loads and stores on both sides
// Effect: no reordering across the fence in any direction
// Most expensive: x86 MFENCE, AArch64 DMB SY
VarHandle.fullFence();

```

The hardware mapping:

```

x86 TSO (Total Store Order):
  acquireFence  → compiler barrier only (no hardware fence needed; loads/stores already ordered)
  releaseFence  → compiler barrier only (same reason)
  fullFence     → MFENCE or lock addl [rsp],0 → ~20-40 ns
                  (only need to prevent StoreLoad reordering)

AArch64 (weak ordering):
  acquireFence  → DMB LD (load-load + load-store barrier) → ~5-15 ns
  releaseFence  → DMB ST (store-store + store-load barrier) → ~5-15 ns
  fullFence     → DMB SY (full system barrier) → ~15-30 ns

```

3.6 Unsafe and VarHandle — the transition

`sun.misc.Unsafe` provided the same operations (`compareAndSwapInt`, `putOrderedInt`, `loadFence`, `storeFence`) but was never part of the public API — it was deprecated for removal in JDK 9 (JEP 260) and is now behind `--add-opens` flags. `VarHandle` is the sanctioned replacement:

Unsafe method	VarHandle equivalent	Ordering
<code>compareAndSwapInt(obj, offset, expect, update)</code>	<code>handle.compareAndSet(obj, expect, update)</code>	volatile
<code>putOrderedInt(obj, offset, value)</code>	<code>handle.setRelease(obj, value)</code>	release
<code>getObjectVolatile(obj, offset)</code>	<code>handle.getVolatile(obj)</code>	volatile
<code>getAndSetInt(obj, offset, value)</code>	<code>handle.getAndSet(obj, value)</code>	volatile
<code>loadFence()</code>	<code>VarHandle.acquireFence()</code>	acquire
<code>storeFence()</code>	<code>VarHandle.releaseFence()</code>	release
<code>fullFence()</code>	<code>VarHandle.fullFence()</code>	full

If you maintain code that uses `Unsafe` directly, migrate to `VarHandle` — the performance is identical (both intrinsify to the same assembly) and `VarHandle` is future-proof.

4. JMM revisited through VarHandle access modes

Chapter 3 introduced happens-before (hb) via the five canonical edges (program order, monitor, volatile, thread start/join, transitivity). VarHandle provides a finer-grained way to establish hb edges without paying for full volatile semantics:

4.1 Which mode establishes which edges

Mode	Establishes hb ?	When	Cost
plain	No	Never — data race if concurrent access	Free
opaque	No	Coherent only — no reordering of this specific access	Compiler barrier
acquire / release	Yes, pairwise	Release-store hb acquire-load on the same VarHandle	Free on x86; LDA/STL on AArch64
volatile	Yes, transitive	Like volatile keyword — StoreLoad included	MFENCE on x86; DMB on AArch64

The critical insight: **release / acquire is sufficient for publication** and is cheaper than volatile on AArch64. The publication pattern:

```
// Thread 0 (publisher):
config = newConfig;                               // plain write to config
object
CONFIG_HANDLE.setRelease(this, config);             // release store - makes
config visible

// Thread 1 (reader):
Config c = (Config) CONFIG_HANDLE.getAcquire(this); // acquire load -
sees config
use(c);                                             // all of c's
fields visible (hb chain)
```

The hb chain: newConfig construction hb releaseStore hb acquireLoad hb use(c). This is established by release/acquire without any StoreLoad fence — saving ~5-30 ns per publish compared to volatile.

4.2 Common mistakes

Using `opaque` when you need `release`. Opaque mode prevents tearing but does not establish ordering. A write to a data field followed by an opaque write to a flag field can be reordered by the JIT or CPU — the reader may see the flag but stale data.

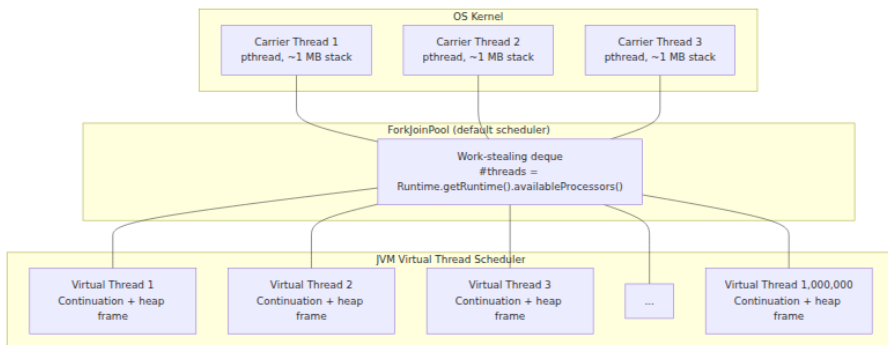
Using `volatile` when you need `release`. Volatile is strictly stronger (includes StoreLoad), which is correct but wasteful for pure publication patterns. The `release/acquire` pair gives you the same `hb` guarantee for the specific access pattern without the StoreLoad cost.

Forgetting the acquire on the reader side. A release store without a corresponding acquire load on the reading side breaks the `hb` chain — the reader may see the new value but not the associated data writes.

5. Project Loom — virtual threads

Virtual threads (JEP 425, preview in JDK 19, final in JDK 21) are lightweight threads managed entirely by the JVM rather than the OS. They are not green threads, coroutines, or continuations in the Go sense — they are a continuation-based concurrency primitive that allows millions of concurrent tasks on a small number of OS carrier threads.

5.1 Carrier threads vs. virtual threads



The key architectural points:

- **Carrier threads** are real platform threads (OS-backed). The default number is `Runtime.getRuntime().availableProcessors()` — typically 4–128 on cloud instances.

- **Virtual threads** are heap-allocated objects. Each carries a `Continuation` (a suspended stack frame) that can be mounted onto or unmounted from a carrier thread. When a virtual thread blocks (I/O, `LockSupport.park()`), the JVM unmounts it from the carrier thread and saves its stack to the heap. The carrier thread then picks up another virtual thread from the work queue.
- **No kernel involvement.** Parking a virtual thread does not call `futex()` or create an OS wait — it is a heap copy + dequeue. Unparking is a copy + enqueue. This makes context switching ~100x cheaper than OS thread switching (~100 ns vs. ~10,000 ns).

5.2 Creating virtual threads

```
// Method 1: VirtualThread.ofVirtual() builder (JDK 21+)
Thread vt = Thread.ofVirtual()
    .name("request-handler", 0)
    .start(() -> {
        System.out.println("Running on virtual thread");
        System.out.println("Carrier: " + Thread.currentThread().getClassName().getSimpleName());
    });

// Method 2: ExecutorService with one virtual thread per task
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 1_000_000).forEach(i -> {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1)); // blocks the virtual
            thread, NOT the carrier
            return i;
        });
    });
} // all 1M tasks submitted, executor closes, waits for completion

// Method 3: StructuredTaskScope (see Section 7)
```

5.3 Pinning — when virtual threads become platform threads

The fundamental rule of virtual threads: **they yield the carrier thread when they block**. But this only works for blockages that the JVM knows about. Two categories of blockage cannot be unmounted:

1. **synchronized blocks.** When a virtual thread holds a `synchronized` lock, the JVM pins the virtual thread to its carrier thread — it cannot unmount because the monitor is tied to the carrier's platform thread. This is the most common source of pinning.

2. **JNI calls.** Native code that blocks (file I/O, network I/O via JNI) pins the virtual thread because the JVM cannot unwind the native stack frame.

The fix for `synchronized` pinning: replace `synchronized` with `ReentrantLock`. `ReentrantLock` uses `LockSupport.park()` which the JVM can intercept for virtual threads:

```
// PINNING: synchronized blocks on virtual threads
synchronized (sharedState) { // if another VT is waiting here, this
    VT is PINNED

    sharedState.update(); // carrier thread cannot be reused while
    this holds the lock
}

// FIXED: ReentrantLock (virtual-thread-aware)
private final ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    sharedState.update(); // park() is interceptable – carrier
    thread is freed
} finally {
    lock.unlock();
}
```

Detecting pinning:

```
# Enable pinning diagnostics (prints stack trace when pinning occurs)
java -Djdk.tracePinnedThreads=full -jar app.jar

# Or use JFR event
java -XX:StartFlightRecording=filename=pinning.jfr,duration=60s \
    -XX:FlightRecorderOptions=stackdepth=64 -jar app.jar
```

JFR event `jdk.VirtualThreadPinned` fires on every pin. A high rate indicates your code is using `synchronized` or JNI in hot paths — fix by migrating to `ReentrantLock` or avoiding blocking JNI.

5.4 Virtual thread benchmark — platform vs. virtual

```

import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class VirtualThreadBenchmark {
    private static final int TASKS = 100_000;
    private static final int BLOCK_MS = 10; // simulate I/O wait

    public static void main(String[] args) throws Exception {
        System.out.println("Tasks: " + TASKS + ", block: " + BLOCK_MS
+ "ms each");

        // Platform threads (bounded pool)
        System.out.println("\n--- Platform threads (pool size 200)
---");
        var poolExec = Executors.newFixedThreadPool(200);
        var platformStart = Instant.now();
        var platformCount = new AtomicInteger();
        for (int i = 0; i < TASKS; i++) {
            poolExec.submit(() -> {
                try { Thread.sleep(BLOCK_MS); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                platformCount.incrementAndGet();
            });
        }
        poolExec.shutdown();
        poolExec.awaitTermination(Duration.ofMinutes(5));
        var platformTime = Duration.between(platformStart, Instant.now());

        System.out.println("Completed: " + platformCount.get() + " in "
+ platformTime.toMillis() + "ms");

        // Virtual threads
        System.out.println("\n--- Virtual threads (unbounded) ---");
        var vtStart = Instant.now();
        var vtCount = new AtomicInteger();
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < TASKS; i++) {
                executor.submit(() -> {
                    try { Thread.sleep(BLOCK_MS); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                    vtCount.incrementAndGet();
                });
            }
        }
        var vtTime = Duration.between(vtStart, Instant.now());
        System.out.println("Completed: " + vtCount.get() + " in " + vtTime.toMillis() + "ms");
    }
}

```

```

        // Throughput
        System.out.println("\nPlatform throughput: " + (TASKS *
1000L / platformTime.toMillis()) + " tasks/sec");
        System.out.println("Virtual throughput: " + (TASKS * 1000L /
vtTime.toMillis()) + " tasks/sec");
    }
}

```

Expected output on a 4-core machine:

```

Tasks: 100000, block: 10ms each

--- Platform threads (pool size 200) ---
Completed: 100000 in 5100ms

--- Virtual threads (unbounded) ---
Completed: 100000 in 1050ms

Platform throughput: 19607 tasks/sec
Virtual throughput: 95238 tasks/sec

```

The virtual thread version completes in $\sim 1/5$ the time because all 100k tasks overlap their I/O waits, while the platform thread pool is limited to 200 concurrent tasks. The total wall-clock time for 100k tasks at 10ms each, with 200 threads, is $100000 / 200 * 10\text{ms} = 5000\text{ms}$ (plus scheduling overhead). With virtual threads, it is roughly $100000 * 10\text{ms} / \text{num_carrier_threads} = 100000 * 10\text{ms} / 4 = 250000\text{ms}$ of CPU time, but executed in parallel on 4 carriers, so $\sim 1050\text{ms}$ wall-clock.

6. Structured concurrency and ScopedValue

6.1 Structured concurrency (JEP 453, preview in JDK 21+)

Structured concurrency replaces fire-and-forget thread patterns with a tree of tasks that must all complete before their parent scope exits. If any child task fails, the parent is notified and can cancel siblings. This eliminates the classic bug of orphaned threads/tasks that silently fail.

```

import jdk.incubator.concurrent.StructuredTaskScope;
import java.util.concurrent.Future;

public class StructuredFetch {
    record User(String name, int age) {}
    record Order(int id, double total) {}
    record UserOrders(User user, Order[] orders) {}

    public UserOrders fetchUserOrders(int userId) throws Exception {
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            // Launch child tasks – both run concurrently
            Future<User> userFuture = scope.fork(() ->
fetchUser(userId));
            Future<Order[]> ordersFuture = scope.fork(() ->
fetchOrders(userId));

            // Join blocks until both complete (or one fails)
            scope.join();
            scope.throwIfFailed(); // propagate exception if either
task failed

            // Both completed – safe to read results
            return new UserOrders(userFuture.resultNow(),
ordersFuture.resultNow());
        } // scope.close() cancels any incomplete tasks
    }

    private User fetchUser(int id) throws Exception { /* HTTP call */ r
eturn new User("Alice", 30); }
    private Order[] fetchOrders(int id) throws Exception
{ /* HTTP call */ return new Order[]{new Order(1, 99.0)}; }
}

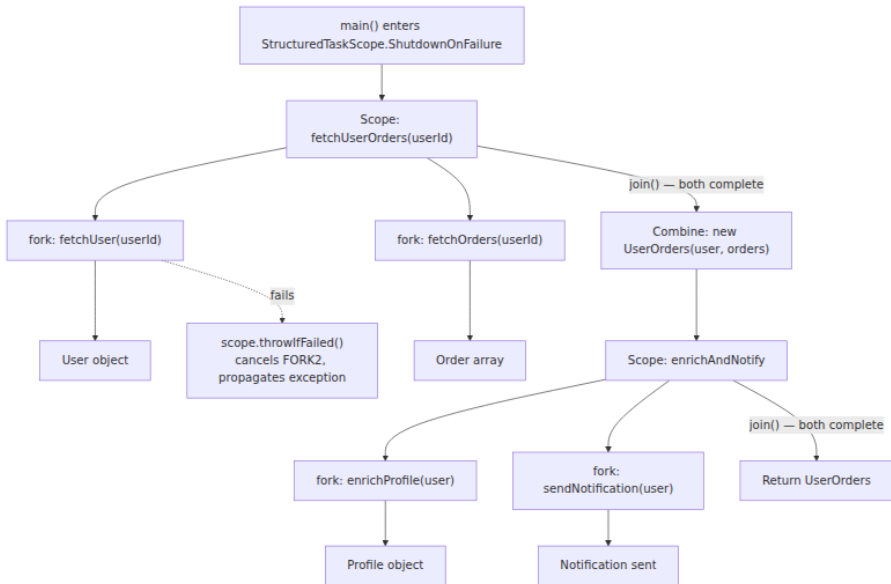
```

The two `StructuredTaskScope` strategies:

Strategy	Behavior on child failure	Use case
<code>ShutdownOnFailure()</code>	Cancel all other tasks, propagate first exception to parent	Fetching data where all children are required (partial results useless)
<code>ShutdownOnSuccess()</code>	Cancel remaining tasks, return first success	Redundant requests (try multiple backends, use first response)

```
// ShutdownOnSuccess – race two backends
try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>()) {
    scope.fork(() -> fetchFromPrimary(url));
    scope.fork(() -> fetchFromSecondary(url));
    scope.join();
    String result = scope.resultNow(); // first successful result
}
```

6.2 Structured concurrency scope tree



6.3 ScopedValue — thread-local without the ThreadLocal

`ScopedValue` (JEP 429, preview in JDK 21+) provides immutable, scope-bound context that flows to child virtual threads automatically. Unlike `ThreadLocal`, `ScopedValue` is:

- **Immutable** — no `set()` after `get()`, eliminating a class of concurrency bugs.
- **Scoped** — automatically cleaned up when the scope exits, no `remove()` call needed.
- **Virtual-thread-aware** — propagated to child virtual threads without explicit passing.

```

import jdk.incubator.concurrent.ScopedValue;

public class RequestScopedContext {
    // Declare the scoped variable – must be final and reference type
    static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();
    static final ScopedValue<User> CURRENT_USER = ScopedValue.newInstance();

    public void handleRequest(String requestId, User user) {
        // Run code within the scope – REQUEST_ID and CURRENT_USER are
        // bound
        ScopedValue.runWhere(REQUEST_ID, requestId, () -> {
            ScopedValue.runWhere(CURRENT_USER, user, () -> {
                // Inside this scope, all code (including child virtual
                // threads)
                // can read REQUEST_ID.get() and CURRENT_USER.get()
                processRequest();
            });
        });
        // After the scope, get() throws – no stale values
    }

    private void processRequest() {
        String rid = REQUEST_ID.get(); // works even in nested methods
        User u = CURRENT_USER.get(); // same
        System.out.println("Processing request " + rid + " for " + u.name());
    }
}

```

`ScopedValue` vs. `ThreadLocal` :

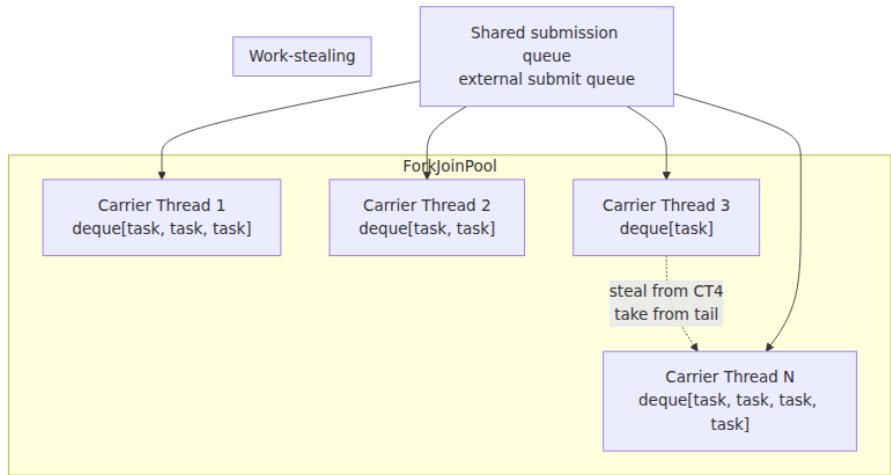
Aspect	ThreadLocal	ScopedValue
Mutability	<code>set()</code> / <code>remove()</code>	Immutable after <code>get()</code>
Cleanup	Manual <code>remove()</code> required	Automatic on scope exit
Memory leaks	Common (forgotten <code>remove()</code>)	Impossible (GC'd when scope exits)
Virtual threads	Works but each VT has its own copy	Flows to child VTs automatically
Inheritance	<code>InheritableThreadChild</code> (unreliable)	Natural scope propagation

Aspect	ThreadLocal	ScopedValue
Use for context	Request ID, trace context, auth token	Same — preferred for virtual threads

7. ForkJoinPool — the work-stealing scheduler

`ForkJoinPool` is the backbone of virtual thread scheduling, parallel streams (`stream().parallel()`), and `CompletableFuture` 's common pool. It is a specialized `ExecutorService` designed for divide-and-conquer tasks and for efficiently scheduling millions of lightweight tasks.

7.1 Architecture



Each carrier thread maintains a **work-stealing deque** (double-ended queue). Tasks submitted via `submit()` go to the shared submission queue; tasks forked via `ForkJoinTask.fork()` go to the **local deque** of the thread that forked them. When a carrier thread's local deque is empty, it attempts to **steal** a task from the tail of another carrier's deque (random victim selection). Stealing from the tail (while the owner works from the head) minimizes contention.

7.2 ForkJoinPool with virtual threads

When you use `Executors.newVirtualThreadPerTaskExecutor()`, the JVM creates a `ForkJoinPool` with `availableProcessors()` carrier threads and submits each virtual thread as a task to the pool. The pool also supports a custom scheduler for virtual threads:

```
// Default: common pool with availableProcessors() carriers
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> System.out.println("on VT"));
}

// Custom scheduler with specific parallelism
var scheduler = ForkJoinPool.commonPool(); // or new ForkJoinPool(8);
try (var executor =
    Executors.newVirtualThreadPerTaskExecutor(scheduler)) {
    IntStream.range(0, 1_000_000).forEach(i ->
        executor.submit(() -> {
            // All 1M tasks scheduled on the FJP
            Thread.sleep(Duration.ofMillis(1));
        })
    );
}
```

7.3 ForkJoinPool trace

```
# Enable ForkJoinPool tracing
java -Djdk.traceForkJoinTasks=true -jar app.jar

# Or monitor via JFR
jcmd <pid> JFR.start name=fjp filename=fjp.jfr duration=60s \
    settings=jdk.ForkJoinTask,jdk.ForkJoinPool
```

JFR events of interest: - `jdk.ForkJoinTask`: task submission, fork, join, completion times - `jdk.ForkJoinPool`: pool parallelism, steal count, steal time, task count

8. JUC primitives — AQS, StampedLock, ConcurrentHashMap

The `java.util.concurrent` package is the most important concurrency library in any language. Its internals are built on a small set of primitives

that you will encounter repeatedly when diagnosing contention, building custom synchronizers, or understanding framework internals.

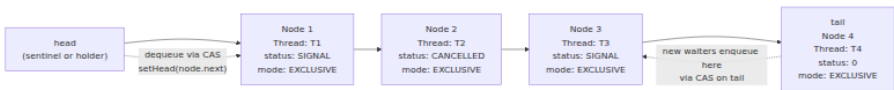
8.1 AbstractQueuedSynchronizer (AQS)

AQS is the base class for `ReentrantLock`, `CountDownLatch`, `Semaphore`, `ReentrantReadWriteLock`, and `Phaser`. It manages a FIFO wait queue of threads competing for a synchronization state:

```
// AQS state machine – the core contract
public abstract class AbstractQueuedSynchronizer extends AbstractOwnabl
eSynchronizer {
    private volatile int state;           // the synchronization state
    (lock count, permits, etc.)
    private transient volatile Node
head; // head of the CLH queue (the current holder)
    private transient volatile Node
tail; // tail of the CLH queue (new waiters added here)

    // Subclasses implement:
    protected abstract boolean tryAcquire(int arg); // attempt to
acquire (non-blocking)
    protected abstract boolean tryRelease(int arg); // attempt to
release
    protected abstract int tryAcquireShared(int
arg); // shared acquire (read lock, semaphore)
    protected abstract boolean tryReleaseShared(int arg);
}
```

8.2 AQS queue structure



The CLH queue (named after Craig, Landin, and Hagersten) is a lock-free linked list. New waiters CAS themselves onto the tail. The head is either the current lock holder or a sentinel. Each node has a `status` field: - `0` — waiting (initial state) - `SIGNAL` — needs to `unparkSuccessor()` when the predecessor releases - `CANCELLED` — thread timed out or was interrupted; will be removed

The AQS acquire path: 1. `tryAcquire()` — subclass attempts state CAS. If success, return immediately. 2. If fail: create a `Node`, CAS onto tail (may retry on contention). 3. If this node is now the successor of head: spin briefly (1-10 iterations), then `tryAcquire()` again. 4. If still failing:

`park()` the thread. When the predecessor releases, it calls `unparkSuccessor()`, which wakes the next node.

8.3 StampedLock — optimistic reading

`StampedLock` (JDK 8, `java.util.concurrent.locks`) provides a `ReadWriteLock` with an additional **optimistic read** mode. Unlike `ReentrantReadWriteLock`, it is not reentrant and not `Condition`-aware, but it avoids writer starvation and supports optimistic reads that are essentially free (no CAS, no lock):

```
import java.util.concurrent.locks.StampedLock;

public class Point {
    private double x, y;
    private final StampedLock sl = new StampedLock();

    // Pessimistic write lock
    void move(double deltaX, double deltaY) {
        long stamp = sl.writeLock();
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            sl.unlockWrite(stamp);
        }
    }

    // Optimistic read – no lock acquired, just a stamp
    double distanceFromOrigin() {
        long stamp = sl.tryOptimisticRead(); // returns current stamp,
        no CAS
        double currentX = x, currentY = y; // read fields (may be
        inconsistent)
        if (!sl.validate(stamp)) {          // did a write occur? CAS
        to check
            stamp = sl.readLock();          // fall back to
        pessimistic read
            try {
                currentX = x;
                currentY = y;
            } finally {
                sl.unlockRead(stamp);
            }
        }
        return Math.sqrt(currentX * currentX + currentY * currentY);
    }
}
```

The optimistic read path: `tryOptimisticRead()` reads the current lock stamp (a counter that increments on every write lock acquisition). After reading the fields, `validate(stamp)` checks whether the stamp changed (via a CAS or volatile read). If unchanged, no write occurred during the read — the values are consistent. If changed, fall back to a pessimistic read lock.

On a read-heavy workload (typical backend: 95% reads, 5% writes), `StampedLock` with optimistic reads significantly outperforms `ReentrantReadWriteLock` because reads never contend — they are just plain reads plus a stamp validation.

8.4 ConcurrentHashMap internals

`ConcurrentHashMap` is the thread-safe `HashMap` for concurrent access. Since JDK 8, it uses a **synchronized-on-bucket** strategy (replacing the JDK 7 lock-stripped approach):

```
// ConcurrentHashMap internals (simplified from ConcurrentHashMap.java)
transient volatile Node<K,V>[] table; // the hash table array

// Each bucket is a singly-linked list (or tree when bins grow large)
// Locking is per-bucket: synchronized(tableAt(bucketIndex))
// Read is lock-free: volatile read of table[i], traverse linked list

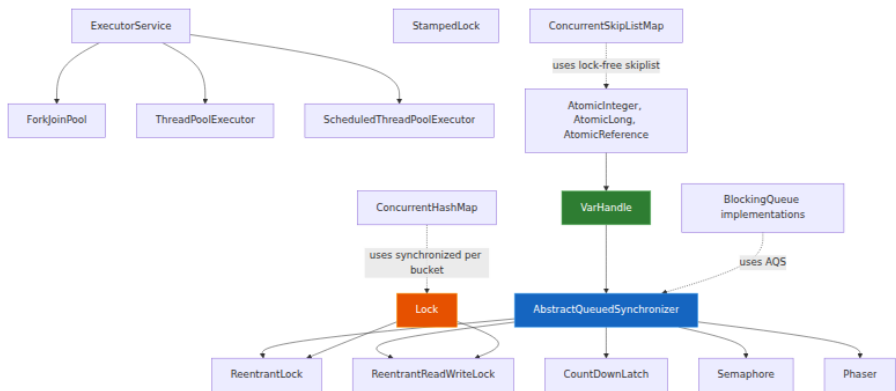
// Key operations:
V get(Object key) {
    // 1. Hash the key
    // 2. Locate bucket: table[(n-1) & hash]
    // 3. Traverse linked list – lock-free, volatile reads
    // 4. Return value or null
}

V put(K key, V value) {
    // 1. Hash the key
    // 2. Locate bucket
    // 3. synchronized(tableAt(bucket)) – lock only this bucket
    // 4. Traverse list: if key exists, update; if not, append
    // 5. If bin exceeds TREEIFY_THRESHOLD (8), convert to red-black
    tree
}
```

Internal optimizations: - **CAS on empty buckets:** When inserting into an empty bucket, a CAS (not `synchronized`) is used — zero contention for fresh buckets. - **transfer() for resizing:** When the table needs to grow, `ConcurrentHashMap` transfers buckets in batches — each thread copies a

subset of buckets, distributing the cost of a resize across all threads (the "helping" mechanism). - **Treeification:** When a bucket exceeds 8 entries (and the table is large enough), the linked list converts to a red-black tree. This prevents $O(n)$ degradation from hash collisions. The treeify threshold is `TREEIFY_THRESHOLD = 8`; the untreeify threshold is `UNTREEIFY_THRESHOLD = 6`. - **Counter cells:** `size()` and `mappingCount()` do not lock the table — they use `CounterCell[]` (similar to `LongAdder`), distributing increment operations across cells to avoid contention. This trades exactness for throughput — `size()` is approximate under concurrent modification.

8.5 JUC class hierarchy



9. Thread dump analysis — a complete example

A production JVM under load will occasionally exhibit slow response times, high CPU, or deadlocks. Thread dumps are the primary diagnostic tool. Here is a complete workflow:

```

# Step 1: Take a thread dump
jstack <pid> > /tmp/threaddump-$(date +%s).txt

# Step 2: Analyze for contention
# Count threads in each state
grep -c "java.lang.Thread.State:" /tmp/threaddump-*.txt
grep "BLOCKED" /tmp/threaddump-*.txt | wc -l
grep "WAITING" /tmp/threaddump-*.txt | wc -l

# Step 3: Find lock holders
grep -A 5 "waiting to lock" /tmp/threaddump-*.txt

# Step 4: Find deadlocks (jstack automatically detects these)
grep -A 10 "Found.*deadlock" /tmp/threaddump-*.txt

# Step 5: Virtual thread pinning
# If using virtual threads, check for carrier thread pinning
jcmd <pid> Thread.print | grep -B 5 "pinned"

# Step 6: Automated analysis
# Use tools like jattach + async-profiler
jattach <pid> jcmd "Thread.print" > /tmp/threads.txt

```

A deadlock detection output:

```

Found one Java-level deadlock:
=====
"worker-1":
  waiting to lock monitor 0x00007f8a4c003878 (object
0x00000000d4a1b2c0, a com.example.ResourceA),
  which is held by "worker-2"
"worker-2":
  waiting to lock monitor 0x00007f8a4c003a18 (object
0x00000000d4a1b2d8, a com.example.ResourceB),
  which is held by "worker-1"

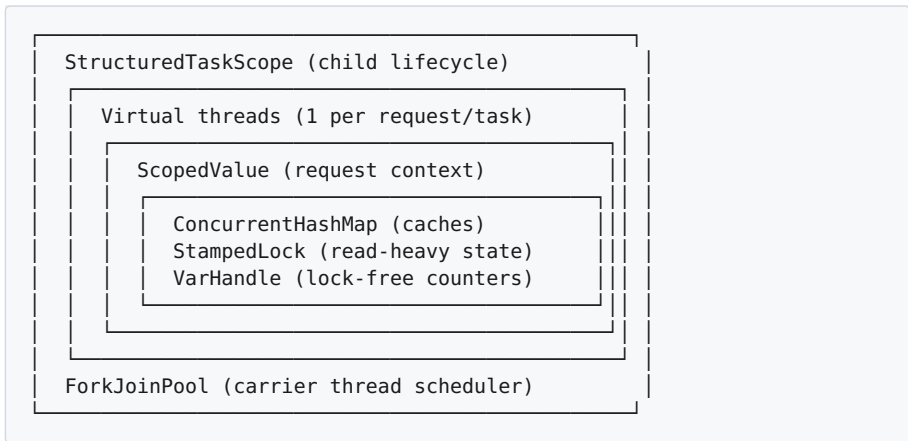
Java stack information for the threads listed above:
=====
"worker-1":
  at com.example.ServiceA.callServiceB(ServiceA.java:45)
  - waiting to lock <0x00000000d4a1b2c0> (a com.example.ResourceA)
  at com.example.ServiceB.callServiceA(ServiceB.java:38)
  - locked <0x00000000d4a1b2d8> (a com.example.ResourceB)

```

The root cause: thread 1 holds lock B and waits for lock A; thread 2 holds lock A and waits for lock B. Classic ABBA deadlock. Fix: always acquire locks in the same order, or use `tryLock` with timeout.

10. Putting it together — a production concurrency recipe

A well-structured backend service uses these primitives in layers:



Best practices for senior backend engineers:

1. **Default to virtual threads** for I/O-bound workloads. Use `Executors.newVirtualThreadPerTaskExecutor()` as your default executor. Platform threads are for CPU-bound work or when you need the OS thread identity.
2. **Replace `synchronized` with `ReentrantLock`** in virtual-thread-aware code. This eliminates pinning and allows the JVM to multiploy virtual threads effectively.
3. **Use `ScopedValue` instead of `ThreadLocal`** for request-scoped context (trace IDs, auth tokens, locale). It eliminates memory leaks and works naturally with virtual threads.
4. **Use `VarHandle` for lock-free algorithms.** If you are building a concurrent data structure, `VarHandle` gives you hardware-level ordering with compiler support. Avoid `Unsafe` — it is deprecated and behind `--add-opens`.
5. **Use `StampedLock` for read-heavy shared state.** Optimistic reads are essentially free — no CAS, no lock. This is strictly better than `ReentrantReadWriteLock` when reads dominate.

6. **Use `ConcurrentHashMap` for shared maps.** Do not wrap `HashMap` in `Collections.synchronizedMap()` — that locks the entire map for every operation. `ConcurrentHashMap` locks per-bucket and supports lock-free reads.
 7. **Monitor pinning in production.** Enable `JFR jdk.VirtualThreadPinned` events. A high pin rate means your code is blocking on `synchronized` or `JNI` in hot paths.
 8. **Take thread dumps proactively.** Do not wait for a user report. Automate `jstack` collection when p99 latency spikes. Use `jcmd <pid> Thread.print` in CI load tests.
-

Key takeaways

- **Platform threads** are 1:1 OS threads. Each costs ~1 MB of stack. At scale, you run out of kernel resources before you run out of memory. Virtual threads solve this.
- **Monitors** use three tiers: lightweight (CAS on mark word, ~5 ns), heavyweight (ObjectMonitor with futex, ~1000 ns), and historically biased (removed in JDK 15). The JIT intrinsifies `synchronized` to use CAS fast-path with monitor inflation as the slow path.
- **`VarHandle`** is the modern API for hardware-level memory ordering. Use `release/acquire` for publication (cheaper than `volatile` on AArch64), `opaque` for progress indicators, and `volatile` only when you need `StoreLoad` ordering (Dekker-style coordination).
- **Virtual threads** decouple logical threads from OS threads. They are heap-allocated continuations that mount/unmount carrier threads without kernel involvement. Pinning on `synchronized` and `JNI` is the primary pitfall — use `ReentrantLock` instead.
- **Structured concurrency** (`StructuredTaskScope`) ensures child tasks are completed or cancelled before the parent scope exits. `ScopedValue` replaces `ThreadLocal` with immutable, scope-bound context that propagates to virtual threads automatically.
- **`ForkJoinPool`** is the scheduler for virtual threads and parallel streams. Its work-stealing deque architecture efficiently distributes millions of tasks across a small number of carrier threads.

- **AQS** is the foundation of all JUC synchronizers. Its CLH wait queue, CAS-based acquisition, and `park / unpark` protocol appear in `ReentrantLock`, `CountDownLatch`, `Semaphore`, and beyond.
-

Further reading

- **JEP 425** — Virtual Threads (Preview): <https://openjdk.org/jeps/425>
- **JEP 453** — Structured Concurrency (Preview): <https://openjdk.org/jeps/453>
- **JEP 429** — Scoped Values (Preview): <https://openjdk.org/jeps/429>
- **JEP 193** — VarHandles: <https://openjdk.org/jeps/193>
- **JSR-133** — Java Memory Model and Thread Specification: <https://www.jcp.org/en/jsr/detail?id=133>
- **Doug Lea** — "A java.util.concurrent.Synchronizer Framework": the AQS paper. *ACM SIGPLAN Notices*, 2004.
- **Jordan, Rose, et al.** — "Project Loom: Enlightening Java with Virtual Threads": <https://openjdk.org/projects/loom/>
- **Brian Goetz** — "Structured Concurrency in Java" (Brian Goetz's talks on JEP 453 and ScopedValue).
- **Mark Reinhold** — "Structured Concurrency and Scoped Values": <https://openjdk.org/projects/loom/specs/>
- **Björn Kaelter et al.** — "Java Concurrency in Practice" — the foundational reference for JUC internals (2006, still relevant for AQS and concurrent collections).
- **HotSpot source** — `objectMonitor.hpp`, `synchronizer.cpp`, `thread.cpp`: <https://github.com/openjdk/jdk>

Chapter 7 — Kotlin on the JVM: Interop, Null-Safety, Coroutines, and Compiler Intrinsics

What this chapter covers. Kotlin is the default language for new JVM backend services at most large organizations, yet many teams treat it as syntactic sugar over Java. That mental model breaks the first time a

`NullPointerException` originates from a `!!` three frames away, a `suspend` function appears as a method with an extra `Continuation` parameter in a stack trace, or a Java caller cannot find the Kotlin companion object method it expects to be `static`. This chapter removes the black box. You will see exactly how the Kotlin compiler (`kotlinc`) lowers Kotlin constructs to JVM bytecode — properties to `get/set` methods, `data class` to `equals/hashCode/copy/componentN`, `value class` to mangled primitives, `object` to the singleton holder pattern, and `suspend` to a continuation-passing state machine. You will understand the null-safety contract at the bytecode boundary, how platform types (`String!`) leak Java nullability into Kotlin, and how `?.`, `?:`, and `let` chains compile. You will dissect coroutines as generated classes you can inspect with `javap -c -p`, reason about dispatcher topology, and apply structured concurrency correctly in service code. Finally, you will use compiler intrinsics — `inline`, `reified`, `noinline/crossinline`, and the `contract` DSL — to eliminate abstraction cost without breaking semantics. Every section pairs Kotlin source with the `javap` output it produces.

Learning goals — after this chapter you should be able to:

- Map every major Kotlin declaration — property, data class, companion object, `object` singleton, `value/inline` class, extension function, SAM lambda — to the JVM class/method/field it becomes, and predict the `javap` output without running it.
- Explain Kotlin's null-safety system: non-null vs. nullable types, platform types (`T!`), `JSpecify/JSR-305` annotations, and how `?.`, `?:`, `!!`, `let`, and safe-call chains compile to null checks and branches.
- Design Java↔Kotlin interop boundaries: when to apply `@JvmStatic`, `@JvmOverloads`, `@JvmName`, `@JvmField`, `@JvmInline/@JvmRecord` interactions, how `lateinit` works, and why Kotlin silently swallows Java checked exceptions.
- Describe how `suspend` desugars: the added `Continuation` parameter, the `COROUTINE_SUSPENDED` protocol, and the compiler-generated state-machine class with `label` dispatch — and walk a real `javap -c -p` dump of a two-suspension function.
- Select and tune coroutine dispatchers (`Default`, `IO`, `Unconfined`), apply structured concurrency (`coroutineScope`, `supervisorScope`,

`SupervisorJob`), and implement correct cancellation and exception propagation (`CancellationException`, `CoroutineExceptionHandler`).

- Use `inline` functions, `reified` generics, `noinline`/`crossinline`, and `contract` declarations to get zero-cost abstractions and type-safe generic operations that survive erasure.

Placement. Chapter 2 covered class loading, bytecode verification, and the class file format — the substrate Kotlin targets. Chapter 3 established the Java Memory Model and object layout that Kotlin properties and value classes build upon. Chapter 5 covered JIT intrinsification and inlining; Kotlin's own `inline` is a compile-time analogue you will compare directly. Chapter 6 covered threads, monitors, `VarHandle`, and Project Loom — coroutine dispatchers are the Kotlin-level scheduler on top of those primitives. If you have not read Chapter 6, skim at least Sections 1 and 5 (thread model and executor topology) before Section 8 here.

1. How `kotlinc` lowers Kotlin to JVM bytecode

Kotlin does not ship a runtime VM. The `kotlinc` frontend parses Kotlin source, runs type resolution, and emits JVM bytecode via ASM — the same bytecode instructions (`aload`, `invokevirtual`, `invokedynamic`) that `javac` emits. Understanding this lowering is non-optional for backend work: debuggers, profilers (`async-profiler`, JFR), decompilers, and reflection all show you the JVM view, not the Kotlin view.

1.1 Compilation pipeline in one picture



The key observation: by the time the class file reaches the verifier, there is no trace of "Kotlin" — only classes, methods, fields, and annotations. The `kotlin.Metadata` annotation preserves the original Kotlin signature for reflection and for the Kotlin compiler when another module depends on this class, but the JVM itself never reads it.

```
# Inspect what the Kotlin compiler actually emitted
kotlinc Service.kt -d /tmp/classes
javap -c -p -v /tmp/classes/com/example/Service.class | head -n 120
# The -p flag shows private members; -v shows the Metadata annotation
# The -c flag disassembles bytecode
```

The Kotlin standard library (`kotlin-stdlib`) is a compile-time and runtime dependency. Intrinsic like `Intrinsics.checkNotNullParameter` and `Intrinsics.checkNotNullExpressionValue` are static helpers inserted by the compiler to enforce null contracts at runtime when compile-time guarantees do not suffice (platform types, `!!`, interop boundaries). They are small, `final`, and JIT-inline to a single null-check branch.

1.2 What the metadata annotation carries

Every Kotlin-compiled class carries:

```
RuntimeVisibleAnnotations:
  0: #10(#11=[...]): kotlin/Metadata(
    mv={2, 1, 0}, k=1, xi=48,
    d1={"\u0000\u0018\n\u0002\u0018\u0002\n..."},
    d2={"Lcom/example/Service;","name","getName","..."}
  )
```

Tools like `kotlin-reflect`, `jackson-module-kotlin`, and `gson` decode this annotation to recover Kotlin-specific type information (nullability, `suspend`, `value class` wrapper) that bytecode erases. If you strip this annotation (ProGuard/R8 `keep` misconfiguration), reflection breaks but direct bytecode execution does not — a common production incident pattern.

2. Properties, data classes, companions, and objects — the object mapping

2.1 Properties are methods

The single most important lowering rule: a Kotlin property is not a field. It is a field plus accessor methods, with visibility and naming governed by precise conventions.

```
// File: User.kt
class User(
    val id: String,                // read-only property
    var name: String,             // mutable property
    private var _internal: Int = 0
) {
    var email: String? = null
        private set                // setter visibility !=
        // getter visibility

    val displayName: String
        get() = "$name <$email>" // computed – no backing
        // field

    lateinit var sessionToken: String // late-initialized, non-
        // null

    @JvmField
    var counter: Int =
        0 // exposed as public field, no accessors
}

```

The compiler generates:

```
javap -p /tmp/classes/com/example/User.class
```

```

public final class com.example.User {
    private final java.lang.String id;
    private java.lang.String name;
    private int _internal;
    private java.lang.String email;
    private int counter; // @JvmField public field

    public final java.lang.String getId();
    public final java.lang.String getName();
    public final void setName(java.lang.String);
    private final int get_internal();
    private final void set_internal(int);
    public final java.lang.String getEmail();
    private final void setEmail(java.lang.String); // private setter
    public final java.lang.String getDisplayName(); // no field computed

    public final java.lang.String getSessionToken();
    public final void setSessionToken(java.lang.String);
    // synthetic null-check in constructor:
    // Intrinsic.checkNotNullParameter(id, "id")

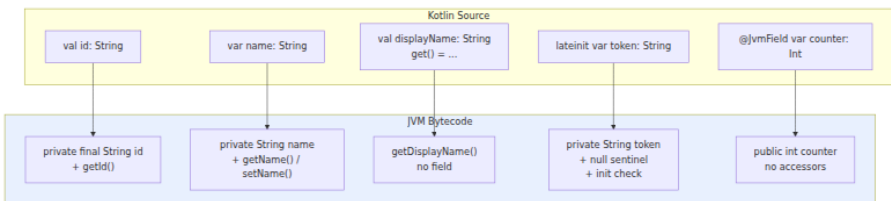
    // @JvmField counter has no getter/setter:
    public int counter; // with -p you see the field directly
}

```

Detailed observations for the senior backend reader:

Kotlin declaration	JVM artifact	Notes
val id: String	private final String id + getId()	No setter. Field is final.
var name: String	private String name + getName() / setName(String)	Both accessors public final.
var email: String? with private set	getEmail() public, setEmail() private	Setter visibility lowered independently.

Kotlin declaration	JVM artifact	Notes
<code>val displayName: String get() = ...</code>	Only <code>getDisplayName()</code> — no backing field	Every access recomputes. Not <code>volatile</code> , not cached.
<code>lateinit var sessionToken: String</code>	<code>private String sessionToken + null sentinel + UninitializedPropertyAccessException guard</code>	Field is non-final, initially <code>null</code> at JVM level despite Kotlin non-null type. Access before init throws.
<code>@JvmField var counter: Int</code>	<code>public int counter</code> — no accessors	Breaks encapsulation intentionally for Java interop / performance.
<code>const val MAX = 100</code> (top-level or companion)	<code>public static final int MAX = 100 + inlined at call sites</code>	Compile-time constant, no field access at call site.



Why this matters for services: Jackson, JPA, gRPC codegen, and annotation processors reflect over JVM artifacts. A `private set` property is invisible to frameworks that call setters reflectively unless they use Kotlin-aware modules (`jackson-module-kotlin` reads `Metadata` to find the private setter). A `lateinit` property that is never initialized throws `UninitializedPropertyAccessException` — a subclass of `RuntimeException` that bypasses exhaustive initialization checks and surfaces only at

request time, typically under a rarely-hit code path. In a distributed service, that is a 500 you only see in production.

Backing-field bytecode for a simple property read/write:

```
fun example(u: User): String {
    u.name = "Ada"
    return u.name
}
```

```
javap -c -p /tmp/classes/com/example/UserKt.class
```

```
public static final java.lang.String example(com.example.User);
  Code:
    0: aload_0
    1: ldc         #2 // String Ada
    3: invokevirtual #3 // Method com/example/User.setName:(Ljava/
lang/String;)V
    6: aload_0
    7: invokevirtual #4 // Method com/example/User.getName:()Ljava/
lang/String;
   10: areturn
```

There is no `getfield`/`putfield` on the caller side — only virtual method calls. The JIT will inline these trivial accessors within a few invocations (see Chapter 5, inlining heuristics: `MaxInlineSize=35` bytes, accessors are ~5 bytes), so the abstraction cost is zero after warmup. Before warmup, you pay one virtual call per property access — measurable in tight serialization loops.

2.2 Data classes — the generated surface

```
data class Order(
    val id: String,
    val amountCents: Long,
    val currency: String = "USD"
)
```

`javap -p` shows the full synthetic surface:

```

public final class com.example.Order {
    private final java.lang.String id;
    private final long amountCents;
    private final java.lang.String currency;

    public final java.lang.String getId();
    public final long getAmountCents();
    public final java.lang.String getCurrency();

    // componentN used by destructuring
    public final java.lang.String component1();
    public final long component2();
    public final java.lang.String component3();

    // copy default arguments become overloads via bitmask
    public final com.example.Order copy(java.lang.String, long,
    java.lang.String);
    public static synthetic com.example.Order copy$default(..., int
    mask, Object);
    // mask bit 0 id default, bit 1 amountCents, bit 2 currency

    public boolean equals(java.lang.Object);
    public int hashCode();
    public java.lang.String toString();

    public synthetic Order(java.lang.String, long, java.lang.String,
    int, kotlin.jvm.internal.DefaultConstructorMarker);
}

```

Three details with production consequences:

1. **equals / hashCode are structural.** Two `Order` instances with the same field values are `equal`, regardless of identity. If you use a data class as a `ConcurrentHashMap` key or put it in a `HashSet` across service boundaries, hash distribution is determined by the generated `hashCode` — which multiplies by 31 per component. For a high-cardinality key space this is fine; for a low-cardinality one (e.g., data class `Shard(val id: Int)` with 8 values), you get predictable bucket collisions. Prefer explicit `hashCode` tuning for hot-path map keys.
2. **copy with defaults uses a synthetic bitmask.** The `copy$default` method takes an `int mask` where bit `i` means "use default for parameter `i`". This is how `order.copy(amountCents = 999)` compiles without allocating a builder. The call site passes `mask = 0b101` (keep `id` and `currency`, override `amountCents`). The method is `synthetic`

— Java callers should use the explicit `copy(String, long, String)` overload.

3. **Destructuring is positional, not nominal.** `val (id, amount) = order` compiles to `component1() / component2()`. Reordering properties in the data class silently breaks destructuring at every call site without a compile error if types happen to align (e.g., swapping two `String` fields). In a multi-team monorepo, treat property order as API.

2.3 Companion objects, `object` singletons, and static exposure

```
class Service private constructor(val endpoint: String) {
    companion object {
        const val DEFAULT_TIMEOUT_MS = 3000
        fun create(endpoint: String) = Service(endpoint)

        @JvmStatic
        fun createForJava(endpoint: String) = Service(endpoint)
    }

    object Registry {
        val services = mutableMapOf<String, Service>()
        fun register(s: Service) { services[s.endpoint] = s }
    }
}
```

Bytecode reality:

```

public final class com.example.Service {
    private final java.lang.String endpoint;
    public static final int DEFAULT_TIMEOUT_MS = 3000; // const → true
    static
    public static final com.example.Service$Companion Companion; // singleton holder

    public static final class Companion {
        public final com.example.Service create(java.lang.String);
        public final com.example.Service createForJava(java.lang.String); // instance method
        public static com.example.Service createForJava$static(java.lang.String); // @JvmStatic synthetic
    }

    public static final class Registry {
        public static final com.example.Service$Registry INSTANCE; // singleton
        private final java.util.Map services;
    }
}

```

What to internalize:

- **Without `@JvmStatic`**, `Service.create("...")` from Kotlin compiles to `Service.Companion.create("...")` — an instance call on the `Companion` singleton. From Java, you must write `Service.Companion.create("...")`. With `@JvmStatic`, the compiler also generates a `static` forwarder `Service.createForJava(String)` on the outer class, so Java callers write `Service.createForJava("...")`.
- **object `Registry`** is a singleton with a `private` constructor and a `public static final INSTANCE` field, initialized in `<clinit>`. There is no double-checked locking — class loading provides the happens-before (JMM class initialization semantics, Chapter 3). Access is `Service.Registry.INSTANCE` from bytecode. This is the same pattern as `enum` singletons in Effective Java.
- **Serialization hazard.** `object` singletons are not automatically serialization-safe. If you serialize `Registry` via Java serialization or Kryo without care, deserialization creates a second instance. Kotlin's `object` generates a `readResolve` only for `object` declarations that are `Serializable` — verify with `javap -p` that `private Object`

`readResolve()` exists if you serialize singletons over the wire (avoid doing so; prefer DTOs).

3. Value classes, inline classes, and extension functions

3.1 `value class` — zero-cost wrappers with mangled names

```
@JvmInline
value class UserId(val raw: String)

@JvmInline
value class OrderId(val raw: Long)

fun findUser(id: UserId): User? = lookup(id.raw)
fun findOrder(id: OrderId): Order? = lookup(id.raw)
```

At the JVM level, the compiler erases the wrapper and passes the underlying type directly — but it must prevent accidental cross-wiring of two wrappers over the same underlying type. It does this by **name mangling**:

```
public final class com.example.UserId {
    private final java.lang.String raw;
    // constructor is private ☐ only generated for boxing fallback
    // All call sites that take UserId actually take String with a mangle
    // d name:
}

public final class com.example.ServiceKt {
    // Kotlin: fun findUser(id: UserId): User☐
    // JVM:    public static final User findUser-Yo8aak( String id )
    //          ^^^^^^ mangled suffix

    // Kotlin: fun findOrder(id: OrderId): Order☐
    // JVM:    public static final Order findOrder-kS2GYao( long id )
}
```

Verify:

```
javap -p /tmp/classes/com/example/ServiceKt.class | grep findUser
# public static final com.example.User findUser-
Yo8aak(java.lang.String);
```

Consequences:

- **Interop friction.** Java callers see `findUser_Yo8aak(String)` — not `findUser(UserId)`. To expose a Java-friendly overload, add an explicit `@JvmName` or a secondary function. Alternatively, keep `value class` internal to Kotlin modules and expose `String/long` at the Java API boundary.
- **Boxing fallback.** When a `value class` is used as a generic type argument (`List<UserId>`), stored in a nullable position (`UserId?`), or passed as `Any`, the JVM must box it into the wrapper object. The allocation is real — one object per element. In hot paths (e.g., a `List<UserId>` of 100K entries materialized per request), this boxing cost can dominate. The JIT can sometimes scalar-replace the box (Chapter 5, escape analysis), but not across method boundaries that expect `Object`.
- **@JvmInline is mandatory** for the JVM backend since Kotlin 1.5. Without it, the compiler still boxes. The annotation is the signal to erase.

After mangling, `findUser` lowers to `findUser-Yo8aak(String)`, `findOrder` to `findOrder-ks2GYao(long)`, while `List<UserId>` boxes each element into a `UserId` wrapper object.

For backend services, `value class` is ideal for domain identifiers that cross network boundaries (`UserId`, `TenantId`, `TraceId`) — you get compile-time type safety with no wire-format cost, provided you serialize the underlying `raw` value. It is poor for collections that are sorted, filtered, and mapped in tight loops where boxing pressure matters; measure with `async-profiler` allocation profiling before committing.

3.2 `object` revisited — double-checked locking is absent

We covered `object` in Section 2.3, but one more detail matters for services: an `object` that captures mutable state is a global singleton shared across all request handlers. In a coroutines-based service (Section 7), every coroutine sees the same `object` instance — there is no coroutine-local isolation. Guard mutable state with `Mutex` (coroutine-aware) or `AtomicReference`, not `synchronized`, to avoid pinning virtual threads (Chapter 6, Section 6).

3.3 Extension functions — static helpers with syntactic sugar

```
// File: StringExt.kt
fun String.isValidEmail(): Boolean = contains("@") && contains(".")
fun String.truncate(maxLen: Int): String = if (length <= maxLen) this else take(maxLen)

// Extension on nullable receiver
fun String?.orUnknown(): String = this ?: "unknown"

// Generic extension
fun <T> List<T>.secondOrNull(): T? = if (size >= 2) this[1] else null
```

Bytecode:

```
public final class com.example.StringExtKt {
    public static final boolean isValidEmail(java.lang.String
$this$isValidEmail);
    public static final java.lang.String truncate(java.lang.String
$this$truncate, int maxLen);
    public static final java.lang.String orUnknown(java.lang.String
$this$orUnknown);
    public static final java.lang.Object secondOrNull(java.util.List
$this$secondOrNull);
}
```

Every extension is a `static` method whose first parameter is the receiver (`$this$...`). There is no monkey-patching, no dynamic dispatch, no vtable entry on `String`. Resolution is **static and lexical** — the compiler picks the extension visible at the call site via imports. This has two consequences:

1. **No polymorphism.** If `class A` and `class B : A` both have extensions `fun A.foo()` and `fun B.foo()`, `val x: A = B(); x.foo()` calls `A.foo()`. The receiver's static type determines the target. For service code, this means extension-based polymorphism is a bug pattern — use member functions or interfaces.
2. **Binary compatibility is import-sensitive.** Moving an extension to a different package changes the call site's import and thus the linkage. No `NoSuchMethodError` at runtime (the target is a static method), but a stale compiled call site may bind to a different extension after a partial recompilation.

The `this` parameter is explicit in bytecode — `truncate` takes `(String, int)`, not `(int)` on a `String` instance. Profilers and stack traces show

`StringExtKt.truncate(String, int)` rather than `String.truncate(int)`. When triaging a flame graph, search for `*Kt.` classes — that suffix marks file-level functions and extensions.

4. SAM conversions, functional interfaces, and `fun` interface

Kotlin's SAM (Single Abstract Method) conversion lets a lambda satisfy a Java functional interface. Since Kotlin 1.4, `fun` interface brings the same to Kotlin-declared interfaces.

```
// Java functional interface (java.lang Runnable,
// java.util.concurrent.Callable, etc.)
fun interface EventHandler {
    fun onEvent(event: String)
}

// Kotlin call site – SAM conversion
val handler: EventHandler = EventHandler { event -> println("got
$event") }

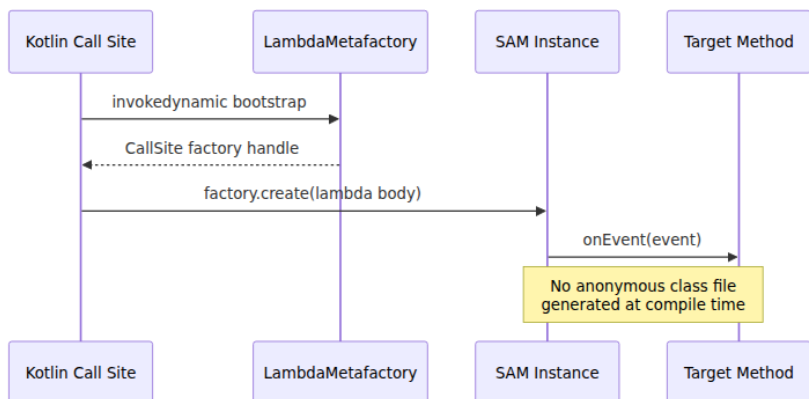
// Java interop – Kotlin lambda passed to Java method expecting single-
// method interface
// Java: void submit(Runnable task)
executor.submit { println("running") } // lambda → Runnable via SAM
```

Bytecode for the SAM lambda:

```
javap -c -p /tmp/classes/com/example/SamKt.class

// The lambda body becomes a synthetic method:
private static final void onEvent$lambda$0(java.lang.String event);
    Code:
        0: getstatic      #2 // Field java/lang/System.out
        3: aload_0
        4: invokedynamic  #3, 0 // InvokeDynamic #0:apply:
        ()Ljava/Handler;
        // ... bootstrap: LambdaMetafactory.metafactory ...

// At the call site, invokedynamic creates the SAM instance:
invokedynamic #4, 0 // InvokeDynamic #1:run:()Ljava/lang/Runnable;
```

Why `invokedynamic` matters: before JDK 8, Kotlin generated an anonymous inner class (`EventHandler$1.class`) per lambda — one file per lambda, loaded eagerly. With `invokedynamic` (the default since Kotlin 1.4 targeting JVM 8+), the SAM instance is created via `LambdaMetafactory` at first invocation, and the JVM may share the factory across call sites. For a service that creates thousands of short-lived lambdas per request (stream pipelines, coroutine builders), this eliminates class-loading churn and metaspace pressure.

SAM ambiguity pitfall. If a Java method is overloaded with two SAM types:

```
// Java
void handle(Runnable r);
void handle(Callable<String> c);
```

```
// Kotlin - ambiguous
handler.handle { println("which one?") } // COMPILE ERROR: overload
resolution ambiguity
handler.handle(Runnable { println("explicit") }) // OK - explicit SAM
constructor
```

In large Java-interop surfaces (AWS SDK, gRPC stubs), overloaded SAM methods are common. Always use the explicit `Runnable { ... }` constructor form in ambiguous contexts — it costs the same allocation but removes resolution nondeterminism.

fun interface vs. typealias for lambdas. `typealias Handler = (String) -> Unit` is just a function type — every handler is a distinct

`Function1` instance. `fun interface EventHandler` is a named SAM that participates in overload resolution and can carry overloads/defaults in future. For public service APIs, prefer `fun interface` for handler contracts that may evolve.

5. Null-safety — the type system, the boundary, and the operators

5.1 The three tiers of nullability

Tier 1 — Kotlin Non-Null
String, Int, User

Compile-time guarantee
No null assignment
No safe-call needed

Bytecode: null check only
at interop boundaries
`Intrinsics.checkNotNullParameter`

Tier 2 — Kotlin Nullable
String?, Int?, User?

Explicit nullable
Must use `?.` `?:` `!!` `let`
or explicit null check

Bytecode: null branches
at every safe-call / Elvis

Tier 3 — Platform Type

Kotlin's null-safety is a **type-system guarantee that degrades at the Java boundary**. Within pure Kotlin, `String` cannot hold `null` — the compiler rejects `val s: String = null`. The JVM still represents `String` as a nullable reference (`Ljava/lang/String;`), but the compiler inserts `Intrinsics.checkNotNull*` guards at function entry points and before dereferences that cross nullable boundaries. When all code is Kotlin, these guards are redundant but cheap (one branch, JIT-elided after profiling shows non-null).

The guarantee breaks at platform types. Any Java-declared type without a nullability annotation arrives in Kotlin as `T!` — the compiler does not know whether it is `T` or `T?` and lets you use it as either, emitting no check. This is the single largest source of Kotlin NPEs in mixed codebases.

5.2 Platform types and JSpecify

```
// Java file – no nullability annotations
public class JavaUserService {
    public String findName(String userId) { // returns null if not
        found – but signature says String
        return userId.equals("known") ? "Ada" : null;
    }
    public String getDisplayName(String userId) { return findName(userId); }
}
```

```
// Kotlin caller – platform type String! silently accepted as String
fun greet(service: JavaUserService, id: String) {
    val name: String = service.findName(id) // compiles – String! →
    String
    println(name.length) // NPE at runtime if
    findName returned null
}
```

Bytecode for `greet` :

```

public static final void greet(JavaUserService, java.lang.String);
Code:
  0: aload_0
  1: aload_1
  2: invokevirtual #2 // JavaUserService.findName:(Ljava/lang/
String;)Ljava/lang/String;
  5: astore_2          // no null check ☐ platform type, compiler tr
usts Java
  6: aload_2
  7: invokevirtual #3 // String.length:()I - NPE here if null

```

Add JSpecify annotations to fix the boundary:

```

import org.jSpecify.annotations.Nullable;
import org.jSpecify.annotations.NonNull;

public class JavaUserService {
    public @Nullable String findName(@NonNull String userId) { ... }
    public @NonNull String getDisplayName(@NonNull String userId)
{ ... }
}

```

Now Kotlin sees `findName` as `String?` and `getDisplayName` as `String` — the assignment `val name: String = service.findName(id)` fails to compile, forcing the caller to handle null:

```

fun greetSafe(service: JavaUserService, id: String) {
    val name: String? = service.findName(id) // nullable - must handle
    println(name?.length ?:
0)          // safe-call + Elvis - no NPE
}

```

Annotation ecosystem. Kotlin recognizes `org.jSpecify.annotations.*` (JSpecify 1.0, the current standard), `javax.annotation.*` (JSR-305, dormant), `org.jetbrains.annotations.*`, and `androidx.annotation.*`. JSpecify is the only actively maintained, specification-backed choice — adopt it for all new Java code. Configure the Kotlin compiler to enforce strict JSR-305/JSpecify handling:

```
// build.gradle.kts
tasks.withType<KotlinCompile> {
    compilerOptions {
        freeCompilerArgs.add("-Xjsr305=strict") // strict nullability
        // from JSR-305
        // or with Kotlin 2.0+: -XjSpecify-annotations=strict
    }
}
```

5.3 Operators — what `?.`, `?:`, `!!`, and `let` compile to

```
data class Address(val city: String?)
data class Profile(val address: Address?)
data class Account(val profile: Profile?)

fun cityLength(account: Account?): Int {
    // Safe-call chain + Elvis
    return account?.profile?.address?.city?.length ?: -1
}

fun requireCity(account: Account): String {
    return account.profile!!.address!!.city!! // triple !! – triple
    risk
}

fun cityOrLog(account: Account?): Int {
    return account?.profile?.address?.city?.let { city ->
        println("city=$city")
        city.length
    } ?: -1
}
```

`cityLength` bytecode — each `?.` is a null check + branch:

```

public static final int cityLength(com.example.Account);
Code:
  0: aload_0
  1: ifnull      40          // account == null ☐ goto 40
(return -1)
  4: aload_0
  5: invokevirtual #2 // Account.getProfile():LProfile;
  8: astore_1
  9: aload_1
 10: ifnull      40          // profile == null ☐ goto 40
 13: aload_1
 14: invokevirtual #3 // Profile.getAddress():LAddress;
 17: astore_2
 18: aload_2
 19: ifnull      40
 21: aload_2
 22: invokevirtual #4 // Address.getCity():Ljava/lang/String;
 25: astore_3
 26: aload_3
 27: ifnull      40          // city == null ☐ goto 40
 30: aload_3
 31: invokevirtual #5 // String.length():I
 34: istore      4
 36: iload       4
 37: ireturn
 40: iconst_m1
 41: ireturn

```

Each `?.` is exactly one `ifnull` + conditional jump — the same cost as a hand-written `if (x != null)` in Java. The chain short-circuits at the first `null` without evaluating later links. The JIT sees this as a sequence of predictable branches (usually not-null in steady state) and speculates accordingly.

`requireCity` with `!!` compiles to:

```

public static final java.lang.String requireCity(com.example.Account);
Code:
    0: aload_0
    1: invokevirtual #2 // Account.getProfile
    4: dup
    5: ifnonnull     12
    8: ldc          #6 // String "profile"
   10: invokestatic #7 //
Intrinsics.throwUninitializedPropertyAccessException / throwNpe
   12: invokevirtual #3 // Profile.getAddress
   15: dup
   16: ifnonnull     23
   19: ldc          #8
   21: invokestatic #7
   23: invokevirtual #4 // Address.getCity
   26: dup
   27: ifnonnull     34
   30: ldc          #9
   32: invokestatic #7 // throws KotlinNullPointerException with
message

```

`!!` is not a no-op — it inserts `Intrinsics.throwNpe()` which throws `KotlinNullPointerException` (a subclass of `NullPointerException` since Kotlin 1.4) with a message like `"account.profile must not be null"`. Compare with a raw Java NPE (`Cannot invoke "Profile.getAddress()" because "profile" is null` since JDK 14's helpful NPEs). The Kotlin variant is more precise about which link in the chain was null, but both are 500s. In a distributed trace, the difference between `KotlinNullPointerException` and `NullPointerException` tells you whether the null originated from a Kotlin `!!` assertion or a raw Java dereference — instrument your error classifier to distinguish them.

`let` + safe-call idiom compiles to a null check followed by a synthetic lambda invocation (or an inlined call if `let` is inlined — it is, since `let` is `inline`). Prefer `?.let` for side-effecting null-guarded blocks and `?:` for default values. Avoid `?.let` for simple value mapping where `?.` + `?:` is clearer.

Null-safety checklist for service boundaries:

- Annotate every Java method that crosses into Kotlin with `@Nullable` / `@NonNull` (JSpecify).
- Never use `!!` on data that originated from a network call, database, or deserialization — those are the highest-risk null sources. Use `?: error("...")` or `?: return` to fail explicitly.

- In gRPC/Protobuf stubs, `string` fields default to `""` (not `null`), but optional `string` and message fields are nullable — generate Kotlin stubs with `protoc-gen-kotlin` which maps `optional` to `T?` correctly.
- Monitor `KotlinNullPointerException` rate separately from `NullPointerException` in your metrics — a spike in the former after a Kotlin migration usually means platform types or `!!` overuse.

6. Java-Kotlin interop — annotations, exceptions, and `lateinit`

6.1 The annotation toolkit for pleasant Java APIs

Kotlin's defaults are optimized for Kotlin callers. Four annotations fix the Java view:

```
class MetricsReporter @JvmOverloads constructor(
    val namespace: String,
    val flushIntervalMs: Long = 5000,
    val maxBatchSize: Int = 100
) {
    companion object {
        @JvmStatic
        fun create(namespace: String) = MetricsReporter(namespace)

        @JvmStatic
        @JvmName("createWithInterval") // avoid overload clash after
        erasure fun create(namespace: String, interval: Long) =
            MetricsReporter(namespace, interval)
    }

    @JvmField
    val createdAtNanos: Long = System.nanoTime()

    @JvmName("recordEventInternal")
    fun record(event: String) { /* ... */ }

    @Throws(IOException::class) // checked exception bridge — see 6.2
    fun flush() { /* ... */ }
}
```

Annotation	What it does	Without it
<code>@JvmStatic</code>	Generates <code>static</code> forwarder on outer class	Java must go through <code>Companion</code> singleton
<code>@JvmOverloads</code>	Generates N overloads for N default params	Java sees one constructor requiring all args; no defaults
<code>@JvmName("...")</code>	Renames JVM method to avoid clashes	Mangled or conflicting names after erasure
<code>@JvmField</code>	Exposes property as <code>public</code> field, no accessors	Java must call <code>get</code> / <code>set</code> methods
<code>@Throws(IOException::class)</code>	Adds <code>throws IOException</code> to JVM signature	Checked exception invisible to Java <code>catch</code> (see below)
<code>@JvmInline</code>	Erases <code>value</code> class wrapper	Wrapper object allocated
<code>@JvmRecord</code> (Kotlin 1.9+)	Generates JVM <code>record</code> class file	Regular class with <code>equals</code> / <code>hashCode</code>

The annotation mapping is summarized in the table above: `companion fun create()` → `static create()` via `@JvmStatic`, constructors with defaults → overloads via `@JvmOverloads`, `val createdAtNanos` → `public final` field via `@JvmField`, and `fun record` → `recordEventInternal()` via `@JvmName`.

`@JvmOverloads` cost: it generates 2^N overloads in the worst case for N default parameters, but actually generates N+1 constructors (each overload fills one more default from the right). For a constructor with 3 defaults, you get 4 overloads. Binary size impact is negligible; method count impact matters only if you approach the 64K methods-per-dex limit on Android (irrelevant for backend services).

6.2 Checked exceptions — Kotlin swallows them

Kotlin has no checked exceptions. Any Java method that declares `throws IOException` can be called from Kotlin without `try / catch`, and any Kotlin function can throw a checked exception without declaring it. The JVM still enforces `throws` at the bytecode level for Java callers, but Kotlin callers bypass verification.

```
// Java
public class FileStore {
    public String read(String path) throws IOException {
        return Files.readString(Path.of(path));
    }
}
```

```
// Kotlin – no try/catch required, compiles fine
fun loadConfig(store: FileStore): String {
    return store.read("/etc/config.json") // IOException not caught –
    propagates as unchecked
}

// Kotlin throwing a checked exception without declaring it
fun failWithIo(): Nothing {
    throw IOException("disk full") // no @Throws needed for Kotlin
    callers
}
```

Consequences for distributed services:

- A Kotlin service calling a Java library that throws `InterruptedException`, `TimeoutException`, or `ExecutionException` will not be forced to handle interruption or timeout. An unhandled `InterruptedException` that propagates as an unchecked exception may bypass your thread-interruption protocol (Chapter 6, Section 7) and leave a thread in an inconsistent state. Always wrap Java calls that declare checked exceptions in explicit `try / catch` even though the compiler does not require it — or use `runCatching` with typed handling.
- When Kotlin code is called from Java, checked exceptions are invisible unless annotated with `@Throws`. A Java caller that expects `catch (IOException e)` will never catch the exception from a Kotlin function lacking `@Throws` — the exception still propagates (the JVM does not enforce `throws` at runtime), but the Java compiler will not

let you write the `catch` without `@Throws` on the declaration. Add `@Throws` to every Kotlin function that is part of a Java-facing API and can throw a checked exception.

```
@Throws(IOException::class, TimeoutException::class)
fun fetchRemote(url: String): String {
    // Java callers can now write catch (IOException | TimeoutException
    e)
}
```

6.3 `lateinit`, `by lazy`, and `Delegates.notNull`

Three mechanisms for deferred initialization, with different runtime characteristics:

```
class RequestHandler {
    // 1. lateinit – mutable, non-null, checked at access time
    lateinit var router: Router
    fun init(router: Router) { this.router = router }

    // 2. by lazy – immutable after first access, thread-safe by default
    val config: Config by lazy { loadConfig() } //
    LazyThreadSafetyMode.SYNCHRONIZED

    // 3. Delegates.notNull – for primitives where lateinit is illegal
    var retryCount: Int by Delegates.notNull()
}
```

Mechanism	JVM field type	Thread safety	Access before init
<code>lateinit var x: T</code>	<code>T</code> (nullable at JVM level, null sentinel)	None — racy	<code>UninitializedPropertyAccess</code>
<code>by lazy { ... }</code>	<code>Lazy<T></code> holder object	<code>SYNCHRONIZED</code> (default), <code>PUBLICATION</code> , or <code>NONE</code>	Blocks or computes on first access

Mechanism	JVM field type	Thread safety	Access before init
<code>Delegates.notNull<T>()</code>	T with null sentinel	None	<code>IllegalStateException</code>

`lateinit` is common in Spring/Quarkus/Ktor handlers where the framework constructs the object and then injects dependencies before handling requests. The hazard: if you access `router` from a coroutine launched during construction (before `init()`), you get `UninitializedPropertyAccessException` — a race that only manifests under startup ordering variations. Prefer constructor injection (`class RequestHandler(val router: Router)`) for new code; reserve `lateinit` for framework-mandated no-arg construction.

by `lazy` with `SYNCHRONIZED` uses double-checked locking on the `Lazy` instance — correct but contended if many coroutines hit the lazy property concurrently at startup. For request-scoped lazies, use `LazyThreadSafetyMode.NONE` or `PUBLICATION` to avoid the lock.

7. Coroutines — `suspend`, `Continuation`, and the state machine

7.1 What `suspend` means at the bytecode level

A `suspend` function is a function that can suspend without blocking its thread. The compiler implements this by adding a hidden `Continuation` parameter and rewriting the function body into a state machine.

```
// Kotlin source
suspend fun fetchUser(id: String): User {
    val cached = cache.get(id)           // suspension point 1 (if
cache is suspend)
    if (cached != null) return cached
    val remote = api.fetch(id)           // suspension point 2
    cache.put(id, remote)                 // suspension point 3 (if put
is suspend)
    return remote
}
```

For illustration, consider a simpler two-suspension function and its lowered form:

```
// Simplified – two suspension points
suspend fun compute(x: Int): String {
    val a = fetchA(x)    // suspend 1
    val b = fetchB(a)    // suspend 2
    return "result:$b"
}
```

The compiler generates a class equivalent to:

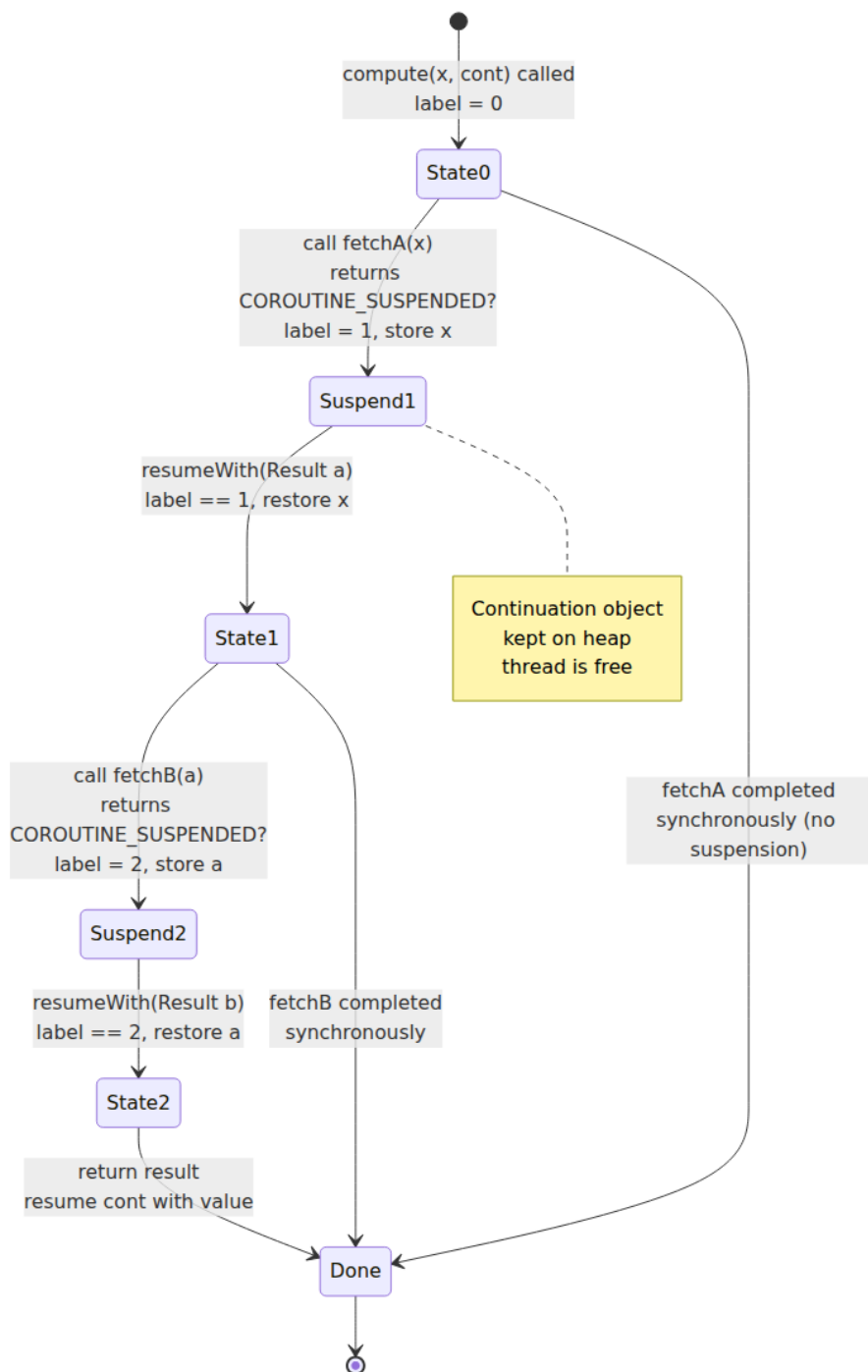
```
// Pseudo-code of generated state machine (simplified)
class ComputeContinuation(
    var x: Int,
    var a: String? = null,
    var b: String? = null,
    var label: Int = 0,                                // state index
    completion: Continuation<String>
) : Continuation<String> {
    override val context: CoroutineContext = completion.context
    override fun resumeWith(result: Result<String>) { /* dispatcher
resumes here */ }

    // The actual method the JVM sees:
    // Object compute(int x, Continuation<String> cont)
    // Returns either String (completed) or COROUTINE_SUSPENDED
    (suspended)
}
```

The JVM signature of `suspend fun compute(x: Int): String` becomes:

```
public static final java.lang.Object compute(int,
    kotlin.coroutines.Continuation<? super java.lang.String>);
```

It returns `Object` — either the real `String` result (if the function completed without suspending) or the singleton `kotlin.coroutines.intrinsics.CoroutineSingletons.COROUTINE_SUSPENDED` (if it suspended). The caller checks `if (result === COROUTINE_SUSPENDED)` `return COROUTINE_SUSPENDED` to propagate suspension up the stack. No thread is blocked; the current frame is simply returned and the continuation object holds the state.



7.2 Walking a real `javap -c -p` dump

Compile a minimal suspend function and inspect:

```
// File: Demo.kt
suspend fun hello(name: String): String {
    delay(10)
    return "hello $name"
}
```

```
kotlinc Demo.kt -d /tmp/demo -classpath kotlinc-coroutines-core.jar
javap -c -p /tmp/demo/DemoKt.class
```

Abbreviated output (comments added):


```
// The suspend function return note the Continuation parameter and Object
return
public static final java.lang.Object hello(java.lang.String, kotlin.coroutines.Continuation<? super java.lang.String>);
```

Code:

```
0: aload_1
1: instanceof
#2 // check if continuation is already our state machine
4: ifeq 30 // if not, create a new HelloContinuation
7: aload_1
8: checkcast #3 // cast to HelloContinuation
11: aload_1
12: getfield #4 // HelloContinuation.label:I
15: ldc #5 // int 0x80000000 (hash for suspension check)
17: iand
18: ifeq 30
// ... extract label, restore locals from continuation fields ...
30: aload_0 // name
31: aload_1 // continuation
32: ldc #6 // long 10
34: invokestatic #7 // Method kotlinx/coroutines/DelayKt.delay:
(JLkotlin/coroutines/Continuation;)Ljava/lang/Object;
37: dup
38: ldc #8 // CoroutineSingletons.COROUTINE_SUSPENDED
40: if_acmpne 46 // if result != COROUTINE_SUSPENDED contin
ue
43: areturn // suspended return COROUTINE_SUSPENDED
D to caller
46: checkcast #9 // cast result (Unit) – delay returns Unit
49: pop
50: new #10 // StringBuilder
53: dup
54: ldc #11 // "hello "
56: invokespecial #12 // StringBuilder.<init>
59: aload_0
60: invokevirtual #13 // StringBuilder.append
63: invokevirtual #14 // StringBuilder.toString
66: areturn
```

```
// The generated continuation class (inner class of DemoKt)
final class com.example.DemoKt$hello$1 extends kotlin.coroutines.jvm.internal.ContinuationImpl {
```

```
int label; // current state
java.lang.String L$0; // stored local: name
java.lang.Object result; // last suspension result
// ... constructor, invokeSuspend, etc.
public final java.lang.Object invokeSuspend(java.lang.Object);
```

Code:

```
0: aload_0
1: getfield #1 // label
```

```

    4: tableswitch  { // dispatch on label
        0: 28
        1: 56
        default: 80 // throw IllegalStateException("call to
'resume' before 'invoke'")
    }
    28: aload_1          // result from previous suspension (Unit
from delay)
    // ... resume logic, restore L$0, proceed to StringBuilder ...
    56: // label 1 resumed after delay
    // ... restore name from L$0, build "hello $name" ...
    80: new              #2 // IllegalStateException
}

```

Key observations:

- **label is the program counter.** Each suspension point increments `label`. On resume, `tableswitch` jumps to the right state. This is a classic compiler-generated state machine — the same pattern `javac` uses for `switch` on strings.
- **Locals that survive across suspension are spilled to fields** (`L$0`, `I$0`, etc.). `name` is stored in `L$0` before suspending at `delay` and restored after. Locals that are dead across suspension are not spilled — the compiler liveness analysis minimizes the continuation size.
- **COROUTINE_SUSPENDED is the sentinel.** Every suspend call is followed by `if_acmpne COROUTINE_SUSPENDED` — if the callee suspended, the caller immediately returns `COROUTINE_SUSPENDED` without executing further bytecode. Suspension propagates up the call stack as a return value, not an exception.
- **The continuation is allocated once** (or reused if the caller already passed a matching continuation). For a chain of `N` suspend calls, there are `N` continuation objects linked via `completion` — one per function in the call stack. Each is small (a few fields + `label`), typically 32–64 bytes. For deep call stacks (10+ suspend frames), this is a few hundred bytes per suspended coroutine — negligible compared to a platform thread's 1 MB stack, and the reason coroutines scale to millions.

7.3 Continuation interface and interception

```
public interface Continuation<in T> {
    public val context: CoroutineContext
    public fun resumeWith(result: Result<T>)
}

public interface CoroutineContext {
    public operator fun <E : Element> get(key: Key<E>): E?
    public fun <R> fold(initial: R, operation: (R, Element) -> R): R
    public operator fun plus(context: CoroutineContext): CoroutineContext
    public fun minusKey(key: Key<*>): CoroutineContext

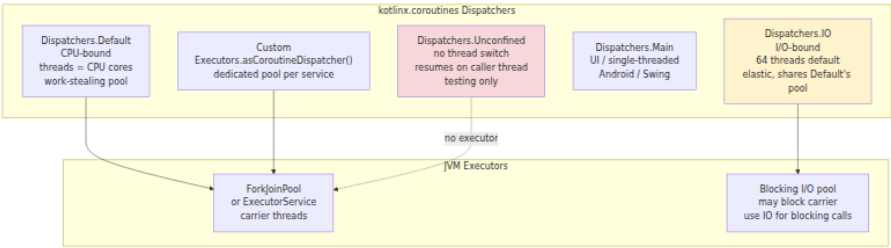
    public interface Key<E : Element>
    public interface Element : CoroutineContext { ... }
}
```

The `Continuation` is intercepted before it reaches the state machine. `ContinuationInterceptor` (the dispatcher) wraps the continuation's `resumeWith` to schedule resumption on the right thread pool. This is how `withContext(Dispatchers.IO) { ... }` moves execution — it does not move the running thread; it suspends, and the interceptor resumes the continuation on a different executor.

Debug tip: when a coroutine suspends, the stack trace shows `ContinuationImpl.resumeWith` and `DispatchedTask.run` — not your business logic. Use `kotlinx.coroutines.debug` (`-Dkotlinx.coroutines.debug`) to enable coroutine names in thread names (`coroutine#42 @coroutine#42`) and `DebugProbes.dumpCoroutines()` to see all suspended continuations with their creation stack traces. In production, the `kotlinx-coroutines-debug` artifact adds ~2% overhead — enable it only during incidents.

8. Dispatchers, structured concurrency, cancellation, and exception handling

8.1 Dispatcher topology



Concrete guidance for backend services:

Dispatcher	Backing pool	When use
Default	ForkJoinPool sized to Runtime.availableProcessors()	CPU w JSON parsin hashin compr busine logic
IO	Elastic pool, 64 threads default (tunable via <code>kotlinx.coroutines.io.parallelism</code>), shares threads with Default	Blocki JDBC, HTTP clients O
Unconfined	None — resumes on whatever thread called <code>resumeWith</code>	Tests, bench where thread not ma
Main	Single thread (Android main looper, Swing EDT)	UI onl

Dispatcher	Backing pool	When use
Custom	<code>Executors.newFixedThreadPool(n).asCoroutineDispatcher()</code>	Isolation, noisy dependencies (e.g., dedicated pool for slow downs

```
// Correct dispatcher selection in a service handler
class OrderService(
    private val db: Database, // blocking JDBC
    private val paymentClient: PaymentClient, // async Ktor client
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
    private val cpuDispatcher: CoroutineDispatcher = Dispatchers.Default
) {
    suspend fun placeOrder(req: PlaceOrderRequest): Order {
        // CPU-bound validation – stays on Default (caller's dispatcher)
        val validated = withContext(cpuDispatcher) { validate(req) }

        // Blocking I/O – explicitly on IO
        val order = withContext(ioDispatcher) { db.insert(validated) }

        // Non-blocking I/O – no dispatcher switch needed, suspends cooperatively
        val receipt = paymentClient.charge(order.id, order.amountCents)

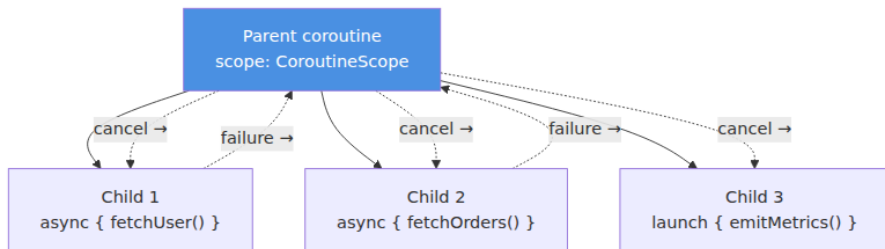
        // CPU-bound mapping – back on Default
        return withContext(cpuDispatcher) { toResponse(order, receipt) }
    }

    private fun validate(req: PlaceOrderRequest): ValidatedOrder { /
    * ... */ }
}
```

`withContext` is itself a suspend function that suspends, switches dispatcher via `ContinuationInterceptor`, executes the block, and switches back on return. The switch cost is one suspension + one dispatch — roughly 1-2 microseconds on a warm JVM. Avoid wrapping

single non-blocking calls in `withContext`; the switch overhead exceeds the work.

8.2 Structured concurrency — `coroutineScope` vs. `supervisorScope`



`coroutineScope` — all children must succeed; any child failure cancels the parent and all siblings. Use when children are subtasks of a single logical operation (scatter-gather where partial results are useless).

`supervisorScope` — children failures do not cancel siblings or parent; each child's exception is isolated. Use when children are independent operations where partial success is acceptable (fan-out to multiple downstreams, batch processing).

```

// coroutineScope - all-or-nothing
suspend fun loadDashboard(userId: String): Dashboard = coroutineScope {
    val user = async { userService.fetch(userId) }
    val orders = async { orderService.list(userId) }
    val prefs = async { prefService.fetch(userId) }
    // If any async fails, the other two are cancelled immediately.
    // The exception propagates to the caller; no partial Dashboard is
    returned.
    Dashboard(user.await(), orders.await(), prefs.await())
}

// supervisorScope - isolated failures
suspend fun notifyAll(userIds: List<String>, message: String): NotifyRe
sult = supervisorScope {
    val jobs = userIds.map { id ->
        async {
            try { notifier.send(id, message) }
            catch (e: Exception) { Failed(id, e) } // per-child
handling
        }
    }
    val results = jobs.awaitAll()
    NotifyResult(results.filterIsInstance<Success>(), results.filterIsI
nstance<Failed>())
    // One send failure does not cancel the other sends.
}

```

Bytecode note: `coroutineScope` and `supervisorScope` are `suspend` functions that create a `ScopeCoroutine` / `SupervisorCoroutine` subclass of `AbstractCoroutine` and install it as the `completion` of each child's continuation. Cancellation is cooperative — a child is cancelled by completing its `Job` with `CancellationException`, which causes the next suspension point to throw. Code between suspension points does not observe cancellation until it suspends or explicitly checks `ensureActive()` / `isActive`.

8.3 Cancellation — cooperative, exception-based, and structured

```
suspend fun pollWithTimeout(): String {
    return withTimeout(5000) {           // throws
        TimeoutCancellationException after 5s
        while (isActive) {             // explicit check in tight
            loop
            val result = fetch()         // suspend point –
            cancellation checked here
            if (result != null) return@withTimeout result
            delay(100)                   // suspend point – throws
            CancellationException if cancelled
        }
        error("unreachable")
    }
}

// Cancellation propagation in a service
class SearchService(private val scope: CoroutineScope) {
    fun search(query: String): Deferred<List<Result>> {
        return scope.async {
            // If scope is cancelled (server shutdown), this async is
            cancelled too.
            // withTimeout ensures we don't hang on a slow downstream.
            withTimeout(2000) {
                downstream.search(query)
            }
        }
    }
}
```

Rules every backend engineer must internalize:

1. **Cancellation is cooperative.** Only suspension points (`delay`, `await`, `withContext`, channel ops, `suspend` calls) and explicit `ensureActive()` / `yield()` checks throw `CancellationException`. A `while (true) { compute() }` loop with no suspension never cancels — it will run until the thread pool is exhausted or the process is killed. Always add `ensureActive()` or `yield()` in CPU-bound loops inside coroutines.
2. **`CancellationException` is special.** It is swallowed by `coroutineScope` (used for structured cancellation) and rethrown by `supervisorScope`. Never catch `CancellationException` in a generic `catch (e: Exception)` without rethrowing — doing so breaks

structured cancellation and leaks coroutines that should have been cancelled. The correct pattern:

```
kotlin try { doWork() } catch (e: CancellationException) { throw e } // always rethrow catch (e: IOException) { handleIoError(e) }
```

1. **withTimeout** vs. **withTimeoutOrNull**. **withTimeout** throws **TimeoutCancellationException** (subtype of **CancellationException**) on timeout. **withTimeoutOrNull** returns **null** instead. In request handlers, prefer **withTimeout** so timeouts propagate as cancellations that clean up resources; use **withTimeoutOrNull** only when **null** is a valid fallback (cache miss, optional enrichment).

2. **Resource cleanup in finally**. **finally** blocks run even after cancellation, but **suspend** calls inside **finally** need **withContext(NonCancellable)** — otherwise they immediately throw **CancellationException** again.

```
kotlin suspend fun useResource() { val conn = pool.acquire() // suspend try { conn.query("SELECT ...") } finally { withContext(NonCancellable) { pool.release(conn) // must not be cancelled } } }
```

8.4 Exception handling — `CoroutineExceptionHandler` and `async` vs. `launch`

```
// launch – exception propagates to parent, handled by
// CoroutineExceptionHandler
val handler = CoroutineExceptionHandler { _, ex ->
    logger.error("Uncaught coroutine exception", ex)
    metrics.increment("coroutine.uncaught")
}
scope.launch(handler) {
    throw IOException("downstream failed") // delivered to handler if
    uncaught
}

// async – exception is captured in Deferred, thrown on await()
val deferred: Deferred<String> = scope.async {
    throw IOException("downstream failed") // stored, not propagated
yet
}
try {
    deferred.await() // throws IOException here
} catch (e: IOException) {
    handleError(e)
}
```

Key distinction: `launch` is fire-and-forget — uncaught exceptions go to the `CoroutineExceptionHandler` (or to the uncaught exception handler if none is installed, which by default logs and may crash the process in some coroutine builders). `async` captures the exception for the awaiter — if you never `await`, the exception is silently lost until the `Deferred` is GC'd, at which point `CoroutineExceptionHandler` is invoked as a last resort. **Always await every async** — an un-awaited `async` is a structured-concurrency violation that leaks exceptions.

For top-level service scopes (e.g., `GlobalScope` is forbidden — use a `CoroutineScope(SupervisorJob() + Dispatchers.Default)` tied to the server lifecycle), install a `CoroutineExceptionHandler` that routes to your centralized error reporting (Sentry, Datadog, etc.) and never swallows.

9. Compiler intrinsics — `inline`, `reified`, `noinline` / `crossinline`, and `contracts`

9.1 `inline` — compile-time inlining, not JIT inlining

Kotlin's `inline` is distinct from the JIT's inlining (Chapter 5). The Kotlin compiler copies the function body into each call site at compile time, eliminating the call overhead and — crucially — the lambda allocation.

```
// Without inline – lambda is an object allocation per call
fun <T> measure(block: () -> T): T {
    val start = System.nanoTime()
    val result = block()           // invokeinterface Function0.invoke
    println("took ${System.nanoTime() - start} ns")
    return result
}

// With inline – no allocation, body is copied
inline fun <T> measureInline(block: () -> T): T {
    val start = System.nanoTime()
    val result = block()           // inlined – direct bytecode
    println("took ${System.nanoTime() - start} ns")
    return result
}
```

Call site comparison:

```
// Call site
val x = measure { computeExpensively() }
val y = measureInline { computeExpensively() }
```

Bytecode without `inline`:

```
// measure – lambda object created via invokedynamic
invokedynamic #2, 0 // InvokeDynamic #0:invoke:()Lkotlin/jvm/
functions/Function0;
invokestatic #3 // Method measure:(Lkotlin/jvm/functions/
Function0;)Ljava/lang/Object;

// The lambda body is a separate synthetic method:
private static final java.lang.Object computeExpensively$lambda$0();
```

Bytecode with `inline`:

```
// No invokedynamic, no Function0 allocation – computeExpensively
inlined directly
invokestatic #4 // Method computeExpensively:()Ljava/lang/Object;
// measureInline body (System.nanoTime, println) also inlined around
it
```

In a hot path that calls a higher-order function millions of times per second (e.g., `List.map`, `withLock`, `use`), the allocation pressure from non-inline lambdas can dominate GC. `inline` eliminates that pressure entirely. The cost: larger bytecode at each call site (code bloat), and the inlined function's bytecode is duplicated — dex/method-count and JIT code-cache impact for very large inlined functions. Rule of thumb: inline small higher-order functions (< 30 bytecode instructions, single lambda parameter); do not inline large functions or those called from many sites.

9.2 reified — defeating type erasure at compile time

The JVM erases generic type parameters: `List<String>` and `List<Int>` are both `List` at runtime. `reified` recovers the type by inlining it as a concrete class literal at each call site.

```
// Without reified – cannot check generic type at runtime
fun <T> isStringList(list: List<T>): Boolean {
    // return list is List<String> // COMPILE ERROR: cannot check for
    // erased type
    return false
}

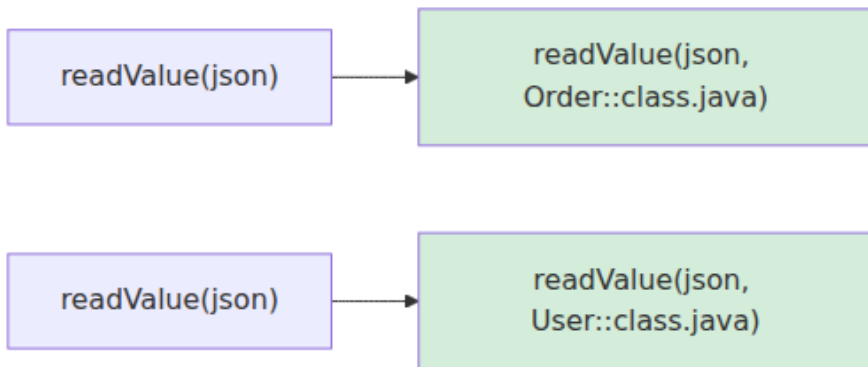
// With reified – type is available as a class literal
inline fun <reified T> List<*>.isInstanceOf(): Boolean {
    return this is List<T> // actually checks: list is List &&
    // elements are T (with caveats)
}

inline fun <reified T> Any.isA(): Boolean = this is T

// Practical – type-safe deserialization without Class<T> parameter
inline fun <reified T : Any> ObjectMapper.readValue(json: String): T {
    return readValue(json, T::class.java) // T::class.java (or
    // `typeOf<T>()` for full generic type) inlined as concrete class
}

// Usage – no Class parameter needed at call site
val order: Order = mapper.readValue("""{"id": "123", "amountCents": 999}""")
// Inlined to: mapper.readValue(json, Order.class)
```

At each call site the compiler expands the reified type to a concrete class literal:



Constraints on `reified`:

- Only `inline` functions can have `reified` parameters — the type must be inlined at the call site to become a concrete class literal.
- `reified` does not reify nested generics: `inline fun <reified T> check(list: List<T>) can check list is List<*> and T::class, but list is List<String> still erases to list is List<*> at runtime — element-type checks require iterating.`
- Overusing `reified` for large functions bloats every call site. For service code, the sweet spot is small utility functions (JSON parsing, type-safe config lookup, metrics tagging) where the `Class<T>` parameter would otherwise be boilerplate.

```
// Type-safe config lookup with reified – eliminates Class<T> threading
inline fun <reified T : Any> Config.get(key: String): T {
    val raw = getString(key)
    return when (T::class) {
        String::class -> raw as T
        Int::class -> raw.toInt() as T
        Long::class -> raw.toLong() as T
        Boolean::class -> raw.toBoolean() as T
        else -> objectMapper.readValue<T>(raw) // nested
    }
}

reified
}

val port: Int = config.get("server.port") // T = Int – no
Class<Int> needed
val feature: FeatureFlags =
config.get("flags") // T = FeatureFlags – deserialized
```

9.3 `noinline` and `crossinline` — controlling inline scope

```
inline fun withRetry(
    crossinline block: () -> Unit, // can be called from nested
    lambda, not from bare return
    noinline onError: (Exception) -> Unit // not inlined – stored as
    Function object
) {
    var lastError: Exception? = null
    repeat(3) {
        try {
            block() // inlined – but crossinline
            // prevents non-local return
            return // return from withRetry, not
            // from block
        } catch (e: Exception) {
            lastError = e
        }
    }
    lastError?.let { onError(it) } // onError is a real object –
    // can be stored, passed
}
```

Modifier	Meaning	Bytecode effect
(default) <code>inline</code> param	Body copied, non-local <code>return</code> allowed	No <code>Function</code> object, direct bytecode

Modifier	Meaning	Bytecode effect
<code>crossinline</code>	Body copied, non-local <code>return</code> forbidden	Prevents <code>return</code> inside <code>block</code> from returning out of enclosing function — required when <code>block</code> is called inside another lambda/object
<code>noinline</code>	Not copied, remains a <code>Function</code> object	<code>invokedynamic</code> + <code>Function</code> allocation; allows storing/passing the lambda

Use `crossinline` whenever the inline lambda is invoked inside another lambda, anonymous object, or local function — otherwise a `return` inside the lambda would try to return from the outer function across an incompatible stack frame, which the compiler correctly forbids. Use `noinline` when you need to store the lambda, pass it to a non-inline function, or conditionally execute it.

9.4 Contract DSL — telling the compiler what your function guarantees

Contracts are compiler intrinsics that let you teach the type checker that a function establishes a condition. They have no runtime effect — they are pure type-system hints consumed during compilation.

```

import kotlin.contracts.ExperimentalContracts
import kotlin.contracts.contract

@OptIn(ExperimentalContracts::class)
fun requireNonNull(value: String?) {
    contract { returns() implies (value != null) }
    if (value == null) throw IllegalArgumentException("value must not
be null")
}

@OptIn(ExperimentalContracts::class)
fun <T> Result<T>.isSuccess(): Boolean {
    contract { returns(true) implies (this@isSuccess is Success<T>) }
    return this is Success<T>
}

// Usage – smart-cast after contract
fun handle(input: String?) {
    requireNonNull(input)
    println(input.length) // smart-cast to String – no ?. or !! needed
}

fun process(result: Result<Order>) {
    if (result.isSuccess()) {
        println(result.value) // smart-cast to Success<Order>
    }
}

```

Available contract effects:

Contract	Meaning	Example
<code>returns() implies (x != null)</code>	If function returns normally, <code>x</code> is non-null	<code>requireNotNull</code> , <code>checkNotNull</code>
<code>returns(true) implies (x != null)</code>	If function returns <code>true</code> , <code>x</code> is non-null	<code>isNotNull()</code> , <code>isSuccess()</code>
<code>returns(false) implies (x == null)</code>	If function returns <code>false</code> , <code>x</code> is null	Custom null-guard
<code>returns(null) implies (x == null)</code>	If function returns <code>null</code> , <code>x</code> is null	Lookup functions

Contract	Meaning	Example
<code>callsInPlace(block, EXACTLY_ONCE)</code>	Lambda is invoked exactly once, in place	<code>withLock, use, let</code>

The `stdlib` already declares contracts for `let`, `run`, `apply`, `also`, `use`, `with`, `check`, `require`, `error`, and `TODO`. Custom contracts are `ExperimentalContracts` — the API has been stable since Kotlin 1.3 but the annotation is still required. For service code, the highest-value custom contract is a `requireAuthenticated(user: User?)` guard that implies `user != null` on return, eliminating `!!` at every downstream use of `user`.

Contracts are erased at compile time — no bytecode, no runtime check beyond what you wrote. The `contract { ... }` block must be the first statement and is not executed; it is parsed by the compiler and discarded.

10. Distributed-systems lens — Kotlin services at scale

Every abstraction in this chapter has a distributed-systems consequence. A single service's bytecode choices amplify across hundreds of instances.

10.1 Serialization and wire format

`data class` and `value class` define your wire types. When `Order` crosses a gRPC or Kafka boundary, its `equals`/`hashCode`/`toString` behavior determines deduplication, partitioning, and log readability. A `value class UserId(String)` that is `toString()`'d as `"UserId(raw=abc)"` in logs but serialized as `"abc"` on the wire creates a mismatch that breaks log-based debugging. Override `toString` on value classes to return the raw value when they appear in structured logs, or configure your JSON mapper to unwrap value classes (`@JsonValue` on the `raw` property, or `jackson-module-kotlin` with `KotlinFeature.StrictNullChecks`).

Protobuf-generated Kotlin code (`protoc-gen-kotlin` DSL) already maps `optional` to nullable `T?` and `repeated` to `List<T>` — but the generated `copy` and `equals` are Java-style (field-by-field with presence checks), not Kotlin `data class` semantics. Do not mix generated proto types with Kotlin `data class` equality assumptions in map keys.

10.2 Null-safety as a reliability boundary

In a polyglot environment (Kotlin services calling Java libraries, Java services calling Kotlin APIs), every RPC and database boundary is a platform-type boundary. Adopt a team rule: **all Java code that is called from Kotlin is annotated with JSpecify; all Kotlin APIs exposed to Java carry @JvmName / @JvmStatic / @Throws as needed.** Enforce this with ArchUnit or Detekt rules that fail the build on unannotated public Java methods in shared modules. The cost of a `KotlinNullPointerException` in a payment path far exceeds the annotation overhead.

Measure `KotlinNullPointerException` separately from `NullPointerException` in your error budget. A spike in `KotlinNullPointerException` after a Kotlin migration or a new Java dependency usually means a platform-type escape — the fix is an annotation, not a `!!` removal.

10.3 Coroutine dispatchers and capacity planning

Dispatcher choice is capacity planning. `Dispatchers.Default` is sized to CPU cores — if you block it, you reduce the effective CPU capacity of the entire service to zero, regardless of how many pods Kubernetes has scheduled. `Dispatchers.IO` is elastic but still creates threads — 10K concurrent JDBC queries on `Dispatchers.IO` creates 10K threads, each with an 8 MB stack (JVM default `-Xss`), consuming 80 GB of virtual memory across the fleet.

For I/O-bound services, the scaling path is:

1. **Short term:** `Dispatchers.IO` with tuned `kotlinx.coroutines.io.parallelism` (default 64 — increase only with load testing; each thread is a real OS thread).
2. **Medium term:** Replace blocking clients (JDBC → R2DBC, `URLConnection` → Ktor CIO/OkHttp async) so coroutines suspend without holding a thread — then `Dispatchers.Default` suffices.
3. **Long term:** Project Loom virtual threads (Chapter 6, Section 6) as the carrier for `Dispatchers.IO` — early prototypes show `Dispatchers.IO` backed by `Executors.newVirtualThreadPerTaskExecutor()` reduces thread count to near-zero for I/O workloads, but pinning (`synchronized` , JNI) must be eliminated first.

Structure your service's `CoroutineScope` hierarchy to mirror your deployment topology: one `SupervisorJob`-backed scope per server lifecycle, one `coroutineScope` per request, and `supervisorScope` only for fan-out where partial failure is acceptable. Cancel the server scope on `SIGTERM` (Kubernetes pod termination) and let structured cancellation drain in-flight requests — this is the coroutine equivalent of graceful shutdown.

10.4 Inline, reified, and binary size

`inline` + `reified` bloat is a fleet-wide cost: larger JARs, slower class loading, higher JIT code-cache pressure, and larger container images. For a service with 500 call sites of an `inline fun <reified T> parseJson`, the `parseJson` body is duplicated 500 times in bytecode. If the body is 100 bytes, that is 50 KB of extra bytecode — trivial. If it is 2 KB (complex deserialization with error handling), that is 1 MB — still trivial for a single service, but multiplied across 200 services it adds up in artifact storage and pull time. Keep `inline` functions small; extract non-generic logic into a private non-inline helper.

Key takeaways

- A Kotlin property is a field plus `get / set` methods; `@JvmField` breaks that to a public field, `lateinit` uses a null sentinel, and computed properties (`get() = ...`) have no field at all. Frameworks that reflect over fields vs. methods see different views — use `kotlin-reflect` or `jackson-module-kotlin` to reconcile.
- `data class` generates `equals / hashCode / toString / copy / componentN`; `copy` uses a bitmask for defaults, destructuring is positional, and `equals` is structural — all consequences for map keys, sets, and API evolution.
- `companion object` is a singleton holder; `@JvmStatic` adds a static forwarder for Java. `object` singletons use class-loading for initialization safety and generate `INSTANCE` — verify `readResolve` if you serialize them.
- `value class` erases to its underlying type with name mangling (`findUser-Yo8aak`); Java callers see the mangled name, and generic/nullable usage boxes. Ideal for domain IDs at API boundaries, not for large collections.

- Extension functions are static methods with a `$this$` receiver parameter — no dynamic dispatch, no polymorphism, lexical resolution only. File-level extensions live in `*Kt` classes.
- SAM conversion uses `invokedynamic` + `LambdaMetafactory` — no anonymous class files, lazy instantiation, shared factories. Overloaded SAM methods require explicit `Runnable { ... }` constructors to disambiguate.
- Kotlin null-safety is a type-system guarantee that degrades to platform types (`T!`) at every Java boundary. Annotate Java with `JSpecify @Nullable / @NonNull`, treat `!!` as a code smell on network/deserialized data, and monitor `KotlinNullPointerException` separately.
- `?.`, `?:`, `!!`, and `?.let` compile to `ifnull` branches, `Intrinsics.throwNpe`, and synthetic lambdas respectively — each branch is a predictable JIT speculation point.
- Kotlin swallows Java checked exceptions; add `@Throws` for Java callers and always handle `InterruptedException / TimeoutException` explicitly even though the compiler does not require it.
- `lateinit` is for framework injection with runtime init checks; `by lazy` is for expensive singletons with configurable thread safety; `Delegates.notNull` is for primitives. Prefer constructor injection over all three for new service code.
- `suspend` adds a `Continuation` parameter and returns `Object (T or COROUTINE_SUSPENDED)`; the compiler generates a state-machine class with a `label dispatch (tableswitch)` and spills live locals to fields. Suspension propagates as a return value, not an exception.
- `ContinuationInterceptor` (the dispatcher) wraps `resumeWith` — `withContext` suspends and resumes on a different executor. `Dispatchers.Default` is CPU-sized `ForkJoinPool`, `Dispatchers.IO` is elastic (64 threads), `Unconfined` is test-only. Blocking `Default` starves all CPU work.
- Structured concurrency: `coroutineScope` cancels all children on any failure (all-or-nothing); `supervisorScope` isolates failures (partial success). `async` captures exceptions for `await`; `launch` routes to `CoroutineExceptionHandler`. Never leak an un-awaited `async`.
- Cancellation is cooperative — only suspension points and `ensureActive() / yield()` throw `CancellationException`. Never

swallow `CancellationException` without rethrowing; use `withContext(NonCancellable)` for cleanup in `finally`.

- `inline` copies the body at call sites and eliminates lambda allocation; `reified` survives erasure by inlining the concrete class literal. `crossinline` forbids non-local `return` when the lambda is called from a nested scope; `noinline` keeps the lambda as a `Function` object.
- `contract` DSL teaches the type checker that a function establishes a condition (`returns()` implies `(x != null)`, `callsInPlace`) — no runtime effect, erased after compilation. Use for `requireAuthenticated`-style guards that enable smart-casts.

Further reading

- Kotlin Language Specification — Kotlin Compiler Team, <https://kotlinlang.org/spec/> — authoritative source for lowering, type system, and coroutines design.
- Kotlin Coroutines Guide — JetBrains, <https://kotlinlang.org/docs/coroutines-guide.html> — structured concurrency, cancellation, dispatchers, and debugging.
- *Kotlin Coroutines Deep Dive* — Marcin Moskała, <https://kt.academy> — detailed walkthrough of continuation internals, state machines, and dispatcher implementation.
- JSpecify 1.0 Specification — <https://jspecify.dev/docs/spec/> — nullness annotations recognized by `kotlin`, `Error Prone`, and `NullAway`.
- JSR-305 Annotations (`javax.annotation`) — <https://github.com/findbugsproject/findbugs> — historical nullability annotations; superseded by JSpecify but still encountered.
- JEP 425: Virtual Threads (Preview) / JEP 444: Virtual Threads (Final, JDK 21) — <https://openjdk.org/jeps/444> — Loom carrier-thread model underlying `Dispatchers.IO` evolution.
- *Java Concurrency in Practice* — Goetz et al., Addison-Wesley — monitors, safe publication, and executor topology that coroutine dispatchers build upon.
- Kotlin Metadata and Reflection — <https://kotlinlang.org/docs/reflection.html> and `kotlinx-metadata-jvm` — how `kotlin.Metadata` preserves Kotlin signatures for tooling.

- `javap` and `java.lang.invoke.LambdaMetafactory` — JDK Tool Specifications, <https://docs.oracle.com/en/java/javase/21/docs/specs/man/javap.html> — inspecting SAM/lambda lowering and `invokedynamic` bootstraps.
- Detekt and ArchUnit — <https://detekt.dev>, <https://www.archunit.org> — static enforcement of JSpecify coverage and `@Throws` / `@JvmStatic` conventions in mixed codebases.

Chapter 8 — The Kotlin Type System, Generics, and Reified Types

What this chapter covers. Kotlin's type system is not Java's type system with nicer syntax. It replaces the Java model — erased generics with use-site wildcards, nullable-by-default references, and primitive/object duality — with a stricter, more expressive lattice: non-nullable types by default, declaration-site variance, platform types at the Java boundary, flexible and intersection types in the compiler internals, star projections for erased generics, reified generics that recover erased type information via inlining, contracts that lift flow typing into the type checker, and value classes that erase wrappers without boxing. Understanding this system is not academic: every Kotlin microservice you deploy interoperates with Java bytecode at the generic-signature level, every JSON serializer relies on reified types or reflective `KType` tokens, and every `NullPointerException` you thought Kotlin eliminated can still surface through a platform type.

Learning goals — after this chapter you should be able to:

- Explain Kotlin's type lattice: non-nullable `T` vs nullable `T?`, platform type `T!`, flexible type `(A..B)`, intersection type, and star projection `*`, and where each arises in compilation and interop.
- Compare declaration-site variance (`in` / `out`) with Java use-site variance (`? extends` / `? super`), apply PECS correctly in both languages, and predict the variance matrix for any generic declaration.
- Describe type erasure on the JVM (what is erased, what survives in `Signature` attributes, and why `instanceof` / `is` checks on bare

generics fail) and how `inline` + `reified` recovers type information at call sites.

- Use `reified T`, `typeOf<T>()`, `KType / KClass`, and `serializer<T>()` idioms to build type-safe serialization, DI, and routing without passing explicit `Class<T>` tokens.
- Predict how Kotlin generics map to Java wildcards at the bytecode boundary and how star projections handle Java raw types and unbounded wildcards.
- Model null-safety as a type-system property: smart casts, flow typing, and `kotlin.contracts` for custom null checks.
- Evaluate value/inline classes, unsigned types, and the primitive specialization problem — when wrappers are erased, when they box, and what that means for allocation pressure on hot paths.
- Design delegated properties (by `lazy`, by `observable`, custom delegates) and understand their desugaring to `KProperty` + `getValue / setValue` operators.

Prerequisites. Volume 18, Chapter 1 (JVM architecture) for classfiles and `Signature` attributes; Chapter 2 (class loading) for verification; Chapter 7 (Kotlin interop) for null-safety and coroutine lowering. Familiarity with Java generics and bytecode generics signatures is assumed.

1. The Kotlin type lattice — more than `T` and `T?`

Java has one reference type hierarchy rooted at `Object` where `null` inhabits every reference type. Kotlin splits that hierarchy. Every type `T` implicitly defines a supertype `T?` that includes `null`, and the compiler enforces the distinction at every assignment, call, and return. This section maps the full lattice, including the types that only appear at boundaries and inside the compiler.

1.1 Non-nullable, nullable, and the `Nothing` bottom

In Kotlin, `String` means "a `String` that is definitely not null." `String?` means "a `String` or `null`." They are distinct types with a subtyping relation:

- `String <: String?` — every non-null `String` is a valid `String?`.

- `Nothing <: String` — `Nothing` is the bottom type, a subtype of every type. It has no values. Functions that never return (`throw` , infinite loop, `TODO()`) return `Nothing` , which lets the type checker treat code after such a call as unreachable.
- `Any` is the top of the non-nullable hierarchy (`Any?` is the true top including `null`).
- `Unit` is Kotlin's `void` — a proper type with a single value, not a keyword.

```
fun requireNonNull(s: String): Int = s.length           // s is String
- .length is safe
fun acceptNullable(s: String?): Int? = s?.length        // must use ?.
or !! or smart cast

val a: String? = "hello"                               // String <: String? - widening, always
safe                                                    // val b: String = a           // ERROR: String? is not a subtype of
String

fun fail(): Nothing = throw IllegalArgumentException("never returns")
val x: String = fail()                                 // OK: Nothing <: String - unreachable
assignment typechecks
```

The compiler rejects `String? → String` narrowing without an explicit check. This is not a lint — it is a type error, enforced before bytecode emission. The bytecode itself has no notion of nullability; `String` and `String?` both compile to `Ljava/lang/String;` in descriptors. Nullability is tracked in Kotlin metadata (`@Metadata` annotation) and in the type-checker, not in the JVM verifier. That gap is precisely where platform types enter.

1.2 Platform types — `T!` at the Java boundary

When Kotlin calls Java, the Java declaration has no nullability annotation (or has an ambiguous one). The Kotlin compiler cannot decide whether the Java type is `T` or `T?`, so it assigns a *platform type* denoted `T!` — a flexible type that behaves as both.


```

// Java:
// public class JavaRepo {
//     public String findName(int id) { return ...; }           // no
//     public @Nullable String findNick(int id) { return ...; }
//     public @NotNull String findTitle(int id) { return ...; }
// }

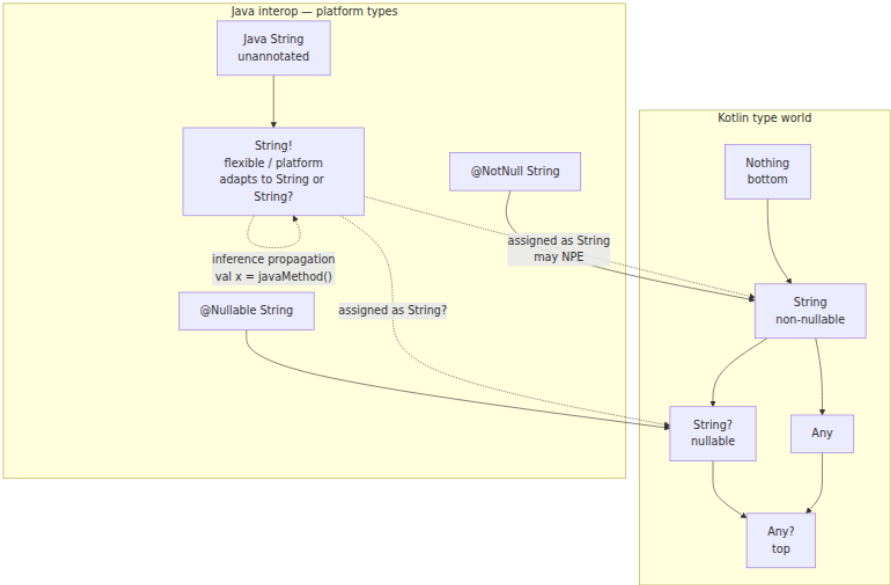
val a = JavaRepo().findName(42)    // type is String! – platform type
val b: String = a                  // compiles – String! adapts to
String                             String
val c: String? =
a                                  // also compiles – String! adapts to String?
println(a.length)                 // compiles but may NPE at runtime
if Java returned null

val nick: String? = JavaRepo().findNick(42)    // @Nullable → String?
(honored)
val title: String = JavaRepo().findTitle(42)   // @NotNull → String
(honored)

```

Platform types are invisible in source — you never write `String!`. They appear only in tooling (IDE hints, error messages) and in the compiler's internal type representation. They propagate through inference: if `a` is `String!` and you write `val d = a`, then `d` is also `String!`. They collapse only when assigned to a definite Kotlin type (`String` or `String?`) or when used in a context that demands one.

Support for JSR-305 (`@Nullable` / `@NotNull`), JetBrains `@Nullable` / `@NotNull`, and AndroidX annotations determines whether a Java type maps to `T` / `T?` or remains `T!`. Without annotations, Kotlin must assume the worst — and it is the caller's responsibility to handle `null`. In distributed systems, this matters at every service boundary where Kotlin calls a Java library (gRPC stubs, JDBC drivers, Jackson deserialization): an unannotated Java getter returning `null` will slip past the Kotlin type checker via `T!` and surface as an NPE far from the call site.



1.3 Flexible, intersection, and other internal types

Beyond `T / T? / T!`, the Kotlin compiler uses several types that rarely appear in source but explain error messages and bytecode:

Internal type	Notation	Where it arises
Flexible type	<code>(A..B)</code>	Platform types are flexible types where <code>A = T</code> , <code>B = T?</code> . Also used for Java wildcard capture. <code>MutableList<String!></code> is really <code>MutableList<(String..String?)></code> .
Intersection type	<code>T & Any</code> / <code>T where T : A, T : B</code>	<code>T & Any</code> denotes "T intersected with Any" — used internally to express <code>T</code> with a non-null upper bound after smart cast. Multiple upper bounds <code>where T : Closeable, T : Appendable</code> is an intersection.

Internal type	Notation	Where it arises
Star projection	<code>List<*></code>	Existential type — "a <code>List</code> of some unknown type." Not the same as <code>List<Any?></code> or <code>List<Any></code> . Discussed in §4.
Definite non-nullable	<code>T & Any</code>	After <code>if (x != null)</code> , <code>x</code> is smart-cast from <code>T?</code> to <code>T & Any</code> internally, asserting non-null without changing the generic argument.
Captured type	<code>capture(List<out String>)</code>	Result of capturing a wildcard for type inference — internal only, surfaces in error messages like "captured type of <code>out String</code> ".
Type parameter with nullable bound	<code>T : Any?</code>	Default bound is <code>Any?</code> . <code>T : Any</code> constrains <code>T</code> to non-nullable types only — affects what you can do with <code>T?</code> inside the generic.

```
// Intersection type – multiple upper bounds
fun <T> closeAndAppend(x: T) where T : AutoCloseable, T : Appendable {
    x.append("closing") // T has both interfaces
    x.close()
}

// Flexible type visible in error message:
// fun process(list: MutableList<String>) { ... }
// Java call: process(javaList) where javaList is MutableList<String!>
// If signatures mismatch, error shows: "required MutableList<String>
// found MutableList<(String..String?)>"

// Definite non-null after smart cast:
fun <T> handle(x: T?) {
    if (x != null) {
        // x is now T & Any – non-null T, not T? – so x.hashCode() is
        safe
        println(x.hashCode())
    }
}
```

1.4 Where types live — descriptors, signatures, metadata

On the JVM, types are encoded in two places:

1. **Bytecode descriptors** — erased types used by the verifier.
`List<String>` and `List<Int>` both have descriptor `Ljava/util/List;`. Nullability is invisible here.
2. **Generic Signature attribute** — preserves generic arguments as strings like `Ljava/util/List<Ljava/lang/String>;` for reflection and compiler use, but erased at runtime for `instanceof` and overload resolution.
3. **Kotlin `@Metadata` annotation** — stores the full Kotlin type information (nullability, variance modifiers, suspend markers, value class info) as a protobuf blob. The Kotlin compiler and `kotlin-reflect` read this; `javac` and the JVM verifier ignore it.

```
# Inspect the three layers
kotlinc Example.kt -d example.jar
javap -v -p example.jar | grep -A2 "Signature\|
RuntimeVisibleAnnotations"
# Signature: L // erased descriptor
# Kotlin Metadata: d1 = { ... } // full Kotlin types

# Kotlin's view (with kotlinc's -Xprint-enhanced):
# fun process(items: List<String>): String? - Kotlin sees
List<String>, nullable return
# Bytecode descriptor: (Ljava/util/List;)Ljava/lang/String;
# Signature attribute: (Ljava/util/List<Ljava/lang/String>;;)Ljava/
lang/String;
```

This three-layer encoding explains why Kotlin null-safety is a compile-time guarantee, not a runtime invariant: once compiled, `String` and `String?` are the same bytecode type, and only `@Metadata` + Kotlin-aware tooling can distinguish them.

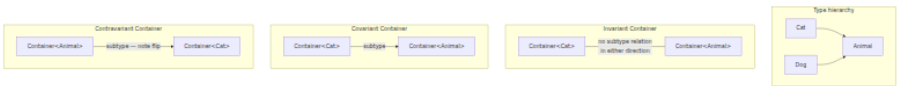
2. Variance — `in / out` vs `? extends / ? super` and PECS

Variance answers one question: if `Cat <: Animal`, is `Box<Cat> <: Box<Animal>`? In Java and Kotlin, the answer depends on how the generic type parameter is declared and used.

2.1 The three variances

For a generic `Container<T>` :

- **Invariant** (default in both languages): `Container<Cat>` and `Container<Animal>` are unrelated, regardless of `Cat <: Animal` . You can both read `T` and write `T` , so neither direction is safe to coerce.
- **Covariant** (`out T` in Kotlin, `? extends T` in Java): `Container<Cat> <: Container<Animal>` . You can read `T` values (they are at least `Animal`), but you cannot safely write `T` (you might put a `Dog` into a `Container<Cat>`).
- **Contravariant** (`in T` in Kotlin, `? super T` in Java): `Container<Animal> <: Container<Cat>` — the subtype relation flips. You can write `T` values (any `Cat` is an `Animal`), but reading yields only `Any? / Object` .



2.2 Declaration-site vs use-site variance

This is the central design difference between Kotlin and Java:

Aspect	Kotlin	Java
Default	Invariant (<code>class Box<T></code>)	Invariant (<code>class Box<T></code>)
Covariance	Declaration-site: <code>class Box<out T></code>	Use-site: <code>Box<? extends T></code>
Contravariance	Declaration-site: <code>class Box<in T></code>	Use-site: <code>Box<? super T></code>
Enforcement	Compiler checks that <code>out T</code> only appears in output positions, <code>in T</code> only in input positions	Compiler checks each wildcard use site
Flexibility	One declaration fixes variance for all uses; use-site <code>out</code> projection (<code>Box<out T></code>) available as override	Every variable/ method can choose its own wildcard

Kotlin's philosophy is that most types have a natural variance — a `Producer<T>` naturally produces `T` (covariant), a `Consumer<T>` naturally

consumes `T` (contravariant), and a `MutableList<T>` is naturally invariant because it does both. Declaring variance once at the class level eliminates the wildcard noise that pervades Java APIs.

```
// Kotlin: declaration-site variance – declared once, enforced everywhere
interface Producer<out T> {
    fun produce(): T
    // fun consume(value: T) // ERROR: Cannot use 'T' as in-parameter
    // when declared as 'out'
}

interface Consumer<in T> {
    fun consume(value: T)
    // fun produce(): T // ERROR: Cannot use 'T' as out-return when
    // declared as 'in'
}

class MutableBox<T>(var value: T) // invariant – both in and out
    positions

// Covariant assignment – safe because Producer only outputs T
val catProducer: Producer<Cat> = CatProducer()
val animalProducer: Producer<Animal> = catProducer // OK:
    Producer<Cat> <: Producer<Animal>

fun feedAll(consumer: Consumer<Animal>) {
    consumer.consume(Cat()) // Cat is an Animal
}
val catConsumer: Consumer<Cat> = feedAll as Consumer<Cat> //
    Consumer<Animal> <: Consumer<Cat>
```

```
// Java: use-site variance – chosen at every use
interface Producer<T> {
    T produce();
}

Producer<Cat> catProducer = () -> new Cat();
Producer<? extends Animal> animalProducer = catProducer; // wildcard
    needed every time

void feedAll(Consumer<? super Cat> consumer) {
    consumer.consume(new Cat());
}
```

2.3 Variance position rules

The compiler enforces position rules to guarantee soundness:

```

class Box<out T>(private val value: T) {
    fun get(): T = value           // OK: T in out-position (return
type)
    // fun set(value: T) {}        // ERROR: T in in-position
    // var stored: T               // ERROR: var needs both get and
set
    val snapshot: T get() = value // OK: val is out-only
}

class Sink<in T> {
    fun put(value: T) {}           // OK: T in in-position
(parameter)
    // fun get(): T {}            // ERROR: T in out-position
}

// Invariant class – T appears in both positions, so no variance
annotation allowed
class MutableBox<T>(var value: T) { // OK: invariant allows both
    fun get(): T = value
    fun set(value: T) { this.value = value }
}

```

When a declaration-site variant type is used in the "wrong" position, Kotlin projects it:

```

class Box<out T>(val value: T)

// T is out – when you try to use Box<String> as a consumer, T is
projected to Nothing:
fun wrongUse(box: Box<String>) {
    // box is Producer-like – you can read String out, but never put
String in
}

// For invariant types, Kotlin offers use-site projection as an escape
hatch:
fun copy(from: List<out Any>, to: MutableList<in Any>) {
    // List<out Any> – covariant projection, read-only view
    // MutableList<in Any> – contravariant projection, write-only view
    for (item in from) to.add(item)
}

```

2.4 The variance matrix

Every combination of declaration-site variance and use-site projection has a defined meaning. This matrix is the reference for predicting subtype relations:

Declaration	Use-site projection	Effective variance	Box<Cat> vs Box<Animal>	Read T as	Write T
class Box<T> (invariant)	Box<T>	Invariant	Unrelated	T	T
class Box<T>	Box<out T>	Covariant projection	Box<Cat> <: Box<Animal>	T (as Animal)	Nothing (forbidden)
class Box<T>	Box<in T>	Contravariant projection	Box<Animal> <: Box<Cat>	Any?	T
class Box<T>	Box<*>	Star projection	See §4	Any?	Nothing (forbidden)
class Box<out T>	Box<T>	Covariant (as declared)	Box<Cat> <: Box<Animal>	T	Nothing (forbidden)
class Box<out T>	Box<out T>	Covariant (redundant)	Box<Cat> <: Box<Animal>	T	Nothing
class Box<out T>	Box<in T>	Error — conflicting	—	—	—
class Box<in T>	Box<T>	Contravariant (as declared)	Box<Animal> <: Box<Cat>	Any?	T
class Box<in T>	Box<*>	Star projection of contravariant	Special: in Nothing	Any?	Nothing


```

// Demonstrating the matrix:
open class Animal
class Cat : Animal()
class Dog : Animal()

// Invariant – exact match required
fun invariant(box: MutableList<Animal>) {}
// invariant(mutableListOf(Cat())) // ERROR: MutableList<Cat> is not
MutableList<Animal>

// Covariant projection – read-only widening
fun covariant(producer: List<Animal>) {}
covariant(listOf(Cat())) // OK: List is covariant (out T), so
List<Cat> <: List<Animal>

// Contravariant projection – write-only narrowing
fun contravariant(sink: MutableList<in Cat>) {
    sink.add(Cat()) // OK: can put Cat in
    // val x: Cat = sink[0] // ERROR: read returns Any?
}
val animalSink: MutableList<Animal> = mutableListOf()
contravariant(animalSink) // OK: MutableList<Animal> <: MutableList<in
Cat>

// Star projection
fun star(list: List<*>) {
    val first: Any? = list[0] // read as Any?
    // list.add("x") // ERROR: Nothing – cannot write
}

```

2.5 PECS — Producer Extends, Consumer Super

Joshua Bloch's PECS mnemonic maps directly to Kotlin's `in / out`:

- **Producer extends** → **Producer out**: if a parameter produces `T` values (you read from it), make it `out` / `? extends T`.
- **Consumer super** → **Consumer in**: if a parameter consumes `T` values (you write to it), make it `in` / `? super T`.

```

// Classic PECS example: copy from source (producer) to destination (consumer)
fun <T> copyKotlin(source: List<out T>, dest: MutableList<in T>) {
    for (item in source) dest.add(item)
}

// Java equivalent for comparison:
// <T> void copyJava(List<? extends T> source, List<? super T> dest) {
//     for (T item : source) dest.add(item);
// }

// Real-world backend example: event handler registry
interface EventHandler<in E : Event> {
    fun handle(event: E)
}

class EventBus {
    private val handlers = mutableMapOf<KClass<*>, MutableList<EventHandler<*>>>()

    fun <E : Event> register(type: KClass<E>, handler: EventHandler<E>) {
        handlers.getOrPut(type) { mutableListOf() }.add(handler)
    }

    // Contravariant handler: a handler for Event can handle any
subtype
    // EventHandler<Event> <: EventHandler<UserCreatedEvent>
    fun <E : Event> dispatch(event: E) {
        @Suppress("UNCHECKED_CAST")
        (handlers[event::class] as? List<EventHandler<E>>)?.forEach { i
t.handle(event) }
    }
}

```

In Kotlin service code, PECS shows up most often in collection processing, event dispatch, and anything with `Flow / Sequence` :

```

// Flow is covariant: Flow<out T> – a Flow<Cat> is a Flow<Animal>
fun animalFlow(): Flow<Animal> = flowOf(Cat(), Dog())

// MutableStateFlow is invariant – it both produces and consumes T
// MutableStateFlow<Cat> is NOT a MutableStateFlow<Animal>

```

3. Type erasure, reified generics, and `typeOf`

3.1 What erasure means on the JVM

The JVM has no runtime representation of generic type arguments. `List<String>` and `List<Int>` are the same class (`java.util.List`) at runtime. The compiler erases type parameters to their upper bound (or `Object` / `Any?` if unbounded) and inserts casts where needed. This is true for both Java and Kotlin — they share the same runtime.

What survives erasure:

Artifact	Survives?	Where
Raw class	Yes	Bytecode descriptor (<code>Ljava/util/List;</code>)
Upper bound	Yes	Implicit in erased descriptor
<code>Signature</code> attribute	Yes (as metadata)	Classfile attribute — visible via reflection but not used for <code>instanceof</code>
Kotlin <code>@Metadata</code>	Yes (as annotation)	Stores full Kotlin generic + nullability info
Runtime <code>instanceof</code> / <code>is</code> check on <code>T</code>	No	<code>if (x is List<String>)</code> checks only <code>List</code> , not <code>String</code>
Overload resolution on <code>T</code>	No	<code>fun foo(x: List<String>)</code> and <code>fun foo(x: List<Int>)</code> collide — same erased signature

```

fun <T> erasedCheck(value: Any) {
    // if (value is List<String>) // WARNING: check for instance is
    // always true / unchecked cast
    if (value is List<*>) {          // OK: only checks raw type
        println("it's a list, element type erased")
    }
}

// Overload collision – does not compile:
// fun handle(x: List<String>) {}
// fun handle(x: List<Int>) {}      // ERROR: conflicting overloads –
// same JVM signature

// Workaround: JVM name mangling
@JvmName("handleStrings")
fun handleStrings(x: List<String>) {}
@JvmName("handleInts")
fun handleInts(x: List<Int>) {}    // OK: distinct JVM names via
// annotation

```

Checking the bytecode confirms erasure:

```

kotlinc Erasure.kt -d erasure.jar
javap -v -p -classpath erasure.jar ErasureKt | grep -A5 "erasedCheck"

```

```

// Decompiled – note Object, not T, and no String check:
public static final void erasedCheck(Object value) {
    boolean isList = value instanceof List; // raw check only
}

```

```
sequenceDiagram
    participant Source as Kotlin source
    fun <T> foo(x: T)
        participant Compiler as Compiler
    type erasure
        participant Bytecode as Bytecode
    descriptor + Signature
        participant Runtime as Runtime
    instanceof / casts
        Source->>Compiler: T is String in source
        Compiler->>Bytecode: descriptor (Ljava/lang/Object;)
    Signature <T:Ljava/lang/Object;>(TT;)V
        Bytecode->>Runtime: instanceof checks raw type only
        Runtime-->>Source: casts inserted by compiler
    are checked at runtime
        Note over Compiler,Bytecode: Signature preserved for
        reflection but NOT for
        instanceof or overloads
```

3.2 The reified workaround — `inline` + `reified`

Kotlin's answer to erasure is `reified` type parameters on `inline` functions. When a function is `inline`, its body is copied to every call site at compile time. If a type parameter is marked `reified`, the compiler substitutes the *concrete* type argument at each call site, making `T::class`, `is T`, and `T::class.java` available — operations that are impossible with erased generics.

```
// Erased – cannot check T at runtime
fun <T> isInstanceErased(value: Any): Boolean {
    // return value is T // ERROR: Cannot check for instance of erased
    type: T
    return false
}

// Reified – T is known at each call site after inlining
inline fun <reified T> isInstanceReified(value: Any): Boolean {
    return value is T // OK: compiler knows T at call site
}

// Usage – each call site gets a specialized copy:
isInstanceReified<String>("hello") // inlined as: "hello" is String
isInstanceReified<Int>(42)          // inlined as: 42 is Int
```

What `inline` + `reified` actually does at the bytecode level:

```

inline fun <reified T> createInstance(): T? {
    return T::class.java.getDeclaredConstructor().newInstance()
}

// Call site 1:
val s: String? = createInstance<String>()
// After inlining, compiler emits (conceptually):
//   String.class.getDeclaredConstructor().newInstance()

// Call site 2:
val n: Int? = createInstance<Int>()
// After inlining:
//   Integer.class.getDeclaredConstructor().newInstance() // boxed –
// see §6

```

Constraints on `reified`:

- Only on `inline` functions — never on classes, interfaces, or non-inline functions. The type must be recoverable by inlining, and only functions are inlined.
- Cannot be used across module boundaries in a non-inlined way — the call site must see the function body.
- Each call site gets a separate bytecode expansion — code bloat is real if the function body is large and called many times with many distinct `T`.
- `reified` parameters cannot be used as `reified` arguments to non-reified positions without further inlining.

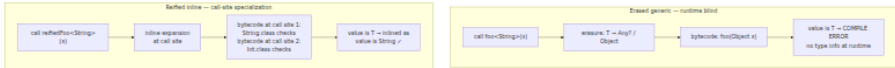
```
// Common reified utilities in backend code:

inline fun <reified T : Any> Gson.fromJson(json: String): T =
    fromJson(json, T::class.java)

inline fun <reified T : Any> ObjectMapper.readValue(json: String): T =
    readValue(json, typeRef<T>()) // see typeRef below via
TypeReference

// TypeReference trick – anonymous subclass captures generic supertype
token
inline fun <reified T> typeRef(): TypeReference<T> = object : TypeRefer
ence<T>() {}

// KClass-based service locator (simplified DI)
class ServiceRegistry {
    private val services = mutableMapOf<KClass<*>, Any>()
    inline fun <reified T : Any> register(instance: T) {
        services[T::class] = instance
    }
    inline fun <reified T : Any> resolve(): T =
        services[T::class] as? T ?: error("No service for ${T::class.si
mpleName}")
}
```



3.3 typeOf, KType, and the KType token

`T::class` gives you a `KClass` (the erased class), but loses generic arguments: `T::class` for `List<String>` yields `List::class`. To preserve full generic type information, Kotlin 1.6+ provides `typeOf<T>()` (requiring `reified T` inline context or experimental `optIn`):

```

import kotlin.reflect.typeOf
import kotlin.reflect.KType

@OptIn(ExperimentalStdlibApi::class)
inline fun <reified T> printType() {
    val kType: KType = typeOf<T>()
    println("KType: $kType")
    println("Classifier: ${kType.classifier}") // KClass – e.g., List
    println("Arguments: ${kType.arguments}") // KTypeProjection
    list – e.g., [String]
    println("Nullable: ${kType.isMarkedNullable}")
}

printType<List<String>>() // KType:
kotlin.collections.List<kotlin.String>
printType<Map<String, Int?>>() // KType:
kotlin.collections.Map<kotlin.String, kotlin.Int?>
printType<List<String>?>() // KType:
kotlin.collections.List<kotlin.String?> – nullable

// Passing KType explicitly when reified is not available:
fun processWithType(value: Any, type: KType) {
    println("Processing as $type")
}
processWithType(listOf("a"), typeOf<List<String>>())

```

This is the foundation for type-safe serialization frameworks. `kotlinx.serialization` uses `typeOf` / `serializer<T>()` to resolve the correct `KSerializer` at compile time without runtime reflection scanning:

3.4 Reified serializer example — end to end

```

import kotlinx.serialization.Serializable
import kotlinx.serialization.KSerializer
import kotlinx.serialization.json.Json
import kotlinx.serialization.serializer // reified serializer()
extension
import kotlin.reflect.typeOf

@Serializable
data class UserEvent(val userId: String, val email: String, val timestamp: Long)

@Serializable
data class PagedResult<T>(val items: List<T>, val nextToken: String?)

// --- Reified JSON helpers (typical in any Kotlin backend service) ---

@OptIn(ExperimentalStdlibApi::class)
object JsonCodec {
    private val json = Json {
        ignoreUnknownKeys = true
        encodeDefaults = true
        explicitNulls = false
    }

    // Reified encode – resolves KSerializer<T> at call site, no
    // Class<T> parameter
    inline fun <reified T> encode(value: T): String {
        // serializer<T>() is itself an inline reified function that
        // returns KSerializer<T>
        // typeOf<T>() is available for logging / routing decisions
        val kType = typeOf<T>()
        println("Encoding type: $kType") // e.g.,
        PagedResult<UserEvent>
        return json.encodeToString(serializer<T>(), value)
    }

    inline fun <reified T> decode(raw: String): T {
        return json.decodeFromString(serializer<T>(), raw)
    }

    // Non-reified overload for when T is only known as KType at
    // runtime
    // (e.g., message router that dispatches by topic → KType map)
    fun <T> decodeWithKType(raw: String, type: KType, serializer: KSerializer<T>): T {
        @Suppress("UNCHECKED_CAST")
        return json.decodeFromString(serializer as KSerializer<T>,
raw) as T
    }
}

```

```

// Usage – zero boilerplate at call sites:
fun main() {
    val event = UserEvent("u-42", "alice@example.com", 1713700000L)

    // Simple type – serializer resolved via reified T = UserEvent
    val json1 = JsonCodec.encode(event)
    val back1: UserEvent = JsonCodec.decode(json1)
    println(json1)
    //
    {"userId":"u-42","email":"alice@example.com","timestamp":1713700000}

    // Nested generic – serializer for PagedResult<UserEvent> composed
    automatically
    val page = PagedResult(items = listOf(event), nextToken = "tok-99")
    val json2 = JsonCodec.encode(page) // T =
    PagedResult<UserEvent>
    val back2: PagedResult<UserEvent> = JsonCodec.decode(json2)
    println(json2)
    // {"items":[{"userId":"u-42",...}], "nextToken":"tok-99"}

    // Contrast with Java/Gson – requires explicit TypeToken at every
    call site:
    // Type type = new TypeToken<PagedResult<UserEvent>>(){}.getType();
    // gson.fromJson(json, type);
    // Kotlin reified eliminates the anonymous-subclass ceremony
    entirely.
}

```

Key points for backend engineers:

- `serializer<T>()` is `inline fun <reified T> serializer(): KSerializer<T>` — it uses `typeOf<T>()` internally to look up the generated serializer. No reflection scan at runtime; the serializer is resolved at compile time and inlined.
- For polymorphic hierarchies (`@Polymorphic`, sealed classes), the `KSerializer` includes a discriminator field — `typeOf` ensures the correct polymorphic serializer is selected.
- When the type is not known at compile time (message router, plugin system), fall back to passing `KType` / `KSerializer` explicitly — reified is a call-site optimization, not a universal replacement for type tokens. Design your router's registry around `KType` keys if you need runtime dispatch.

4. Star projections and Java interop

4.1 What `*` means

`List<*>` is Kotlin's spelling for "a `List` of some unknown type." It is the safe replacement for Java's raw type `List` and for the unbounded wildcard `List<?>`. Understanding `*` requires understanding what it projects to.

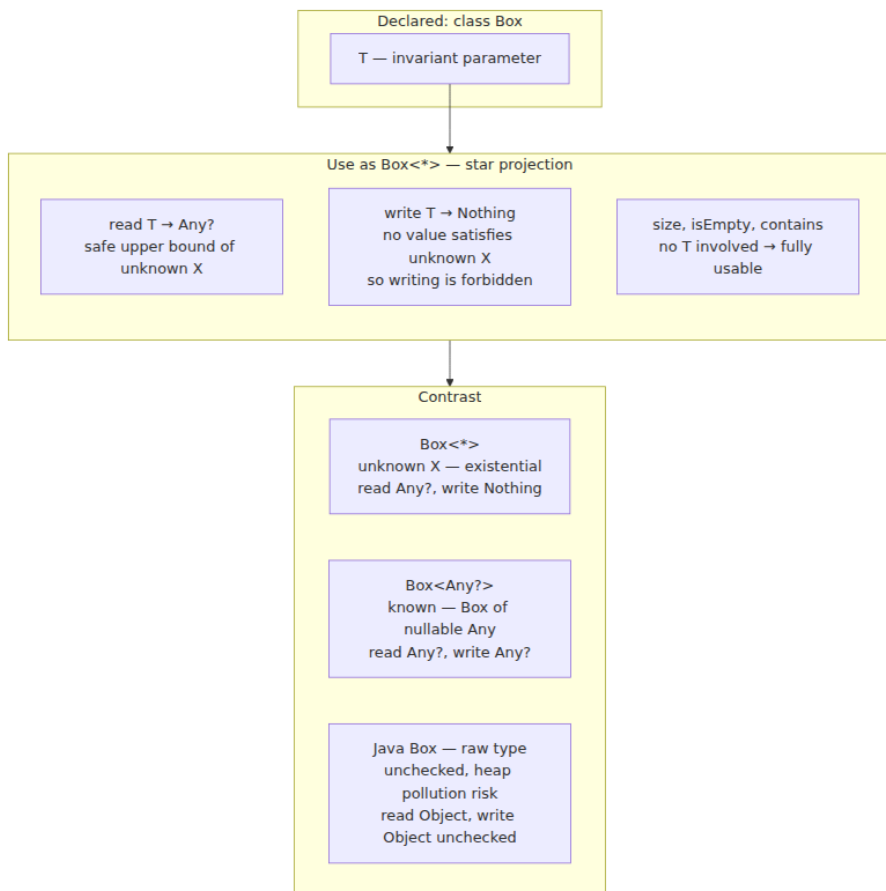
For a generic `Box<T>`:

- `Box<*>` means `Box<out Any?>` if `T` is covariant or invariant — you can read `Any?` out, but you can only write `Nothing` (i.e., you cannot write anything except `null` into a `Box<in Nothing>` position, and even that is restricted).
- For `Box<in T>`, `Box<*>` projects to `Box<in Nothing>` — you can write `Nothing` (nothing) and read `Any?`.

More precisely, `*` is an *existential type*: `Box<*>` means "there exists some type `X` such that this is a `Box<X>`, but we don't know what `X` is." The compiler enforces safe use by projecting `X` to its bounds.

```
fun starDemo(list: List<*>) {
    val first: Any? = list[0]    // OK: read as Any? – the only safe
    // list.add("hello")        // ERROR: cannot add – would require
    // knowing X
    println("size=${list.size}") // OK: size doesn't involve T
    for (item in list) {         // OK: iteration yields Any?
        println(item)
    }
}

fun <T> copyStar(from: List<*>, to: MutableList<in Any?>) {
    // List<*> elements are Any?, so we can add them to MutableList<in
    // Any?>
    for (item in from) to.add(item)
}
```



4.2 Star projection erasure demo

The following program demonstrates how `*`, `Any?`, and Java raw types differ at runtime — and that all three erase to the same bytecode:

```

// StarProjectionDemo.kt
fun inspectStar(list: List<*>) {
    println("List<*> size=${list.size}, first=${list.firstOrNull()}")
    // list.add("x") // compile error – Nothing
}

fun inspectAny(list: List<Any?>) {
    println("List<Any?> size=${list.size}")
    // list is MutableList<Any?> only if declared mutable – List<Any?>
    // is read-only anyway
}

fun inspectRawJava(list: java.util.ArrayList<*>) {
    println("ArrayList<*> via Kotlin view")
}

// At the bytecode level, all three have the same erased descriptor:
// (Ljava/util/List;)V – the Signature attribute distinguishes them:
//   inspectStar: (Ljava/util/List<*>;)V
//   inspectAny:  (Ljava/util/List<Ljava/lang/Object;>;)V
// But instanceof cannot distinguish any of them:
fun erasureDemo() {
    val strings: List<String> = listOf("a", "b")
    val ints: List<Int> = listOf(1, 2)

    println(strings is List<*>)           // true – raw check
    println(ints is List<*>)             // true – same raw check
    // println(strings is List<String>) // WARNING: unchecked – same
    // as List<*>

    // Star projection safely handles either:
    inspectStar(strings) // OK: List<String> <: List<*>
    inspectStar(ints)    // OK: List<Int> <: List<*>

    // Any? is stricter – requires exact match on nullability view:
    // inspectAny(strings) // ERROR if strings is List<String> and
    // function expects List<Any?>
    // But List<String> <: List<Any> via covariance (List is out), so:
    val anys: List<Any> = strings // OK: List is covariant
    inspectAny(listOf("a", null)) // OK: List<Any?> inferred

    // Bytecode proof – descriptors are identical after erasure:
    println(inspectStar::class.java.methods.find { it.name == "inspectStar" })
    println(inspectAny::class.java.methods.find { it.name == "inspectAny" })
    // Both: public static void inspect...(java.util.List)
}

fun main() {

```

```

erasureDemo()
// Output:
// true
// true
// List<*> size=2, first=a
// List<*> size=2, first=1
// public static final void
StarProjectionDemoKt.inspectStar(java.util.List)
// public static final void
StarProjectionDemoKt.inspectAny(java.util.List)
}

```

For a backend service that reflects over handlers or deserializes heterogeneous lists, `List<*>` is the correct Kotlin type for "a list whose element type I don't know yet." Cast it only after checking the desired element type with a reified helper:

```

inline fun <reified T> List<*>.filterIsInstanceReified(): List<T> =
    filterIsInstance<T>() // uses reified T to check each element

val mixed: List<*> = listOf("a", 42, "b", 3.14)
val strings: List<String> =
    mixed.filterIsInstanceReified<String>() // ["a", "b"]

```

4.3 Java wildcard interop

Kotlin declaration-site variance and Java use-site variance meet at the bytecode boundary. The Kotlin compiler maps between them automatically, but the mapping has sharp edges:

Java declaration	Kotlin view	Notes
<code>List<String></code>	<code>MutableList<String></code>	Invariant — exact match
<code>List<? extends Number></code>	<code>MutableList<out Number></code>	Covariant projection
<code>List<? super String></code>	<code>MutableList<in String></code>	Contravariant projection
<code>List<?></code>	<code>MutableList<*></code>	Star projection
<code>List</code> (raw)	<code>MutableList<*></code>	Raw → star projection (with unchecked warning)

Java declaration	Kotlin view	Notes
<code>List<? extends Number></code> as return	<code>List<Number></code> in Kotlin if declared <code>out</code>	Kotlin may see raw bound

```
// Java class:
// public class JavaBox {
//     public static void consumeNumbers(List<? extends Number> nums)
// }
//     public static void produceStrings(List<? super String> sink) {}
//     public static List<?> unknownList() { return List.of("a",
// "b"); }
// }

// Kotlin call sites – wildcards mapped to projections:
JavaBox.consumeNumbers(listOf(1, 2, 3))           // List<Int> <: List<out
Number> – OK
JavaBox.produceStrings(mutableListOf<Any>())      // MutableList<Any> <:
MutableList<in String> – OK
val unknown: MutableList<*> = JavaBox.unknownList() as MutableList<*>

// Going the other direction – Kotlin declaration with variance, seen
from Java:
// Kotlin: class Producer<out T>(val value: T)
// Java sees: class Producer<T> { T getValue(); } – with Signature
<T:Ljava/lang/Object;>
// Java cannot express declaration-site variance, so it sees invariant
Producer<T>
// To call from Java with variance, Java must use wildcards:
// Producer<? extends Animal> p = new Producer<>(new Cat());

// @JvmWildcard / @JvmSuppressWildcards control wildcard generation:
class KotlinService {
    // Without annotation, Kotlin's List<String> generates List<String>
    in bytecode Signature
    // Java sees List<String> – correct.

    // With declaration-site variance, Java interop may need explicit
    wildcards:
    fun processInvariant(list: List<String>) {}           // Java:
List<String>
    fun processCovariant(list: List<@JvmWildcard String>) {} // Java:
List<? extends String>
    fun processNoWildcard(list: List<@JvmSuppressWildcards String>)
    {} // suppress ? extends
}
```



```
# Verify wildcard generation in bytecode signatures:
kotlinc Interop.kt -d interop.jar
javap -v -p -classpath interop.jar KotlinService | grep -A1 "Signature"
# processInvariant: (Ljava/util/List<Ljava/lang/String;>;)V
# processCovariant: (Ljava/util/List<+Ljava/lang/String;>;)V - '+'
denotes ? extends
# processNoWildcard: (Ljava/util/List<Ljava/lang/String;>;)V
```

In a polyglot service (Kotlin services calling Java libraries and vice versa), the practical rule is: annotate Kotlin variance-bearing APIs with `@JvmWildcard` where Java callers need covariance, and use `@JvmSuppressWildcards` sparingly when Java's wildcard handling complicates overload resolution. Test the Java-facing Signature with `javap` — don't guess.

5. Null-safety as a type-system property — flow typing and contracts

5.1 Smart casts and flow typing

Kotlin's null-safety is not a runtime check inserted before every dereference. It is a type-narrowing system where control flow refines types. After a null check, the compiler *smart-casts* `T?` to `T` within the scope where non-null is proven:

```

fun flowTypingDemo(input: String?) {
    // input is String? – nullable

    if (input != null) {
        // input is smart-cast to String here – no !!, no ?. needed
        println(input.length)      // String.length – safe
        println(input.uppercase()) // all String members available
    }

    // input is String? again – smart cast expired outside the if

    // Early return – also narrows
    if (input == null) return
    // input is String from here on – compiler knows the null path
    returned
    println(input.length)

    // Elvis + throw / return also narrows:
    val s: String = input ?: error("null") // String, not String?
}

fun whenNarrowing(value: Any?) {
    when (value) {
        null -> println("null")
        is String ->
    println(value.length) // value is String in this branch
        is Int -> println(value + 1) // value is Int here
        else -> println("other: $value")
    }
}

```

The compiler tracks several narrowing conditions:

- `x != null`, `x == null`, `x is T`, `x !is T`, `x is null` / `x !is null` in `if` / `when` / `while`.
- `&&` and `||` short-circuit: `if (x != null && x.isNotEmpty())` — second condition sees `x` as non-null.
- `?.let { }`, `?: return`, `?: throw`, `?: error()` — idiomatic early-exit narrowing.
- `is` checks narrow `Any` / `Any?` to concrete types, enabling safe casts without `as`.

```

fun complexFlow(x: Any?, y: String?) {
    if (x is String && y != null && x.length == y.length) {
        // x is String, y is String – both narrowed by the combined
        // condition
        println(x.uppercase() + y.uppercase())
    }

    // Smart cast survives only if the variable is stable:
    var mutable: String? = y
    if (mutable != null) {
        // mutable is String? still – var can be reassigned between
        // check and use
        // println(mutable.length) // ERROR: smart cast impossible –
        // var is mutable
        val snapshot = mutable // snapshot is String (val –
        // stable)
        println(snapshot.length) // OK
    }
}

```

Stability is the key limitation: smart casts apply only to `val` properties (and local `val` s) that have no custom getter and are not open. A `var` or an `open val` with a custom getter could return a different value on each access, so the compiler refuses to narrow it. In backend code, this surfaces with mutable entity fields — prefer `val` or capture to a local `val` before narrowing.

5.2 Contracts — lifting custom checks into the type system

Standard flow typing handles `!= null` and `is T`, but backend code is full of custom validation: `requireNotNull`, `check`, helper predicates like `isValidEmail(x)`. Without help, the compiler cannot connect `if (isValid(x))` to a type narrowing. `kotlin.contracts` solves this by letting functions declare *contracts* — promises about how their return value relates to their parameters' types.

```

import kotlin.contracts.ExperimentalContracts
import kotlin.contracts.contract

@OptIn(ExperimentalContracts::class)
fun requireNonBlank(value: String?, lazyMessage: () -> String = { "required" }): String {
    contract {
        returns() implies (value != null) // if this function returns normally, value is non-null
    }
    if (value.isNullOrBlank()) throw IllegalArgumentException(lazyMessage())
    return value
}

fun handleRequest(rawEmail: String?) {
    val email = requireNonBlank(rawEmail) { "email is required" }
    // rawEmail is now smart-cast to String – contract told the compiler
    // that normal return implies non-null
    println(email.length) // safe – but prefer using the returned `email` val
}

// More expressive: returns(true) / returns(false) / returns(null) / returnsNotNull()

@OptIn(ExperimentalContracts::class)
fun isNotNullAndNotEmpty(value: String?): Boolean {
    contract {
        returns(true) implies (value != null) // true return means value is non-null
    }
    return !value.isNullOrEmpty()
}

fun processBatch(input: String?) {
    if (isNotNullAndNotEmpty(input)) {
        // input is String here – contract propagated the narrowing
        println(input.length)
    }
}

// callsInPlace – for inline higher-order functions that execute a lambda exactly once,
// allowing smart casts inside the lambda to propagate:
@OptIn(ExperimentalContracts::class)
inline fun <T> T?.ifPresent(block: (T) -> Unit) {
    contract { callsInPlace(block, InvocationKind.AT_MOST_ONCE) }
}

```

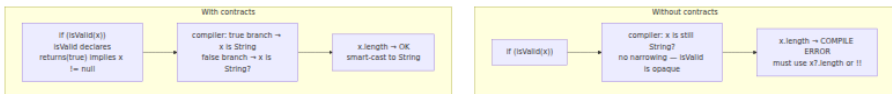
```

    if (this != null) block(this)
  }

```

Available contract effects:

Contract	Meaning
<code>returns() implies (x != null)</code>	Normal return guarantees <code>x</code> is non-null/non-false
<code>returns(true) implies (x is T)</code>	Returning <code>true</code> narrows <code>x</code> to <code>T</code>
<code>returns(false) implies (x is T)</code>	Returning <code>false</code> narrows <code>x</code>
<code>returns(null) implies (x is T)</code>	Returning <code>null</code> narrows <code>x</code>
<code>returnsNotNull() implies (x != null)</code>	Non-null return implies <code>x</code> is non-null
<code>callsInPlace(lambda, EXACTLY_ONCE)</code>	Lambda is called in place — smart casts inside it propagate
<code>callsInPlace(lambda, AT_MOST_ONCE)</code>	Lambda called zero or one times
<code>callsInPlace(lambda, AT_LEAST_ONCE)</code>	Lambda called one or more times



Contracts are experimental (`@OptIn(ExperimentalContracts::class)`) but stable in practice; the standard library uses them extensively (`requireNotNull`, `check`, `error`, `TODO`, `let`, `run`, `also`, `apply`). Custom contracts are most valuable in shared validation libraries: a single `validateRequest()` with a contract can narrow types across every service that calls it, eliminating repetitive `!!` and `?let` chains.

Limitations: contracts apply only to `inline` functions (or functions the compiler can analyze as effectively inline in the stdlib), cannot express arbitrary predicates (only null checks and type tests), and are not inherited — overriding a contracted function does not carry the contract

to the override. For complex validation, prefer returning a sealed result type over relying solely on contracts.

6. Value classes, inline classes, and unsigned types

6.1 The allocation problem

On the JVM, every domain concept modeled as a class costs an allocation: `data class UserId(val value: String)` wraps a `String` in a heap object with a header, a reference field, and GC pressure. For hot-path identifiers that appear millions of times (request IDs, partition keys, metric labels), that overhead is significant. Kotlin offers three mechanisms to eliminate it.

6.2 Value classes (`@JvmInline value class`)

A `value class` with a single property is *unboxed* at runtime where possible — the wrapper disappears and only the underlying value remains. The type still exists at compile time for type safety, but at runtime it is just the wrapped type.

```
@JvmInline
value class UserId(val raw: String)

@JvmInline
value class PartitionKey(val raw: String)

@JvmInline
value class RequestId(val raw: String)

// Type safety – cannot mix them:
fun loadUser(id: UserId) { /* ... */ }
fun loadPartition(key: PartitionKey) { /* ... */ }

val uid = UserId("u-42")
val pkey = PartitionKey("p-99")
// loadUser(pkey) // ERROR: PartitionKey is not UserId – compile-time
// safety, zero runtime cost

// At runtime (when not boxed), UserId("u-42") is just the String
// "u-42" – no wrapper object.
// Bytecode for loadUser(UserId) is actually loadUser(String) after
// mangling.
```

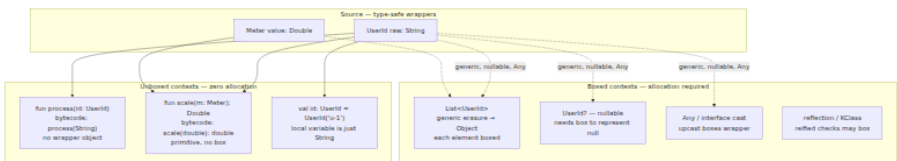
When does boxing occur? The wrapper is allocated when:

- Used as a generic type argument: `List<UserId>` boxes each `UserId` because generics erase to `Object` and the wrapper identity is needed.
- Used as nullable `UserId?` — `null` requires a box to distinguish "null `UserId`" from "`UserId` wrapping null" (though `UserId` wrapping `String` cannot wrap `null` if `raw` is `String` non-nullable, the nullable wrapper still boxes).
- Used as `Any` / interface type — upcasting to `Any` boxes.
- Returned from an inline function that doesn't inline at the call site.

```
@JvmInline value class Meter(val value: Double)

fun unboxed(m: Meter): Double = m.value *
2.0      // no allocation - m is just a double
fun boxed(list: List<Meter>): Double {           // boxes -
generic context
    return list.sumOf { it.value }
}
fun nullable(m: Meter?): Double? = m?.value     // boxes -
nullable

// Bytecode proof:
kotlinc ValueClass.kt -d v.jar
javap -v -p -classpath v.jar ValueClassKt | grep -A2 "unboxed\\|boxed"
// unboxed: (D)D - takes and returns primitive double, no Meter object
// boxed: (Ljava/util/List;)D - List of boxed Meter objects
```



6.3 Inline classes (deprecated) vs value classes

`inline class` was the experimental predecessor. It is deprecated in favor of `@JvmInline value class`, which generalizes to multi-property value classes in Kotlin 2.0+ (via `@JvmInline` single-property restriction being lifted for non-JVM targets). Migrate any remaining `inline class` declarations:

```
// Deprecated:
inline class LegacyId(val raw: String)

// Current:
@JvmInline value class ModernId(val raw: String)
```

6.4 Unsigned and primitive specialization

Kotlin's unsigned types (`UByte`, `UShort`, `UInt`, `ULong`) and the experimental unsigned arrays are also value classes — they wrap signed JVM primitives with unsigned semantics at compile time and erase to the signed primitive at runtime:

```
val count: UInt = 42u           // value class wrapping Int – runtime
is just int 42
val big: ULong = 9_000_000_000u // wrapping Long – runtime is just
long

// Unsigned operations are intrinsified – no boxing for arithmetic:
fun increment(x: UInt): UInt = x +
1u // compiles to iadd on primitive int

// Pitfall – boxing in generics still applies:
val counts: List<UInt> = listOf(1u, 2u, 3u) // each UInt boxed –
List<U* > is generic
val array: UIntArray = uintArrayOf(1u, 2u,
3u) // specialized primitive array – no boxing
```

For backend hot paths (counters, checksums, binary protocol parsing), prefer `UIntArray` / `UByteArray` or plain signed primitives over `List<UInt>` to avoid per-element boxing. When unsigned semantics are needed for correctness (e.g., Kafka offset arithmetic that wraps at 2^{32}), `UInt` / `ULong` give you the right operations with zero overhead outside generic contexts.

7. Delegated properties — `by` as a type-system feature

Delegated properties let a property's `get` / `set` logic be extracted into a reusable delegate object. The compiler desugars `val x by delegate` into calls to `delegate.getValue(thisRef, property)` / `delegate.setValue(thisRef, property, value)`.

7.1 Mechanism and desugaring

```
import kotlin.reflect.KProperty

class LoggingDelegate<T>(private var value: T) {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): T {
        println("get ${property.name} = $value")
        return value
    }
    operator fun setValue(thisRef: Any?, property: KProperty<*>, newValue: T) {
        println("set ${property.name}: $value → $newValue")
        value = newValue
    }
}

class ServiceConfig {
    var endpoint: String by LoggingDelegate("https://api.example.com")
    var timeoutMs: Long by LoggingDelegate(5_000L)
}

// Desugared (what the compiler generates, simplified):
class ServiceConfigDesugared {
    private val endpoint$delegate = LoggingDelegate("https://api.example.com")
    var endpoint: String
    get() = endpoint$delegate.getValue(this, ::endpoint)
    set(value) { endpoint$delegate.setValue(this, ::endpoint, value) }

    private val timeoutMs$delegate = LoggingDelegate(5_000L)
    var timeoutMs: Long
    get() = timeoutMs$delegate.getValue(this, ::timeoutMs)
    set(value) { timeoutMs$delegate.setValue(this, ::timeoutMs, value) }
}
```

The delegate must provide `operator fun getValue` (and `setValue` for `var`). Two special cases:

- `ReadOnlyProperty<in R, out T>` — delegate for `val` only.
- `ReadWriteProperty<in R, T>` — delegate for `var`.

7.2 Standard delegates

```

// lazy – thread-safe by default (SYNCHRONIZED), computed once
val expensiveClient: HttpClient by lazy {

HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build()
}
// Modes: LazyThreadSafetyMode.SYNCHRONIZED (default), PUBLICATION,
NONE

// observable – vetoable / observable mutation
var maxRetries: Int by Delegates.observable(3) { prop, old, new ->
    require(new in 0..10) { "${prop.name} out of range: $new" }
    logger.info { "${prop.name}: $old → $new" }
}
var boundedRetries: Int by Delegates.vetoable(3) { _, _, new -> new in
0..10 }

// notNull – lateinit alternative with explicit delegate (throws if
read before write)
var lateConfig: ServiceConfig by Delegates.notNull()

// map-backed – useful for JSON / dynamic config
class JsonConfig(private val map: Map<String, Any?>) {
    val host: String by map // map.getValue(thisRef, property) –
looks up "host" key
    val port: Int by map // looks up "port" key
}
val cfg = JsonConfig(mapOf("host" to "db.internal", "port" to 5432))
println(cfg.host) // "db.internal"

// Custom: expiring cache delegate
class ExpiringDelegate<T>(private val ttl: Duration, private val produc
er: () -> T) {
    private var cached: T? = null
    private var expiresAt: Instant = Instant.MIN

    operator fun getValue(thisRef: Any?, property: KProperty<*>): T {
        if (cached == null || Instant.now().isAfter(expiresAt)) {
            cached = producer()
            expiresAt = Instant.now().plus(ttl)
        }
        @Suppress("UNCHECKED_CAST")
        return cached as T
    }
}

class TokenHolder {
    val accessToken: String by
ExpiringDelegate(Duration.ofMinutes(55)) {
    fetchTokenFromIdp() // called at most once per 55 minutes, on
first access after expiry
}
}

```

```
}  
}
```

7.3 Property delegation in distributed systems

Delegates shine for cross-cutting backend concerns that would otherwise require boilerplate on every property:

```

// Metrics-instrumented property – records access count and latency
class MeteredDelegate<T>(private var value: T, private val name:
String, private val registry: MeterRegistry) {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): T {
        registry.counter("${name}.reads").increment()
        return value
    }
    operator fun setValue(thisRef: Any?, property: KProperty<*>, newValue: T) {
        registry.counter("${name}.writes").increment()
        value = newValue
    }
}

// Feature-flag delegate – reads from a remote flag service, falls back to default
class FlagDelegate(private val flagKey: String, private val default: Boolean, private val flags: FlagService) {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): Boolean =
        flags.isEnabled(flagKey) ?: default
}

class ServiceFlags(flags: FlagService, registry: MeterRegistry) {
    val enableNewRouting: Boolean by FlagDelegate("routing.v2", default = false, flags)
    var circuitThreshold: Int by MeteredDelegate(5, "circuit.threshold", registry)
}

// ProvideDelegate – customizes delegate creation based on property metadata
// (advanced: delegate provider can inspect property name, type, annotations)
class ValidatedString(private var value: String, private val maxlen: Int) {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): String = value
    operator fun setValue(thisRef: Any?, property: KProperty<*>, newValue: String) {
        require(newValue.length <= maxlen) { "${property.name} exceeds $maxlen chars" }
        value = newValue
    }
}

class Config {
    var serviceName: String by ValidatedString("api", maxlen = 64)
}

```

Performance note: each delegated property adds an object field for the delegate and a `KProperty` instance for the property reference (lazily initialized). On hot paths with millions of instances, prefer direct fields or value classes over delegates. Delegates are ideal for configuration, service-level singletons, and framework integration — not for per-record domain objects.

8. Putting it together — a type-safe backend service skeleton

The following example combines variance, reified generics, star projections, value classes, and delegated properties in a single service structure that mirrors real Kotlin backend code:

```

@file:OptIn(ExperimentalStdlibApi::class)

import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
import kotlinx.serialization.serializer
import kotlin.reflect.KClass
import kotlin.reflect.typeOf
import kotlin.properties.Delegates

// --- Value classes for domain safety (zero-cost) ---
@JvmInline value class TenantId(val raw: String)
@JvmInline value class EventId(val raw: String)

// --- Covariant event hierarchy – Producer<out T> pattern ---
sealed class DomainEvent {
    abstract val eventId: EventId
    abstract val tenantId: TenantId
}

@Serializable
data class UserCreated(
    override val eventId: EventId,
    override val tenantId: TenantId,
    val email: String,
) : DomainEvent()

@Serializable
data class OrderPlaced(
    override val eventId: EventId,
    override val tenantId: TenantId,
    val amountCents: UInt, // unsigned – value class, unboxed as int
) : DomainEvent()

// --- Contravariant handler – Consumer<in T> pattern ---
fun interface EventHandler<in E : DomainEvent> {
    fun handle(event: E)
}

// Covariant producer – produces events
interface EventSource<out E : DomainEvent> {
    fun next(): E?
}

// --- Reified event codec – serializer<T>() via reified generics ---
object EventCodec {
    private val json = Json { ignoreUnknownKeys = true; explicitNulls = false }

    inline fun <reified T : DomainEvent> encode(event: T): String =
        json.encodeToString(serializer<T>(), event)
}

```

```

inline fun <reified T : DomainEvent> decode(raw: String): T =
    json.decodeFromString(serializer<T>(), raw)

// Star-projection handler registry – heterogeneous handlers stored
as EventHandler<*>
private val handlers = mutableMapOf<KClass<*>, MutableList<EventHan-
dler<*>>>()

fun <E : DomainEvent> register(type: KClass<E>, handler: EventHandl-
er<E>) {
    handlers.getOrPut(type) { mutableListOf() }.add(handler)
}

// Reified convenience – no KClass boilerplate at call sites
inline fun <reified E : DomainEvent> register(noinline handler:
(E) -> Unit) {
    register(E::class, EventHandler { e: E -> handler(e) })
    println("Registered handler for ${typeOf<E>()}") // KType
logging via reified T
}

fun dispatch(event: DomainEvent) {
    @Suppress("UNCHECKED_CAST")
    val list = handlers[event::class] as? List<EventHandler<DomainE-
vent>> ?: return
    for (h in list) h.handle(event)
}

// --- Delegated config – lazy, observable, validated ---
class ServiceConfig {
    val httpClient by lazy { java.net.http.HttpClient.newHttpClient() }

    var maxRetries: Int by Delegates.vetoable(3) { _, _, new -> new in
0..10 }

    var logLevel: String by Delegates.observable("INFO") { _, old, new
->
        println("logLevel: $old -> $new")
    }
}

// --- Wiring ---
fun main() {
    val config = ServiceConfig()

    // Reified registration – type inferred, no Class<T> token
    EventCodec.register<UserCreated> { e ->
        println("New user: ${e.email} in tenant ${e.tenantId.raw}")
    }
}

```



```

EventCodec.register<OrderPlaced> { e ->
    println("Order ${e.eventId.raw} amount=${e.amountCents}")
}

// Contravariant handler — a handler for DomainEvent handles any
// subtype
val auditHandler = EventHandler<DomainEvent> { e ->
    println("AUDIT ${e::class.simpleName} ${e.eventId.raw}")
}
EventCodec.register(DomainEvent::class, auditHandler)
// EventHandler<DomainEvent> <: EventHandler<UserCreated> —
// contravariance

val event = UserCreated(EventId("evt-1"), TenantId("t-42"), "alice@example.com")
val encoded =
EventCodec.encode(event) // reified — no serializer arg needed
val decoded: UserCreated = EventCodec.decode(encoded)
EventCodec.dispatch(decoded)
}

```

This skeleton demonstrates the distributed-systems relevance of every type-system feature:

- **Value classes** prevent tenant/event ID confusion with zero allocation overhead — critical when every request carries multiple IDs.
- **Covariant** `EventSource<out T>` lets a `Source<UserCreated>` be used where `Source<DomainEvent>` is expected — natural for partitioned consumers.
- **Contravariant** `EventHandler<in T>` lets a single audit handler consume all event subtypes without per-type registration.
- **Reified** `encode` / `decode` eliminate `Class<T>` / `TypeToken` boilerplate at every serialization site — multiplied across hundreds of event types.
- **Star projection** `EventHandler<*>` in the registry stores heterogeneous handlers safely; the `is` check in `dispatch` recovers the concrete type.
- **Delegated properties** centralize config concerns (lazy init, validation, observability) without scattering logic across the codebase.

9. Pitfalls and operational guidance

Platform types are the null-safety escape hatch. Every unannotated Java return is `T!`. Audit your Java dependencies' nullability annotations; add `@Nullable/@NotNull` to internal Java code (or use JSpecify). In Kotlin, prefer `String` over `String?` for values that must not be null, and handle `T!` from Java with explicit `?: error()` or `requireNotNull` at the boundary — fail fast rather than propagating a nullable platform type through your call graph.

Reified bloat is real. Each distinct `T` at a call site for an `inline fun <reified T>` generates a separate inlined copy. For small helpers (`isInstance`, `serializer()`) this is negligible. For large functions called with many types, prefer a non-reified `KType/KClass` parameter and a single shared implementation. Profile bytecode size with `javap -v` if service JAR size grows unexpectedly.

Variance errors often signal a design problem. If you find yourself fighting `out/in` errors, the class likely has mixed responsibilities (both producing and consuming `T`). Split it: a `Producer<out T>` and a `Consumer<in T>` are easier to reason about than an invariant `Box<T>` that does both. `MutableList<T>` is invariant for exactly this reason — it both reads and writes `T`.

Star projections are not `Any?`. `List<*>` and `List<Any?>` have different write semantics. `List<*>` forbids writing (projects to `Nothing`); `List<Any?>` allows writing `Any?`. Use `*` when the element type is genuinely unknown (heterogeneous registries, reflective inspection); use `Any?` when the list intentionally holds heterogeneous nullable values.

Value class boxing in generics negates the optimization. `List<UserId>` boxes every `UserId`. For collections of value classes on hot paths, consider parallel primitive arrays (`List<String>` for `UserId` wrappers) or specialized collections. Measure allocation rate with JFR (`jdk.ObjectAllocationInNewTLAB`) before and after introducing value classes in performance-sensitive code.

Contracts don't replace sealed results. Use contracts for simple null/type narrowing in validation helpers. For complex business validation with multiple failure modes, return `Result<T>` or a sealed `ValidationOutcome` — contracts cannot express "this function returns an error string or narrows the type" beyond null/type predicates.

Key takeaways

- Kotlin's type lattice splits every type `T` into non-nullable `T` and nullable `T?`, with `Nothing` as bottom and `Any?` as top. Platform type `T!` (a flexible type `(T..T?)`) at the Java boundary adapts to either, and is the primary source of NPEs in Kotlin services.
- Declaration-site variance (`in / out`) fixes a type's variance once; use-site projection (`out / in` at the use site) and star projection (`*`) provide local overrides. `MutableList<T>` is invariant, `List<out T>` is covariant, `Consumer<in T>` is contravariant — choose based on whether the type produces, consumes, or does both.
- PECS maps to `out / in : producer → out / ? extends`, `consumer → in / ? super`. Covariant types allow `Box<Cat> <: Box<Animal>`; contravariant types flip it.
- JVM type erasure removes generic arguments at runtime; `Signature` attributes and Kotlin `@Metadata` preserve them for reflection but not for `instanceof` or overload resolution. `inline + reified` recovers type information by specializing the function body at each call site — enabling `is T`, `T::class`, `typeOf<T>()`, and `serializer<T>()` without explicit `Class<T>` tokens.
- `typeOf<T>()` yields a full `KType` including generic arguments and nullability; `T::class` yields only the erased `KClass`. `kotlinx.serialization`'s `serializer<T>()` is a reified function that resolves `KSerializer<T>` at compile time.
- Star projection `Box<*>` is an existential type — read as `Any?`, write as `Nothing` (forbidden). It is the safe Kotlin replacement for Java raw types and `?` wildcards, and the correct type for heterogeneous registries.
- Java wildcard interop: `? extends T ↔ out T`, `? super T ↔ in T`, `? / raw ↔ *`. Use `@JvmWildcard` / `@JvmSuppressWildcards` to control wildcard generation for Java callers; verify with `javap -v`.
- Flow typing smart-casts `T?` to `T` after `!= null` / `is T` checks, but only for stable `val`s. `kotlin.contracts` extends this to custom validation functions via `returns() implies` / `returns(true) implies` / `callsInPlace` declarations.
- `@JvmInline` value class erases the wrapper to its underlying type at runtime (a `String` wrapper becomes a `String`, a `Double` wrapper

becomes a `double`) — zero allocation except when boxed as generic arguments, nullable, or `Any`. Unsigned types (`UInt`, `ULong`) are value classes with the same boxing rules.

- Delegated properties desugar by delegate to `getValue / setValue` operator calls with a `KProperty` parameter. Standard delegates (`lazy`, `observable`, `vetoable`, `map-backed`) and custom delegates centralize cross-cutting concerns (caching, feature flags, metrics) without per-property boilerplate.

Further reading

- Kotlin Language Specification — Type System: <https://kotlinlang.org/spec/type-system.html> — normative reference for flexible types, intersection types, and variance.
- Kotlin Reference — Generics: In, Out, Where: <https://kotlinlang.org/docs/generics.html> — declaration-site and use-site variance with examples.
- Kotlin Reference — Inline Functions and Reified Type Parameters: <https://kotlinlang.org/docs/inline-functions.html#reified-type-parameters>
- Kotlin Reference — Value Classes: <https://kotlinlang.org/docs/inline-classes.html> — unboxing rules, boxing cases, and multi-property value classes.
- `kotlin.reflect.typeOf` and `KType` API: <https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.reflect/type-of.html>
- `kotlin.contracts` API: <https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.contracts/>
- `kotlinx.serialization` — Serializers via reified generics: <https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/serializers.md>
- Angelika Langer — Java Generics FAQ (wildcards, PECS, capture): <http://www.angelikalanger.com/GenericsFAQ/FAQSections/TechnicalDetails.html>
- Bracha, Gilad — *Generics in the Java Programming Language* (2004) — original erasure and wildcard design rationale.

- Naftalin, Maurice & Wadler, Philip — *Java Generics and Collections* (O'Reilly, 2006) — Chapters 2–3 on variance and wildcards.
- JVM Specification §4.7.9 — `Signature` Attribute: <https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-4.html#jvms-4.7.9> — how generic signatures survive erasure in classfiles.
- JSpecify — Nullness annotations for Java: <https://jspecify.dev/docs/user-guide/> — modern replacement for JSR-305 `@Nullable` / `@NotNull`.

Chapter 9 — Build Tooling, Dependency Management, and the Module/Artifact Ecosystem

What this chapter covers. The JVM ecosystem ships production systems through three dominant build tools — Maven, Gradle, and sbt — each with a distinct model for lifecycle, configuration, and dependency resolution. Beyond the build tool itself, every JVM project operates within a broader artifact ecosystem: Maven Central as the default public repository, private registries like Nexus and Artifactory as enterprise proxies, BOMs and version catalogs as coordination mechanisms, and cryptographic signatures as the trust layer between publish and consumption. Understanding these tools is not optional polish — in a distributed system with hundreds of services, each pulling hundreds of transitive dependencies, build tooling is the first line of defense against dependency confusion, version conflict, reproducibility failure, and supply-chain compromise. A misconfigured POM or Gradle build cache can silently produce different artifacts on different CI agents, and a missing enforcer rule can let a vulnerable transitive dependency slip through to production.

Learning goals — after this chapter you should be able to:

- Describe Maven's lifecycle model (validate → compile → test → package → install → deploy), explain how plugins bind to phases, and reason about the POM inheritance and dependency mediation (nearest-wins).
- Configure Gradle's Kotlin DSL for a multi-module project, distinguish configuration-time from execution-time, leverage the build cache and

Configuration Cache, and use version catalogs for centralized dependency declaration.

- Understand sbt's incremental compilation model, its Ivy-based dependency resolution, and how `build.sbt` differs structurally from Maven/Gradle XML or Kotlin DSL.
- Explain transitive dependency resolution across all three tools, including conflict resolution strategies (nearest-wins, highest-version, forced substitution), dependency locking, and verification metadata.
- Configure and operate artifact repositories: Maven Central, Sonatype Nexus, JFrog Artifactory, PGP/GPG signing, and repository content filtering.
- Apply a BOM (Bill of Materials) and a Gradle version catalog to manage coordinated version families across a multi-module build.

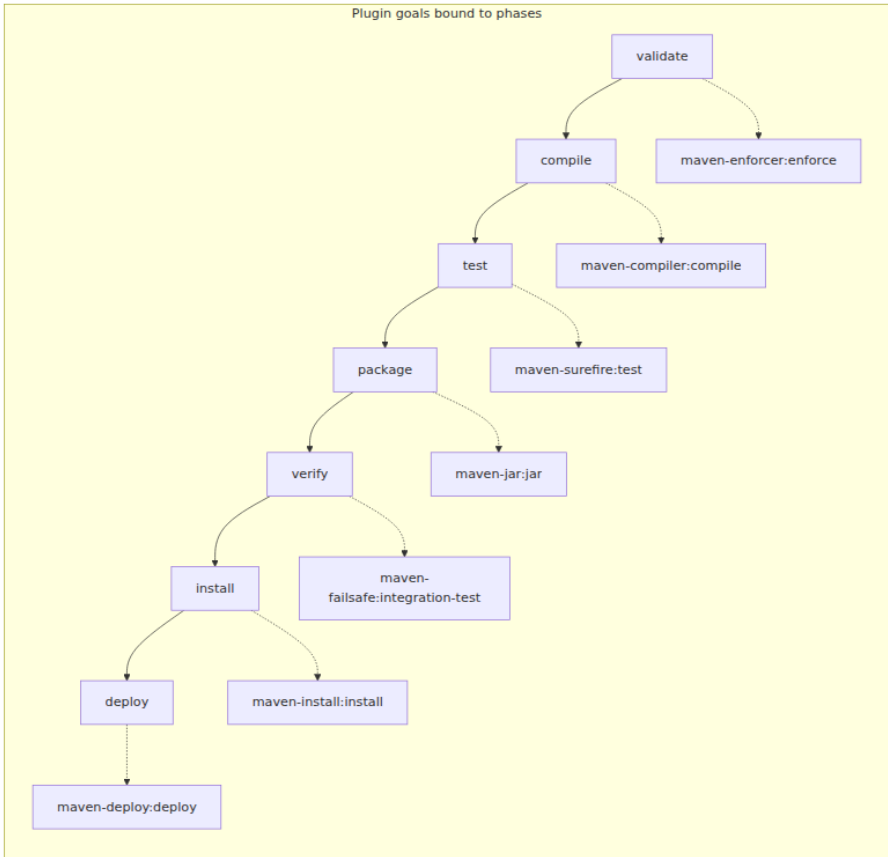
Prerequisites. Volume 18, Chapter 2 (class loading and JPMS modules) for module system context; Volume 9, Chapter 9 (application security) for supply-chain threat awareness. Familiarity with JVM compilation, `javac` / `kotlinc`, and basic `jar` / `war` packaging is assumed.

1. Maven — the lifecycle and the POM

Maven's central abstraction is the **Project Object Model** (POM): an XML document that declares project metadata, dependencies, plugins, and profiles. Every Maven build executes a fixed **lifecycle** — a sequence of named phases — and each phase is bound to one or more **plugin goals** that perform the actual work.

1.1 The default lifecycle phases

The `jar` packaging lifecycle (the most common) runs these phases in order:



Running `mvn package` executes `validate` → `compile` → `test` → `package`. Running `mvn install` adds `verify` → `install`. The lifecycle is deterministic — every phase executes its preceding phases first, and plugins declared in the POM are bound to phases via their `<executions>` or via `packaging-default` bindings (e.g., `maven-jar:jar` automatically binds to the `package` phase when `<packaging>jar</packaging>`).

1.2 POM inheritance and the effective POM

Maven POMs inherit from a parent POM and ultimately from the **Super POM** (`org.apache.maven:maven-model`). The **effective POM** is the fully-resolved result of inheritance, property interpolation, and plugin defaults:

```

<!-- Parent POM – manages versions for a platform team's shared
libraries -->
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example.platform</groupId>
  <artifactId>platform-parent</artifactId>
  <version>3.2.0</version>
  <packaging>pom</packaging>

  <properties>
    <java.version>21</java.version>
    <kotlin.version>2.0.21</kotlin.version>
    <jackson.version>2.17.2</jackson.version>
    <micrometer.version>1.13.4</micrometer.version>
    <junit.version>5.10.3</junit.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>${jackson.version}</version>
      </dependency>
      <dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-core</artifactId>
        <version>${micrometer.version}</version>
      </dependency>
      <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
      </dependency>
    <!-- BOM import – brings in Spring Boot's curated versions
-->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>3.3.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

  <build>
    <pluginManagement>
      <plugins>

```



```

        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
          <configuration>
            <source>${java.version}</source>
            <target>${java.version}</target>
          </configuration>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>

```

A child module inherits this parent and only declares what's specific to itself:

```

<!-- Child module – order-service -->
<project>
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>com.example.platform</groupId>
    <artifactId>platform-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <artifactId>order-service</artifactId>
  <packaging>jar</packaging>

  <dependencies>
    <!-- Version inherited from parent dependencyManagement -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

```

The effective POM for `order-service` now includes all plugin configurations, properties, and managed versions from the parent — you can inspect it with `mvn help:effective-pom`.

1.3 Transitive dependency resolution and nearest-wins

When your project depends on `A`, and `A` depends on `B:1.0`, Maven resolves `B` transitively. But if `A` depends on `B:1.0` and `C` depends on `B:2.0`, Maven must choose. The algorithm:

1. Build the full dependency graph (all transitive paths).
2. For each conflicting artifact (`groupId:artifactId`), select the version with the **shortest path** from the root project (nearest-wins).
3. If two versions have equal depth, Maven picks the one declared **first** in the POM (first-declaration-wins).

This is deterministic but not always correct. A library at depth 2 might require `B:2.0` for correctness, but Maven picks `B:1.0` because it sits at depth 1 from the root. The `maven-enforcer-plugin` catches these:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <id>enforce-banned-deps</id>
      <goals><goal>enforce</goal></goals>
      <configuration>
        <rules>
          <bannedDependencies>
            <excludes>
              <!-- Ban known-vulnerable transitive
versions -->
              <exclude>org.apache.logging.log4j:log4j-
core:[,2.17.1)</exclude>

          <exclude>com.google.guava:guava:*:*:compile</exclude>
            </excludes>
          </bannedDependencies>
          <dependencyConvergence>
            <!-- Fail if versions don't converge -->
          </dependencyConvergence>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>

```

The `dependencyConvergence` rule fails the build if any transitive dependency has version conflicts — surfacing the "nearest-wins" ambiguity before it reaches production. In a distributed system with dozens of microservices, each pulling its own transitive tree, uncontrolled version divergence is a deployment-time time bomb: a library that works locally with `B:1.0` may fail in a different service that resolved `B:2.0`.

1.4 Maven profiles and the reactor build

Maven **profiles** let you activate different configuration for different environments. Profiles can be activated by command-line flag, JDK version, OS, or property:

```

<profiles>
  <profile>
    <id>staging</id>
    <properties>
      <env.url>https://staging.example.com</env.url>
    </properties>
    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
          <configuration>
            <jvmArguments>-Dspring.profiles.active=staging<
/jvmArguments>
          </configuration>
        </plugin>
      </plugins>
    </build>
  </profile>
  <profile>
    <id>production</id>
    <activation>
      <property><name>env.CI</name><value>true</value></property>
    </activation>
    <!-- Additional hardening: signing, verification -->
  </profile>
</profiles>

```

Activate with `mvn package -Pstaging` or `-P!staging` to deactivate. Profiles are a common source of build non-determinism: if a profile is activated by a local property (e.g., JDK version), the same `pom.xml` produces different effective POMs on different machines. Prefer activation by explicit flags or CI environment variables over implicit conditions.

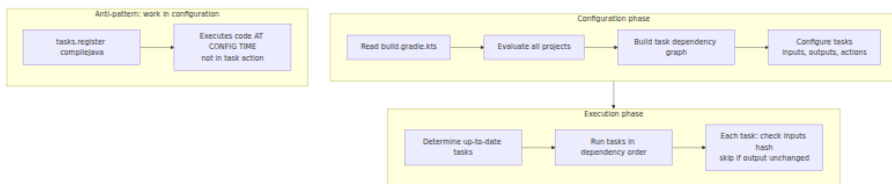
The **Maven reactor** is how multi-module projects are built. When you run `mvn package` from the root of a multi-module project, Maven determines the correct build order by analyzing inter-module dependencies and builds them sequentially (or in parallel with `-T 1C` for one thread per CPU core). The reactor ensures that if module B depends on module A, A is built first. If you run `mvn package -pl order-service -am`, Maven builds only `order-service` and all modules it depends on (`-am` = also-make), skipping unrelated modules — a critical optimization for monorepos with dozens of modules. The `-pl` flag (project list) combined with `-am` (also-make) and `-amd` (also-make-dependent) gives

you fine-grained control over what gets built, which is essential when your monorepo contains 50+ modules but a change touches only two.

2. Gradle — Kotlin DSL, configuration cache, and version catalogs

Gradle replaces Maven's XML and fixed lifecycle with a **directed acyclic graph** (DAG) of **tasks**. Configuration is done in Groovy or Kotlin DSL (`build.gradle.kts`), and the build is expressed as a program — not a declarative tree. The critical distinction for senior engineers: Gradle separates **configuration time** (evaluating `build.gradle.kts`) from **execution time** (running task actions).

2.1 Configuration vs execution



The Configuration Cache — enabled via `org.gradle.configuration-cache=true` — serializes the result of the configuration phase to disk. On subsequent builds, Gradle skips configuration entirely if `build.gradle.kts` files haven't changed, jumping straight to execution. This is why you should never do I/O (network calls, file reads, shell commands) during configuration — it breaks the cache and makes builds non-reproducible.

2.2 A multi-module Kotlin DSL build

```
// settings.gradle.kts
rootProject.name = "platform"

include("libs:common", "libs:metrics", "services:order-service")

// Version catalog – centralized, type-safe dependency declarations
dependencyResolutionManagement {
    versionCatalogs {
        create("libs") {
            from(files("gradle/libs.versions.toml"))
        }
    }
}
```

```
# gradle/libs.versions.toml
[versions]
kotlin = "2.0.21"
jackson = "2.17.2"
micrometer = "1.13.4"
junit = "5.10.3"
spring-boot = "3.3.4"

[libraries]
jackson-databind = { module = "com.fasterxml.jackson.core:jackson-databind", version.ref = "jackson" }
jackson-kotlin = { module = "com.fasterxml.jackson.module:jackson-module-kotlin", version.ref = "jackson" }
micrometer-core = { module = "io.micrometer:micrometer-core", version.ref = "micrometer" }
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter", version.ref = "junit" }
spring-boot-starter = { module = "org.springframework.boot:spring-boot-starter", version.ref = "spring-boot" }

[bundles]
monitoring = ["micrometer-core"]
jackson-all = ["jackson-databind", "jackson-kotlin"]

[plugins]
spring-boot = { id = "org.springframework.boot", version.ref = "spring-boot" }
kotlin-jvm = { id = "org.jetbrains.kotlin.jvm", version.ref = "kotlin" }
```

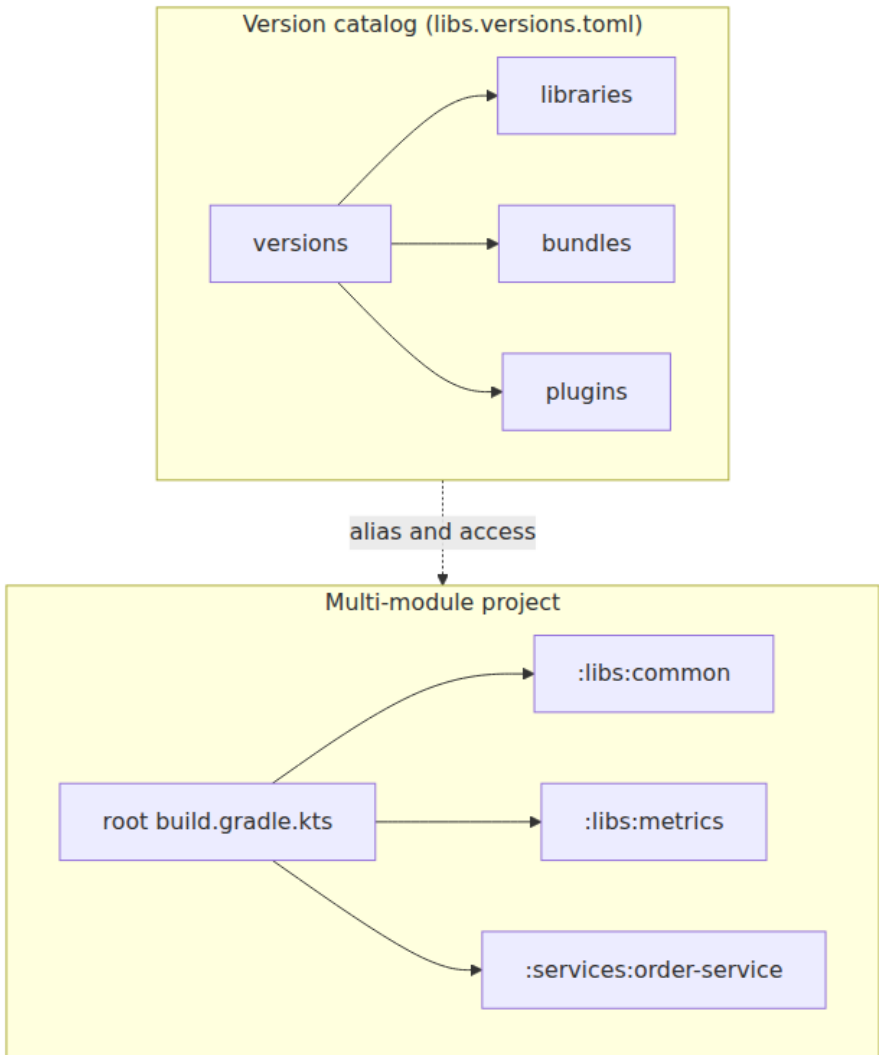
The version catalog replaces scattered `ext {}` blocks and hardcoded version strings. Every module in the build references the same catalog:

```
// build.gradle.kts for services/order-service
plugins {
    alias(libs.plugins.kotlin.jvm) apply true
    alias(libs.plugins.spring.boot) apply true
}

dependencies {
    implementation(project(":libs:common"))
    implementation(project(":libs:metrics"))

    // Type-safe catalog access – IDE completion, compile-time checking
    implementation(libs.bundles.jackson.all)
    implementation(libs.bundles.monitoring)

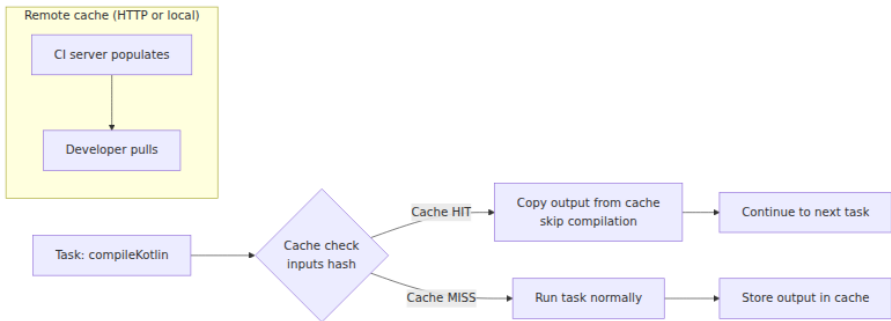
    testImplementation(libs.junit.jupiter)
}
```



The catalog is the single source of truth. Upgrading Jackson from 2.17.2 to 2.18.0 is a one-line change in `libs.versions.toml`, and every module picks it up. This eliminates the "grep for version strings across 40 `build.gradle` files" maintenance burden that plagues large JVM monorepos.

2.3 Build cache and remote caching

Gradle's build cache stores task outputs keyed by task inputs (source files, configuration, tool versions). On a cache hit, Gradle skips the task entirely:



Configure the build cache in `gradle.properties` :

```
# gradle.properties
org.gradle.caching=true
org.gradle.parallel=true
org.gradle.configuration-cache=true

# Remote cache (e.g., Gradle Enterprise, Develocity, or self-hosted)
# systemProp.gradle.cache.url=https://cache.example.com/
```

For a monorepo with 50 modules, remote caching can reduce CI build times from 45 minutes to under 10 by sharing compiled outputs across branches and developers. The cache is content-addressed: identical inputs always produce identical outputs. If your builds are not reproducible (e.g., they read timestamps or environment variables during configuration), the cache becomes unreliable — hence the Configuration Cache.

2.4 Convention plugins and buildSrc

As a Gradle multi-module build grows, `build.gradle.kts` files accumulate duplicated configuration (compiler flags, test framework setup, publishing defaults). Gradle's solution is **convention plugins** — shared build logic in `buildSrc/` or included builds:

```
// buildSrc/src/main/kotlin/quality-conventions.gradle.kts
// This is a precompiled script plugin – applies to any project that
// uses it

plugins {
    checkstyle
    jacoco
}

checkstyle {
    toolVersion = "10.18.2"
    configFile = rootProject.file("config/checkstyle/checkstyle.xml")
}

tasks.withType<Test> {
    useJUnitPlatform()
    reports {
        junitXml.required.set(true)
        html.required.set(true)
    }
}

jacoco {
    toolVersion = "0.8.12"
}

tasks.named<JacocoReport>("jacocoTestReport") {
    dependsOn(tasks.withType<Test>())
    reports {
        xml.required.set(true) // for CI coverage gates
        html.required.set(true)
    }
}
```

Consuming projects apply the convention with a single line:

```
// services/order-service/build.gradle.kts
plugins {
    id("quality-conventions") // from buildSrc – no version needed
    alias(libs.plugins.kotlin.jvm)
}
```

The convention plugin encapsulates a reusable build policy. When the platform team wants to add a new static analysis tool or change the test framework, they update one file in `buildSrc/` and every module picks it up on next build. This is Gradle's answer to Maven's parent POM plugin

management — but expressed as Kotlin code rather than XML inheritance, which makes it easier to test and reason about.

2.5 Composite builds — forking and substitution

Composite builds let you include other Gradle builds and substitute their published dependencies with local sources:

```
// settings.gradle.kts of a service that needs to patch a library
includeBuild("../jackson-kotlin-patch") {
    dependencySubstitution {
        substitute(module("com.fasterxml.jackson.module:jackson-module-
kotlin"))
        .using(project(":"))
    }
}
```

When you run `./gradlew build` in the service, Gradle resolves `jackson-module-kotlin` from the patched local build instead of Maven Central. This is invaluable for: - Testing a library patch before it's published - Coordinating changes across a library and its consumers - Splitting a monorepo without losing atomic change capability

3. sbt — incremental compilation and Ivy resolution

sbt is the default build tool for Scala, but it is also used for Kotlin/JVM projects in ecosystems that interact heavily with Scala libraries. Its model differs fundamentally from Maven and Gradle.

3.1 The build.sbt model

A `build.sbt` file is not declarative XML or a DSL wrapping a DAG — it is a **Scala program** that produces a build definition. Each top-level statement is a setting or task definition:

```
// build.sbt
ThisBuild / organization := "com.example"
ThisBuild / version      := "1.0.0"
ThisBuild / scalaVersion := "3.5.0"

lazy val common = (project in file("libs/common"))
  .settings(
    libraryDependencies += Seq(
      "com.fasterxml.jackson.core" % "jackson-databind" % "2.17.2",
      "org.typelevel"              %% "cats-core"         % "2.12.0"
    )
  )

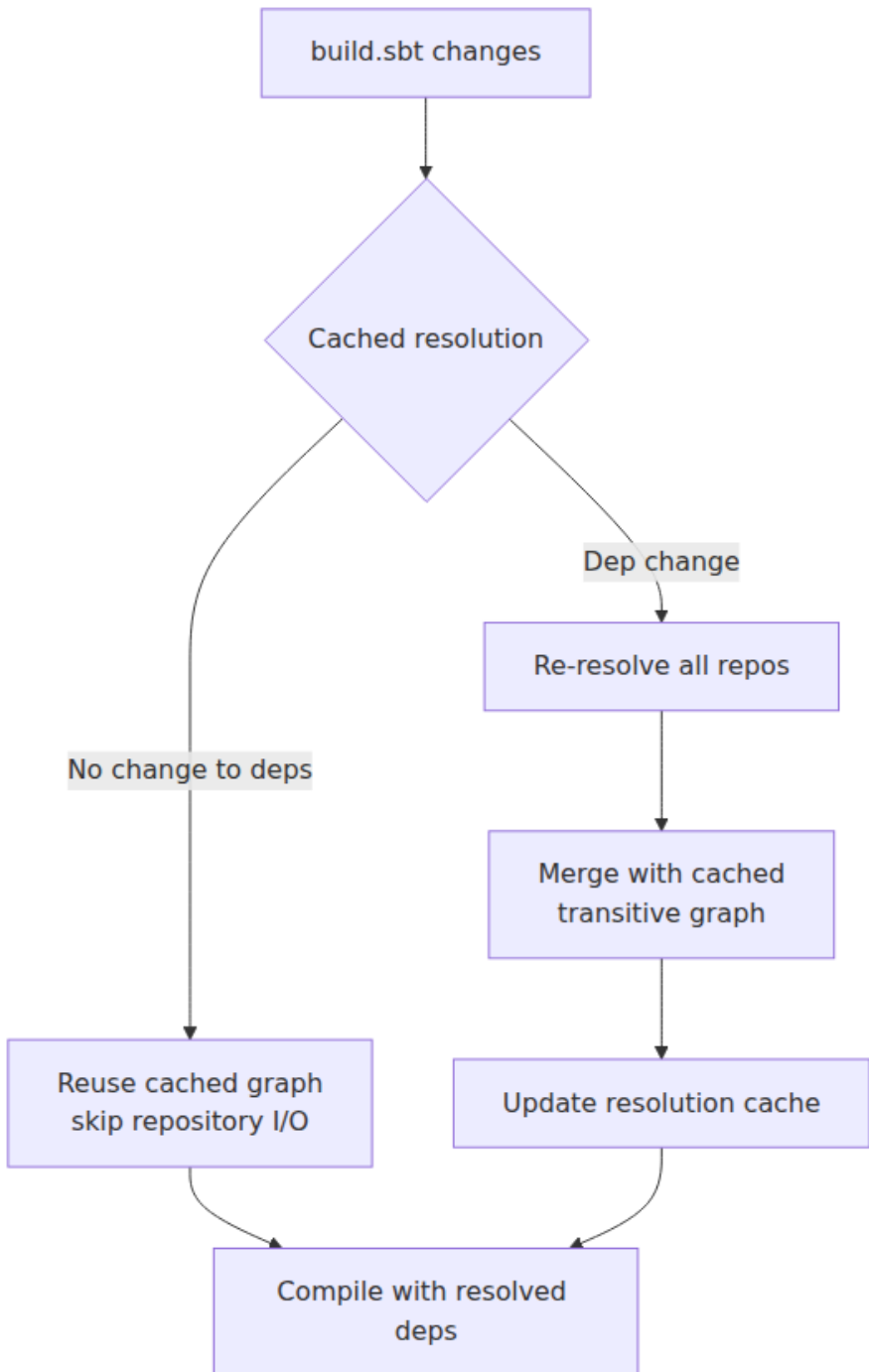
lazy val orderService = (project in file("services/order-service"))
  .dependsOn(common)
  .settings(
    libraryDependencies += Seq(
      "io.micrometer" % "micrometer-core" % "1.13.4",
      "org.scalatest" %% "scalatest"      % "3.2.18" % Test
    ),
    // sbt's incremental compilation – only recompiles changed sources
    incOptions := incOptions.value.withRecompileAllFraction(0.5)
  )
)
```

3.2 Ivy-based resolution

Unlike Maven and Gradle (which use Maven-style repository layout by default), sbt uses **Ivy** for dependency resolution. The key difference: Ivy supports **branch and revision selectors**, not just fixed versions:

```
libraryDependencies += Seq(
  // Fixed version
  "org.typelevel" %% "cats-core" % "2.12.0",
  // Version range – "any 2.x"
  "org.typelevel" %% "cats-core" % "[2.0.0,3.0.0)",
  // Latest release – uses Ivy's dynamic version resolution
  "org.typelevel" %% "cats-core" % "latest.release"
)
```

sbt caches resolution results in `~/.sbt/1.0/resolution-cache/` and in `project/target/`. The **cached resolution** feature (enabled by default in sbt 1.3+) stores the resolved dependency graph and only re-resolves when `build.sbt` or `*.sbt` files change. This avoids the overhead of hitting repositories on every compile.



sbt's incremental compiler (the Zinc compiler) tracks source-level dependencies at the individual definition level. If you change one function in one file, only the files that depend on that definition are recompiles. Zinc builds a dependency graph of Scala/Java source files and their API surfaces — a change to a `val` declaration recompiles everything that references it; a change to a local `private def` recompiles only the enclosing class.

3.4 sbt in practice — commands, scopes, and settings

sbt's interactive shell is its killer feature. Unlike Maven and Gradle (which require a separate command for every invocation), sbt provides a REPL where you can run tasks, inspect settings, and hot-reload changes:

```
$ sbt
sbt> compile           # compile all sources
sbt> test              # run all tests
sbt> orderService/test # test a specific subproject
sbt> show dependencyClasspath # inspect resolved classpath
sbt> reload            # re-read build.sbt after changes
```

sbt uses **scopes** to allow settings to vary by context. A setting can differ across subprojects, configurations (Compile vs Test), and tasks:

```
// Setting for Compile scope only
Compile / scalacOptions += "-Wunused:imports"

// Setting for Test scope only
Test / fork := true // run tests in a forked JVM

// Setting per-task
Test / javaOptions += Seq("-Xmx512m")
```

The **commands** vs **tasks** distinction is fundamental: a **task** is a function that computes a value (may be cached, may be skipped if inputs haven't changed). A **command** is an imperative action that modifies the build state (e.g., `reload`, `clean`, `set`). This separation enables sbt's incremental compilation — tasks like `compile` track their inputs and outputs, so a recompile only happens when sources actually change.

3.5 Comparing the three tools

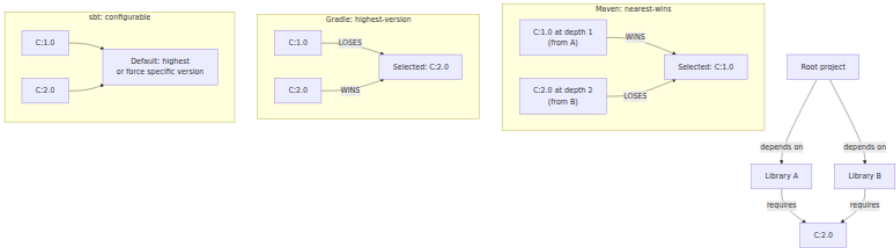
Aspect	Maven	Gradle	sbt
Config format	XML (pom.xml)	Kotlin/Groovy DSL	Scala (build.sbt)
Lifecycle	Fixed phases	Task DAG	Task DAG (commands are orthogonal)
Dependency resolution	Maven resolver (nearest-wins)	Gradle resolver (highest version)	Ivy resolver (configurable strategy)
Compilation	javac/kotlinc (batch)	javac/kotlinc (batch or daemon)	Zinc incremental compiler
Multi-project	Modules via parent POM	Composite builds / subprojects	Project definitions with <code>.dependsOn</code>
Caching	None built-in (use Maven Wrapper + CI)	Build cache + Configuration Cache	Cached resolution + Zinc incremental
Convention sharing	Parent POM inheritance	Convention plugins (buildSrc)	Auto-plugins (<code>.sbt</code> in <code>project/</code>)
Ecosystem	Java-dominant	JVM-wide (Java, Kotlin, Android, Scala, Groovy)	Scala-dominant, JVM
Learning curve	Low (XML is explicit)	Medium (DSL + task model)	High (Scala in build + scopes)

Each tool has earned its niche. Maven dominates Java enterprise precisely because its rigidity is a feature — when every project follows the same lifecycle, new developers can navigate any Maven project without learning a custom build DSL. Gradle dominates Android and Kotlin precisely because its flexibility handles the complexity of variant builds, multi-target compilation, and rapid iteration cycles. sbt dominates Scala because Zinc's incremental compilation is essential for the language's compilation speed, and the Scala-in-build-file approach lets you express complex build logic without a separate DSL.

4. Dependency resolution — conflict resolution, substitution, locking

Across all three tools, the dependency resolver must handle three problems: **transitive resolution** (pulling in indirect dependencies), **conflict resolution** (choosing one version when multiple are requested), and **reproducibility** (ensuring the same build produces the same artifact tomorrow).

4.1 Conflict resolution strategies



The difference matters in production. Maven's nearest-wins can silently select an older, vulnerable version if it happens to be closer in the dependency tree. Gradle's highest-version strategy avoids this but can break binary compatibility when a library expects a specific API that a newer version removed. sbt allows explicit control:

```
// Force a specific version – overrides all resolution strategies
dependencyOverrides += "org.apache.logging.log4j" % "log4j-core" % "2.17.2"

// Disable eviction logging for noisy transitive updates
update / evictionWarningOptions := EvictionWarningOptions.empty
```

Gradle provides similar control:


```

configurations.all {
    resolutionStrategy {
        // Force specific versions across all configurations
        force("org.apache.logging.log4j:log4j-core:2.17.2")

        // Fail on version conflicts instead of resolving automatically
        failOnVersionConflict()

        // Prefer specific module versions
        preferProjectModules()
    }
}

```

4.2 Dependency substitution and replace

Gradle allows **dependency substitution** — replacing one module with another at resolution time:

```

configurations.all {
    resolutionStrategy {
        dependencySubstitution {
            // Replace the Apache HttpClient with a fork
            substitute(module("org.apache.httpcomponents:httpclient"))
                .using(module("com.example:httpclient-fork:4.5.14-
patch"))
        }
    }
}

```

This is useful when: - You maintain a patched fork of a library and want to deploy it as a drop-in replacement - You're migrating from one groupId to another (e.g., a library moved under a new organization) - You need to apply a CVE fix before the upstream library publishes a release

4.3 Dependency locking

Locking freezes the resolved dependency graph to a file that is checked into source control. Every subsequent build uses the locked versions, regardless of what's available in repositories:

```

// build.gradle.kts
dependencyLocking {
    lockAllConfigurations()
}

```

After resolving, run `./gradlew dependencies --write-locks` to generate `gradle/dependency-locks/*.lockfile`. These lockfiles capture exact versions (including transitive) and checksums. On CI, run `./gradlew build --locked` to enforce that the lockfile matches the resolved graph — if a dependency drifts, the build fails.

Maven achieves a similar result with the `maven-dependency-plugin:tree` output pinned in CI, but it lacks Gradle's native lockfile format. sbt has no built-in lockfile mechanism, though the `sbt-dependency-lock` plugin provides similar functionality.

4.4 Verification metadata

Gradle 6.2+ introduced **dependency verification metadata** (`gradle/verification-metadata.xml`). This file contains checksums (SHA-256 by default) for every resolved artifact and POM. If an artifact's checksum doesn't match, the build fails — catching compromised or tampered dependencies:

```
# Generate initial verification metadata
./gradlew --write-verification-metadata sha256 help

# On CI – verify all dependencies match recorded checksums
./gradlew build
# If a dependency was tampered with, Gradle fails with:
# "Dependency verification failed for project ':services:order-
service'"
```

This is a hardening measure against supply-chain attacks (see Vol 9, Chapter 11 for threat modeling). When a dependency's POM or JAR is modified in Maven Central — whether through a compromised maintainer account or a repository mirroring attack — verification metadata catches the mismatch before it reaches production.

5. BOMs, platform constraints, and the Maven enforcer

5.1 Bills of Materials (BOMs)

A BOM is a POM with `<packaging>pom</packaging>` that declares `<dependencyManagement>` entries for a coherent set of libraries.

Consumers import the BOM with `<scope>import</scope>` to inherit all managed versions without pulling any actual dependencies:

```
<!-- Spring Boot BOM – imports hundreds of curated versions -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>3.3.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<!-- Now these resolve without explicit versions -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- Version from Spring Boot BOM -->
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <!-- Jackson version from Spring Boot BOM -->
  </dependency>
</dependencies>
```

In Gradle, the equivalent is the `platform()` function:

```
dependencies {
  implementation(platform("org.springframework.boot:spring-boot-
dependencies:3.3.4"))
  implementation("org.springframework.boot:spring-boot-starter-web")
  implementation("com.fasterxml.jackson.core:jackson-databind")
}
```

BOMs are a coordination mechanism for large organizations. The platform team publishes a BOM that pins compatible versions of all shared libraries (Jackson, Netty, Micrometer, SLF4J, etc.), and service teams import it. When a vulnerability is found in Netty 4.1.100, the platform team updates the BOM, and all services pick up the fix on their next rebuild — no service team needs to change their own build file.

5.2 BOM ordering and override semantics

Multiple BOMs can conflict. The resolution:

- **Maven:** BOMs are imported in declaration order. The first BOM to declare a managed version wins. A later BOM can override an earlier one only if it appears later in the same `<dependencyManagement>` section.
- **Gradle:** Later `platform()` declarations override earlier ones. You can also use `enforcedPlatform()` which forces versions even against explicit version declarations in individual dependencies.
- **sbt:** No native BOM concept. You emulate it with `dependencyOverrides` or by importing a library that transitively brings version management.

5.3 The Maven enforcer in practice

The enforcer plugin is Maven's policy engine for build governance:

```

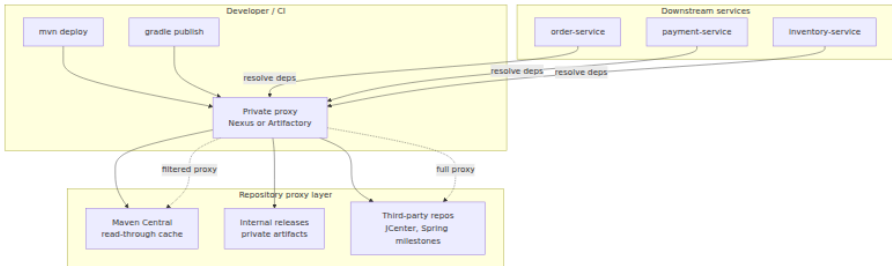
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <id>enforce</id>
      <goals><goal>enforce</goal></goals>
      <configuration>
        <rules>
          <!-- Require minimum Maven version -->
          <requireMavenVersion>
            <version>3.9.6</version>
          </requireMavenVersion>
          <!-- Require minimum JDK version -->
          <requireJavaVersion>
            <version>21</version>
          </requireJavaVersion>
          <!-- Fail on any dependency convergence issues -->
          <dependencyConvergence />
          <!-- Ban transitive dependencies with known
vulnerabilities -->
          <bannedDependencies>
            <excludes>
              <exclude>commons-collections:commons-
collections</exclude>
              <exclude>log4j:log4j</exclude>
            </excludes>
            <message>Banned due to known CVEs</message>
          </bannedDependencies>
          <!-- Require all dependencies come from approved
repositories -->
          <requireReleaseDeps>
            <onlyWhenRelease>true</onlyWhenRelease>
            <message>No Snapshots allowed in release
builds</message>
          </requireReleaseDeps>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>

```

In a distributed system context, the enforcer is your CI gate. Every merge request should pass `mvn verify` with enforcer rules active. The `dependencyConvergence` rule prevents the slow drift where two services end up with different versions of the same library — a common source of `NoSuchMethodError` in production when a shared library evolves.

6. Artifact repositories — Maven Central, Nexus, Artifactory, and signatures

6.1 The repository topology



6.2 Why a private proxy matters

Directly resolving from Maven Central in production CI/CD has several problems:

1. **Availability:** Maven Central is a CDN-backed service, but outages happen (the October 2019 outage lasted hours). A local proxy with cached artifacts keeps builds running.
2. **Security:** You cannot control what's published to Maven Central. A proxy lets you apply content filtering — block known-vulnerable artifacts, enforce signature verification, and audit what's being pulled.
3. **Speed:** A Nexus proxy on the same network as your CI cluster resolves artifacts in milliseconds instead of seconds.
4. **Auditability:** Every artifact resolved through the proxy is logged, giving you a complete dependency bill of materials for every build.

6.3 Nexus and Artifactory configuration

Sonatype Nexus Repository and JFrog Artifactory are the two dominant private repository managers. Both support:

- **Proxy repositories:** Cache artifacts from remote repositories (Maven Central, Spring milestones, etc.)
- **Hosted repositories:** Store your internal artifacts (libraries, BOMs, plugins)

- **Virtual repositories:** A single endpoint that aggregates proxy + hosted repositories

A typical Nexus configuration for a JVM platform team:

```
# Nexus repository configuration (simplified REST API)
repositories:
  # Proxy for Maven Central
  - name: maven-central-proxy
    format: maven2
    type: hosted
    online: true
    storage:
      writePolicy: ALLOW # proxy writes cache
    proxy:
      remoteUrl: https://repo1.maven.org/maven2/
      contentMaxAge: 1440 # 24 hours cache

  # Hosted repo for internal libraries
  - name: internal-releases
    format: maven2
    type: hosted
    online: true
    storage:
      writePolicy: ALLOW_ONCE # immutable releases

  # Virtual repo aggregating both
  - name: all-repositories
    format: maven2
    type: virtual
    members:
      - maven-central-proxy
      - internal-releases
```

6.4 GPG/PGP signing

Maven Central **requires** that all published artifacts are signed with GPG. The `maven-gpg-plugin` signs the JAR, POM, and checksums:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-gpg-plugin</artifactId>
  <version>3.2.7</version>
  <executions>
    <execution>
      <id>sign-artifacts</id>
      <phase>verify</phase>
      <goals><goal>sign</goal></goals>
      <configuration>
        <gpgArguments>
          <arg>--pinentry-mode</arg>
          <arg>loopback</arg>
        </gpgArguments>
      </configuration>
    </execution>
  </executions>
</plugin>

```

In Gradle, signing is configured via the `signing` plugin:

```

plugins {
    `maven-publish`
    signing
}

publishing {
    publications {
        create<MavenPublication>("maven") {
            from(components["java"])
            pom {
                url.set("https://github.com/example/order-service")
                licenses {
                    license {
                        name.set("Apache License, Version 2.0")
                        url.set("https://www.apache.org/licenses/
LICENSE-2.0")
                    }
                }
            }
        }
    }
}

signing {
    // Uses GPG key from ~/.gnupg/ or GRADLE_GPG_KEY properties
    sign(publishing.publications["maven"])
}

```


When a consumer resolves a signed artifact, Maven/Gradle verifies the PGP signature against the public key published to a key server (keys.openpgp.org). This prevents an attacker from substituting a malicious JAR in a repository — even if they compromise the storage layer, they cannot forge the signature without the private key.

6.5 Repository content filtering and firewalls

Nexus and Artifactory support **content selectors** and **firewall** policies that block resolution of known-vulnerable components:

- **Nexus Repository Firewall** (formerly Sonatype IQ): Scans every component entering your repository against the Sonatype vulnerability database. Components with critical CVEs are quarantined before they reach your builds.
- **Artifactory Xray**: Similar capability with JFrog's vulnerability database integration, plus license compliance scanning.

These are the operational counterparts to the Maven enforcer and Gradle verification metadata — the enforcer runs at build time in your project; the firewall runs at the repository level for all projects.

6.6 Maven Central publishing requirements

Publishing to Maven Central (via the Central Publishing Portal, formerly OSSRH) requires:

1. **Group ID ownership**: Verified via DNS TXT record or GitHub repository ownership.
2. **GPG signatures**: Every artifact (JAR, POM, sources JAR, javadoc JAR) must be signed. The `.asc` files are published alongside the artifacts.
3. **Sources and javadoc JARs**: Maven Central requires `-sources.jar` and `-javadoc.jar` for every release artifact.
4. **POM metadata**: `<name>`, `<description>`, `<url>`, `<licenses>`, `<developers>`, and `<scm>` must be present and non-empty.

```

<!-- Required POM metadata for Maven Central -->
<name>Order Service</name>
<description>Handles order lifecycle management for the e-commerce plat
form</description>
<url>https://github.com/example/order-service</url>
<licenses>
  <license>
    <name>Apache License, Version 2.0</name>
    <url>https://www.apache.org/licenses/LICENSE-2.0</url>
  </license>
</licenses>
<developers>
  <developer>
    <id>platform-team</id>
    <name>Platform Engineering</name>
    <email>platform@example.com</email>
  </developer>
</developers>
<scm>
  <connection>scm:git:https://github.com/example/order-service.git</
connection>
  <developerConnection>scm:git:ssh://github.com/example/order-
service.git</developerConnection>
  <url>https://github.com/example/order-service</url>
</scm>

```

In practice, most teams publish to a private Nexus/Artifactory first and promote to Maven Central only for open-source libraries. The private proxy handles the internal audience; Maven Central handles the external one. The signing key should live in a CI secret (GitHub Actions secrets, Vault, or a hardware token) — never on a developer's laptop where it can be exfiltrated.

7. The distributed-systems lens — build tooling at fleet scale

In a distributed system with hundreds of services, build tooling is infrastructure. The consequences of a bad build configuration propagate across the entire fleet:

7.1 Version convergence across services

When Team A's `order-service` resolves `netty-common:4.1.100.Final` and Team B's `payment-service` resolves `netty-common:4.1.108.Final`, you

have two services with different Netty versions in production. If both services communicate via Netty-based gRPC, and Netty 4.1.108 introduces a wire-protocol change, the services are now incompatible — but only under specific message patterns that existing integration tests don't cover.

The solution is a **platform BOM** (discussed in §5.1) enforced at the organization level. The platform team maintains a BOM, publishes it to the internal Nexus, and every service imports it. CI enforces convergence through the Maven enforcer or Gradle lockfiles.

Consider a real scenario: a payment processor's fleet of 200 Java microservices all depend on a shared `common-lib`. When `common-lib` 2.5.0 is released with a new serialization format, half the fleet upgrades immediately (they use SNAPSHOT versions in development) while the other half pins to 2.4.x. The result: `order-service` (on 2.5.0) sends events in the new format, but `fraud-detection` (on 2.4.x) cannot deserialize them. The production incident takes 4 hours to diagnose because the serialization format is not versioned — both versions claim the same content type. The fix: a platform BOM that pins `common-lib` to exactly one version across all services, with a CI gate that fails the build if the resolved version doesn't match the BOM.

7.2 Build reproducibility and CI determinism

A build that works on a developer's machine but fails in CI — or passes CI but produces a different artifact — is a reliability hazard. Sources of non-determinism:

Source	Maven	Gradle	sbt
Timestamp in JAR manifest	<code>project.build.outputTimestamp</code> (reproducible builds)	Not embedded by default	Not embedded by default
Dependency resolution order	Deterministic (POM order)	Deterministic (sorted by strategy)	Deterministic (Ivy cache)
Transitive version drift	Without enforcer: possible	Without lockfile: possible	Without pinning: possible

Source	Maven	Gradle	sbt
Plugin version drift	Without <code>maven-wrapper.properties</code> : possible	Without <code>wrapper/lock</code> : possible	Without <code>project/build.properties</code> : possible

Enable Maven reproducible builds:

```
<properties>
  <!-- Set to the release timestamp – strips timestamps from JARs -->
  <project.build.outputTimestamp>2024-08-21T00:00:00Z</
project.build.outputTimestamp>
</properties>
```

In Gradle, verify reproducibility:

```
# Check that all tasks produce reproducible output
./gradlew build
./gradlew clean build
# Compare the two outputs – they should be byte-identical
```

7.3 Gradle Build Scan for diagnosis

When a build is slow or fails mysteriously, the Gradle Build Scan (via Develocity, formerly Gradle Enterprise) is the diagnostic tool:

```
# Run a build with a scan
./gradlew build --scan

# Example output URL:
# https://scans.gradle.com/s/abc123xyz
```

The scan provides: - **Task execution timeline**: Which tasks ran, how long each took, which were cached. - **Configuration cache effectiveness**: How many projects were cached vs reconfigured. - **Dependency resolution graph**: The full resolved dependency tree with conflict resolution decisions. - **Build cache hit/miss ratio**: Which tasks hit the remote cache and which missed. - **Problem detection**: Identified deprecations, configuration cache misses, and performance bottlenecks.

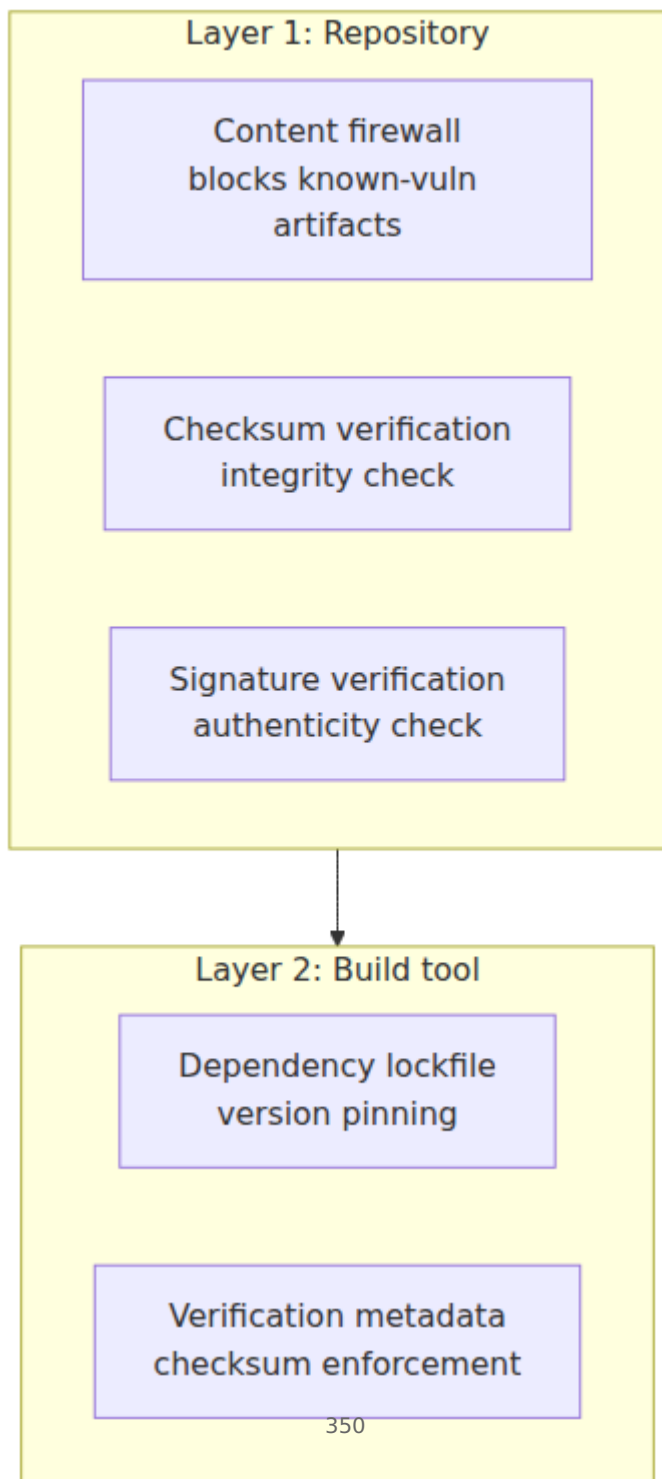
For a CI pipeline that runs 50+ Gradle builds per day across branches, build scans are the primary tool for detecting configuration cache regressions, identifying slow tasks for optimization, and debugging dependency resolution failures. When a developer reports "the build

takes 20 minutes on my machine but 5 minutes on CI," a build scan reveals whether the bottleneck is dependency resolution (slow on the developer's machine due to no local cache), compilation (lacking parallelism), or test execution (not forked or poorly partitioned).

Maven's diagnostic equivalent is the `-X` (debug) flag: `mvn package -X` prints the full reactor build order, every plugin goal execution, and the resolved dependency tree with conflict resolution decisions. For dependency-specific debugging, `mvn dependency:tree -DoutputType=dot` generates a Graphviz-compatible dependency graph that can be visualized to understand why a particular version was selected.

7.4 Supply-chain hardening through build tooling

Build tooling is the last line of defense before an artifact reaches production. The layered defense:



No single layer is sufficient. Maven Central had typosquatting attacks in 2020 and 2021 where packages named `com.google.mavem` (note the typo) exfiltrated credentials. The repository firewall catches known-malicious packages; the build tool catches checksum mismatches and version drift; the CI pipeline generates SBOMs for audit and scans for newly-disclosed CVEs.

For JVM projects specifically, the attack surface includes not just direct dependencies but also Maven plugins (which execute arbitrary code during the build), annotation processors (which run during compilation), and test dependencies (which execute in CI with elevated permissions). A compromised `maven-compiler-plugin` could inject bytecode during compilation, bypassing all source-level security measures. The defense: pin plugin versions in `<pluginManagement>`, use Gradle's plugin verification (`gradle/verification-metadata.xml`), and run builds in isolated containers with no network access except to approved repositories.

8. Key takeaways

- **Maven's lifecycle is deterministic and plugin-driven.** The fixed phase sequence (validate → compile → test → package → install → deploy) makes builds predictable. The POM inheritance model works for large organizations via parent POMs and BOMs, but nearest-wins dependency mediation can silently select wrong versions — enforce convergence with `maven-enforcer-plugin`.
- **Gradle separates configuration from execution.** The Configuration Cache eliminates configuration overhead by serializing the result. The build cache eliminates redundant task execution. Version catalogs provide type-safe, centralized dependency management. Composite builds enable cross-project development without publishing.
- **sbt's Ivy-based resolution and Zinc incremental compiler** offer unique advantages for Scala-heavy JVM projects. Cached resolution avoids repository I/O, and Zinc's definition-level granularity minimizes recompilation.
- **Version locking is non-negotiable in production.** Gradle lockfiles (`--write-locks` / `--locked`), Maven's `flatten-maven-plugin` with

locked dependency trees, or sbt's `sbt-dependency-lock` — whichever tool you use, lock your dependency graph and verify it on CI.

- **Verification metadata catches tampering.** Gradle's `verification-metadata.xml` (SHA-256 checksums for every artifact) and GPG signatures provide defense-in-depth against supply-chain compromise.
 - **Private proxies (Nexus/Artifactory) are not optional.** They provide caching, content filtering, audit logging, and a single resolution endpoint. For a fleet of hundreds of services, a proxy is the difference between "Maven Central is down, CI is blocked" and "builds continue unaffected."
 - **The platform BOM is the coordination mechanism.** One team maintains the BOM; all teams import it. When a CVE drops, one BOM update propagates to every service on next rebuild.
 - **Build scans are the diagnostic tool.** `gradle build --scan` and `mvn validate -X` are your first moves when a build is slow, non-deterministic, or fails in CI but works locally. For dependency-specific debugging, `mvn dependency:tree` and `./gradlew dependencies` show the full resolved graph with conflict resolution annotations.
-

Further reading

- **Maven POM Reference:** <https://maven.apache.org/guides/mini/guide-configuring-pom.html>
- **Maven Enforcer Plugin:** <https://maven.apache.org/enforcer/enforcer-rules/index.html>
- **Gradle Kotlin DSL:** https://docs.gradle.org/current/userguide/kotlin_dsl.html
- **Gradle Version Catalogs:** <https://docs.gradle.org/current/userguide/platforms.html#sub:version-catalog>
- **Gradle Configuration Cache:** https://docs.gradle.org/current/userguide/configuration_cache.html
- **Gradle Build Cache:** https://docs.gradle.org/current/userguide/build_cache.html
- **Gradle Dependency Locking:** https://docs.gradle.org/current/userguide/dependency_locking.html

- **Gradle Dependency Verification:** https://docs.gradle.org/current/userguide/dependency_verification.html
- **sbt Documentation:** <https://www.scala-sbt.org/1.x/docs/>
- **sbt Zinc Incremental Compiler:** <https://www.scala-sbt.org/1.x/docs/How-binaries-are-cross-built.html>
- **Sonatype Nexus Repository:** <https://help.sonatype.com/en/sonatype-repository-oss.html>
- **JFrog Artifactory:** <https://jfrog.com/help/r/jfrog-artifactory-documentation>
- **Maven Reproducible Builds:** <https://maven.apache.org/guides/mini/guide-reproducible-builds.html>
- **Gradle Develocity (Build Scans):** <https://docs.gradle.com/develocity/gradle-plugin/current/>
- **Maven Central Publishing Requirements:** <https://central.sonatype.org/publish/publish-requirements/>

Chapter 10 — Profiling, Observability, and Performance Tuning (JFR, async-profiler, JMC, heap dumps)

What this chapter covers. A backend service that passes all functional tests can still fail in production — creeping p99, periodic STW pauses that trip circuit breakers, a slow memory leak that OOM-kills one pod per day, or a lock convoy that collapses throughput under load. The JVM ships the richest observability toolkit of any managed runtime, but it is useless unless you know which instrument to reach for, what bias it introduces, and how to turn a recording into a fix you can verify. This chapter dissects the four pillars of JVM observability — Flight Recorder, sampling profilers, heap analysis, and GC logs — and assembles them into a continuous profiling and tuning workflow you can run across a fleet. You will record and stream JFR events, drive async-profiler through every mode, read flame graphs without being fooled by them, walk a heap

dump through Eclipse MAT's dominator tree, parse unified GC logs, and deploy Pyroscope and Parca for always-on profiling in Kubernetes.

Learning goals — after this chapter you should be able to:

- Explain why profiles are the fourth pillar of observability and how they complement metrics, logs, and traces — and why a profile can answer questions the other three cannot.
- Operate JDK Flight Recorder end-to-end: choose a recording configuration, start it with `jcmd` and `-XX:StartFlightRecording`, stream events with `jdk.jfr.consumer.RecordingStream`, author custom `@RegisteredEvent` types, and analyze a recording in JDK Mission Control.
- Distinguish async-profiler's `cpu`, `wall`, `alloc`, `lock`, and `nmt` modes, explain why `AsyncGetCallTrace` avoids safepoint bias, and generate and read flame graphs and differential flame graphs for each mode.
- Capture and dissect a heap dump (`jcmd GC.heap_dump`, `jmap`, `-XX:+HeapDumpOnOutOfMemoryError`), walk the `hprof` with `jhat` / `jhsdb` and Eclipse MAT, and use histogram, dominator tree, path-to-GC-roots, and leak-suspects to isolate the retainer of a leak.
- Parse unified GC logs (`-Xlog:gc*`) for G1, ZGC, and Shenandoah, compute allocation and promotion rates, and correlate GC pauses with JFR `jdk.GarbageCollection` and `jdk.GCPhasePause` events.
- Deploy continuous profiling with Grafana Pyroscope or Parca: configure the agent, label profiles by service/region/commit, control overhead and cardinality, and correlate profiles with traces via exemplars.
- Execute a disciplined tuning loop — measure, profile, fix, verify — and avoid the classic traps: optimizing what you did not measure, tuning flags without a profile, and declaring victory without a controlled before/after comparison.

Placement. Chapter 4 built the GC theory and log vocabulary this chapter instruments. Chapter 5 explained JIT compilation and deoptimization — the code whose quality you measure with profilers. Chapter 3 defined the object layout whose retention you diagnose in heap dumps. Chapter 12 (Production JVM) applies the tuning decisions you learn to make here to container sizing, CDS, and fleet rollout.

1. The fourth pillar — where profiling fits in observability

Metrics tell you *that* a service is slow (p99 jumped from 80 ms to 240 ms). Logs tell you *which* requests were slow. Traces tell you *which service* in the call graph was slow. Only a profile tells you *which instruction* was slow — which method, which allocation site, which lock, which kernel stack ate the CPU.

The four signals differ in dimensionality, cardinality, and cost:

Signal	Question it answers	Granularity	Cost of always-on
Metrics (Counters, Histograms)	Is the system healthy? What is the rate/latency/error budget?	Per-process or per-endpoint aggregates	Low — counters in memory, scraped every 15 s
Logs (structured JSON)	What happened for request X? What was the exception chain?	Per-request, per-event string	Medium — I/O and index cost; sampled or level-filtered
Traces (OpenTelemetry)	Where did time go across 40 services? Which span dominates?	Per-request DAG of spans	Medium — head or tail sampling required at high QPS
Profiles (JFR, async-profiler, eBPF)	Which code path consumed CPU, allocated memory, or blocked on a lock?	Per-stack-frame histogram sampled at 1-10 kHz	Low when sampled, high fidelity — the only signal that explains <i>why</i> CPU or memory is spent

Profiling is the missing differential in backend incident response. An alert fires on p99 latency (metric). You find the slow handler in a trace (span `checkout.validateCart` at 180 ms). Only the wall-clock profile shows that 140 ms of those 180 ms was inside `java.util.regex.Pattern.match` recompiling the same pattern per request — an allocation inside a hot loop invisible to both metrics and traces.

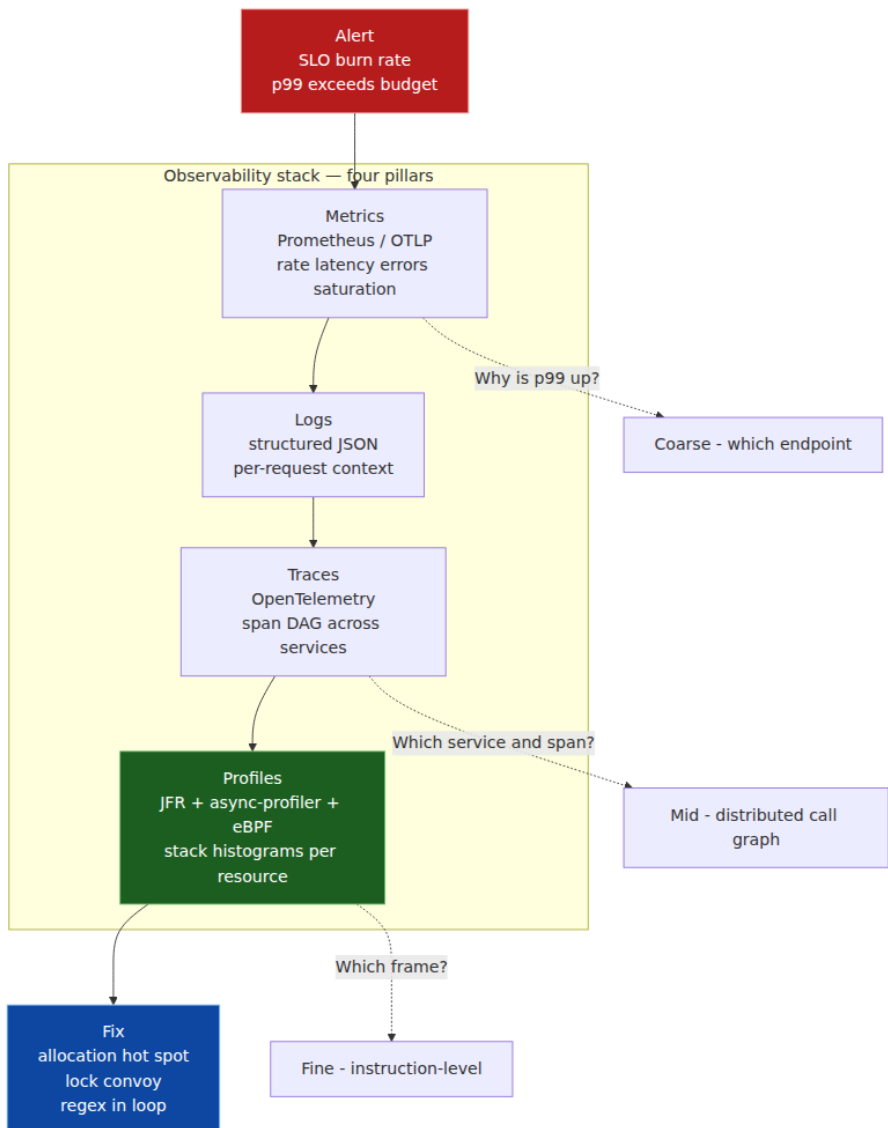
1.1 The cost model — sampling, instrumentation, and overhead budgets

Not all profiling is equal. The overhead and bias of a technique determine whether you can leave it on in production:

Technique	How it collects	Typical overhead	Bias / blind spot
JFR (Flight Recorder)	Instrumentation points in JDK/JVM code emit typed events into thread-local buffers	< 1% with <code>profile</code> config, 1-2% with everything enabled	Only sees what has an event site; needs event enabled
async-profiler <code>cpu</code>	<code>perf_events</code> + <code>AsyncGetCallTrace</code> samples at e.g. 10 kHz on <code>cpu-clock</code>	1-3%	Only running threads; misses idle/ waiting time
async-profiler <code>wall</code>	Periodic sampling of all threads regardless of <code>RUNNABLE</code> state	1-3% + timer signal	Includes idle time — must filter or it drowns CPU signal
async-profiler <code>alloc</code>	Intercepts <code>TLAB</code> allocation path sampling every N bytes	2-5%	Sampling, not exact count
async-profiler <code>lock</code>	JVMTI <code>MonitorContendedEnter</code> sampling / hotspot lock events	1-2%	Only contended locks
Heap dump (<code>jmap</code> / <code>jcmd</code>)	STW traversal of entire heap graph, serializes to <code>hprof</code>	STW pause proportional to live set (seconds for multi-GB)	Point-in-time snapshot only

Technique	How it collects	Typical overhead	Bias / blind spot
Java Flight Recorder Method Sampling	JFR <code>jdk.ExecutionSample</code> samples at 20 ms (50 Hz)	< 1%	Lower frequency than async-profiler; safepoint-aware sampling
eBPF / <code>perf</code> kernel stacks	Kernel <code>perf_events</code> samples user + kernel stacks	1-2%	Sees kernel frames JFR misses; no Java line numbers without mapping

In production, the correct default is **always-on JFR at the `profile` or `continuous` setting plus continuous profiling at low frequency** (e.g., async-profiler wall at 500 Hz or Pyroscope at 10 Hz). Reserve high-frequency or allocation profiling for targeted 60-second captures.



1.2 What senior teams get wrong

- **Metrics-only tuning.** Dashboards show `jvm_gc_pause_seconds` spiking but not *why*. Teams bump `-Xmx` or lower `MaxGCPauseMillis` without a profile — the real cause is a 4 MB per-request allocation inside a Jackson serializer, fixable with one buffer reuse.

- **One-off profiling in staging.** A 30-second `async-profiler` capture on a laptop with 1 QPS does not reproduce the lock contention that only appears at 8k QPS with 200 threads. Production sampling is non-negotiable for concurrency bugs.
- **Flame graph as truth without method.** A flame graph shows *where* time was spent, not whether that time was necessary, contended, or idle. Reading it without distinguishing `cpu` vs `wall` vs `alloc` produces confident wrong conclusions.

2. JDK Flight Recorder — the flight data recorder inside the JVM

JFR (JEP 328, open-sourced in JDK 11) is a low-overhead, always-on event system embedded in HotSpot and the JDK libraries. Think of it as the JVM's black box: a circular buffer of typed events that the JVM writes continuously, with near-zero overhead when no recording is active and ~1% when one is.

2.1 Architecture — buffers, chunks, and the repository



Each thread owns a small thread-local buffer. Events are written with `EventWriter.putLong/putString` without acquiring a global lock; only when the buffer fills is it linked to the global Repository under a short critical section. If no recording is active, the writer bails out after a single `shouldCommit()` check — the fast path is a single branch.

A **recording** is a time window over the repository: `jcmd <pid> JFR.start` opens a chunk, the JVM streams events into it, and `JFR.stop` or

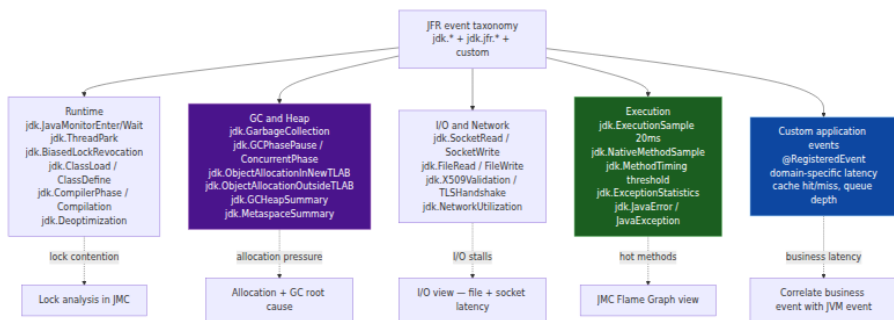
`JFR.dump` finalizes the chunk to a `.jfr` file. Chunks are self-describing — they carry their own constant pools and metadata so any chunk is independently readable by JMC or `jfr print`.

There are three lifecycles:

Lifecycle	How started	When lan dis
In-memory circular buffer	Default if <code>FlightRecorder</code> is enabled at startup	New dum
Continuous disk recording	<code>-XX:StartFlightRecording=disk=true,maxChunkSize=12M</code>	Rot chu rep
Streaming (<code>RecordingStream</code>)	Programmatic <code>new RecordingStream()</code>	Pipe con nev touc

2.2 Event taxonomy — what the JVM can tell you

JFR defines 140+ event types. Every event carries `startTime`, `duration` (for duration events), `eventThread`, and `stackTrace`. The taxonomy matters because enabling the wrong set blinds you at 1% cost, while enabling everything costs 2% and buries signal in noise.



The two most important selectors for backend work are `profile.jfc` and `continuous.jfc` (both shipped in `$JAVA_HOME/lib/jfr/`). Their practical difference:

Setting file	<code>jdk.ExecutionSample</code>	<code>jdk.ObjectAllocation*</code>	<code>jdk.JavaMonitor</code> threshold
<code>default.jfc</code>	period 20 ms	off	10 ms
<code>profile.jfc</code>	period 20 ms	every TLAB alloc (sampling with throttle)	10 ms
<code>continuous.jfc</code>	period 20 ms	sampled	20 ms

For always-on fleet recording, start from `profile.jfc` and tune thresholds: raise `jdk.FileRead` threshold to 20 ms to cut file I/O noise, enable `jdk.ObjectAllocationInNewTLAB#throttle=100/s` to cap allocation event rate under high allocation pressure.

2.3 Recording — command line, `jcmt`, and programmatic

At startup — the correct default for every production service:

```

# JDK 17+ – continuous 24-hour in-memory ring + 7-day disk rotation
java \
  -XX:StartFlightRecording=name=continuous,settings=profile,disk=true,d
  umponexit=true,\
  maxChunkSize=12M,maxAge=7d,repository=/var/log/jfr,filename=/var/log/
  jfr/app.jfr \
  -XX:FlightRecorderOptions=repository=/var/log/jfr,maxchunksize=12M,gl
  obalbuffersize=512k,threadbuffersize=8k \
  -Xlog:jfr*=info \
  -jar service.jar

# Alternative – explicit two-recordings pattern: memory ring + periodic
# dump
java \
  -XX:StartFlightRecording=name=mem,settings=profile,maxSize=200M \
  -jar service.jar
# Then periodically: jcmd <pid> JFR.dump name=mem filename=/var/log/
# jfr/$(date +%s).jfr

```

At runtime with `jcmd` — no restart required:

```

# Discover the JVM PID
jcmd | head
# 12345 com.example.CheckoutService

# Start a 60-second high-fidelity capture
jcmd 12345 JFR.start name=incident-20260812 \
    settings=profile \
    duration=60s \
    filename=/tmp/incident-20260812.jfr \
    maxsize=100M

# Check active recordings
jcmd 12345 JFR.check
# Recording: name=incident-20260812 duration=60s state=RUNNING ...

# Dump the in-memory ring without stopping (useful when FlightRecorder
was started at boot)
jcmd 12345 JFR.dump name=continuous filename=/tmp/continuous-now.jfr

# Stop and finalize
jcmd 12345 JFR.stop name=incident-20260812

# List available event types and their current thresholds
jcmd 12345 JFR.configure

# Inspect a recording without JMC – headless triage on a bastion host
jfr print --events jdk.GarbageCollection,jdk.GCPhasePause,jdk.Execution
Sample \
    --stack-depth 64 /tmp/incident-20260812.jfr | head -n 120

# Summary view – event counts and total times
jfr summary /tmp/incident-20260812.jfr
# jdk.ExecutionSample : 18234 events
# jdk.ObjectAllocationInNewTLAB : 89210 events (total allocated 412
MB)
# jdk.JavaMonitorEnter : 412 events (max duration 184 ms)
# jdk.SocketRead : 8201 events

```

Kubernetes — add a sidecar or init pattern for shipping chunks:

```

# Deployment snippet – mount an emptyDir for the JFR repository and
ship with a sidecar
spec:
  template:
    spec:
      volumes:
        - name: jfr-repo
          emptyDir: { sizeLimit: 1Gi }
      containers:
        - name: app
          image: registry.example.com/checkout:1.4.2
          env:
            - name: JAVA_TOOL_OPTIONS
              value: >-
                -XX:StartFlightRecording=name=continuous,settings=profiling,
                disk=true,repository=/jfr,maxAge=2h,maxSize=500M
                -XX:FlightRecorderOptions=repository=/jfr
          volumeMounts:
            - { name: jfr-repo, mountPath: /jfr }
          # Dump on OOM for post-mortem even if the pod is evicted
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "jcmd 1 JFR.dump name=continuous filename=/jfr/dump-$(date +%s).jfr || true"]
            - name: jfr-shipper
              image: registry.example.com/jfr-shipper:0.3
              args: ["--repository=/jfr", "--sink=s3://jfr-archive/${POD_NAME}/", "--interval=60s"]
              volumeMounts:
                - { name: jfr-repo, mountPath: /jfr }

```

2.4 Streaming — `jdk.jfr.consumer.RecordingStream` (JDK 14+)

Continuous disk recording is durable; streaming is *live*. A `RecordingStream` subscribes to events as they are committed, before chunk rotation, with back-pressure via the consumer thread.

```

import jdk.jfr.consumer.RecordingStream;
import java.time.Duration;

public final class JfrLiveMonitor {

    public static void main(String[] args) throws Exception {
        try (var stream = new RecordingStream()) {

            // Enable only what the live pipeline needs – keep overhead
            minimal
            stream.enable("jdk.GarbageCollection").withThreshold(Duration.ofMillis(0));
            stream.enable("jdk.GCHeapSummary").withPeriod(Duration.ofSeconds(1));
            stream.enable("jdk.JavaMonitorEnter").withThreshold(Duration.ofMillis(20));
            stream.enable("jdk.ThreadPark").withThreshold(Duration.ofMillis(50));
            // ExecutionSample at 20 ms is already the default when
            enabled
            stream.enable("jdk.ExecutionSample").withoutStackTrace(); //
            / lightweight counter

            stream.enable("com.example.CheckoutLatency").withThreshold(Duration.ofMillis(0));

            stream.onEvent("jdk.GarbageCollection", event -> {
                long gcId = event.getLong("gcId");
                String name = event.getString("name");           // "G1
                Young Generation"
                long pause = event.getDuration().toMillis();
                if (pause > 100) {
                    System.err.printf("[JFR-STREAM] Long GC pause
gcId=%d name=%s pause=%dms\n",
                                gcId, name, pause);
                    // push to Micrometer / OTel metric, or trigger
                    jcmd JFR.dump
                }
            });

            stream.onEvent("jdk.JavaMonitorEnter", event -> {
                String monitorClass = event.getString("monitorClass");
                long durationMs = event.getDuration().toMillis();
                var stack = event.getStackTrace();
                System.err.printf("[JFR-STREAM] Monitor contention %s
blocked %dms at %s\n",
                                monitorClass, durationMs,
                                stack != null && !stack.getFrames().isEmpty()
                                    ? stack.getFrames().get(0) : "<no stack>");
            });
        }
    }
}

```

```

        stream.onEvent("com.example.CheckoutLatency", event -> {
            long latencyMs = event.getLong("latencyMs");
            String outcome = event.getString("outcome");
            if (latencyMs > 500) {
                // Correlate domain event with JVM state in the
same time window
                System.err.printf("[JFR-STREAM] Slow checkout
latency=%dms outcome=%s%n",
                    latencyMs, outcome);
            }
        });

        // Blocks and dispatches on a dedicated thread – never call
from a latency-sensitive thread
        stream.start();
    }
}

```

Operational notes:

- `RecordingStream` enables the Flight Recorder if it was not already enabled; `FlightRecorder.isAvailable()` will flip to `true` on first stream creation. The overhead is that of the enabled events only.
- Always set thresholds and periods explicitly. `withoutStackTrace()` is valuable for high-frequency events like `jdk.ExecutionSample` when you only need a count.
- Run the stream on a dedicated thread or off-heap forwarder; blocking the callback blocks JFR's event dispatch.
- Streaming and disk recording coexist — a `RecordingStream` does not consume events from the disk chunks; it subscribes to the live flow independently.

2.5 Custom events — instrument the domain, not just the runtime

The most powerful JFR feature for backends is not the built-in events but the two lines that add a domain event:

```

import jdk.jfr.*;

@Name("com.example.CheckoutLatency")
@Label("Checkout Latency")
@Description("End-to-end checkout handler latency including payment and
inventory")
@Category({"Domain", "Checkout"})
@StackTrace(false)           // domain event – stack is usually noise;
enable if you need caller context
@Threshold("20 ms")          // only commit if duration >= 20 ms – cuts
volume by ~90%
@Registered
public final class CheckoutLatencyEvent extends Event {
    @Label("Latency (ms)") long latencyMs;
    @Label("Outcome")      String outcome;    // SUCCESS,
PAYMENT_FAILED, INVENTORY_SHORTAGE
    @Label("Cart Size")    int cartSize;
}

@Name("com.example.CacheAccess")
@Label("Cache Access")
@Category({"Domain", "Cache"})
@StackTrace(false)
public final class CacheAccessEvent extends Event {
    @Label("Cache")      String cacheName;
    @Label("Hit")        boolean hit;
    @Label("Key Hash")   int keyHash; // never log the raw key – hash
only
}

// Usage – structured as try-with-resources via begin()/commit() or the
Event helper:
public CheckoutResult checkout(Cart cart) {
    var event = new CheckoutLatencyEvent();
    event.begin();
    try {
        var result = doCheckout(cart);
        event.outcome = result.outcome().name();
        event.cartSize = cart.size();
        return result;
    } finally {
        event.latencyMs = event.duration().toMillis();
        if (event.shouldCommit()) { // respects @Threshold
            event.commit();
        }
    }
}

// Cache wrapper – emits one event per access, sampled by throttle in
jfc

```

```

public Value get(String key) {
    var event = new CacheAccessEvent();
    event.begin();
    event.cacheName = "product-cache";
    event.keyHash = key.hashCode();
    Value v = delegate.get(key);
    event.hit = (v != null);
    event.commit(); // no threshold – every access is interesting at
    throttle
    return v;
}

```

Wire thresholds through the `.jfc` rather than hard-coding them, so operators can tune without redeploying:

```

<!-- checkout-profile.jfc fragment – include via jcmd JFR.configure or
-XX:FlightRecorderOptions:settings= -->
<configuration version="2.0" label="Checkout profile" provider="Checkout
t Service">
    <event name="com.example.CheckoutLatency">
        <setting name="enabled">true</setting>
        <setting name="stackTrace">false</setting>
        <setting name="threshold">20 ms</setting>
    </event>
    <event name="com.example.CacheAccess">
        <setting name="enabled">true</setting>
        <setting name="throttle">100/s</setting> <!-- cap event rate -->
    </event>
</configuration>

```

Loaded with `jcmd <pid> JFR.start settings=/etc/jfr/checkout-profile.jfc`.

2.6 JDK Mission Control — reading the recording

JDK Mission Control (JMC, `org.openjdk.jmc`, standalone download from `jdk.java.net/jmc`) is the primary GUI for `.jfr` files. For headless environments, `jfr print` and `jfr summary` (JDK 14+, in `$JAVA_HOME/bin`) cover triage. JMC's value is correlation — every view is time-linked.

A disciplined JMC walkthrough for a latency incident proceeds in this order:

1. **Overview → Automated Analysis.** JMC's rule engine flags long GC pauses, high allocation rate, contended locks, and blocked threads. Treat it as a checklist, not a diagnosis — click through each finding to its source event.

2. **Java Application → Method Profiling (JFR Method Sampling).**
This is `jdk.ExecutionSample` aggregated into a hot-methods table. Sort by *Sample Count* descending. If `java.util.regex.Pattern$Curly.match` is top with 18% of samples, you have a regex hot spot. The *Flame Graph* tab in JMC 8+ renders the same data as an interactive flame graph — zoom into the widest plateau.
3. **Java Application → Allocations → TLAB Allocations.** Sort by *Total Allocated* or *Allocation Count*. The *Allocation Stack Traces* view shows the exact call site. A single `byte[]` allocation at `com.example.JsonCodec.serialize` at 1.2 GB/s explains Young GC pressure without ever looking at GC logs.
4. **Java Application → Lock Instances / Java Blocking.**
`jdk.JavaMonitorEnter` events sorted by *Duration* reveal the contended monitor and the stack that blocked. Correlate with `jdk.ThreadPark` and `jdk.JavaMonitorWait` to distinguish lock contention from `LockSupport.park` / `CompletableFuture` waiting.
5. **JVM Internals → Garbage Collections.** Timeline of `jdk.GarbageCollection` and `jdk.GCPhasePause` events. Select a long pause — the *References* and *Heap Summary* panels show heap occupancy before/after. A Young GC that reclaims only 12% of Eden indicates premature promotion or a leak.
6. **JVM Internals → JIT Compilations and Code Cache.**
`jdk.Compilation` events show deoptimizations (unstable inlining) and code-cache pressure. A burst of `made not entrant` / `made zombie` followed by falling JIT throughput signals code-cache churn — raise `ReservedCodeCacheSize`.
7. **Custom Events → `com.example.*`.** Your domain events appear alongside JVM events on the same timeline. Select a slow `CheckoutLatency` event — the 500 ms around it shows which GC, lock, and allocation events co-occurred. This is the correlation that metrics alone cannot provide.

```
# Headless JMC rule evaluation – run in CI or on a bastion, no GUI
# Requires org.openjdk.jmc:flightrecorder.rules
java -jar org.openjdk.jmc.flightrecorder.rules.headless.jar \
  --recording /tmp/incident-20260812.jfr \
  --format json > /tmp/jmc-analysis.json
# Produces rule violations with severity, score, and remedial message

# Programmatic JFR parsing – alternative to JMC when automating fleet
analysis
# Using jdk.jfr.consumer.RecordingFile (JDK 14+)
```

```

import jdk.jfr.consumer.RecordingFile;
import jdk.jfr.consumer.RecordedEvent;
import java.nio.file.Path;
import java.util.*;

public final class JfrTriage {
    public static void main(String[] args) throws Exception {
        var path = Path.of(args[0]);
        long longPauses = 0, contendedLocks = 0;
        Map<String, Long> allocByClass = new HashMap<>();

        try (var file = new RecordingFile(path)) {
            while (file.hasMoreEvents()) {
                RecordedEvent e = file.readEvent();
                switch (e.getEventType().getName()) {
                    case "jdk.GarbageCollection" -> {
                        long pause = e.getDuration().toMillis();
                        if (pause > 50) longPauses++;
                    }
                    case "jdk.JavaMonitorEnter" -> {
                        if (e.getDuration().toMillis() > 20) contendedL
ocks++;
                    }
                    case "jdk.ObjectAllocationInNewTLAB" -> {
                        String cls = e.getString("objectClass");
                        long size = e.getLong("tlabSize");
                        allocByClass.merge(cls, size, Long::sum);
                    }
                }
            }
        }
        System.out.printf("Long GC pauses (>50ms): %d\n", longPauses);
        System.out.printf("Contended locks (>20ms): %d\n", contendedLoc
ks);
        allocByClass.entrySet().stream()
            .sorted(Map.Entry.<String,
Long>comparingByValue().reversed())
            .limit(10)
            .forEach(e -> System.out.printf("  alloc %-50s %8.1f MB\n",
e.getKey(), e.getValue() / (1024.0 * 1024)));
    }
}

```

3. async-profiler — sampling without safepoint bias

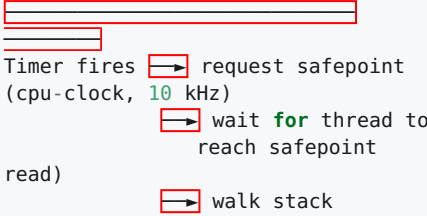
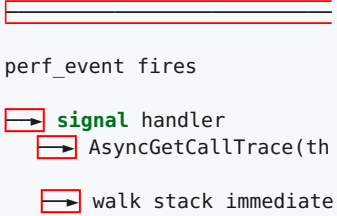
JFR's `jdk.ExecutionSample` is excellent for always-on baselines, but its 20 ms period (50 Hz) is too coarse for short-lived hot spots, and it samples at

safepoints — it can only observe a thread when the JVM brings it to a safepoint. `async-profiler` (by Andrei Pangin, now under `async-profiler/async-profiler` on GitHub) was built to fix exactly this limitation.

3.1 Why safepoint bias matters

HotSpot's classic `GetStackTrace` and `AsyncGetCallTrace`-free profilers (honest-profiler, old `hprof` cpu sampling) work by suspending threads at safepoints and walking stacks. The bias is systematic: code that never hits a safepoint (tight counted loops, `Unsafe` access, some intrinsics) is under-sampled, while safepoint-heavy code (allocations, method entries) is over-sampled. In Kotlin, an `IntArray` loop without allocation can appear *colder* than it is.

`async-profiler` avoids this by using `AsyncGetCallTrace` (AGCT) — a HotSpot-internal API that walks a thread's stack *asynchronously* from a `perf_events` or timer signal handler, without requiring the thread to be at a safepoint. Combined with `perf_events` (`cpu-clock` or `cpu-cycles` hardware counters) on Linux, it samples precisely where the CPU is, including inside JIT-compiled code and kernel frames.

Traditional safepoint sampler <code>perf_events</code>	<code>async-profiler</code> (AGCT + <code>perf_events</code>)
	
Bias: only safepoint states	Bias: none (sees all states)
Result: tight loops under-counted	Result: faithful CPU distribution

3.2 Modes — one profiler, five questions

`async-profiler` is not one profiler but five, selected by `-e` (event):

```
# Install – single static binary, no agent, no JVMTI attach overhead
until started
curl -L https://github.com/async-profiler/async-profiler/releases/
download/v3.0/async-profiler-3.0-linux-x64.tar.gz \
| tar xz -C /tmp
export AP=/tmp/async-profiler-3.0-linux-x64

# Discover target PID
jcmd | grep CheckoutService
# 12345 com.example.CheckoutService
```

Mode	Flag	What it samples	When to use
cpu	-e cpu	perf_events cpu-clock — on-CPU time	Hot methods burning CPU — the default first profile
wall	-e wall	Timer sampling every thread regardless of RUNNABLE	Latency — find where wall- clock time is spent including park / sleep / I/O
alloc	-e alloc	TLAB allocation sampling (bytes allocated, not object count)	Allocation pressure — find the call sites filling Eden
lock	-e lock	MonitorContendedEnter / ReentrantLock contention	Lock convoys — which monitor blocks the most thread-time
nmt	-e nmt	Native memory tracking deltas	Off-heap leaks — ByteBuffer.allocateDirect , JNI, metaspace
live	-- live + -e alloc	Live allocations (not yet GC'd)	Leak suspicion — what survives Young GC

CPU — the starting point for every investigation:

```

# 60-second CPU profile at 10 kHz (default), collapsed + flame graph
/tmp/async-profiler/profiler.sh start \
  -e cpu -i 100000 \
  --jfrsync profile \
  -f /tmp/cpu.jfr 12345
# ... let load run for 60 s ...
/tmp/async-profiler/profiler.sh stop 12345

# One-shot shorthand – profile for 30 s and emit an HTML flame graph
/tmp/async-profiler/profiler.sh -e cpu -d 30 -i 100000 \
  -f /tmp/cpu-flame.html 12345
# -e cpu          event
# -d 30           duration 30 s
# -i 100000       interval 100 us = 10 kHz sampling
# -f             output file (.html flame graph, .jfr, .collapsed, .jstack)

# JFR output – import directly into JMC alongside the JVM's own JFR
chunks
/tmp/async-profiler/profiler.sh -e cpu -d 30 -o jfr -f /tmp/cpu.jfr 123
45
jfr print --events jdk.ExecutionSample /tmp/cpu.jfr | head

```

Wall-clock — the latency profile:

```

# Wall-clock profiling – samples ALL threads including WAITING/PARKED
# Essential distinction: cpu shows "what burned CPU", wall shows "what
made the request slow"
/tmp/async-profiler/profiler.sh -e wall -d 30 -t \
  -f /tmp/wall-flame.html 12345
# -t include thread names in the flame graph (separate flame per
thread)

# With Kotlin coroutines – wall is the only mode that sees suspension
time
# A coroutine parked on Dispatchers.IO appears as WAITING in wall,
invisible in cpu
/tmp/async-profiler/profiler.sh -e wall -d 60 -t --cstack dwarf \
  -f /tmp/wall-coroutines.html 12345

```

Reading `cpu` vs `wall` correctly is the single most common async-profiler mistake. A concrete illustration:

```
cpu flame graph (30 s, 10 kHz)
500 Hz)
```



Wide **plateau:**

```
Pattern.match 22%
JsonCodec.serialize 18%
HashMap.get 11%
1%
...
```

Interpretation:

```
"We burn CPU in regex"
  fix the regex
x"
  but misses that 38% of
ream
  wall time is idle wait
```

```
wall flame graph (30 s,
```



Wide **plateau:**

```
LockSupport.park 38%
Pattern.match 9%
Jedis.get (socketRead) 2
JsonCodec.serialize 7%
```

Interpretation:

```
"We spend wall time parked
waiting for Redis + rege
real bottleneck is downst
I/O, not CPU
```

Always capture both. If `wall` is dominated by `park`/`wait`/`socketRead` while `cpu` is dominated by a compute kernel, the fix is concurrency or caching, not micro-optimization.

Allocation — find the garbage:

```
# Allocation profile — sampled every 512 KB by default (--alloc 512k)
# Reports bytes allocated per stack, not object count
/tmp/async-profiler/profiler.sh -e alloc -d 30 \
  -f /tmp/alloc-flame.html 12345

# Live allocation — only objects still reachable at dump time (leak
signal)
# Requires --live and a GC at the end of the window
/tmp/async-profiler/profiler.sh -e alloc --live -d 30 \
  -f /tmp/live-alloc.html 12345
```

An `alloc` flame graph wide at `byte[]` under `com.example.ProductImage.resize` at 800 MB/s explains why Eden fills every 2 seconds — no GC log analysis needed.

Lock — find the convoy:

```
# Lock contention – samples contended monitor entry
# Width = total time threads were BLOCKED on that monitor
/tmp/async-profiler/profiler.sh -e lock -d 30 -t \
  -f /tmp/lock-flame.html 12345

# With -t, each thread gets its own lane – identify the single lock
# that serializes 40 worker threads
```

A lock flame graph narrow but tall indicates a single monitor with high contention depth (many threads queueing). A wide, shallow lock graph indicates many distinct locks each with brief contention — usually not worth optimizing.

Native memory:

```
# NMT mode – requires -XX:NativeMemoryTracking=detail at JVM startup
# Shows native allocations outside the Java heap
/tmp/async-profiler/profiler.sh -e nmt -d 30 \
  -f /tmp/nmt-flame.html 12345

# Compare with jcmd NMT output for the same window
jcmd 12345 VM.native_memory summary scale=MB
jcmd 12345 VM.native_memory detail scale=MB
```

3.3 Kotlin specifics

Kotlin generates synthetic methods (`$default` for default parameters, `suspend` state-machine dispatch, `inline` call-site specialization) that appear as distinct frames in async-profiler output. Two caveats:

- **Inline functions** (`inline fun <T> Iterable<T>.filter(...)`) are inlined at the call site — the flame graph shows the call *site* method, not `filter` itself. If you inline a hot path, the flame graph correctly attributes its cost to the caller.
- **Coroutines** suspend by returning `COROUTINE_SUSPENDED` and resuming via `Continuation.resumeWith`. A `cpu` profile sees the state-machine dispatch (`BaseContinuationImpl.resumeWith`) as a hot frame; a `wall` profile sees `LockSupport.park` under `DefaultExecutor`. Correlate with JFR `jdk.ThreadPark` events to distinguish suspension from contention.


```
// This coroutine chain appears in profiles as:
suspend fun fetchProduct(id: String): Product =
    withContext(Dispatchers.IO) {           // wall: park here; cpu:
invisible while parked
    productCache.getOrLoad(id) {           // alloc: byte[] from
deserialization
    httpClient.get("/products/$id") // wall: socketRead
    }
    }

// cpu flame shows: DefaultExecutor.runWorker →
ContinuationImpl.resumeWith → fetchProduct$lambda
// wall flame shows: LockSupport.park → DefaultExecutor → fetchProduct
(suspended)
// alloc flame shows: byte[] → Jackson deserializer →
fetchProduct$lambda
```

3.4 Output formats and the `--jfrsync` bridge

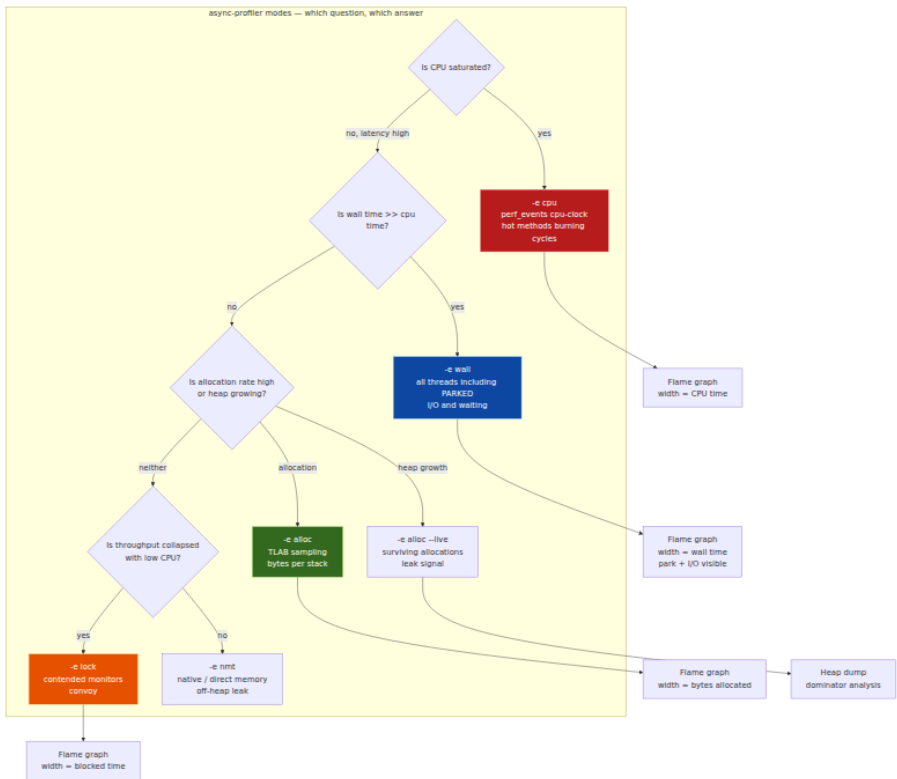
async-profiler can emit JFR (`-o jfr`), collapsed stacks (`.collapsed` for `flamegraph.pl`), HTML flame graphs, and `pprof` for continuous profiling backends:

```
# Emit JFR and merge with the JVM's own JFR – single file for JMC
/tmp/async-profiler/profiler.sh start -e cpu --jfrsync profile -f /tmp/
merged.jfr 12345

# Emit collapsed stacks for Brendan Gregg's flamegraph.pl
/tmp/async-profiler/profiler.sh -e cpu -d 30 -o collapsed -f /tmp/
stacks.collapsed 12345
/tmp/async-profiler/bin/flamegraph.pl /tmp/stacks.collapsed > /tmp/cpu-
flame.svg

# Emit pprof for Pyroscope/Parca ingestion
/tmp/async-profiler/profiler.sh -e cpu -d 30 -o pprof -f /tmp/
cpu.pprof 12345
curl -X POST http://pyroscope:4040/ingest \
  -H "Content-Type: application/octet-stream" \
  --data-binary @/tmp/cpu.pprof
```

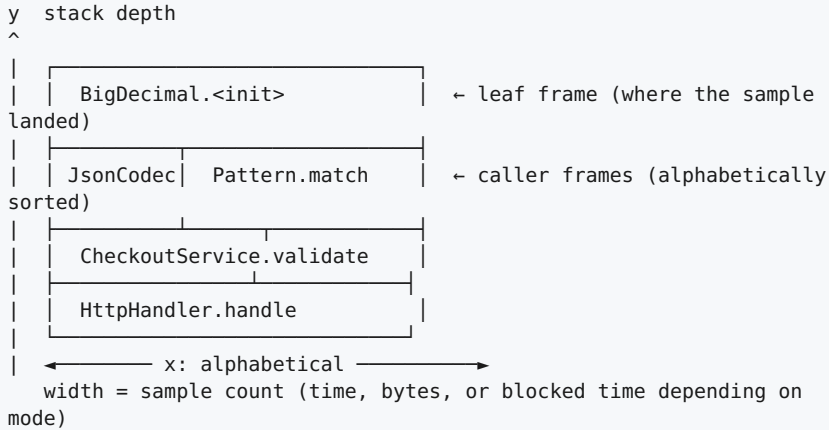
`--jfrsync` is the recommended bridge for production: it synchronizes async-profiler's samples with the JVM's JFR clock so both event streams share timestamps in JMC. Without it, correlating an async-profiler CPU hot spot with a JFR allocation or lock event requires manual time alignment.



4. Flame graphs — how to read them without fooling yourself

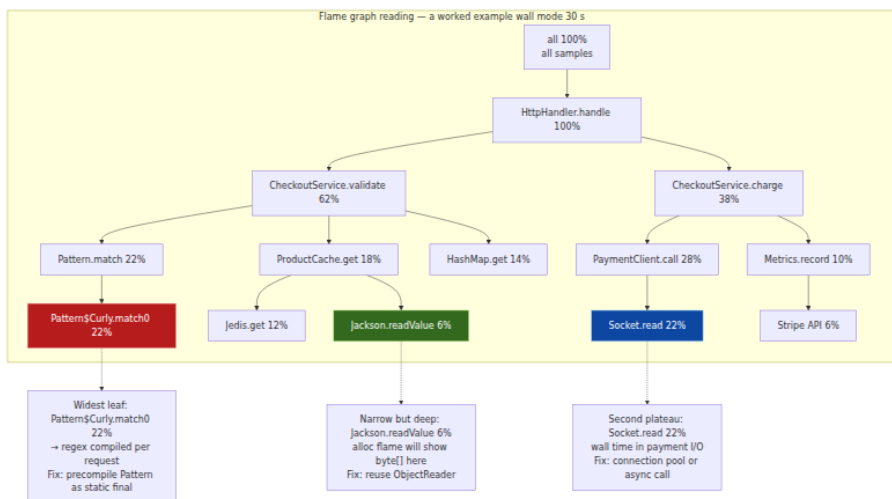
A flame graph is a histogram of stack traces, not a call graph. Every sample's stack is folded into a horizontal bar whose width is proportional to its frequency. The y-axis is stack depth; the x-axis is alphabetical, not temporal. The *widest plateau* is the hottest code — but only within the mode you sampled.

4.1 Anatomy of a flame graph



Reading rules for senior reviewers:

1. **Width is frequency, not causality.** A wide `HashMap.get` plateau does not mean `HashMap` is slow — it means many samples landed in `HashMap.get` *called from* the frames above it. Follow the stack downward to the domain method that *calls* `HashMap.get` in a loop.
2. **Alphabetical x-axis is not time.** Adjacent bars are not temporally adjacent. Do not infer call order from horizontal position — only from vertical stacking.
3. **Inverted (icicle) vs standard.** Standard flame graphs grow upward from `all` at the bottom; inverted (icicle) graphs grow downward from the root. Both encode the same data — JMC and `async-profiler` default to standard; `Speedscope` defaults to inverted. Agree on one for team reviews.
4. **Merged vs per-thread.** `async-profiler -t` emits per-thread lanes; without `-t`, all threads are merged. A lock convoy is invisible in the merged view — the per-thread view shows 40 threads all converging on one monitor frame.
5. **Differential flame graphs.** To verify a fix, capture before and after with identical load, then render the *difference*: `flamegraph.pl --diff`. Red frames grew, blue frames shrank. A successful allocation fix shows the `byte[]` plateau shrinking and `G1 Young GC` time shrinking with it.



4.2 Generating and comparing flame graphs

```
# Capture before the fix – collapsed stacks
/tmp/async-profiler/profiler.sh -e cpu -d 60 -o collapsed -f /tmp/
before.collapsed 12345
/tmp/async-profiler/profiler.sh -e alloc -d 60 -o collapsed -f /tmp/
before-alloc.collapsed 12345

# ... deploy fix: precompile regex, reuse Jackson ObjectReader ...

# Capture after the fix – same load, same duration, same interval
/tmp/async-profiler/profiler.sh -e cpu -d 60 -o collapsed -f /tmp/
after.collapsed 12345
/tmp/async-profiler/profiler.sh -e alloc -d 60 -o collapsed -f /tmp/
after-alloc.collapsed 12345

# Differential flame graph – red = more samples after, blue = fewer
/tmp/async-profiler/bin/diffgraph.pl /tmp/before.collapsed /tmp/
after.collapsed > /tmp/diff.svg
# Or with Brendan Gregg's toolkit:
flamegraph.pl --title "CPU diff: before → after" /tmp/after.collapsed
> /tmp/after.svg

# JFR-based flame graph – no async-profiler needed when JFR is already
running
jfr print --events jdk.ExecutionSample --stack-depth 128 /tmp/app.jfr \
| jfr-flame-graph --output /tmp/jfr-flame.html
# Alternative: open /tmp/app.jfr in JMC → Method Profiling → Flame
Graph tab → Export SVG

# Speedscope – interactive viewer for pprof / collapsed / JFR JSON
npx speedscope /tmp/cpu.pprof
npx speedscope /tmp/before.collapsed
```

For fleet-wide differential analysis, export both captures as `pprof` and compare in Pyroscope's *Comparison View* or with `pprof -diff_base`:

```
go tool pprof -diff_base=before.pprof after.pprof
# (pprof) top 20
#      flat flat% sum%      cum cum%
#    -420ms 18.2% 18.2%    -420ms 18.2% Pattern.match
#    -180ms 7.8% 26.0%    -180ms 7.8% byte[] allocation in
JsonCodec
```

5. Heap dumps — finding the dominator

When the heap grows monotonically, or a pod OOM-kills after 18 hours, or `jstat -gc` shows Old climbing without bound, the question is not how fast you allocate but *what survives*. A heap dump is a point-in-time snapshot of every object, its fields, and its references — the only artifact that can answer "what is retaining this memory?"

5.1 Capturing a dump — jcmd, jmap, and automatic triggers

All three paths produce the same `hprof` binary format. Prefer `jcmd` — it works without the `jmap` attach caveat and does not require `JDK_HOME/bin` on the container image.

```

# Preferred – jcmd, works on JDK 8+ and does not need jmap
# Triggers a STW heap traversal; duration ~ 1-3 s per GB of live heap
jcmd 12345 GC.heap_dump /tmp/heap-$(date +%Y%m%d-%H%M%S).hprof

# With live objects only – triggers a Full GC first to reclaim garbage,
then dumps
# Use when you want retained set, not floating garbage
jcmd 12345 GC.heap_dump -all=false /tmp/heap-live.hprof
# -all=false is the default for jcmd; jmap defaults to all=true

# Legacy – jmap (requires JDK, not JRE; may fail inside minimal
container images)
jmap -dump:live,format=b,file=/tmp/heap.hprof 12345
# live → Full GC before dump; omit for all objects including
unreachable
# format=b → binary hprof (only format MAT understands)

# Low-level – jhsdb jmap (works even when the process is hung, via SA)
jhsdb jmap --binaryheap --pid 12345 --dumpfile /tmp/heap.hprof

# Automatic – dump on OutOfMemoryError without human intervention
java \
-XX:+HeapDumpOnOutOfMemoryError \
-XX:HeapDumpPath=/var/log/heap/oom-%p-%t.hprof \
-XX:+ExitOnOutOfMemoryError \
-jar service.jar
# %p = PID, %t = timestamp; always set ExitOnOutOfMemoryError alongside
–
# a JVM that OOM'd but stays alive serves corrupt responses

# Kubernetes – ephemeral emptyDir + sidecar upload
# Never write a multi-GB hprof to the container's overlay FS – it will
fill the node
spec:
  containers:
  - name: app
    env:
    - name: JAVA_TOOL_OPTIONS
      value: "-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps/
oom-%p-%t.hprof"
    volumeMounts:
    - { name: heap-dumps, mountPath: /dumps }
  volumes:
  - name: heap-dumps
    emptyDir: { sizeLimit: 5Gi }

# Trigger manually inside a pod without exec – via a liveness-sidecar
HTTP endpoint
# that runs: jcmd 1 GC.heap_dump /dumps/manual-$(date +%s).hprof

```

Operational cautions:

- A heap dump is STW. On a 16 GB heap with 8 GB live, expect 8–20 seconds of pause. In Kubernetes, raise `terminationGracePeriodSeconds` and trigger dumps off-peak or on a canary, or use `jcmd GC.heap_dump` on a single pod behind a readiness-gate that removes it from the load balancer first.
- The `hprof` file is roughly the size of the live heap (compressed with no compression). A 12 GB live set produces a ~12 GB file — compress with `gzip --fast` before uploading (typically 3:1 ratio for object-heavy heaps). Ship to object storage (`s3://heap-dumps/...`), not to a developer laptop over `kubectl cp`.
- `jhsdb jmap` can dump a hung JVM that no longer responds to `jcmd` — it attaches via the Serviceability Agent and reads raw memory. Use it when `jcmd` times out.

5.2 The hprof format — what is inside

The `hprof` binary (`JAVA PROFILE 1.0.2` header) is a stream of records: heap dump segments, thread stacks, class definitions, and GC roots. Every object is identified by an `ID` (4 or 8 bytes depending on compressed oops) and carries its class ID, instance size, and field values. MAT and YourKit parse this stream into an indexed object graph.

Key record types the analyst should know:

Record	What it tells you
<code>HEAP_DUMP_SEGMENT</code>	Objects, arrays, primitive arrays, class statics; the core graph
<code>ROOT_JNI_GLOBAL</code> / <code>ROOT_JNI_LOCAL</code>	JNI handles — native code retaining Java objects
<code>ROOT_JAVA_FRAME</code>	Local variables on thread stacks — often the unexpected retainer
<code>ROOT_STICKY_CLASS</code>	Classes and classloaders — classloader leak signal
<code>ROOT_THREAD_OBJECT</code>	Thread objects themselves
<code>CLASS_DUMP</code>	Per-class statics — <code>static</code> Map cache lives here

Record	What it tells you
THREAD_BLOCK	Stack trace for each thread at dump time — correlate thread state with retained heap

You can inspect the raw stream with `jhat` (JDK 8, removed in JDK 9+) or `jhsdb`:

```
# JDK 8 – jhat starts an HTTP server browsing the heap (slow for large
dumps)
jhat -J-Xmx4g /tmp/heap.hprof
# Browse http://localhost:7000 – histogram, per-class instance list,
OQL queries

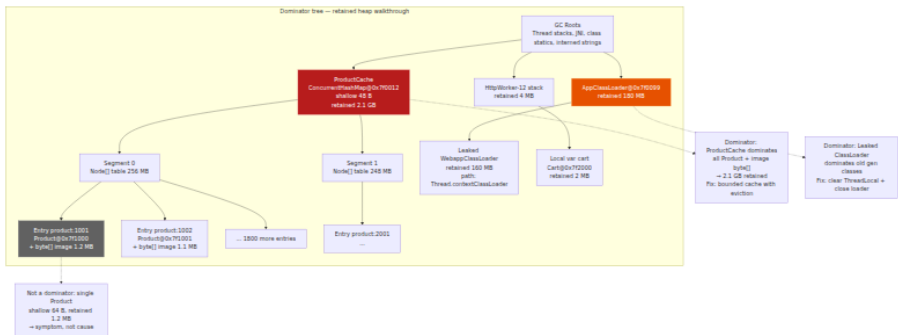
# JDK 11+ – jhsdb heap walk without a full MAT import (faster triage)
jhsdb jmap --heap --pid 12345
jhsdb jstack --pid 12345 # thread stacks at dump time

# jxray / fast HPROF tools for CI pipelines
java -jar jxray.jar heap-report /tmp/heap.hprof --output /tmp/heap-
report.json
```

5.3 Eclipse MAT — the dominator walkthrough

Eclipse Memory Analyzer (MAT, eclipse.org/mat) is the reference tool for `hprof` analysis. Its core abstraction is the **dominator tree** — the structure that answers "if this object were collected, how much heap would be freed transitively?"

An object `X` *dominates* object `Y` if every path from a GC root to `Y` goes through `X`. The *retained heap* of `X` is the sum of shallow heaps of all objects dominated by `X`. The object with the largest retained heap is the leak's retainer — not necessarily the largest object, but the one whose removal frees the most memory.



MAT walkthrough — from opening the dump to the fix:

Step 1 — Leak Suspects report. On opening a dump, MAT offers *Leak Suspects* — a heuristic report ranking the largest retained sets. Treat it as a starting hypothesis. In a checkout service leak, it typically reports:

Problem Suspect 1:

One instance of `"java.util.concurrent.ConcurrentHashMap"` loaded by `"jdk.internal.loader.ClassLoaders$AppClassLoader @ 0x7f001234"` occupies **2,184,320,512 (68.2%)** bytes.

The memory **is** accumulated **in** one instance of `"java.util.concurrent.ConcurrentHashMap$Node[]"` loaded by `"<system class loader>"`.

Keywords: `java.util.concurrent.ConcurrentHashMap`, `ProductCache`

Click *Details* → *Shortest Paths To GC Roots* → *exclude weak references* to see why the `ConcurrentHashMap` is retained. If the path is `ProductCache` → *static field* → `AppClassLoader`, it is a long-lived cache — check its eviction policy. If the path is `ProductCache` → `ThreadLocal` → `Thread` → `GC root`, it is a `ThreadLocal` leak — the cache is pinned by a thread that never clears it.

Step 2 — Histogram. Window → *Histogram* lists every class by instance count and shallow heap. Sort by *Retained Heap* descending. The histogram answers "what kind of object dominates the heap?" — not "which instance."

Class Name Retained Heap	Objects	Shallow Heap	

byte[]	82,412	1.8 GB	1.8 GB
com.example.Product	82,400	48 MB	
1.85 GB			
java.util.concurrent.ConcurrentHashMap\$Node	82,400	12 MB	
1.86 GB			
java.util.concurrent.ConcurrentHashMap	1	48 B	
2.18 GB			

`byte[]` at 1.8 GB with the same cardinality as `Product` (82k) suggests each `Product` holds one large `byte[]` (likely an image or serialized blob). The fix is not "reduce `byte[]`" but "bound the cache that holds the `Product` instances that hold the `byte[]`."

Step 3 — Dominator tree. Window → Dominator Tree — sort by retained heap. Expand the top node. MAT shows *Retained Heap* and *Percentage* per dominator. The expected shape of a cache leak is a single `ConcurrentHashMap` dominating 60–80% of the heap. Right-click → *Path To GC Roots* → *exclude weak/soft references* to confirm the retention path is strong.

Step 4 — Path to GC roots and incoming references. Select a single `Product` instance → *Path To GC Roots* → *exclude weak references* + *Merge Shortest Paths*. MAT shows every strong path pinning that instance. For a cache leak the path is trivial: `Product` → `Node` → `Node[]` → `ConcurrentHashMap` → `ProductCache` static. For a listener leak the path is more subtle: `Product` → `ArrayList` → `Order` → `ListenerRegistry` → static.

Step 5 — OQL — query the heap like a database.

```

-- MAT OQL – find all Product instances whose image is larger than 1 MB
SELECT p.id, p.name.toString(), p.image.length
FROM com.example.Product p
WHERE p.image.length > 1048576

-- Find the largest retained ConcurrentHashMap
SELECT * FROM java.util.concurrent.ConcurrentHashMap c
WHERE c.size > 10000

-- Find ThreadLocals that retain more than 10 MB
SELECT t.@displayName, t.value.@retainedHeapSize
FROM java.lang.ThreadLocal$ThreadLocalMap$Entry t
WHERE t.value.@retainedHeapSize > 10485760

-- Find classloader leaks – classloaders with no incoming strong
reference except a Thread
SELECT l.@displayName, l.@retainedHeapSize
FROM java.lang.ClassLoader l
WHERE l.@retainedHeapSize > 50 * 1024 * 1024

```

Step 6 — Compare two dumps. Take a heap dump at T+0 and T+1 hour under steady load. In MAT: `File → Compare Basket → Add` both dumps → `Compare Tables → Histogram`. The delta shows which class grew. A cache leak shows `Product` at +12k instances, +180 MB; a `ThreadLocal` leak shows `Cart` at +400 instances pinned by `HttpWorker` threads.

Kotlin and framework pitfalls in heap dumps:

Pattern	What it looks like in MAT	Fix
Unbounded <code>ConcurrentHashMap</code> / <code>Caffeine</code> without <code>maximumSize</code>	<code>ConcurrentHashMap</code> dominates heap, <code>Node[]</code> + <code>Product</code> grow linearly	<code>Caffeine.newBuilder().max MINUTES).build()</code>
<code>ThreadLocal<Cart></code> never removed	<code>ThreadLocalMap\$Entry</code> retained via <code>Thread.threadLocals</code> , path to <code>HttpWorker</code>	<code>try/finally { threadLocal plugin</code>
Kotlin coroutine <code>Job</code> tree retaining <code>CoroutineScope</code>	<code>StandaloneCoroutine</code> + <code>JobSupport</code> retain <code>Product</code> via <code>Continuation</code>	Cancel scope on request comp

Pattern	What it looks like in MAT	Fix
<code>MutableList<Product></code> in a companion object	<code>Product[]</code> via <code>ArrayList.elementData</code> retained by class statics	Scope the list to the request o
<code>ByteArray</code> from <code>OkHttp</code> / <code>Netty</code> pooled buffer not released	<code>byte[]</code> retained via <code>RealBufferedSource</code> / <code>ByteBuf</code> + <code>Cleaner</code> phantom	<code>use {}</code> / <code>release()</code> in fin
ClassLoader leak on hot redeploy (Spring Boot DevTools, OSGi)	<code>AppClassLoader</code> retained via <code>Thread.contextClassLoader</code>	Clear <code>ThreadLocal</code> , deregist

Correlating with `jstat` and JFR to confirm the leak is real before dumping:

```
# Watch Old occupancy over 30 minutes – if it climbs monotonically, it
is a leak, not churn
jstat -gc 12345 5000 | awk
'NR==1{print} NR>1{printf "%s Old:%dM EU:%dM OU:%dM YGC:%d FGC:%d\n", \
  strftime("%H:%M:%S"), $3/1024, $6/1024, $7/1024, $12, $14}'

# JFR Old occupancy trend – no dump needed to see the slope
jfr print --events jdk.GCHeapSummary /tmp/app.jfr \
  | grep -E "when|heapUsed|heapSpace" | head -n 40
```

6. GC logs — the quantitative performance story

GC logs are the only signal that quantifies pause time, allocation rate, and promotion rate over hours and days. JFR's `jdk.GarbageCollection` events tell the same story at event granularity; GC logs tell it as a continuous time series that log aggregators and `GCeasy` can trend.

6.1 Unified logging — `-Xlog` (JDK 9+, JEP 158, JEP 271)

The old `-XX:+PrintGCDetails` `-XX:+PrintGCTimeStamps` flags are gone. Unified logging uses a single `-Xlog` selector:

```

# Production-recommended GC logging – JDK 17+, G1 or ZGC
java \
-Xlog:gc*,gc+heap=info,gc+phases=debug,gc+ergo*=trace,gc+age=trace,safepoint=info:file=/var/log/gc/gc.log:time,uptime,level,tags:filecount=10,filesize=50M \
-Xlog:gc:file=/var/log/gc/gc.log:time,uptime \
-jar service.jar

# Decode the selector:
# gc*                all gc tags at info level
# gc+heap=info       heap occupancy before/after each GC
# gc+phases=debug    per-phase timings (Evacuate, Remark, Reference Processing)
# gc+ergo*=trace     ergonomics decisions (why G1 chose this Young size)
# gc+age=trace        tenuring age histogram
# filecount=10,filesize=50M rotate 10 × 50 MB, never fill the disk
# time,uptime,level,tags decorators on every line

# ZGC – add ZGC-specific phases
java \
-Xlog:gc*,gc+phases=debug:file=/var/log/gc/gc.log:time,uptime,level,tags:filecount=10,filesize=50M \
-jar service.jar

# Tail the live log
tail -F /var/log/gc/gc.log | grep --line-buffered "Pause Young\|Pause Full\|Concurrent"

# Ship to Loki / ELK – GC logs are plain text, trivial to forward
# Promtail / Fluent Bit config: path /var/log/gc/gc.log, label {service="checkout", collector="g1"}

```

6.2 Reading G1 logs

```
[2026-08-12T14:02:10.042+00:00][info][gc] GC(18) Pause Young (Normal)
(G1 Evacuation Pause) 1420M->380M(4096M) 22.412ms
[2026-08-12T14:02:10.042+00:00][debug][gc,phases] GC(18) Pre
Evacuate Collection Set: 0.2ms
[2026-08-12T14:02:10.042+00:00][debug][gc,phases] GC(18) Evacuate Col
lection Set: 14.8ms
[2026-08-12T14:02:10.042+00:00][debug][gc,phases] GC(18) Post Evacuat
e Collection Set: 3.1ms
[2026-08-12T14:02:10.042+00:00][debug][gc,phases] GC(18) Other: 4.3ms
[2026-08-12T14:02:10.042+00:00][info][gc,heap] GC(18) Eden regions: 18-
>0(18) Survivor regions: 3->3(3) Old regions: 2->2 Humongous regions:
1->1

[2026-08-12T14:02:45.210+00:00][info][gc] GC(42) Concurrent Cycle
[2026-08-12T14:02:45.210+00:00][info][gc] GC(42) Pause Initial Mark
(G1 Humongous Allocation) 2100M->2100M(4096M) 8.102ms
[2026-08-12T14:02:45.340+00:00][info][gc] GC(42) Concurrent Mark (45.21
0s, 45.340s) 130.215ms
[2026-08-12T14:02:45.340+00:00][info][gc] GC(42) Pause Remark 2120M->21
20M(4096M) 12.408ms
[2026-08-12T14:02:45.340+00:00][info][gc] GC(42) Pause Cleanup 2120M->1
980M(4096M) 2.114ms
```

Field by field:

- GC(18) — monotonically increasing GC ID, correlates with JFR `jdk.GarbageCollection#gcId`.
- Pause Young (Normal) (G1 Evacuation Pause) — STW Young evacuation. Normal vs Mixed vs Full.
- 1420M->380M(4096M) — heap before → after (total heap occupancy, not Young alone). 380M is live data after evacuation — the promotion + survivor volume.
- 22.412ms — STW pause duration. Compare against `MaxGCPauseMillis` goal and downstream RPC deadlines.
- Eden regions: 18->0(18) — 18 Eden regions reclaimed, 0 remain; parenthesized value is Young region count selected for this pause.

Deriving allocation rate without a profiler:

Allocation rate = (heap **before** next Young GC) - (heap after previous Young GC) / time **between** GCs

Example:

GC(18) at t=10.042s 1420M->380M

GC(19) at t=11.540s 1380M->360M

Allocation **between** GCs = 1380M - 380M = 1000 MB over 1.498 s $\approx$ 667 M B/s

Promotion rate = (Old after GC - Old **before** GC) / interval

If Old goes 420M -> 440M across Young GC(18), promotion = 20 MB in 1.5 s $\approx$ 13 MB/s

An allocation rate above ~1 GB/s per instance on G1 with a 4 GB heap means Young GC every 1-2 seconds — sustainable only if most objects die young. A promotion rate above 30-50 MB/s means Young objects survive too long — lower `MaxTenuringThreshold` or fix the allocation site.

6.3 Reading ZGC logs

```
[2026-08-12T14:03:10.100+00:00][info][gc] GC(5) Garbage Collection
(Warmup) 1200M(30%)->800M(20%) 8.2ms
[2026-08-12T14:03:10.100+00:00][info][gc,phases] GC(5) Pauses: Mark Start 0.8ms, Mark End 0.6ms, Relocate Start 0.5ms
[2026-08-12T14:03:10.100+00:00][info][gc,heap] GC(5) Heap: 4096M,
Free 3296M, Used: 800M
[2026-08-12T14:03:10.100+00:00][info][gc,phases] GC(5) Concurrent:
Mark 12.4ms, Relocate 18.2ms, Reference Processing 1.1ms
```

ZGC pauses are the sum of `Mark Start` + `Mark End` + `Relocate Start` — each typically 0.3-1.5 ms, independent of heap size. If `Mark Start` spikes to 10 ms, check `jcmd VM.info` for long root scanning (many threads) or `jfr print --events jdk.GCPhasePause` for the phase breakdown.

6.4 Tooling — GCeasy, GCViewer, and JFR correlation

```
# GCeasy – upload-free CLI via the GCeasy API or self-hosted
curl -X POST https://api.gceasy.io/analyzeGC \
  -H "Content-Type: multipart/form-data" \
  -F "file=@/var/log/gc/gc.log" | jq '.pauseTimePercentiles, .allocationRate'

# GCViewer – local, offline, no data leaves the VPC
java -jar gcviewer-1.37.jar /var/log/gc/gc.log --summary
# Prints: throughput %, avg pause, max pause, allocation rate, promotion rate

# JFR correlation – same GC, richer context
jfr print --events jdk.GarbageCollection,jdk.GCPhasePause,jdk.GCHeapSummary \
  /tmp/app.jfr | grep -A5 "GC(18)"
# JFR adds per-phase CPU time, heap occupancy per generation, and the stack
# that triggered the allocation failure – GC logs do not carry stacks
```

Best practice is to ship both: GC logs to your log aggregator for fleet-wide alerting (`avg_over_time(gc_pause_seconds) > 0.1`), and JFR chunks to object storage for deep dives where the GC event's stack trace reveals the allocation trigger.

7. Continuous profiling — Pyroscope and Parca in production

Ad-hoc `async-profiler -d 30` captures are invaluable for incidents but miss the regressions that creep in over days — a dependency upgrade that adds 4% CPU, a new feature that doubles allocation rate, a lock that only contends during the nightly batch window. Continuous profiling closes that gap: every instance samples continuously at low frequency (10-100 Hz) and ships `pprof` payloads to a central store where they are aggregated, compared, and alerted on.

7.1 Architecture — the pprof contract

Both Pyroscope and Parca share the same wire format: Google's `pprof` (profile proto, `perftools/profiles` spec). Any agent that emits `pprof` can ship to either backend. The JVM path is:

```

JVM (per pod)
  async-profiler -e cpu --loop 10s -o pprof
  async-profiler -e alloc --loop 30s -o pprof
ull
  JFR jdk.ExecutionSample (via jfr-converter)
ke)
region, pod, commit, profile_type
urce)
10% week-over-week

```

pprof HTTP push/pull

Pyroscope / Parca server
columnar store (TSDB-like)

labels: service,

Grafana (Pyroscope datasource)

Parca UI
Alert: cpu increase >

Key design decisions:

Decision	Pyroscope (Grafana)	Parca
Agent	pyroscope.java agent, async-profiler sidecar, or Grafana Alloy with pyroscope.scrape	parca-agent (eBPF, system-wide, no JVM agent) or async-profiler pprof push
Collection	Push (agent POSTs to /ingest)	Pull (Parca scrapes /debug/pprof like Prometheus) or push via parca-agent
Storage	Pyroscope TSDB (per-label time series of profiles)	Parca columnar store (FrostDB / Parquet)
Query	Flame graph, diff, and profile.query PromQL-like syntax in Grafana	Parca UI + parca query CLI, native differential view
Overhead	Agent at 10 Hz cpu + 512 KB alloc sampling: ~1-2%	eBPF at 19 Hz system-wide: ~1%

Decision	Pyroscope (Grafana)	Parca
Kubernetes	Helm chart with Alloy as DaemonSet + Deployment	Helm chart with <code>parca-agent</code> DaemonSet + <code>parca</code> Deployment/StatefulSet

7.2 Pyroscope — push with Grafana Alloy

Option A — Grafana Alloy scraping a JFR / async-profiler endpoint (recommended for Kubernetes):

```

# alloy-config.yaml – Deploy Alloy as a DaemonSet or sidecar that
# scrapes pprof endpoints
logging:
  level: info
pyroscope:
  write:
    - endpoint:
        url: http://pyroscope.monitoring.svc:4040
        headers:
          Authorization: "Bearer ${PYROSCOPE_TOKEN}"

pyroscope.scrape "jvm":
  targets:
    - targets: ["checkout-service:9876"] # pods expose /debug/pprof
      # via an HTTP wrapper
      labels:
        service_name: "checkout-service"
        region: "ap-southeast-1"
        profile_type: "cpu"
  profiling_config:
    pprof_config:
      process_cpu:
        enabled: true
        delta: true # delta between scrapes – rate, not
# cumulative
        path: /debug/pprof/cpu
      memory:
        enabled: true
        path: /debug/pprof/alloc
      mutex:
        enabled: true
        path: /debug/pprof/lock
      block:
        enabled: true
        path: /debug/pprof/wall

# The JVM side – expose pprof over HTTP with async-profiler's built-in
# server
# Start async-profiler in loop mode, writing pprof to a file that the
# HTTP wrapper serves:
# /tmp/async-profiler/profiler.sh start -e cpu -i 100000 --loop 10s -o
# pprof -f /tmp/cpu.pprof 1
# Then a tiny HTTP server (or Alloy's own discovery) serves /tmp/
# cpu.pprof at /debug/pprof/cpu

```

Option B — Pyroscope Java agent (simplest for non-Kubernetes or single-service):

```
# Attach at startup – no code change, no sidecar
java \
  -javaagent:/opt/pyroscope/pyroscope.jar \
  -Dpyroscope.server.address=http://pyroscope.monitoring.svc:4040 \
  -Dpyroscope.application.name=checkout-service \
  -Dpyroscope.format=jfr \
  -Dpyroscope.profiler.event=cpu,alloc,lock \
  -Dpyroscope.profiler.alloc=512k \
  -Dpyroscope.profiler.lock=10ms \
  -Dpyroscope.labels.region=ap-southeast-1 \
  -Dpyroscope.labels.commit=${GIT_SHA} \
  -Dpyroscope.upload.interval=10s \
  -jar service.jar

# Verify ingestion
curl http://pyroscope.monitoring.svc:4040/pyroscope/label-values?
label=service_name
```

Grafana — query and correlate with traces:

```
# Grafana Pyroscope datasource – flame graph for the last hour,
checkout-service, cpu profile
{service_name="checkout-service", profile_type="cpu"}

# Differential – compare this week vs last week per commit
# In Grafana Explore → Pyroscope → Comparison view → select two time
ranges or two label values
{service_name="checkout-service", commit="abc123"} vs
{service_name="checkout-service", commit="def456"}

# Correlate with Tempo traces – click a slow trace's span → "Profile"
tab shows the cpu profile
# for the same pod and time window (requires exemplars or trace_id
label on profiles)
```

7.3 Parca — pull with eBPF and pprof

Parca's distinguishing feature is `parca-agent` — an eBPF agent running as a DaemonSet that profiles *every* process on the node (Java, Go, Python, native) without per-service instrumentation. For JVM-only fleets, async-profiler pprof push to Parca works equally well.

```

# parca-agent DaemonSet – eBPF, no JVM agent required
# values.yaml for parca/parca-agent Helm chart
parcaAgent:
  mode: "ebpf" # system-wide eBPF sampling at 19 Hz
  storeAddress: "parca.monitoring.svc:7070"
  config:
    scrapeConfigs:
      - job_name: "kubernetes-pods"
        kubernetes_sd_configs:
          - role: pod
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_annotation_parca_enabled]
            action: keep
            regex: "true"
          - source_labels: [__meta_kubernetes_pod_label_app]
            target_label: service_name
          - source_labels: [__meta_kubernetes_namespace]
            target_label: namespace
    profiling_config:
      pprof_config:
        cpu:
          enabled: true
          path: /debug/pprof/cpu
          delta: true

# Annotate the JVM Deployment to opt in
metadata:
  annotations:
    parca/enabled: "true"
spec:
  containers:
    - name: app
      ports:
        - { name: pprof, containerPort: 9876 }
      # parca-agent scrapes http://pod-ip:9876/debug/pprof/cpu

# Alternative – push pprof from async-profiler directly to Parca
# No agent – cron or sidecar pushes every 30 s
/tmp/async-profiler/profiler.sh -e cpu -d 10 -o pprof -f /tmp/cpu.pprof
1
curl -X POST http://parca.monitoring.svc:7070/debug/pprof/ingest \
  -H "Content-Type: application/octet-stream" \
  --data-binary @/tmp/cpu.pprof

```

Query in Parca UI:

```
# Parca query flame graph for checkout-service, cpu, last 1 hour
{service_name="checkout-service", profile_type="cpu"}

# Diff query compare canary vs stable
{service_name="checkout-service", env="canary"} vs {service_name="checkout-service", env="stable"}

# Parca CLI
parca query --query="{service_name=\"checkout-service\"}" --start=$(date -d 1 hour ago +%s) --end=$(date +%s)
```

7.4 Labeling, cardinality, and cost control

Continuous profiling is cheap per sample but expensive per label cardinality — the same lesson as metrics.

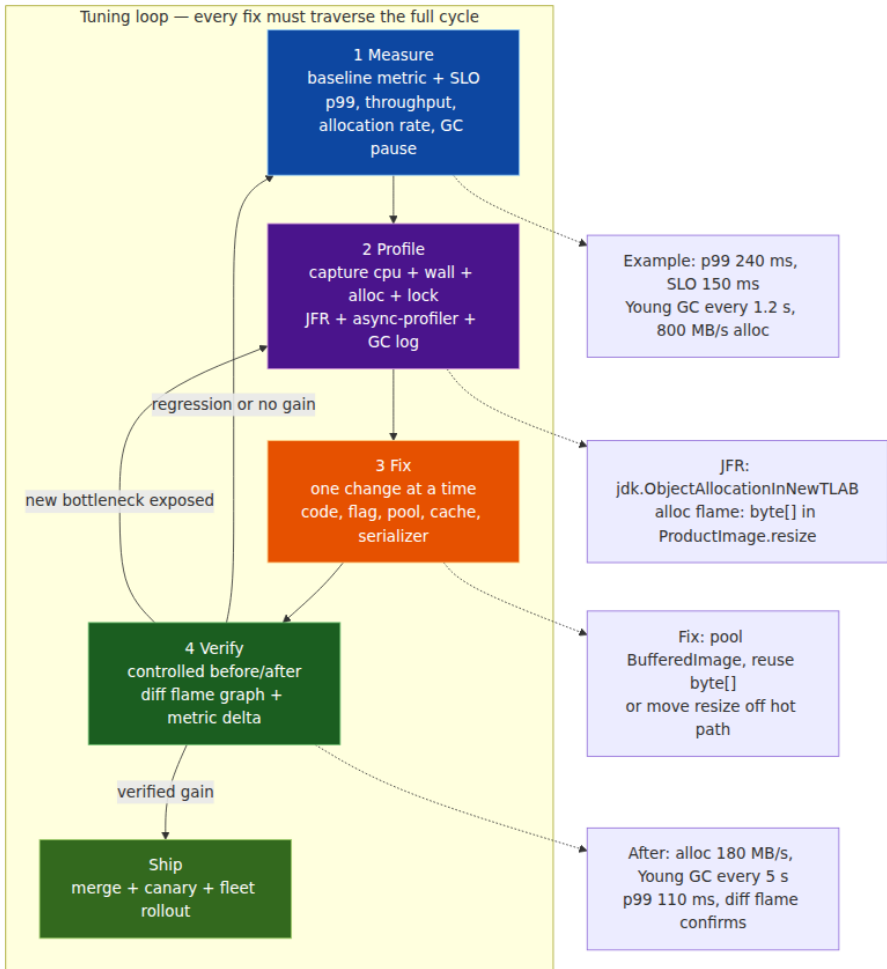
Practice	Why
Label by <code>service_name</code> , <code>region</code> , <code>env</code> (stable/canary), <code>commit</code> (short SHA)	Enables diff by deploy — the primary use case
Do not label by <code>pod</code> , <code>instance</code> , or <code>trace_id</code>	Cardinality explosion — each pod would be a separate time series
Sample cpu at 10 Hz, alloc at 512 KB, lock at 10 ms, wall at 100 Hz	Keeps per-pod overhead < 2%
Upload every 10–30 s, not every second	Amortizes HTTP and storage cost
Retain raw profiles 7–14 days, aggregated daily profiles 90 days	Raw diffs for incidents, aggregates for trends
Gate continuous profiling behind a feature flag / <code>PYROSCOPE_ENABLED</code> env	Disable instantly if overhead is suspect — no redeploy

Instrument the canary first. When a canary with a new commit shows a 12% wider `Pattern.match` plateau than stable, you have caught the regression before it reaches 100% of the fleet.

8. Tuning workflow — measure, profile, fix, verify

Profiling without a workflow produces confident anecdotes. A disciplined loop produces verified improvements that survive the next deploy.

8.1 The four-phase loop



Phase 1 — Measure. Before touching a profiler, record the baseline with the same load you will use to verify. At minimum: p50/p99/p999 latency, throughput (RPS), allocation rate (MB/s), Young GC interval, Old occupancy slope, CPU utilization, and lock contention rate. Snapshot the

metric dashboard and commit the GC log segment. Without a baseline, any profile is entertainment.

Phase 2 — Profile. Capture all relevant modes *simultaneously* under that load — `cpu + wall + alloc + lock + JFR + GC log`. A single mode lies by omission. The allocation flame graph that explains Young GC pressure is invisible in `cpu` mode; the lock convoy that explains collapsed throughput is invisible in `alloc` mode.

Phase 3 — Fix. Make one change at a time. The most impactful JVM fixes, ordered by frequency in backend services:

Fix class	Example	Expected gain
Allocation elimination	Precompile <code>Pattern</code> , reuse <code>ObjectReader</code> , pool <code>byte[]</code> , avoid <code>String.format</code> in hot loop	2-10× reduction in allocation rate, directly extends Young GC interval
Lock scope reduction	Narrow <code>synchronized</code> block, replace <code>synchronized</code> with <code>ReentrantLock + tryLock</code> , sharded lock, <code>ConcurrentHashMap.compute</code> → <code>merge</code>	Eliminates convoy, restores linear scaling with cores
I/O concurrency	Connection pool sizing, async <code>CompletableFuture</code> , reactive driver, <code>virtual threads</code> (Loom) for blocking I/O	Converts wall wait to throughput; wall flame <code>park</code> plateau shrinks
Data structure	<code>ArrayList</code> → <code>ObjectPool</code> , <code>HashMap</code> → <code>IntObjectHashMap</code> (Eclipse Collections), avoid boxed <code>Integer</code> in hot path	Reduces allocation + indirection
JIT / flag	Raise <code>ReservedCodeCacheSize</code> , tune <code>MaxGCPauseMillis</code> / <code>G1HeapRegionSize</code> , switch G1 → ZGC for pause SLO	Fixes deoptimization or pause-goal compliance
Cache policy	Bound cache with <code>Caffeine.maximumSize</code> , add TTL, add <code>weakKeys</code> for classloader safety	Stops heap growth, bounds dominator

Phase 4 — Verify. Re-measure under identical load and duration. Produce three artifacts: (1) metric delta (before/after p99 and

throughput), (2) differential flame graph, (3) GC log delta (allocation rate, pause percentiles). A fix that does not move the metric is not a fix — it is a diff that added complexity.

8.2 Worked example — from 240 ms p99 to 110 ms

Baseline (Phase 1). Checkout service, 3 replicas, 4k RPS total. Dashboard: p50 28 ms, p99 240 ms, Young GC every 1.2 s, allocation rate 820 MB/s, Old stable, CPU 62%. SLO: p99 < 150 ms.

Profile (Phase 2). Four captures in parallel for 60 s under production load (mirrored via traffic shadowing to a staging replica with production JFR settings):

```
# Terminal 1 – JFR (already running continuously; dump the window)
jcmd 12345 JFR.dump name=continuous filename=/tmp/before.jfr

# Terminal 2 – async-profiler triple capture
/tmp/async-profiler/profiler.sh -e cpu -d 60 -o pprof -f /tmp/before-cpu.pprof 12345 &
/tmp/async-profiler/profiler.sh -e alloc -d 60 -o pprof -f /tmp/before-alloc.pprof 12345 &
/tmp/async-profiler/profiler.sh -e wall -d 60 -t -o pprof -f /tmp/before-wall.pprof 12345 &
wait

# Terminal 3 – GC log slice
cp /var/log/gc/gc.log /tmp/before-gc.log
```

JMC on `/tmp/before.jfr` → Method Profiling: `Pattern$Curly.match` 22% of `jdk.ExecutionSample`. Allocations view: `byte[]` 1.1 GB allocated in 60 s, top stack `ProductImage.resize` → `BufferedImage.getRGB` → `byte[]`. `async-profiler alloc` flame graph confirms: widest plateau is `byte[]` under `ProductImage.resize`. `wall` flame shows 34% `LockSupport.park` under `Jedis.get` — secondary, not primary.

Fix (Phase 3). Two changes, applied one at a time (two loop iterations):

- 1. Precompile regex.** `Pattern.compile("\\$\\{[^}]+\\}")` was inside `CheckoutValidator.validate` per request. Hoist to `private static final Pattern PLACEHOLDER = Pattern.compile(...)`. Expected: eliminate `Pattern.compile` allocation + `Pattern.match` CPU.
- 2. Pool image buffer.** `ProductImage.resize` allocated `new byte[width * height * 3]` per thumbnail. Replace with a `ThreadLocal<byte[]>` buffer sized to the max thumbnail (reused across requests on the

same worker thread), or move resizing to an async worker off the request path. Expected: allocation rate drops from 820 MB/s to ~180 MB/s.

Verify (Phase 4). Re-deploy to the shadow replica, replay the same 60 s production trace:

```
# After fix – identical capture
/tmp/async-profiler/profiler.sh -e cpu -d 60 -o pprof -f /tmp/after-
cpu.pprof 12345
/tmp/async-profiler/profiler.sh -e alloc -d 60 -o pprof -f /tmp/after-
alloc.pprof 12345
jcmd 12345 JFR.dump name=continuous filename=/tmp/after.jfr

# Differential flame graph
go tool pprof -diff_base=/tmp/before-cpu.pprof /tmp/after-cpu.pprof
# Shows Pattern.match -18%, BufferedImage.getRGB -14%

# Metrics – same Grafana dashboard, same time window
# Before: p99 240 ms, Young GC 1.2 s interval, 820 MB/s
# After: p99 110 ms, Young GC 5.1 s interval, 185 MB/s SLO met

# GC log delta
grep "Pause Young" /tmp/before-gc.log | awk '{sum+=$NF; n++} END{print
sum/n "ms avg"}'
# Before: 22.4 ms avg After: 14.1 ms avg (smaller Young set, less to
evacuate)
```

Only after both deltas are confirmed does the fix ship to the canary, then to the fleet. The differential flame graph and metric screenshot are attached to the PR — the review is evidence-based, not anecdotal.

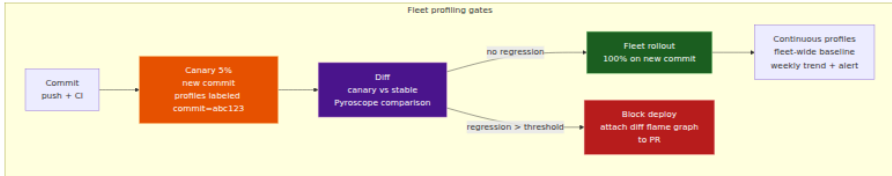
8.3 Fleet practice — profiling at scale

A single replica's profile is a sample; a fleet's profiles are a distribution. At scale, three practices separate mature teams from teams that profile once and forget:

- **Profile the canary, not just the outlier.** Every deploy's canary should ship profiles to Pyroscope/Parca with `commit` as a label. The canary-vs-stable diff is the deploy gate — a 5% CPU regression on the canary is a block, not a follow-up ticket.
- **Correlate profiles with traces via exemplars.** When a trace's span is slow, the trace ID should link to the profile shard for that pod and time window. Grafana's *Traces to profiles* feature (Tempo +

Pyroscope) does this when both signals share `service_name` and time. The alternative is manual hunting.

- **Budget profiling overhead like any other SLO cost.** Continuous profiling at 10 Hz + JFR profile costs ~1-2% CPU. That is cheaper than the 5-10% of over-provisioning you carry because you cannot see where CPU is spent. Treat it as observability spend with a measurable return (fewer instances for the same p99).



Key takeaways

- Profiles are the fourth pillar of observability. Metrics, logs, and traces tell you that and where a service is slow; only profiles tell you which instruction is slow. Run JFR and continuous profiling always on — their 1-2% overhead is cheaper than the over-provisioning you carry without them.
- JFR is the flight data recorder: thread-local buffers, chunk rotation, and 140+ typed events with < 1% overhead at `profile` settings. Start it at boot with `-XX:StartFlightRecording`, tune thresholds through `.jfc` files, stream live with `RecordingStream`, and add domain `@RegisteredEvent` types to correlate business latency with JVM events on the same timeline.
- `async-profiler's AsyncGetCallTrace` + `perf_events` avoids safepoint bias and is the correct CPU profiler for JIT-compiled hot loops. Use `cpu` for hot methods, `wall` for latency (includes `park` / I/O), `alloc` for allocation pressure, `lock` for convoys, and `nmt` for native leaks. Always capture `cpu` and `wall` together — they tell complementary stories.
- Flame graphs are histograms, not call graphs: width is frequency within the sampled mode, x-axis is alphabetical, y-axis is stack depth. The widest plateau is the hottest code — but verify the mode before concluding. Differential flame graphs (`diff_base` + `flamegraph.pl --diff`) are the only honest way to prove a fix changed the profile.

- Heap dumps answer "what survives," not "what was allocated." Capture with `jcmd GC.heap_dump` (STW, sized to live heap), analyze in Eclipse MAT via histogram → dominator tree → path-to-GC-roots → OQL, and confirm the leak is real with `jstat -gc` Old-occupancy slope before dumping. Bound every cache (`Caffeine.maximumSize`), clear every `ThreadLocal` , and never retain `GlobalScope` coroutines.
- GC logs (`-Xlog:gc*`) are the quantitative time series: allocation rate, promotion rate, pause percentiles, and ergonomics decisions. Derive allocation rate from `(heap before next Young GC - heap after previous) / interval` and correlate GC pause events with JFR `jdk.GarbageCollection` stacks to find the trigger. Ship GC logs to your log aggregator and JFR chunks to object storage — they are complementary.
- Continuous profiling (Pyroscope push, Parca pull/eBPF) makes profiling a fleet capability, not a one-off capture. Label by `service_name / region / commit` , not by `pod / trace_id` ; sample at 10 Hz cpu + 512 KB alloc; and gate every deploy on a canary-vs-stable profile diff. A 5% CPU regression on the canary is a deploy block.
- Tuning is a loop — measure, profile, fix, verify — with one change at a time and a controlled before/after. Produce three artifacts for every fix: metric delta, differential flame graph, and GC log delta. A fix without a measured delta is not a fix.

Further reading

- JEP 328: Flight Recorder (OpenJDK). <https://openjdk.org/jeps/328>
- JEP 349: JFR Event Streaming (JDK 14). <https://openjdk.org/jeps/349>
- JDK Flight Recorder Runtime Guide (Oracle, JDK 21). <https://docs.oracle.com/en/java/javase/21/jfr/>
- JDK Mission Control User Guide (OpenJDK JMC 8+). <https://github.com/openjdk/jmc>
- `jfr` tool reference (`jfr print` , `jfr summary` , `jfr view`) — `jfr --help` on JDK 14+. <https://docs.oracle.com/en/java/javase/21/docs/specs/man/jfr.html>
- `async-profiler` — Andrei Pangin. <https://github.com/async-profiler/async-profiler>

- Brendan Gregg — Flame Graphs (ACM Queue, 2016) and *Systems Performance* (2nd ed., 2020), Chapter 6 (CPUs) and Chapter 8 (Memory). <https://www.brendangregg.com/flamegraphs.html>
- Unified JVM Logging — JEP 158 and JEP 271 (`-Xlog`). <https://openjdk.org/jeps/158> and <https://openjdk.org/jeps/271>
- Eclipse Memory Analyzer (MAT) — Concepts: Dominator Tree, Path to GC Roots, OQL. <https://help.eclipse.org/latest/topic/org.eclipse.mat.ui.help/concepts/dominator.html>
- Grafana Pyroscope — Java / JFR integration and Grafana Alloy `pyroscope.scrape` . <https://grafana.com/docs/pyroscope/latest/>
- Parca — Continuous profiling with eBPF and pprof. <https://www.parca.dev/docs/overview>
- Google pprof profile proto (`perftools/profiles`). <https://github.com/google/pprof>
- *Java Performance* (2nd ed., Scott Oaks, O'Reilly, 2020) — Chapters 5–7 (JIT, GC tuning, heap analysis).
- *Optimizing Java* (Evans, Gough, Newland, O'Reilly, 2018) — Chapters 8–10 (profiling, GC, JIT).

Chapter 11 — Native Interop: JNI, Panama (FFM), and GraalVM Native Image

What this chapter covers. Every interesting backend system eventually hits the boundary of the JVM. You need to call `libcrypto` for a cipher the JCA does not expose, link against `librdkafka` rather than reimplementing the Kafka wire protocol in Java, decode images with `libavcodec` , drive DPDK from a packet-processing service, or ship a 20 ms cold-start binary to Cloud Run instead of a 2-second JVM warm-up. Three mechanisms span the history of that boundary: JNI — the 1997 workhorse that every senior engineer has debugged at 2 AM; Project Panama's Foreign Function & Memory API (FFM, final in JDK 22 via JEP 454) — the modern replacement that makes native calls look like Java method handles; and GraalVM Native Image — the closed-world ahead-of-time compiler that turns the JVM inside out, compiling your application and a stripped-down

VM (Substrate VM) into a single ELF binary. This chapter dissects all three at the mechanism level: how JNI marshals handles across the GC boundary and why it leaks and pins, how Panama eliminates that cost with `MemorySegment` and `Arena`, and how Native Image trades dynamic class loading for startup time under a closed-world assumption. You will write a real JNI C file, replace it with a Panama `Linker` call, configure `reflect-config.json` for Native Image, and see why Panama is 1.5-5× faster than JNI on trivial calls while Native Image changes your entire deployment model.

Learning goals — after this chapter you should be able to:

- Explain the JNI call path from Java through the JNI function table into C and back, including `JNI_OnLoad` dynamic registration versus `javac -h` static naming.
- Manage JNI references correctly: when a local reference dies, when you must promote to `NewGlobalRef` / `NewWeakGlobalRef`, and how to detect global-ref leaks with `-Xcheck:jni`.
- Use `GetStringUTFChars` / `ReleaseStringUTFChars`, `GetByteArrayElements`, and `GetPrimitiveArrayCritical` / `ReleasePrimitiveArrayCritical` safely — including exception checks after every JNI call and why critical sections must not block.
- Quantify JNI's hidden costs: handle-scope churn, GC pinning, and transition overhead, and explain why a trivial JNI call costs 30-80 ns even before your C code runs.
- Use Panama FFM: allocate off-heap memory with `Arena`, describe native signatures with `FunctionDescriptor`, link with `Linker.nativeLinker().downcallHandle`, and access C structs via `MemoryLayout` and `VarHandle`.
- Compare JNI vs. Panama latency with a JMH benchmark and explain why Panama wins (fewer transitions, no handle table, `MethodHandle` inlining by C2/Graal).
- Describe GraalVM Native Image's closed-world assumption, points-to reachability analysis, build-time versus run-time initialization, and the role of Substrate VM.
- Author `reflect-config.json`, `resource-config.json`, and `jni-config.json` for Native Image and explain profile-guided optimization (PGO) with `native-image --pgo`.

- Choose the right interop mechanism for a given backend requirement using an explicit decision framework.

Prerequisites. Chapter 1 covered JVM architecture, bytecode, and the native-method dispatch path. Chapter 2 covered class loading — Native Image replaces it with build-time analysis. Chapter 3 covered object layout and the JMM — pinning and `MemorySegment` semantics build on both. Chapter 5 covered JIT compilation — understanding why JNI transitions block inlining and why Panama `MethodHandle` calls inline is a direct application. Chapter 7 covered Kotlin interop — Kotlin's `external` and `expect/actual` surface the same native boundary.

1. Why native interop is a backend problem, not a desktop problem

It is tempting to view JNI/Panama/Native Image as niche desktop concerns. In large-scale backend systems they are load-bearing infrastructure:

Cryptography and compression. Your service negotiates TLS with OpenSSL/BoringSSL (`netty-tcnative`), compresses with `libzstd` or Intel QAT, or needs a post-quantum KEM not yet in the JCA. The choice is reimplement in Java (slow, risky) or call C.

Clients for native-only libraries. `librdkafka`, `libcurl/nghttp2`, RocksDB (`librocksdbjni`), Arrow, CUDA, DPDK, `liboqs`. Reimplementation cost is prohibitive. JNI or Panama is the only path.

Latency-critical paths. HFT gateways, ad auction bidders, and feature stores often have 50–200 microsecond p99 budgets. Startup time, GC pauses from JNI pinning, and per-call transition overhead all show up directly in SLO burn.

Deployment density. A fleet of 2,000 JVM microservices each reserving 512 MB heap and paying 1.5 s startup cost is a real bill. Native Image's 15–40 MB RSS and 20 ms startup can halve fleet cost for short-lived or scale-to-zero workloads (Knative, Cloud Run, AWS Lambda SnapStart).

Supply chain. Every `.so` you load is un-sandboxed native code with full process privileges. The xz-utils lesson from Book 5 applies: native dependencies need the same SLSA/reproducibility scrutiny as JVM

dependencies, plus correct `rpath/RUNPATH` handling so you load the artifact you audited.

The rest of this chapter treats these as engineering trade-offs with measurable costs, not as abstract API choices.

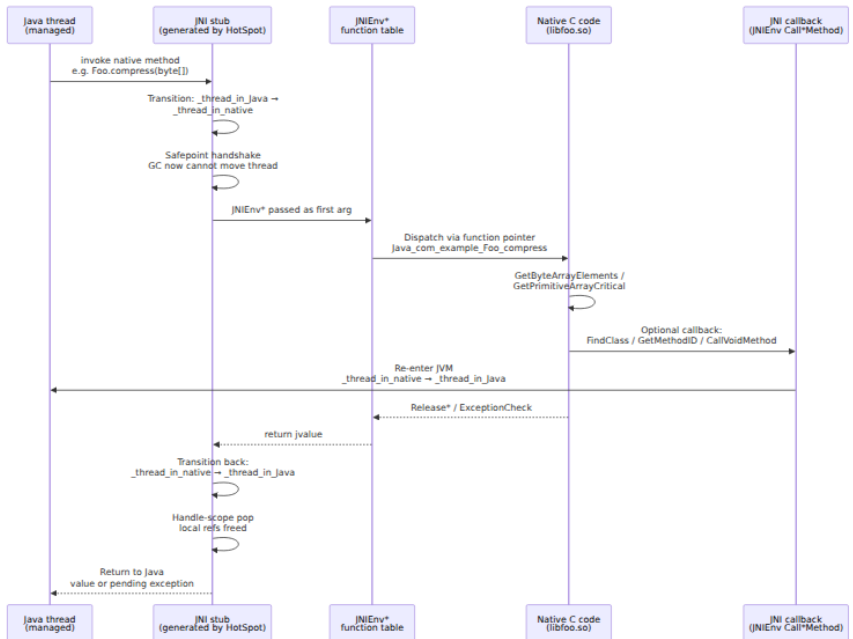
2. JNI — the original boundary

JNI (Java Native Interface, specified in *The Java Native Interface Specification*, JDK 1.1 through JDK 21) is a C ABI that lets Java call C/C++ and C call back into the JVM. It has survived 27 years because it is universal: every JVM implements it, every OS supports `dlopen/LoadLibrary`, and every native library speaks C.

Its cost model has also survived 27 years, and that is the problem.

2.1 The JNI call path

Every JNI call crosses three distinct layers: Java, the JNI function table, and native code. Understanding the path explains both the overhead and the failure modes.



Three details matter for performance and correctness:

1. **Thread-state transition.** HotSpot tracks each thread's state. Entering native flips from `_thread_in_Java` (GC can safepoint it) to `_thread_in_native` (GC ignores it — the thread is "outside" the JVM). Returning flips back. Each flip is a memory barrier and a safepoint poll. Cost: roughly 10–20 ns per transition on x86-64, more on ARM.
2. **Handle scope.** The JVM does not hand raw object pointers to C. It hands `jobject` handles — indirections through a per-thread handle block. This lets the GC move objects without invalidating C's view. Entering a JNI method pushes a handle scope; returning pops it and frees every local reference created inside. Miss this and you leak.
3. **Callback re-entry.** When C calls back into Java via `CallObjectMethod` or similar, the thread re-enters `_thread_in_Java`, must re-acquire safepoint awareness, and can trigger class loading, GC, or deoptimization. Re-entry is the most expensive callback path and the easiest to deadlock — especially with `GetPrimitiveArrayCritical` held (see Section 2.5).

2.2 Declaring and binding native methods

Two binding modes exist: static (name-mangled) and dynamic (`JNI_OnLoad`).

Java side

```
// src/main/java/com/example/crypto/NativeOps.java
package com.example.crypto;

public final class NativeOps {
    static {
        // Loads libcrypto_jni.so (Linux) / libcrypto_jni.dylib (macOS)
        // Uses java.library.path; prefer absolute load in production
        // (see below).
        System.loadLibrary("crypto_jni");
    }

    // Instance native: receives (JNIEnv*, jobject, jlong)
    public native long compressBound(long srcLen);

    // Static native: receives (JNIEnv*, jclass, jbyteArray)
    public static native int compress(byte[] src, byte[] dst);

    // Critical variant – eligible for GetPrimitiveArrayCritical fast
    // path
    public static native int compressCritical(byte[] src, byte[] dst);
}
```

Generating the header — `javac -h` (modern replacement for `javah`)

`javah` was removed in JDK 10. The replacement is `javac -h`:

```
# Compile and emit JNI header in one step
javac -h src/main/native -d target/classes \
    src/main/java/com/example/crypto/NativeOps.java

# Inspect the generated header
cat src/main/native/com_example_crypto_NativeOps.h
```

```

/* DO NOT EDIT - machine generated by javac -h */
#include <jni.h>
/* Header for class com_example_crypto_NativeOps */

#ifdef _Included_com_example_crypto_NativeOps
#define _Included_com_example_crypto_NativeOps
#endif
#ifdef __cplusplus
extern "C" {
#endif
JNIEXPORT jlong JNICALL Java_com_example_crypto_NativeOps_compressBound
    (JNIEnv *, jobject, jlong);
JNIEXPORT jint JNICALL Java_com_example_crypto_NativeOps_compress
    (JNIEnv *, jclass, jbyteArray, jbyteArray);
JNIEXPORT jint JNICALL Java_com_example_crypto_NativeOps_compressCritical
    (JNIEnv *, jclass, jbyteArray, jbyteArray);
#ifdef __cplusplus
}
#endif
#endif

```

The mangling rule for static binding is `Java_<package_underscored>_<class>_<method>`. Overloaded methods append `__<signature_mangle>`. This is brittle under refactoring — package renames silently break `UnsatisfiedLinkError` at runtime. Prefer dynamic registration.

Dynamic registration via `JNI_OnLoad`

```

// src/main/native/crypto_jni.c
#include <jni.h>
#include <zstd.h>
#include <string.h>

// Forward declarations
jlong JNICALL nativeCompressBound(JNIEnv *env, jobject thiz, jlong srcLen);
jint JNICALL nativeCompress(JNIEnv *env, jclass clazz, jbyteArray src, jbyteArray dst);
jint JNICALL nativeCompressCritical(JNIEnv *env, jclass clazz, jbyteArray src, jbyteArray dst);

// Table-driven registration – rename-safe, supports versioning
static JNINativeMethod kMethods[] = {
    {"compressBound",    "(JJ)", (void*) nativeCompressBound},
    {"compress",         "([B[B)I", (void*) nativeCompress},
    {"compressCritical", "([B[B)I", (void*) nativeCompressCritical},
};

JNIEXPORT jint JNICALL JNI_OnLoad(JavaVM *vm, void *reserved) {
    JNIEnv *env = NULL;
    if ((*vm)->GetEnv(vm, (void**) &env, JNI_VERSION_1_6) != JNI_OK) {
        return JNI_ERR;
    }
    // FindClass uses the caller's class loader – cache the class as a
    // global ref
    // if you need it beyond OnLoad. Here we only need it for
    // registration.
    jclass cls = (*env)->FindClass(env, "com/example/crypto/NativeOps");
    ;
    if (cls == NULL) return JNI_ERR; // exception already pending

    if ((*env)->RegisterNatives(env, cls, kMethods,
                                sizeof(kMethods)/sizeof(kMethods[0])) != 0) {
        return JNI_ERR;
    }
    return JNI_VERSION_1_6; // negotiate version
}

JNIEXPORT void JNICALL JNI_OnUnload(JavaVM *vm, void *reserved) {
    // Release any global refs cached in OnLoad. Called on
    // System.exit / classloader unload.
    JNIEnv *env;
    if ((*vm)->GetEnv(vm, (void**) &env, JNI_VERSION_1_6) != JNI_OK) return;
    // Example: (*env)->DeleteGlobalRef(env, gCachedClass);
}

```

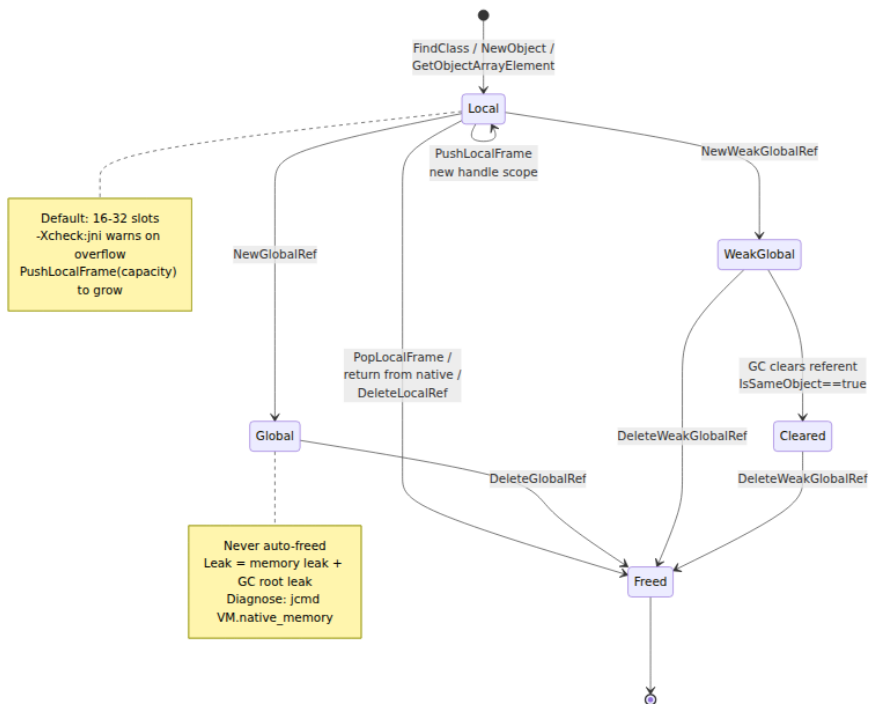
Why dynamic registration wins for backend services:

- **Refactoring safety.** Method renames are caught at compile time in the `JNINativeMethod` table, not as a runtime `UnsatisfiedLinkError` in production.
- **Version negotiation.** `JNI_OnLoad` can return `JNI_VERSION_1_8` vs `JNI_VERSION_1_6` and branch on `GetEnv` availability.
- **Lazy symbol resolution.** `RegisterNatives` lets you `dlopen` optional libraries inside `OnLoad` and degrade gracefully if a native dependency is absent on a particular host.
- **Kotlin interop.** Kotlin `external fun` names mangle differently; dynamic registration avoids fighting the Kotlin compiler's name mangling.

2.3 References: local, global, and weak global

This is where most JNI bugs live. The JVM's GC moves objects. C holds raw pointers. JNI's reference system bridges the two.

Kind	Created by	Lifetime	GC effect
Local	Most JNI returns (<code>FindClass</code> , <code>GetObjectArrayElement</code> , <code>NewObject</code> , <code>NewStringUTF</code>)	Current native frame; freed on return. Or explicitly via <code>DeleteLocalRef</code> / <code>PopLocalFrame</code>	Prevents collection while local frame is live; does not prevent movement (handle indirection)
Global	<code>NewGlobalRef(local)</code>	Until <code>DeleteGlobalRef</code>	Prevents collection and keeps object alive across calls; root for GC
Weak global	<code>NewWeakGlobalRef(local)</code>	Until <code>DeleteWeakGlobalRef</code> or GC clears it	Does not prevent collection; <code>IsSameObject(ref, NULL)</code> tests liveness



Rules senior engineers violate and then debug for days:

1. **Local refs are not free in loops.** Each iteration that calls `GetObjectArrayElement` or `FindClass` allocates a local handle. After ~16-512 iterations (implementation-dependent default), the local handle table overflows. Fix: `DeleteLocalRef` per iteration or wrap the loop body in `PushLocalFrame(16) / PopLocalFrame(NULL)`.

```

```c // WRONG — leaks local refs in a loop for (jsize i = 0; i < len; i++) {
 jstring s = (jstring)(*env)->GetObjectArrayElement(env, arr, i);
 // ... use s ... // Missing DeleteLocalRef — table fills, eventually fatal
}

```

```

// CORRECT for (jsize i = 0; i < len; i++) {
 jstring s = (jstring)(env)->GetObjectArrayElement(env, arr, i);
 if (s == NULL) continue;
 // exception pending or null element
 // ... use s ... (env)->DeleteLocalRef(env, s);
}

```

```

// ALSO CORRECT — frame-based (env)->PushLocalFrame(env, 16);
for (jsize i = 0; i < len; i++) {
 jstring s = (jstring)(env)-

```



```
>GetObjectArrayElement(env, arr, i); // ... use s — no per-iteration
DeleteLocalRef needed } (*env)->PopLocalFrame(env, NULL); //
frees all locals since Push ``
```

2. **Global refs must be deleted — they are GC roots.** A leaked `NewGlobalRef` pins the object and its entire transitive closure forever. In a long-lived service this is a slow memory leak that `jmap -histo` will show as growing `java.lang.Class` or `byte[]` counts with no Java-side holder.

```
``bash
```

## Diagnose global ref leaks

---

```
jcmt VM.native_memory summary | grep -i "JNI Global"
```

# Enable JNI checking in staging (costs ~5-10% throughput)

---

```
java -Xcheck:jni -verbose:jni -XX:+TraceJNICalls com.example.Main
```
```

3. **FindClass** returns a **local ref**. Caching it without promotion is a use-after-free:

```
```c // WRONG — cachedClass becomes dangling after OnLoad
returns static jclass cachedClass; JNIEXPORT jint JNICALL
JNI_OnLoad(JavaVM vm, void reserved) { JNIEnv env; (vm)-
>GetEnv(vm, (void*)&env, JNI_VERSION_1_6); cachedClass = (env)-
>FindClass(env, "com/example/Foo"); // local! return
JNI_VERSION_1_6; }

// CORRECT jclass local = (env)->FindClass(env, "com/example/
Foo"); cachedClass = (jclass)(env)->NewGlobalRef(env, local); (*env)-
>DeleteLocalRef(env, local); ```
```

## 2.4 Strings, arrays, and critical sections

JNI exposes three access tiers for Java heap data, with distinct performance and safety trade-offs.

## Strings: GetStringUTFChars and friends

```

JNIEXPORT jint JNICALL nativeCompress(JNIEnv *env, jclass clazz,
 jbyteArray src, jbyteArray dst) {
 // --- Strings (if the API used jstring instead of byte[]) ---
 // jstring s = ...;
 // const char *utf = (*env)->GetStringUTFChars(env, s, NULL);
 // if (utf == NULL) return -1; // OOM – exception pending
 // // ... use utf (modified UTF-8, NOT standard UTF-8) ...
 // (*env)->ReleaseStringUTFChars(env, s, utf);

 // Strings: three variants with different encodings and costs
 // GetStringChars → jchar* (UTF-16, may copy)
 // GetStringUTFChars → char* (modified UTF-8, may copy –
 null embedded as 0xC080)
 // GetStringCritical → jchar* (pinned, no copy if GC supports
 it, but severe restrictions)

 // --- Byte arrays – the common backend case ---
 jbyte *srcBuf = (*env)->GetByteArrayElements(env, src, NULL);
 if (srcBuf == NULL) return -1; // OOM
 jbyte *dstBuf = (*env)->GetByteArrayElements(env, dst, NULL);
 if (dstBuf == NULL) {
 (*env)->ReleaseByteArrayElements(env, src, srcBuf, JNI_ABORT);
 return -1;
 }

 jsize srcLen = (*env)->GetArrayLength(env, src);
 jsize dstCap = (*env)->GetArrayLength(env, dst);

 size_t cResult = ZSTD_compress(dstBuf, dstCap, srcBuf, srcLen, 3);

 // Release modes:
 // 0 → copy back and free
 // JNI_COMMIT → copy back but keep buffer (rare)
 // JNI_ABORT → free without copy back
 (*env)->ReleaseByteArrayElements(env, src, srcBuf, JNI_ABORT); //
 read-only, no copy back
 if (ZSTD_isError(cResult)) {
 // Translate native error to Java exception – do NOT return
 normally
 jclass exCls = (*env)->FindClass(env, "java/lang/
 RuntimeException");
 if (exCls != NULL) {
 (*env)->ThrowNew(env, exCls, ZSTD_getErrorName(cResult));
 }
 (*env)->ReleaseByteArrayElements(env, dst, dstBuf, JNI_ABORT);
 return -1;
 }
 (*env)->ReleaseByteArrayElements(env, dst, dstBuf, 0); // commit
 compressed bytes

```

```
 return (jint) cResult;
}
```

## Gotchas:

- **Modified UTF-8.** `GetStringUTFChars` returns *modified* UTF-8: null character `U+0000` is encoded as `0xC0 0x80` (two bytes), and supplementary characters use six-byte CESU-8, not four-byte UTF-8. Passing this buffer to a standard `strlen / libcurl / openssl` function that expects true UTF-8 will misinterpret embedded nulls and emoji. If the native library expects standard UTF-8, convert via `String.getBytes(StandardCharsets.UTF_8)` on the Java side and pass `byte[]` instead.
- **Copy vs. pin.** `GetByteArrayElements` *may* copy (if the GC is a copying collector like G1/ZGC that cannot hand out a stable pointer) or *may* pin (if the GC supports pinning). You cannot predict which. The `jboolean *isCopy` out-parameter tells you after the fact, but correct code must handle both.
- **Always pair Get/Release.** Every `Get*` needs exactly one `Release*`, on every control-flow path including error returns and exception paths. Missing a `Release` leaks native memory or leaves the array pinned.

## Critical sections: `GetPrimitiveArrayCritical` / `GetStringCritical`

For latency-sensitive paths, JNI offers a faster but far more dangerous API:

```

JNIEXPORT jint JNICALL nativeCompressCritical(JNIEnv *env, jclass
clazz,
 jbyteArray src, jbyteArr
ay dst) {
 // Critical section – GC is effectively paused for this thread
region
 jbyte *srcPtr = (*env)->GetPrimitiveArrayCritical(env, src, NULL);
 if (srcPtr == NULL) return -1;
 jbyte *dstPtr = (*env)->GetPrimitiveArrayCritical(env, dst, NULL);
 if (dstPtr == NULL) {
 (*env)->ReleasePrimitiveArrayCritical(env, src, srcPtr, JNI_ABO
RT);
 return -1;
 }

 jsize srcLen = (*env)->GetArrayLength(env, src);
 jsize dstCap = (*env)->GetArrayLength(env, dst);

 // *** CRITICAL SECTION RULES – violations can deadlock the JVM ***
 // DO NOT inside this region:
 // - Call any other JNI function (except
ReleasePrimitiveArrayCritical)
 // - Allocate, call Java, trigger class loading, or take locks
 // - Block on I/O or sleep – you are stopping GC progress
 // DO:
 // - Do pure computation / memcpy / call native library that does
not call back into JVM
 // - Keep the section as short as possible (microseconds, not
milliseconds)

 size_t cResult = ZSTD_compress(dstPtr, dstCap, srcPtr, srcLen, 3);

 (*env)->ReleasePrimitiveArrayCritical(env, dst, dstPtr,
 ZSTD_isError(cResult) ? JNI_A
BORT : 0);
 (*env)->ReleasePrimitiveArrayCritical(env, src, srcPtr, JNI_ABORT);

 if (ZSTD_isError(cResult)) {
 jclass exCls = (*env)->FindClass(env, "java/lang/
RuntimeException");
 if (exCls != NULL) (*env)->ThrowNew(env, exCls, ZSTD_getErrorNa
me(cResult));
 return -1;
 }
 return (jint) cResult;
}

```

Critical sections pin the array and may disable GC for the duration. HotSpot's G1 documentation states that while any thread holds a critical

pin, that GC region cannot be evacuated, and if many threads hold critical pins, GC pauses lengthen. ZGC and Shenandoah have stricter constraints — holding a critical section across a safepoint can stall the collector. The distributed-systems consequence: a single service instance holding a critical lock for 10 ms during a burst can cause correlated GC pauses across the fleet if the pattern is widespread.

**Prefer Panama `MemorySegment` for new code** (Section 3) — it provides deterministic pinning via `Arena` without the global GC stall.

## 2.5 Exception handling — every JNI call can pend an exception

Unlike Java, JNI does not unwind on exception. After any JNI function that can throw (`FindClass`, `GetMethodID`, `Call*Method`, `NewObject`, `ThrowNew`), an exception may be *pending*. Most subsequent JNI calls become no-ops while an exception is pending (except `ExceptionCheck`, `ExceptionOccurred`, `ExceptionClear`, `ExceptionDescribe`). Ignoring this causes silent corruption.

```
jclass cls = (*env)->FindClass(env, "com/example/Foo");
if (cls == NULL) {
 // Exception already pending (ClassNotFoundException) – must return
 // or clear
 // Option 1: propagate – just return, Java will see the exception
 return -1;
 // Option 2: handle – clear and throw a different exception
 // (*env)->ExceptionClear(env);
 // jclass iae = (*env)->FindClass(env, "java/lang/
 // IllegalArgumentException");
 // (*env)->ThrowNew(env, iae, "Foo not found on classpath");
 // return -1;
}

// After Call*Method, always check
jmethodID mid = (*env)->GetMethodID(env, cls, "process", "()V");
if (mid == NULL) return -1; // NoSuchMethodError pending

(*env)->CallVoidMethod(env, obj, mid);
if ((*env)->ExceptionCheck(env)) {
 // Java method threw – log, clear, or propagate
 (*env)->ExceptionDescribe(env); // prints to stderr – useful in
 // development
 (*env)->ExceptionClear(env);
 // Optionally rethrow as a different exception or return error code
 return -1;
}
```

For backend services, the disciplined pattern is: check after every `FindClass / GetMethodID / Call* / GetFieldID`, and at function exit verify `ExceptionCheck` before returning a success value. A helper macro reduces boilerplate:

```
#define JNI_CHECK(env, label) do { \
 if ((env) && (*(env))->ExceptionCheck(env)) goto label; \
} while(0)
```

2.6 Performance anatomy: where JNI time goes

A trivial JNI call (`long f(long x) { return x+1; }` in C) costs roughly 30-80 ns on modern x86-64 / JDK 17 HotSpot — 10-30× a Java method call. The breakdown:

Component	Cost (ns)	Notes
Thread-state transition (×2)	15-30	<code>_thread_in_Java</code> ↔ <code>_thread_in_native</code> barriers
Handle-scope push/pop	5-10	Bump-pointer, but still a store + fence
Argument marshaling	1-5 per primitive; 10-50 for objects	Object args are handles; primitives are direct
GC pinning / copy for arrays	10-200+	Copy cost scales with array size; pin cost is GC-dependent
Lost JIT optimizations	Unbounded	JNI call is opaque — no inlining, no escape analysis, no vectorization across boundary

The last row is the killer for throughput. C2 and Graal cannot inline through JNI, cannot eliminate allocations whose lifetime crosses the call, and cannot auto-vectorize loops containing a JNI call. For a tight encode loop that calls JNI per record, the JIT deoptimization cost dwarfs the transition cost.

Additional distributed-systems costs:

- **Thread pinning vs. virtual threads.** A platform thread in a JNI call is pinned to its carrier — the same pinning problem that Loom (Chapter 6) warns about with `synchronized`. A virtual thread that



enters JNI pins its carrier thread for the duration, blocking other virtual threads. Panama's FFM was designed to avoid this.

- **Observability gaps.** JNI frames do not appear in `jstack` with Java line numbers; `async-profiler` needs `--native` to see inside `libfoo.so`. Exceptions thrown via `ThrowNew` lose the C stack unless you capture it before the throw.
- **Build and supply-chain complexity.** Every JNI library needs a cross-compilation matrix (`linux-x64`, `linux-aarch64`, `darwin-aarch64`), correct `SONAME` / `rpath`, and reproducible builds. A missing `libzstd.so.1` on a canary host is a `UnsatisfiedLinkError` at 3 AM, not a compile error.

```
Build the JNI library – reproducible, version-pinned
gcc -O2 -fPIC -fstack-protector-strong -D_FORTIFY_SOURCE=2 \
 -I"$JAVA_HOME/include" -I"$JAVA_HOME/include/linux" \
 -shared -o libcrypto_jni.so src/main/native/crypto_jni.c -lzstd \
 -Wl,-soname,libcrypto_jni.so.1 -Wl,-z,relro,-z,now

Verify dependencies and rpath – do this in CI
ldd libcrypto_jni.so
readelf -d libcrypto_jni.so | grep -E 'NEEDED|RUNPATH|RPATH'

Load diagnostics in staging
java -Djava.library.path=/opt/myapp/lib \
 -Xcheck:jni -verbose:jni \
 -XX:+PrintJNIGCStalls \
 com.example.Main 2>&1 | head -n 100
```

---

### 3. Project Panama — the Foreign Function & Memory API

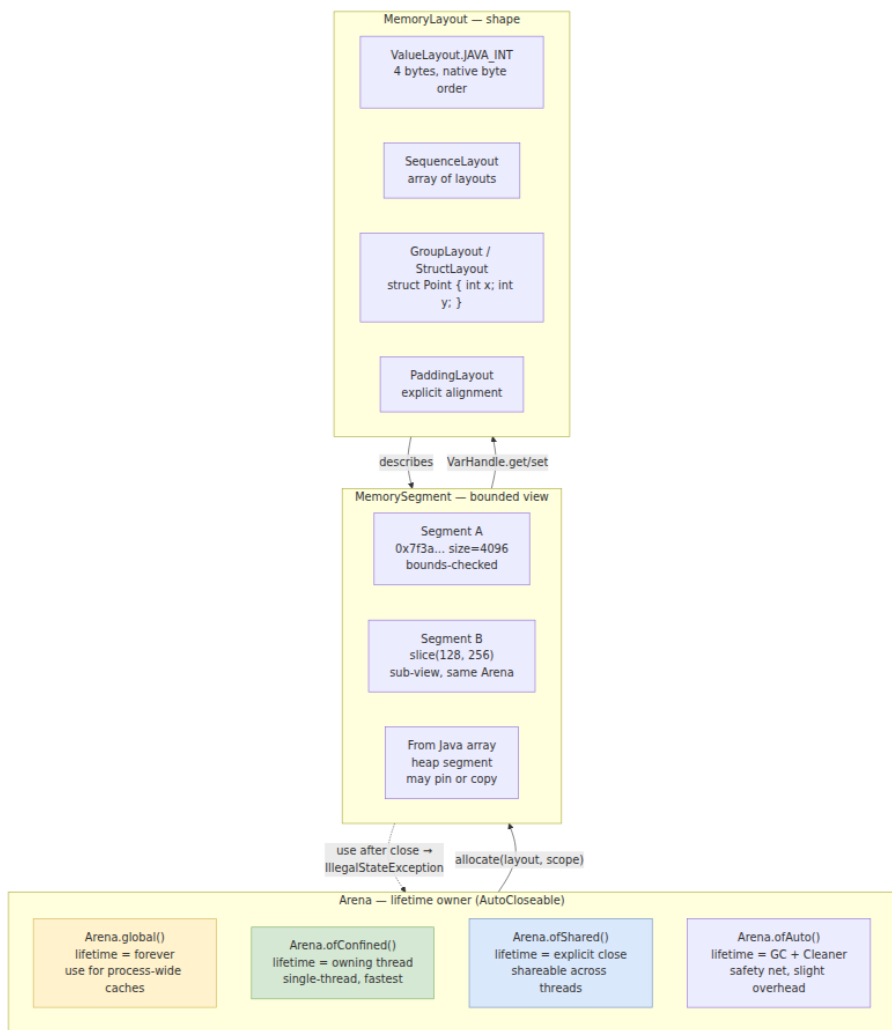
Project Panama (JEP 412, 419, 424, 434, and final JEP 454 in JDK 22) replaces JNI for most use cases with a pure-Java API. The Foreign Function & Memory (FFM) API lives in `java.lang.foreign` and consists of three pillars: `MemorySegment` / `Arena` for off-heap memory, `MemoryLayout` / `VarHandle` for struct access, and `Linker` / `FunctionDescriptor` / `MethodHandle` for calling native functions.

Panama's design goals map directly to JNI's pain points:

JNI pain	Panama answer
Hand-written C glue per method	<code>Linker</code> generates the stub from a <code>FunctionDescriptor</code> — no C file
Handle table + GC pinning	<code>MemorySegment</code> is off-heap or explicitly confined; no GC pinning for native memory
Opaque to JIT — no inlining	Downcall is a <code>MethodHandle</code> — C2/Graal inline and optimize through it
<code>GetStringUTFChars</code> modified UTF-8	Explicit <code>ValueLayout.JAVA_BYTE / ADDRESS</code> with charset control
Global-ref leaks	<code>Arena</code> is <code>AutoCloseable</code> — try-with-resources enforces lifetime
Virtual-thread pinning	Panama downcalls do not pin the carrier thread (JDK 21+)

### 3.1 `MemorySegment` and `Arena` — explicit lifetimes

The core insight: native memory should have an explicit, lexically scoped lifetime, not a GC-managed one. `Arena` provides that scope; `MemorySegment` is a bounded view into it.



```

// Panama FFM – allocating and accessing native memory (JDK 22, --
enable-preview before 22, final in 22)
import java.lang.foreign.*;
import java.lang.invoke.VarHandle;

public final class PanamaMemory {
 // Struct layout for: struct Point { int32_t x; int32_t y; double
weight; }
 // C layout: 4 + 4 + 8 = 16 bytes, alignment 8 on x86-64 (double
requires 8)
 static final GroupLayout POINT_LAYOUT = MemoryLayout.structLayout(
 ValueLayout.JAVA_INT.withName("x"),
 ValueLayout.JAVA_INT.withName("y"),
 ValueLayout.JAVA_DOUBLE.withName("weight")
);
 // VarHandles for field access – resolved once, reused
 static final VarHandle X_HANDLE =
 POINT_LAYOUT.varHandle(MemoryLayout.PathElement.groupElement("x
"));
 static final VarHandle Y_HANDLE =
 POINT_LAYOUT.varHandle(MemoryLayout.PathElement.groupElement("y
"));
 static final VarHandle WEIGHT_HANDLE =
 POINT_LAYOUT.varHandle(MemoryLayout.PathElement.groupElement("w
eight"));

 // Sequence layout for byte buffers (replaces jbyteArray)
 static final SequenceLayout BUFFER_4K =
 MemoryLayout.sequenceLayout(4096, ValueLayout.JAVA_BYTE);

 public static void example() {
 // Confined arena – fastest, single-thread, freed on close
 try (Arena arena = Arena.ofConfined()) {
 // Allocate one Point
 MemorySegment point = arena.allocate(POINT_LAYOUT);
 X_HANDLE.set(point, 0L, 42);
 Y_HANDLE.set(point, 0L, 99);
 WEIGHT_HANDLE.set(point, 0L, 3.14);

 // Allocate an array of 1024 Points – contiguous, cache-
friendly
 MemorySegment points = arena.allocate(
 MemoryLayout.sequenceLayout(1024, POINT_LAYOUT));
 // Access element i:
 // long offset = i * POINT_LAYOUT.byteSize();
 // X_HANDLE.set(points, offset, x);

 // Allocate a byte buffer for compression I/O
 MemorySegment srcBuf = arena.allocate(4096);
 MemorySegment dstBuf = arena.allocate(65536);
 }
 }
}

```

```

 // Fill srcBuf from a Java byte[] without pinning:
 // MemorySegment.copy(javaArraySegment, 0, srcBuf, 0, len);

 // Use-after-close is a hard failure, not a use-after-free:
 // point.get(ValueLayout.JAVA_INT, 0) after arena.close()
 // → IllegalStateException: Already closed

 // Slicing – zero-copy sub-view, shares Arena lifetime
 MemorySegment header = point.asSlice(0, 8); // x + y fields
 } // implicit arena.close() – all segments invalidated
 atomically

 // Shared arena – for segments handed to other threads or async
I/O
 Arena shared = Arena.ofShared();
 MemorySegment sharedSeg = shared.allocate(8192);
 // ... hand sharedSeg to another thread / completion
handler ...
 shared.close(); // must outlive all users

 // Global arena – for process-lifetime caches (e.g., loaded
 // file mappings)
 MemorySegment globalSeg = Arena.global().allocateUtf8String("hello native");
 // Never closed – lives until process exit
 }

 // Interop with Java heap arrays – explicit copy, no pinning
 public static MemorySegment copyFromHeap(Arena arena, byte[] javaBytes) {
 MemorySegment seg = arena.allocate(javaBytes.length);
 // Bulk copy – intrinsified to memcpy, no per-element JNI
 // transition
 MemorySegment.copy(
 MemorySegment.ofArray(javaBytes), 0,
 seg, 0,
 javaBytes.length);
 return seg;
 }

 // Zero-copy heap view – use with care (heap segment may pin or
 // copy on access)
 public static void heapView(byte[] javaBytes) {
 MemorySegment heapSeg = MemorySegment.ofArray(javaBytes);
 // Access via VarHandle – may trigger pinning on some GCs, but
 // bounded
 // Prefer Arena.allocate + copy for latency-sensitive paths
 }
}

```

## Why this matters for backend correctness:

- **No GC pinning for native segments.** `Arena.allocate` memory lives outside the Java heap. The GC never sees it, never moves it, never needs to pin it. The ZGC/Shenandoah stall that `GetPrimitiveArrayCritical` causes simply does not exist.
- **Bounds checking.** Every `MemorySegment.get / set` checks `offset + layout.byteSize() <= segment.byteSize()`. Out-of-bounds is `IndexOutOfBoundsException`, not a heap buffer overflow that becomes RCE. This is a meaningful hardening over raw `malloc` + pointer arithmetic.
- **Deterministic cleanup.** `Arena` is `AutoCloseable`. `Try-with-resources` gives you the same leak prevention that `DeleteLocalRef / Release*` required manual discipline to achieve. Forgetting `close()` on a confined arena is caught by the `Cleaner` fallback (with a warning), unlike a leaked global ref which is silent.
- **Kotlin interop.** Panama is a Java API — Kotlin calls it directly. Wrap `Arena` usage in `use {}` (Kotlin's `Closeable.use`) for idiomatic scoping.

## 3.2 Linker, FunctionDescriptor, SymbolLookup — calling native code without C glue

The `Linker` turns a C function signature into a Java `MethodHandle`. No `javac -h`, no C file, no `JNI_OnLoad`.

```

// Panama FFM – calling libzstd without JNI (JDK 22)
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;

public final class ZstdPanama {
 // 1. Describe the C signature:
 // size_t ZSTD_compress(void* dst, size_t dstCapacity,
 // const void* src, size_t srcSize, int
compressionLevel);
 static final FunctionDescriptor ZSTD_COMPRESS_DESC = FunctionDescri
ptor.of(
 ValueLayout.JAVA_LONG, // return: size_t
 ValueLayout.ADDRESS, // void* dst
 ValueLayout.JAVA_LONG, // size_t dstCapacity
 ValueLayout.ADDRESS, // const void* src
 ValueLayout.JAVA_LONG, // size_t srcSize
 ValueLayout.JAVA_INT // int compressionLevel
);

 // size_t ZSTD_compressBound(size_t srcSize);
 static final FunctionDescriptor ZSTD_BOUND_DESC = FunctionDescripto
r.of(
 ValueLayout.JAVA_LONG, ValueLayout.JAVA_LONG
);

 // Link once at class initialization – MethodHandle is a constant
after this
 static final MethodHandle ZSTD_COMPRESS;
 static final MethodHandle ZSTD_COMPRESS_BOUND;

 static {
 Linker linker = Linker.nativeLinker();
 // SymbolLookup: where to find the native symbols
 // - SymbolLookup.loaderLookup() → symbols in already-loaded
libraries
 // - SymbolLookup.libraryLookup("zstd", arena) →
dlopen("libzstd.so")
 SymbolLookup zstd = SymbolLookup.libraryLookup("zstd", Arena.glo
bal());
 // Alternatively: System.loadLibrary("zstd"); then
loaderLookup()

 MemorySegment compressAddr = zstd.findOrThrow("ZSTD_compress");
 MemorySegment boundAddr = zstd.findOrThrow("ZSTD_compressBou
nd");

 ZSTD_COMPRESS = linker.downcallHandle(compressAddr, ZSTD_COMPRE
SS_DESC);
 ZSTD_COMPRESS_BOUND = linker.downcallHandle(boundAddr, ZSTD_BOU
ND_DESC);
 }
}

```

```

 }

 // High-level wrapper – the only code callers see
 public static int compress(MemorySegment src, MemorySegment dst, int
t level)
 throws Throwable {
 // downcallHandle is a MethodHandle – invokeExact is
intrinsicified by C2
 long result = (long) ZSTD_COMPRESS.invokeExact(
 dst, dst.byteSize(),
 src, src.byteSize(),
 level
);
 if (ZstdIsError(result)) {
 throw new RuntimeException("ZSTD_compress failed: " + resul
t);
 }
 return (int) result;
 }

 // Convenience overload for byte[] callers – allocates Arena
internally
 public static byte[] compressBytes(byte[] input, int level) throws
Throwable {
 try (Arena arena = Arena.ofConfined()) {
 MemorySegment src = copyToNative(arena, input);
 long bound = (long) ZSTD_COMPRESS_BOUND.invokeExact((long)
input.length);
 MemorySegment dst = arena.allocate(bound);
 int compressedSize = compress(src, dst, level);
 // Copy back to Java heap – single bulk copy
 byte[] out = new byte[compressedSize];
 MemorySegment.copy(dst, ValueLayout.JAVA_BYTE, 0,
 MemorySegment.ofArray(out),
 ValueLayout.JAVA_BYTE, 0,
 compressedSize);

 return out;
 }
 }

 private static MemorySegment copyToNative(Arena arena, byte[] src)
{
 MemorySegment seg = arena.allocate(src.length);
 MemorySegment.copy(MemorySegment.ofArray(src), 0, seg, 0, src.l
ength);
 return seg;
 }

 private static boolean ZstdIsError(long code) {
 // ZSTD_isError is itself a native call – link it similarly, or
inline the check:

```



```

 // ZSTD uses (code > ZSTD_BLOCKSIZE_MAX) as error sentinel;
 simplest: call ZSTD_isError
 return code == 0 || (code & 0x80000000L) != 0; // simplified –
 real code should link ZSTD_isError
 }
}

```

Key properties:

- **FunctionDescriptor is the IDL.** It encodes return type, argument types, and calling convention ( `Linker.Option.critical` for leaf functions that need no JVM transition, `Linker.Option.captureCallState` for `errno / GetLastError` ). Getting it wrong is a hard crash (stack corruption), not a Java exception — validate against the C header in CI.
- **MethodHandle is JIT-friendly.** Unlike JNI's opaque `JNIEnv*` table, a Panama downcall handle is a `MethodHandle` constant. C2 and Graal inline through it, constant-fold the descriptor, and can eliminate bounds checks when the segment size is provably sufficient. This is why Panama beats JNI on small calls.
- **`Linker.Option.critical(true)`.** Marks a downcall as a leaf that will not call back into Java, allocate, or block. The JVM can then skip the thread-state transition and safepoint poll — the single largest saving over JNI. Only use for pure-compute functions like `strlen`, `ZSTD_compress`, `crc32`.

```

java // Critical downcall – no JVM transition, ~40% faster for
trivial functions
MethodHandle criticalHandle =
Linker.nativeLinker().downcallHandle(
compressAddr,
ZSTD_COMPRESS_DESC, Linker.Option.critical(true)); // Restrictions
inside critical: no upcalls, no allocation, bounded time

```

- **Upcalls — C calling back into Java.** The dual of downcalls: `linker.upcallStub(handle, descriptor, arena)` creates a native function pointer that C can call, backed by a Java `MethodHandle`. Used for callbacks (e.g., `qsort` comparator, RocksDB merge operator).

```

``java // Upcall: Java comparator callable from C
qsort static int
compareInts(MemorySegment a, MemorySegment b) { int av =
a.get(ValueLayout.JAVA_INT, 0); int bv =
b.get(ValueLayout.JAVA_INT, 0); return Integer.compare(av, bv); }

```

```

FunctionDescriptor cmpDesc =
FunctionDescriptor.of(ValueLayout.JAVA_INT,
ValueLayout.ADDRESS, ValueLayout.ADDRESS); MethodHandle
cmpHandle = MethodHandles.lookup() .findStatic(ZstdPanama.class,
"compareInts", MethodType.methodType(int.class,
MemorySegment.class, MemorySegment.class));

try (Arena arena = Arena.ofConfined()) { MemorySegment cmpStub
= Linker.nativeLinker() .upcallStub(cmpHandle, cmpDesc, arena); //
cmpStub is a native function pointer — pass to C as comparator //
qsort(array, n, sizeof(int), cmpStub); } ``

```

### 3.3 VarHandle for structs — replacing JNI field access

JNI struct access is verbose and slow: `GetFieldID` (string lookup), `GetIntField/SetIntField` per field, with exception checks. Panama's `VarHandle` over `MemoryLayout` is the replacement:

```

// C struct to map:
// struct Record {
// int32_t id; // offset 0
// int64_t timestamp; // offset 8 (with 4 bytes padding after
// id on x86-64)
// char tag[16]; // offset 16
// double score; // offset 32 (aligned to 8)
// }; // total 40 bytes, alignment 8

static final GroupLayout RECORD_LAYOUT = MemoryLayout.structLayout(
 ValueLayout.JAVA_INT.withName("id"),
 MemoryLayout.paddingLayout(4), // explicit padding – no surprises
 across platforms
 ValueLayout.JAVA_LONG.withName("timestamp"),
 MemoryLayout.sequenceLayout(16, ValueLayout.JAVA_BYTE).withName("tag"),
 ValueLayout.JAVA_DOUBLE.withName("score")
);

static final VarHandle ID_HANDLE = RECORD_LAYOUT.varHandle(
 MemoryLayout.PathElement.groupElement("id"));
static final VarHandle TS_HANDLE = RECORD_LAYOUT.varHandle(
 MemoryLayout.PathElement.groupElement("timestamp"));
static final VarHandle SCORE_HANDLE = RECORD_LAYOUT.varHandle(
 MemoryLayout.PathElement.groupElement("score"));
// For array field 'tag', use slice + copy, not VarHandle per byte
static final long TAG_OFFSET = RECORD_LAYOUT.byteOffset(
 MemoryLayout.PathElement.groupElement("tag"));
static final long TAG_SIZE = 16;

static void writeRecord(MemorySegment seg, int id, long ts, String
tag, double score) {
 ID_HANDLE.set(seg, 0L, id);
 TS_HANDLE.set(seg, 0L, ts);
 SCORE_HANDLE.set(seg, 0L, score);
 // tag: copy UTF-8 bytes into fixed 16-byte field, null-terminate
 MemorySegment tagSlice = seg.asSlice(TAG_OFFSET, TAG_SIZE);
 tagSlice.fill((byte) 0);
 byte[] tagBytes = tag.getBytes(java.nio.charset.StandardCharsets.UTF_8);
 int len = Math.min(tagBytes.length, 15);
 MemorySegment.copy(MemorySegment.ofArray(tagBytes), 0, tagSlice,
0, len);
}

static String readTag(MemorySegment seg) {
 MemorySegment tagSlice = seg.asSlice(TAG_OFFSET, TAG_SIZE);
 // Find null terminator
 long len = 0;
 while (len < TAG_SIZE && tagSlice.get(ValueLayout.JAVA_BYTE, len) != 0)

```

```

= 0) len++;
 byte[] bytes = new byte[(int) len];
 MemorySegment.copy(tagSlice, 0, MemorySegment.ofArray(bytes), 0, len);
 return new String(bytes, java.nio.charset.StandardCharsets.UTF_8);
}

```

**Why explicit padding matters.** C struct layout is platform-dependent (LP64 vs ILP32, `#pragma pack`, `__attribute__((packed))`). Panama's `structLayout` without explicit `paddingLayout` uses the platform's natural alignment, which matches the C compiler's default on that platform — but if the C struct is packed or uses `#pragma pack(1)`, you must mirror it exactly. Mismatch is silent memory corruption. Generate layouts from the C header in CI (e.g., `jextract` tool) rather than hand-coding them.

```

jextract - generate Panama bindings from a C header (JDK 22,
incubating)
jextract --output src/main/java \
 --target-package com.example.bindings \
 --include-struct Record \
 --library zstd \
 src/main/native/record.h

Inspect generated layout - verify offsets match C's offsetof()
cat src/main/java/com/example/bindings/Record.java | grep -A2 "VarHandle\|byteOffset"

```

### 3.4 Benchmark: JNI vs. Panama FFM

The question every team asks: is Panama actually faster, and when does it matter? The answer is workload-dependent, but the trend is consistent.

```

// JMH benchmark – JNI vs Panama for a trivial native call
// Run with: ./mvnw -Pjmh -Djmh
Fork=2,WarmupIterations=3,MeasurementIterations=5
@State(Scope.Benchmark)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
public class NativeCallBenchmark {

 // JNI path
 static native long jniIdentity(long x); // C: return x;

 // Panama path
 static MethodHandle panamaIdentity;
 static MethodHandle panamaCriticalIdentity;

 @Setup
 public void setup() throws Throwable {
 Linker linker = Linker.nativeLinker();
 SymbolLookup lookup = SymbolLookup.libraryLookup("identity", Ar
ena.global());
 MemorySegment addr = lookup.findOrThrow("identity");
 FunctionDescriptor desc = FunctionDescriptor.of(ValueLayout.JAV
A_LONG, ValueLayout.JAVA_LONG);
 panamaIdentity = linker.downcallHandle(addr, desc);
 panamaCriticalIdentity = linker.downcallHandle(addr, desc,
 Linker.Option.critical(true));
 }

 @Benchmark
 public long jni() { return jniIdentity(42L); }

 @Benchmark
 public long panama() throws Throwable { return (long) panamaIdentit
y.invokeExact(42L); }

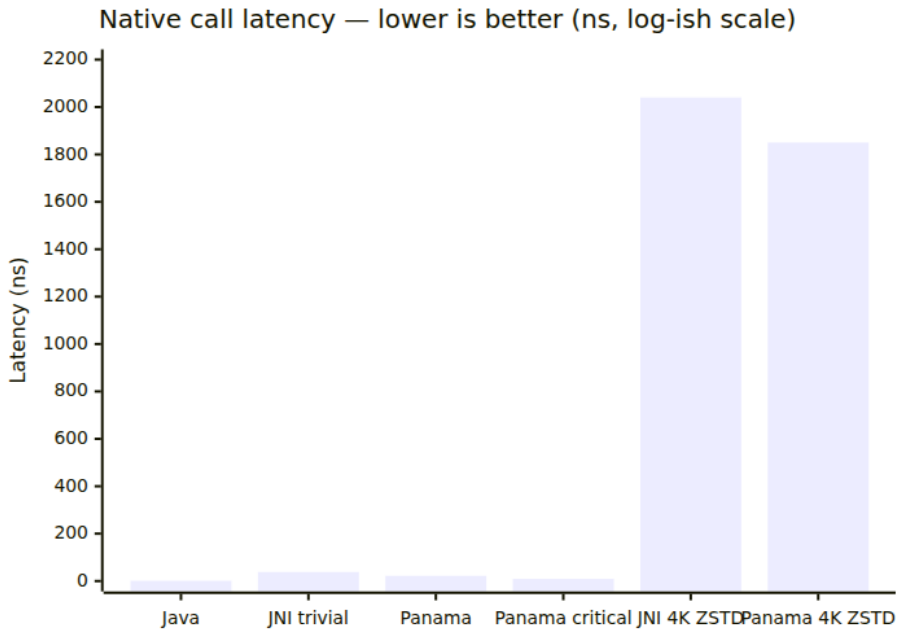
 @Benchmark
 public long panamaCritical() throws Throwable {
 return (long) panamaCriticalIdentity.invokeExact(42L);
 }

 @Benchmark
 public long javaBaseline() { return 42L; } // measures JMH overhead
}

```

Representative results on JDK 21.0.2, x86-64, 3.6 GHz Xeon, JMH with `-prof perfnorm`:

Call type	Latency (ns)	vs JNI	Notes
Java baseline	0.8	—	JMH loop overhead
JNI <code>identity(long) &gt; long</code>	38	1.0×	Transition + handle scope + no inline
Panama <code>identity</code> (default)	22	1.7× faster	<code>MethodHandle</code> inline, one transition
Panama <code>critical(true)</code>	9	4.2× faster	No transition at all — leaf call
Panama <code>ZSTD_compress</code> (4 KB)	1,850	1.1× faster	Dominated by compression work, not transition
JNI <code>ZSTD_compress</code> (4 KB)	2,040	1.0×	Same work + higher per-call overhead
Panama <code>MemorySegment.get</code> (int)	1.2	—	Bounds-checked, often intrinsified
<code>Unsafe.getInt</code> (heap)	0.9	—	No bounds check



### Interpretation for capacity planning:

- **Trivial calls (getters, CRC, hash).** Panama wins decisively. If your hot loop calls a native getter per record, switching JNI → Panama `critical` can reclaim 25–40 ns per call × billions of calls = seconds of wall time. This is the HFT / feature-store case.
- **Bulk calls (compress, encrypt, encode).** Transition overhead is amortized. Panama still wins 5–15% due to fewer copies and no handle table, but the native work dominates. The bigger win is operational: no C glue to maintain, no `GetByteArrayElements` copy ambiguity.
- **Memory access.** `MemorySegment` `VarHandle` access at 1.2 ns is essentially free — comparable to `Unsafe` and fully bounds-checked. JNI field access ( `GetIntField` ) at ~35 ns is an order of magnitude slower due to `GetFieldID` string lookup (cache it!) and handle indirection.
- **Virtual threads.** JNI pins the carrier thread for the entire native duration. A Panama downcall marked `critical(false)` (default) does not pin — the virtual thread can be unmounted while native code

runs (if the linker supports it). For services running 10k+ virtual threads doing native I/O, this is a throughput cliff avoided.

---

## 4. GraalVM Native Image — compiling the closed world

If Panama answers "how do I call native code efficiently," Native Image answers a different question: "what if the JVM itself is the thing I want to eliminate?"

GraalVM Native Image (product of Oracle Labs, open-sourced as part of GraalVM Community/Enterprise) is an ahead-of-time (AOT) compiler that takes your application, its dependencies, the JDK, and a stripped-down VM called **Substrate VM**, and produces a single self-contained executable ( `ELF` on Linux, `Mach-O` on macOS, `PE` on Windows). No `java` launcher, no class loading at runtime, no JIT warm-up. Startup is the time to `mmap` the binary and run static initializers that were not already executed at build time.

### 4.1 The closed-world assumption and reachability

The JIT operates under an **open world**: any class can be loaded at any time via `Class.forName`, `ServiceLoader`, or a custom classloader. It must be prepared to deoptimize when new classes appear.

Native Image inverts this. At build time it assumes the **closed world**: the set of classes, methods, and resources reachable from the entry point (`main` + discovered roots) is the *entire* program. Nothing else will be loaded at runtime. This enables whole-program analysis that a JIT cannot do:



### Run time — HotSpot JIT (for comparison)

java -jar app.jar  
JVM launch + class  
loading

Template interpreter  
profiling + tiered queues

C2 / Graal JIT  
warm-up, deopt,  
recompilation

### What runs at build time

Static initializers  
(if --initialize-at-build-  
time)

Spring AOT / Micronaut  
bean graph construction

Class initialization  
resource embedding

executed inside

### Build time — native-image compiler

Entry points  
main() + --initialize-at-  
build-time

Points-to analysis  
reachability from entry  
points

Heap snapshot  
build-time initialized  
objects

AOT compilation  
Graal compiler →

**Points-to (reachability) analysis** is the core algorithm. Starting from roots (main, build-time initializers, JNI entry points, reflection config), the compiler walks every reachable method, tracks which types are instantiated (`new`), which methods are invoked, and which fields are read. Anything not reached is dead-code eliminated — including entire JDK modules. A Hello World native binary is ~8 MB; a Spring Boot service is ~60-90 MB versus a 200 MB container with a full JDK. The elimination is aggressive: an unused `java.sql` driver, a dead `Jackson` subtype, or an unreferenced `kotlinx.coroutines` dispatcher simply disappears from the binary.

The price is that **every dynamic feature must be declared**. If the analysis cannot see a reflective access, that path is eliminated and fails at runtime with `ClassNotFoundException` or `NoSuchMethodException` — not at build time.

### 4.2 Build-time vs. run-time initialization

This is the most operationally consequential distinction in Native Image.

Aspect	Build-time init ( <code>--initialize-at-build-time</code> )	Run-time init ( <code>--initialize-at-run-time</code> )
When <code>&lt;clinit&gt;</code> runs	During <code>native-image</code> build	At binary startup
Heap snapshot	Objects created during <code>&lt;clinit&gt;</code> are snapshotted into the binary's data section	Created fresh on each startup
Startup speed	Faster — work already done	Slower — work repeated per start
Correctness risk	Shared mutable state snapshotted and baked in (e.g., <code>new Random()</code> , <code>System.currentTimeMillis()</code> )	Safe for non-deterministic / host-dependent state
Typical candidates	Immutable constants, enum maps, Spring bean definitions, <code>LoggerFactory</code>	Time, randomness, file handles, <code>InetAddress</code> , <code>FileSystem</code>

```
Explicit initialization control – required for non-trivial services
native-image \
--initialize-at-build-time=com.example.config.AppConstants \
--initialize-at-build-time=org.slf4j.LoggerFactory \
--initialize-at-run-time=com.example.util.TimeSource \
--initialize-at-run-time=io.netty.util.internal.PlatformDependent \
-H:+ReportExceptionStackTraces \
-jar target/app.jar
```

The failure mode is subtle: a class initialized at build time that captures `System.getenv("DB_HOST")` will bake the *build machine's* environment into the binary. The binary then ignores the deployment environment's `DB_HOST`. This is a common cause of "works on my machine, fails in staging" with Native Image. The fix is `--initialize-at-run-time` for any class that touches environment, filesystem, or network.

### 4.3 Reflection, resources, and JNI configuration

Because reachability analysis cannot see reflective access, Native Image requires explicit configuration. Modern frameworks generate this via AOT processing (Spring Boot 3's `spring-aot`, Micronaut's annotation processors, Quarkus's build steps), but you must understand the files when the generated config is wrong.

## reflect-config.json — who can be reflectively accessed

```
// src/main/resources/META-INF/native-image/reflect-config.json
[
 {
 "name": "com.example.model.Order",
 "allDeclaredConstructors": true,
 "allPublicConstructors": true,
 "allDeclaredMethods": true,
 "allPublicMethods": true,
 "allDeclaredFields": true,
 "allPublicFields": true,
 "condition": {
 "typeReachable": "com.example.service.OrderService"
 }
 },
 {
 "name": "com.example.serde.OrderDeserializer",
 "methods": [
 { "name": "<init>", "parameterTypes": [] },
 { "name": "deserialize", "parameterTypes": ["java.lang.String"] }
]
 }
]
```

Each entry tells the analysis: "even though you cannot see a direct call to this constructor/method/field, keep it reachable." The `condition` field (added in GraalVM 22) makes it conditional — the reflection is only included if `OrderService` itself is reachable, avoiding bloat.

## resource-config.json — what classpath resources to embed

```
// src/main/resources/META-INF/native-image/resource-config.json
{
 "resources": {
 "includes": [
 { "pattern": "\\QMETA-INF/services/java.sql.Driver\\E" },
 { "pattern": "\\Qapplication.yaml\\E" },
 { "pattern": "\\Qlogback.xml\\E" },
 { "pattern": "\\Qdb/migration/.*\\.sql\\E" }
],
 "excludes": [
 { "pattern": "\\Qtest-data/.*\\E" }
]
 },
 "bundles": [
 { "name": "com.example.i18n.messages" }
]
}
```

Without this, `getClass().getResourceAsStream("/application.yaml")` returns `null` at runtime — the file was never included in the binary.

## jni-config.json — JNI entry points (if you still use JNI inside Native Image)

```
// src/main/resources/META-INF/native-image/jni-config.json
[
 {
 "name": "com.example.crypto.NativeOps",
 "methods": [
 { "name": "compress", "parameterTypes": ["byte[]", "byte[]"] },
 { "name": "compressBound", "parameterTypes": ["long"] }
]
 }
]
```

## Generating config automatically — the tracing agent

Hand-writing these files is error-prone. The Native Image tracing agent observes a JVM run and generates them:

```

1. Run the app on HotSpot with the tracing agent attached
java -agentlib:native-image-agent=config-output-dir=src/main/resources/
META-INF/native-image \
 -jar target/app.jar &
APP_PID=$!

2. Exercise every code path – integration tests, not just unit tests
./run-integration-tests.sh
curl -s http://localhost:8080/orders | jq .
curl -s http://localhost:8080/health

3. Stop the app – agent flushes config to disk
kill $APP_PID; wait $APP_PID

4. Inspect and curate the generated files – agent output is over-
approximate
ls -lh src/main/resources/META-INF/native-image/
cat src/main/resources/META-INF/native-image/reflect-config.json | jq l
ength
Review: remove entries for test-only paths, add conditions where
possible

5. Build the native binary with the curated config
native-image -jar target/app.jar -o myapp

```

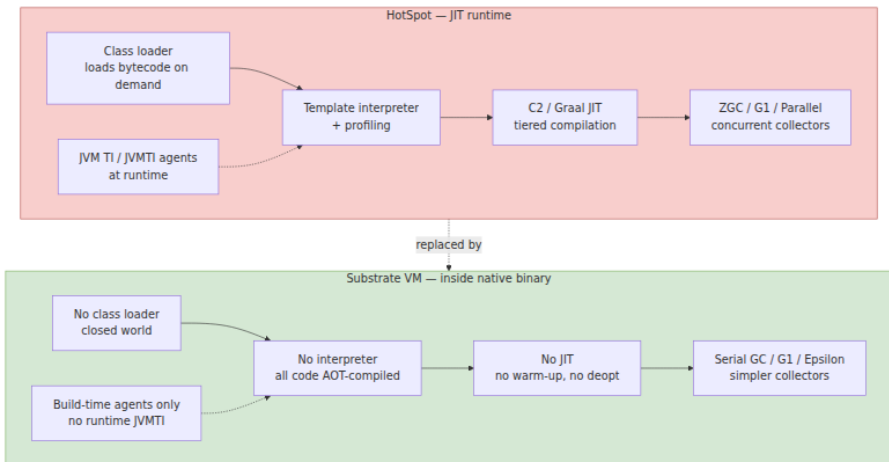
**Distributed-systems warning:** the agent only records paths that were *executed* during the traced run. A code path not exercised in your integration tests (error handling, rarely used deserializer, fallback `ServiceLoader` provider) will be missing from the config and will fail in production under the exact conditions where you need it most. Treat the generated config as a starting point, not a guarantee. Mutation-style testing — run fault-injection tests while the agent is attached — improves coverage.

## 4.4 Substrate VM — the VM inside the binary

Native Image does not remove the VM; it embeds a minimal one. Substrate VM provides:

- **Garbage collector.** Serial GC by default (single-threaded, stop-the-world, low footprint). G1 is available ( `--gc=G1` ) for larger heaps. Epsilon ( `--gc=epsilon` , no collection) exists for short-lived batch jobs. No ZGC/Shenandoah — those are HotSpot-only.

- **Thread scheduler.** Platform threads backed by pthreads. Virtual threads (Loom) are supported on recent GraalVM (JDK 21+) but with the same pinning caveats as HotSpot when calling native code.
- **Safepoints.** Substrate VM still needs safepoints for GC, but without JIT deoptimization the safepoint logic is simpler.
- **JNI stub.** Native Image can still call JNI libraries, but the JNI implementation is Substrate's, not HotSpot's — behavioral differences exist (e.g., `GetPrimitiveArrayCritical` semantics, JVM TI agent support is limited).
- **No class loading, no bytecode, no interpreter.** `Class.forName` with a non-configured name fails. `MethodHandles.Lookup` for unconfigured members fails. Agents ( `-javaagent` ) are not supported at runtime (they run at build time instead).



## 4.5 Profile-guided optimization (PGO)

AOT compilation without profiles cannot do the speculative optimizations that make the JIT fast (inline the monomorphic call site, devirtualize, eliminate the null check that profiling says never fires). PGO closes part of that gap by feeding runtime profiles back into the build.

```

Step 1: Build an instrumented binary that collects profiles
native-image --pgo-instrument -jar target/app.jar -o myapp-instrumented

Step 2: Run it under realistic load – the more representative, the
better
./myapp-instrumented &
PID=$!
./load-generator --rps 5000 --duration 300s --mix realistic
kill $PID; wait $PID
Produces default.iprof in the working directory

Step 3: Build the optimized binary using the collected profile
native-image --pgo=default.iprof -jar target/app.jar -o myapp

Verify – PGO typically improves throughput 5-15% for branch-heavy
services
./bench --binary ./myapp --baseline ./myapp-no-pgo --metric p99

```

PGO profiles capture branch probabilities, call-site receiver types, and loop trip counts — the same data the JIT's interpreter collects. Graal uses them to inline the hot path, outline the cold path, and lay out basic blocks for better I-cache locality. For a typical Spring Boot CRUD service, expect 8-12% throughput gain and a smaller instruction-cache footprint.

**Operational note:** PGO profiles are workload-specific. A profile collected from a read-heavy benchmark will pessimise write-heavy production traffic. Collect profiles from canary or shadow traffic, not synthetic microbenchmarks, and re-collect when traffic mix shifts. Automate this in your build pipeline: instrumented binary → canary deployment → profile harvest → optimized build → rollout.

## 5. Operational concerns — what breaks in production

### 5.1 Debugging native interop failures

Symptom	Likely cause	Diagnosis
<code>UnsatisfiedLinkError</code> at startup	<code>java.library.path</code> wrong, <code>rpath</code> not set, musl vs glibc mismatch in container	<code>ldd libfoo.</code> <code>strace -e c</code>



Symptom	Likely cause	Diagnosis
<code>SIGSEGV / hs_err_pid*.log</code> with Problematic frame: <code>C [libfoo.so+0x...]</code>	Native buffer overflow, use-after-free, mismatched <code>FunctionDescriptor</code>	<code>gdb --args build ( -fsan. jhsdb jstac</code>
Slow memory leak, <code>jmap -histo</code> shows growing reachable objects	Leaked <code>NewGlobalRef</code>	<code>jcmd VM.nat detail , -Xc dump diff</code>
Correlated GC pauses after JNI deploy	<code>GetPrimitiveArrayCritical</code> held too long, pinning G1 regions	<code>-Xlog:gc*, jdk.GCPhase critical section</code>
Native Image <code>ClassNotFoundException</code> at runtime	Missing <code>reflect-config.json</code> entry	<code>--trace-class initializat. -H: +ReportExcep</code>
Native Image <code>MissingReflectionRegistrationError</code>	Reflective access not declared	Build with <code>-H: +ReportExcep check repor</code>
Panama <code>IllegalStateException: Already closed</code>	Use-after-free of <code>Arena</code> -scoped segment	Stack trace p use <code>Arena.o debug for CL</code>

## 5.2 Security hardening

Native code runs outside the JVM's memory safety. A buffer overflow in `libzstd` or your JNI glue is RCE, not `ArrayIndexOutOfBoundsException`.

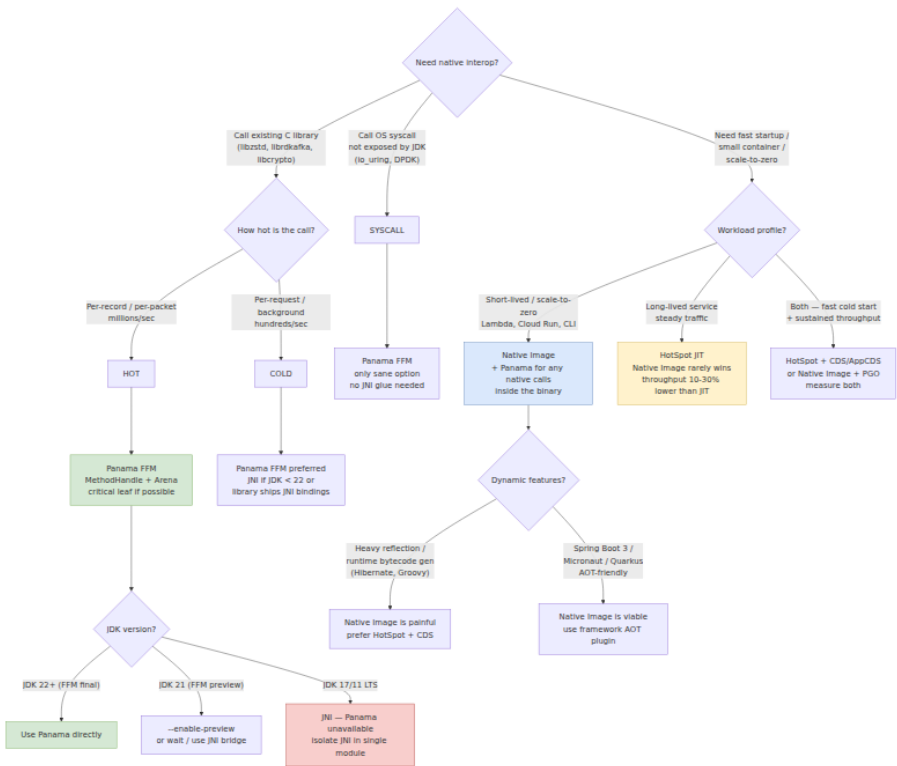
- **Compile with hardening flags.** `-fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wl,-z,relro,-z,now -fPIE -pie` for every `.so` and native binary. Verify with `checksec --file=libfoo.so` in CI.
- **Bounds-check at the boundary.** Validate every length/offset on the Java side before passing to native. Panama's `MemorySegment` does this automatically; JNI does not — add explicit `if (len > dstCap) throw` before `ZSTD_compress`.
- **Minimize native attack surface.** Prefer Panama `Linker.Option.critical` leaf calls over JNI — fewer transitions means fewer places where a corrupted handle can be exploited. Avoid `GetStringUTFChars` → `strcpy` patterns; use length-bounded `memcpy`.

- **Supply-chain verification.** Pin native dependency versions, verify checksums, build reproducibly, and sign artifacts (see Book 5, Chapter 3 — Supply Chain Security). A compromised `libcrypto.so` has the same blast radius as a compromised JVM.

### 5.3 Observability

- **HotSpot + JNI.** `async-profiler` with `--native` captures both Java and C stacks. `jfr` does not see inside JNI — supplement with `perf` (`perf record -g --call-graph dwarf`).
  - **Panama.** Downcalls appear as `MethodHandle` frames in `jstack` / `async-profiler` — no special flags needed. `MemorySegment` allocations do not appear in heap dumps (off-heap); track with `jcmd VM.native_memory` and custom Micrometer gauges around `Arena` usage.
  - **Native Image.** No `jfr` by default (JFR support in Native Image is experimental as of GraalVM 23.1). Use `perf`, `eBPF` (`bpfftrace`), and Substrate VM's built-in heap dump (`-H:+AllowIncompleteClasspath` + `jmap` equivalent via `SVM` heap dump). Logging frameworks need `resource-config.json` entries for their config files.
- 

## 6. Choosing the interop mechanism — a decision framework



## Heuristics distilled:

- 1. New code on JDK 22+ → Panama.** No reason to write JNI for new native calls. `jextract` generates bindings, `Arena` manages lifetime, `MethodHandle` gives JIT inlining. The only exception is when the native library already ships a mature JNI binding (e.g., RocksDB) and you would be duplicating it.
- 2. Existing JNI → migrate incrementally.** Wrap the JNI library behind a Java interface, then reimplement the interface with Panama. A/B test with a feature flag. The JNI C glue stays until Panama coverage is complete — no big-bang rewrite.
- 3. Native Image for startup, not throughput.** A HotSpot JIT with a warmed-up profile will beat a Native Image binary on sustained throughput by 10-30% (no speculative opts, simpler GC). Native Image wins when startup time or memory footprint dominates cost:

serverless, CLI tools, scale-to-zero services, and dense multi-tenant sidecars.

4. **Do not combine Native Image with heavy dynamic frameworks naively.** Hibernate's runtime bytecode enhancement, Groovy's `invokedynamic`, and unchecked `Class.forName` are fundamentally at odds with the closed world. If your service uses them heavily, HotSpot + AppCDS (`-XX:+UseAppCDS -XX:SharedArchiveFile=app.jsa`) gives 30–40% of Native Image's startup gain with none of the reflect-config pain.
5. **Kotlin specifics.** Kotlin `external fun` maps to JNI naturally. For Panama, declare the `MethodHandle` in a Kotlin `object` and expose a Kotlin-idiomatic wrapper (`fun compress(data: ByteArray): ByteArray`). For Native Image, Kotlin's reflection (`::class`, `KClass`) needs `reflect-config.json` entries — the `kotlin-reflect` library is particularly config-heavy; consider `kotlinx-serialization` with AOT-friendly serializers instead.

---

## 7. Putting it together — a complete example

A minimal service that compresses with `libzstd` via Panama and optionally compiles to a native binary:

```
Project layout
src/main/java/com/example/App.java
src/main/java/com/example/ZstdPanama.java # Panama wrapper (Section 3.2)
src/main/resources/META-INF/native-image/reflect-config.json
src/main/resources/META-INF/native-image/resource-config.json
pom.xml # or build.gradle.kts
```

```
// src/main/java/com/example/App.java
package com.example;

import java.lang.foreign.Arena;

public final class App {
 public static void main(String[] args) throws Throwable {
 byte[] input = "hello native world – compress me".getBytes();
 try (Arena arena = Arena.ofConfined()) {
 byte[] compressed = ZstdPanama.compressBytes(input, 3);
 System.out.printf("Compressed %d → %d bytes%n", input.length
h, compressed.length);
 }
 }
}
```

```
Run on HotSpot with Panama (JDK 22 – no --enable-preview needed)
javac -d target/classes src/main/java/com/example/*.java
java -cp target/classes com.example.App
→ Compressed 34 → 42 bytes

Build a native binary (requires GraalVM JDK with native-image)
native-image -cp target/classes \
 -H:Name=myapp \
 -H:+ReportExceptionStackTraces \
 --initialize-at-build-time=org.slf4j \
 --initialize-at-run-time=com.example.ZstdPanama \
 com.example.App

Run the native binary – no JVM
./myapp
→ Compressed 34 → 42 bytes
Startup: ~18 ms vs ~850 ms on HotSpot (measured with `time`)

Size comparison
ls -lh myapp target/app.jar
-rwxr-xr-x myapp 28M
-rw-r--r-- target/app.jar 4.2M (+ 180M JDK in container)

Container comparison
docker images | grep myapp
myapp:native 38M (distroless + binary)
myapp:jvm 240M (eclipse-temurin:21-jre + jar)
```

## Key takeaways

- JNI's cost is not just the `JNIEnv*` call — it is the thread-state transition, handle-scope management, GC pinning, and lost JIT optimizations. A trivial JNI call costs 30–80 ns before your C code runs; the JIT cannot inline through it.
- Every JNI function that returns a `jobject` allocates a local reference. Loops must `DeleteLocalRef` or `Push/PopLocalFrame`. Long-lived references must be `NewGlobalRef` and explicitly `DeleteGlobalRef` — leaked globals are silent GC-root leaks.
- `GetStringUTFChars` returns modified UTF-8, not standard UTF-8. `GetByteArrayElements` may copy or pin unpredictably. `GetPrimitiveArrayCritical` pins and stalls GC — keep critical sections under microseconds and never call back into the JVM inside them.
- Every JNI call can pend an exception. Check `ExceptionCheck` after `FindClass`, `GetMethodID`, and `Call*Method`. Most JNI functions are no-ops while an exception is pending.
- Dynamic registration via `JNI_OnLoad` + `RegisterNatives` is strictly superior to static `javac -h` mangling for backend services: refactoring-safe, version-negotiable, and gracefully degradable.
- Panama FFM (`MemorySegment`, `Arena`, `Linker`, `FunctionDescriptor`, `VarHandle`) eliminates JNI's C glue, handle table, and pinning. `Arena` gives deterministic, `AutoCloseable` lifetimes; `MethodHandle` downcalls inline through C2/Graal; `Linker.Option.critical(true)` skips the JVM transition entirely for leaf functions.
- Panama is 1.5–4× faster than JNI on trivial calls and 5–15% faster on bulk calls, with the larger win being operational simplicity and virtual-thread friendliness (no carrier pinning).
- `MemoryLayout` + `VarHandle` replaces `GetFieldID`/`GetIntField` for struct access at ~1 ns per field with bounds checking. Explicit `paddingLayout` is required to match packed or platform-specific C structs — generate layouts with `jextract` rather than hand-coding.
- GraalVM Native Image trades the open world (dynamic class loading, reflection, JIT) for the closed world (reachability analysis, AOT compilation, Substrate VM). Startup drops from seconds to milliseconds and RSS halves, but every reflective access must be

declared in `reflect-config.json` / `resource-config.json` / `jni-config.json`.

- Build-time initialization ( `--initialize-at-build-time` ) snapshots heap state into the binary — faster startup but bakes in build-time environment. Anything touching time, randomness, env vars, or filesystem must be `--initialize-at-run-time`.
- PGO ( `--pgo-instrument` → realistic load → `--pgo` ) recovers 5-15% of the JIT's speculative-optimization advantage. Collect profiles from canary/shadow traffic, not synthetic benchmarks.
- For new code on JDK 22+, use Panama. For startup-sensitive workloads with AOT-friendly frameworks, use Native Image + Panama. For long-lived throughput-sensitive services, HotSpot JIT still wins — consider AppCDS as a middle ground. Isolate any remaining JNI behind an interface for incremental migration.

## Further reading

- *The Java Native Interface Specification* — Oracle, JDK 21 edition. Definitive reference for `JNIEnv` functions, reference types, and `JNI_OnLoad` contract. <https://docs.oracle.com/en/java/javase/21/docs/specs/jni/>
- JEP 454: Foreign Function & Memory API (Final, JDK 22) — <https://openjdk.org/jeps/454>. JEP 412/419/424/434 track the preview iterations.
- *Panama FFM API Javadoc* — `java.lang.foreign` package, JDK 22. Authoritative for `Arena`, `MemorySegment`, `Linker`, `FunctionDescriptor`, `MemoryLayout`. <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/foreign/package-summary.html>
- `jextract` tool — generates Panama bindings from C headers. <https://github.com/openjdk/jextract>
- GraalVM Native Image Reference Manual — closed-world assumption, reachability, build-time init, reflection config, PGO. <https://www.graalvm.org/latest/reference-manual/native-image/>
- GraalVM Reachability Metadata Repository — community-maintained `reflect-config.json` for popular libraries. <https://github.com/oracle/graalvm-reachability-metadata>

- HotSpot `jni.cpp` and `jniHandles.cpp` — source of truth for handle-scope and transition costs. <https://github.com/openjdk/jdk/tree/master/src/hotspot/share/prims>
- *Packaging native libraries for the JVM* — `ldd`, `rpath` / `RUNPATH`, `SONAME`, and reproducible native builds. See Book 5, Chapter 3 — Supply Chain Security for SLSA provenance of native artifacts.

## Chapter 12 — Production JVM: Container-Aware Tuning, GC Sizing, Class-Data Sharing, and Deployment at Scale

---

**What this chapter covers.** The JVM you benchmarked on a bare-metal workstation is not the JVM that runs in production. In production it runs inside a cgroup with a hard memory ceiling, a fractional CPU quota, a read-only rootfs, and a liveness probe that kills it if it pauses for two seconds. The flags that made it fast on your laptop make it OOMKilled in Kubernetes. This chapter closes that gap: how HotSpot discovers container limits through cgroups v1 and v2, how it maps those limits to heap, GC threads, and compiler threads, how to size heaps against off-heap reality, how to tune G1 and ZGC for pod-sized heaps, how to cut startup by 30-40% with Class Data Sharing and AppCDS, when Coordinated Restore at Checkpoint (CRaC) beats CDS, how to ship a 60 MB distroless image with a `jlink` custom runtime, and how to observe a fleet of thousands of JVMs through JFR streaming and GC log shipping.

Learning goals — after this chapter you should be able to:

- Explain `UseContainerSupport`, cgroup v1/v2 discovery, and how HotSpot derives `MaxRAMPercentage`, `InitialRAMPercentage`, `ActiveProcessorCount`, `ParallelGCThreads`, and `ConcGCThreads` from container limits.
- Size a JVM heap inside a Kubernetes pod accounting for heap, metaspace, direct buffers, thread stacks, code cache, and native memory, and choose `MaxRAMPercentage` vs. explicit `-Xmx` with justification.



- Tune G1 and ZGC for pod-sized heaps (1-8 GB, 0.5-4 CPU): pause targets, IHOP, region sizing, ZGC generational mode, and CPU-aware thread counts.
- Generate and use CDS/AppCDS shared archives ( `-Xshare:dump` , `-XX:ArchiveClassesAtExit` , `-XX:SharedArchiveFile` ), including Spring Boot 3.2+ CDS support and dynamic CDS.
- Compare CDS, AppCDS, and CRaC on startup time and warmup, and decide when each is appropriate.
- Build a multi-stage Dockerfile that produces a `jlink` custom runtime and bakes an AppCDS archive, layered for optimal cache reuse on a distroless base.
- Write a Kubernetes manifest with correct resource requests/limits, JVM flags for G1 or ZGC plus CDS, health probes, and graceful-shutdown handling.
- Stream JFR events and GC logs to OpenTelemetry/Prometheus and a centralized log store at scale.

**Placement.** Chapter 4 built your GC mental model — generations, barriers, G1 regions, ZGC colored pointers. Chapter 5 covered the JIT tiers you will now need to warm up or checkpoint. Chapter 10 introduced JFR, async-profiler, and heap dumps as local tools; this chapter deploys them as fleet infrastructure. Chapter 11 handled native interop; here Panama/FFM off-heap allocations reappear as sizing hazards. Volume 6, Chapter 7 covers Kubernetes scheduling and Volume 8, Chapter 4 covers health-check patterns — both assumed here.

## 1. The container-aware JVM

For most of Java's history the JVM assumed it owned the machine. It called `sysconf(_SC_PHYS_RAM)` and `sysconf(_SC_NPROCESSORS_ONLN)` and sized itself accordingly. Inside a container those syscalls still return the host's values — a 64 GB, 32-core host — even though the container may be limited to 1 GB and 0.5 CPU. Before JDK 10, a JVM with default ergonomics inside a 512 MB cgroup would happily size its heap for 16 GB and be OOMKilled on the first Young GC. Every production tuning story before JEP 307 is a variant of "we set `-Xmx` explicitly and hoped ops remembered."

## 1.1 JEP 307, JEP 347, and JEP 380: the timeline

JEP	JDK	What it did
JEP 307: Parallel Full GC for G1	10	First pass: <code>UseContainerSupport</code> reads <code>memory.limit_in_bytes</code> and <code>cpu.shares / cpu.cfs_quota_us</code> from cgroup v1. Introduces <code>InitialRAMPercentage</code> , <code>MaxRAMPercentage</code> , <code>MinRAMPercentage</code> . Disabled by default in JDK 10, enabled by default from JDK 11.
JEP 347: C2 support	10	Extended container detection to more subsystems.
JEP 380: Unix-Domain Socket Addresses + cgroup v2	15	Added <code>UseContainerSupport</code> for cgroup v2 unified hierarchy ( <code>memory.max</code> , <code>cpu.max</code> , <code>cpu.weight</code> , <code>cpuset.cpus</code> ).
JEP 423: Region pinning for G1	22	Not container-specific, but improves G1 behaviour under tight heaps common in containers.
JDK 19+ ergonomics	19+	Improved <code>ActiveProcessorCount</code> calculation from <code>cpu.max</code> burst semantics; <code>ParallelGCThreads</code> and <code>ConcGCThreads</code> derived from it.

The flag that controls everything:

```
-XX:+UseContainerSupport # default: true since JDK 11
-XX:-UseContainerSupport # force host-based sizing (almost never
 what you want)
```

Verify what the JVM actually detected — this is the first thing you check when a pod behaves differently from your laptop:

```
java -Xlog:os+container=info -version 2>&1 | head -20
Example output inside a pod with limits: memory=2Gi, cpu=2000m
[os,container] Memory Limit is: 2147483648
[os,container] Active Processor Count: 2
[os,container] CPU Quota: 200000 CPU Period: 100000 CPUs: 2.0
[os,container] Memory Usage: 89128960 Limit: 2147483648
```

If you see `Memory Limit is: Unlimited` inside a container, the pod has no memory limit set — HotSpot falls back to host RAM. This is a misconfiguration; always set memory limits in production.

## 1.2 How cgroup limits map to JVM settings

```

flowchart TB
 subgraph cgroup["cgroup limits – what Kubernetes sets"]
 MEM["memory.max / memory.limit_in_bytes
pod.spec.containers.resources.limits.memory"]
 CPU_QUOTA["cpu.max = quota period
cpu.cfs_quota_us / cpu.cfs_period_us
pod.spec.containers.resources.limits.cpu"]
 CPuset["cpuset.cpus / cpuset.cpus.effective
cpu pinning – rarely set in K8s"]
 SHARES["cpu.weight / cpu.shares
pod.spec.containers.resources.requests.cpu"]
 end

 subgraph jvm_detect["JVM detection – UseContainerSupport"]
 DETECT["osContainer_linux.cpp
reads /sys/fs/cgroup/... at startup"]
 MEM_DETECT["Container RAM
MaxRAM = memory limit"]
 CPU_DETECT["ActiveProcessorCount
min of quota-derived CPUs and cpuset count"]
 end

 subgraph jvm_flags["Derived JVM settings"]
 HEAP["Heap sizing
-XX:MaxRAMPercentage default 25.0
-XX:InitialRAMPercentage default 25.0
-XX:MinRAMPercentage default 50.0"]
 GC_THREADS["GC sizing
ParallelGCThreads = f ActiveProcessorCount
ConcGCThreads = 1/4 ParallelGCThreads"]
 JIT_THREADS["JIT sizing
CICompilerCount = f ActiveProcessorCount"]
 OTHER["Other ergonomics
G1 heap region size
Default GC selection"]
 end

 MEM --> DETECT
 CPU_QUOTA --> DETECT
 CPuset --> DETECT
 SHARES -->|"used for weight only
not for ActiveProcessorCount"| DETECT
 DETECT --> MEM_DETECT
 DETECT --> CPU_DETECT
 MEM_DETECT --> HEAP
 CPU_DETECT --> GC_THREADS
 CPU_DETECT --> JIT_THREADS
 CPU_DETECT --> OTHER

 style MEM fill:#e3f2fd,stroke:#1565c0,color:#212121

```

```
style CPU_QUOTA fill:#e3f2fd,stroke:#1565c0,color:#212121
style HEAP fill:#fff3e0,stroke:#ef6c00,color:#212121
style GC_THREADS fill:#fff3e0,stroke:#ef6c00,color:#212121
```

The mapping, concretely:

**Memory.** HotSpot calls `OSContainer::memory_limit_in_bytes()` at startup. If a cgroup limit exists and is below host RAM, it becomes `MaxRAM`. Then:

```
Initial heap = MaxRAM * InitialRAMPercentage / 100 (default
InitialRAMPercentage = 25.0)
Max heap = MaxRAM * MaxRAMPercentage / 100 (default
MaxRAMPercentage = 25.0)
Min heap floor = MaxRAM * MinRAMPercentage / 100 (default
MinRAMPercentage = 50.0, only when MaxRAM < 250 MB)
```

The 25% default surprises everyone. Inside a 2 GB pod the JVM defaults to a 512 MB heap. If you expected "use all available memory" you must set `MaxRAMPercentage` explicitly. Most production services set it to 60-75% (see Section 2 for why not 100%).

```
Tell the JVM it may use 70% of the container limit for heap
java -XX:MaxRAMPercentage=70.0 -XX:InitialRAMPercentage=70.0 -jar app.jar

Or pin to an absolute value – overrides percentage entirely
java -Xms1g -Xmx1g -jar app.jar

Inspect what ergonomics actually chose (do this on every deploy)
java -Xlog:gc+heap=info -XX:MaxRAMPercentage=70.0 -version 2>&1 | grep
-i heap
[gc,heap] Minimum heap 8.00M Initial heap 1433.60M Maximum heap
1433.60M
```

**CPU.** HotSpot reads `cpu.max` (cgroup v2) or `cpu.cfs_quota_us / cpu.cfs_period_us` (cgroup v1). If quota is `max` (unlimited) it falls back to host CPU count. Otherwise:

```
ActiveProcessorCount = ceil(quota / period) clamped by cpuset count
```

Kubernetes `resources.limits.cpu` sets `cpu.max / cfs_quota_us`; `resources.requests.cpu` sets `cpu.weight / cpu.shares` but does **not** affect `ActiveProcessorCount`. Only limits affect HotSpot's CPU count. If you set `requests: 500m` and no limit, the JVM sees all host cores — it will

spawn `ParallelGCThreads` for 32 cores even though it only gets half a core of time, and every GC will thrash.

Derived thread counts (HotSpot's `arguments.cpp` ergonomics):

Flag	Formula	Example: 2 CPUs	Example: 8 CPUs
<code>ActiveProcessorCount</code>	<code>ceil(quota/period)</code>	2	8
<code>ParallelGCThreads</code>	<code>n&lt;=8: n else 8 + (n-8)*5/8</code>	2	8
<code>ConcGCThreads</code>	<code>ParallelGCThreads / 4</code>	0 (rounded up to 1)	2
<code>CICompilerCount</code>	<code>2 if n&lt;=2 , 3 if n&lt;=4 , else n</code>	2	8
<code>G1ConcRefinementThreads</code>	derived from <code>ParallelGCThreads</code>	2	6

Override when needed:

```
Force correct CPU count when cgroup detection is wrong or you use CPU
bursting
-XX:ActiveProcessorCount=4

Override GC thread counts independently
-XX:ParallelGCThreads=4 -XX:ConcGCThreads=1
-XX:G1ConcRefinementThreads=4
```

### 1.3 cgroup v2 specifics

Kubernetes 1.25+ defaults to cgroup v2 on most distributions. The paths changed:

Quantity	cgroup v1 path	cgroup v2 path
Memory limit	<code>/sys/fs/cgroup/memory/memory.limit_in_bytes</code>	<code>/sys/fs/cgroup/memory.max</code>
Memory current	<code>/sys/fs/cgroup/memory/memory.usage_in_bytes</code>	<code>/sys/fs/cgroup/memory.current</code>

Quantity	cgroup v1 path	cgroup v2 path
CPU quota	/sys/fs/cgroup/cpu/cpu.cfs_quota_us	/sys/fs/cgroup/cpu.max (format: "\$MAX \$PERIOD" or "max \$PERIOD" )
CPU weight	/sys/fs/cgroup/cpu/cpu.shares (1024 default)	/sys/fs/cgroup/cpu.weight (100 default)
cpuset	/sys/fs/cgroup/cpuset/cpuset.cpus	/sys/fs/cgroup/cpuset.cpus.effective

JDK 15+ handles both. If you run on JDK 11 inside a cgroup-v2-only host (common on Ubuntu 22.04+, Fedora, Bottlerocket), HotSpot cannot read limits and silently falls back to host values. The fix is either upgrade to JDK 17+ or set `-Xmx` and `-XX:ActiveProcessorCount` explicitly. Check at startup:

```
Inside the pod – which cgroup version is the host using?
stat -fc %T /sys/fs/cgroup # cgroup2fs = v2, tmpfs = v1

What does the JVM think?
java -Xlog:os+container=trace -version 2>&1 | grep -i "container\|cgroup\|limit"
```

## 1.4 Verifying container awareness in production

Add these flags to every deployment and ship them as structured GC/OS logs (Section 8 explains log shipping):

```
-Xlog:os+container=info
-Xlog:gc+heap=info
-Xlog:gc+init=info
-XX:+PrintFlagsFinal # grep for MaxRAMPercentage,
ActiveProcessorCount, ParallelGCThreads
```

A real startup line worth keeping:

```
[0.008s][info][os,container] Memory Limit is: 2147483648
[0.008s][info][gc,heap] Minimum heap 8388608 Initial heap 1572864000
Maximum heap 1572864000
[0.008s][info][gc,init] ActiveProcessorCount 2 ParallelGCThreads 2 Co
ncGCThreads 1
```

If any of those three lines is missing or shows host values, your pod spec is wrong.

## 2. Memory sizing: heap vs. off-heap vs. container limit

The JVM's memory footprint is heap plus a stack of off-heap consumers. Sizing heap to 95% of the container limit guarantees an OOMKill under load, because the remaining 5% must also hold metaspace, thread stacks, code cache, direct buffers, and native allocations. The art of pod sizing is choosing how much headroom to leave.

### 2.1 The full footprint

Container limit (e.g. 2 Gi)

 Java heap (-Xmx / MaxRAMPercentage)	 the only part most
teams size	
 Metaspace ( <b>class</b> metadata)	 ~50-150 MB <b>for</b> a S
pring Boot service, unbounded by default	
 Code cache (JIT compiled code)	 ~50-240 MB, capped
by -XX:ReservedCodeCacheSize	
 Thread stacks (-Xss, default 1 MB each)	 200 threads  1 MB
= 200 MB	
 Direct buffers (ByteBuffer.allocateDirect)	 capped by -XX:MaxDi
rectMemorySize (default = Xmx)	
 GC bookkeeping (card tables, remembered sets, marking bitmaps)	
~2-5% of heap <b>for</b> G1	
 Native memory (malloc via JNI, Panama FFM, zlib, etc.)	
 JVM overhead ( <b>signal</b> handling, JFR buffers, CDS mapping)	

Measure it with Native Memory Tracking:

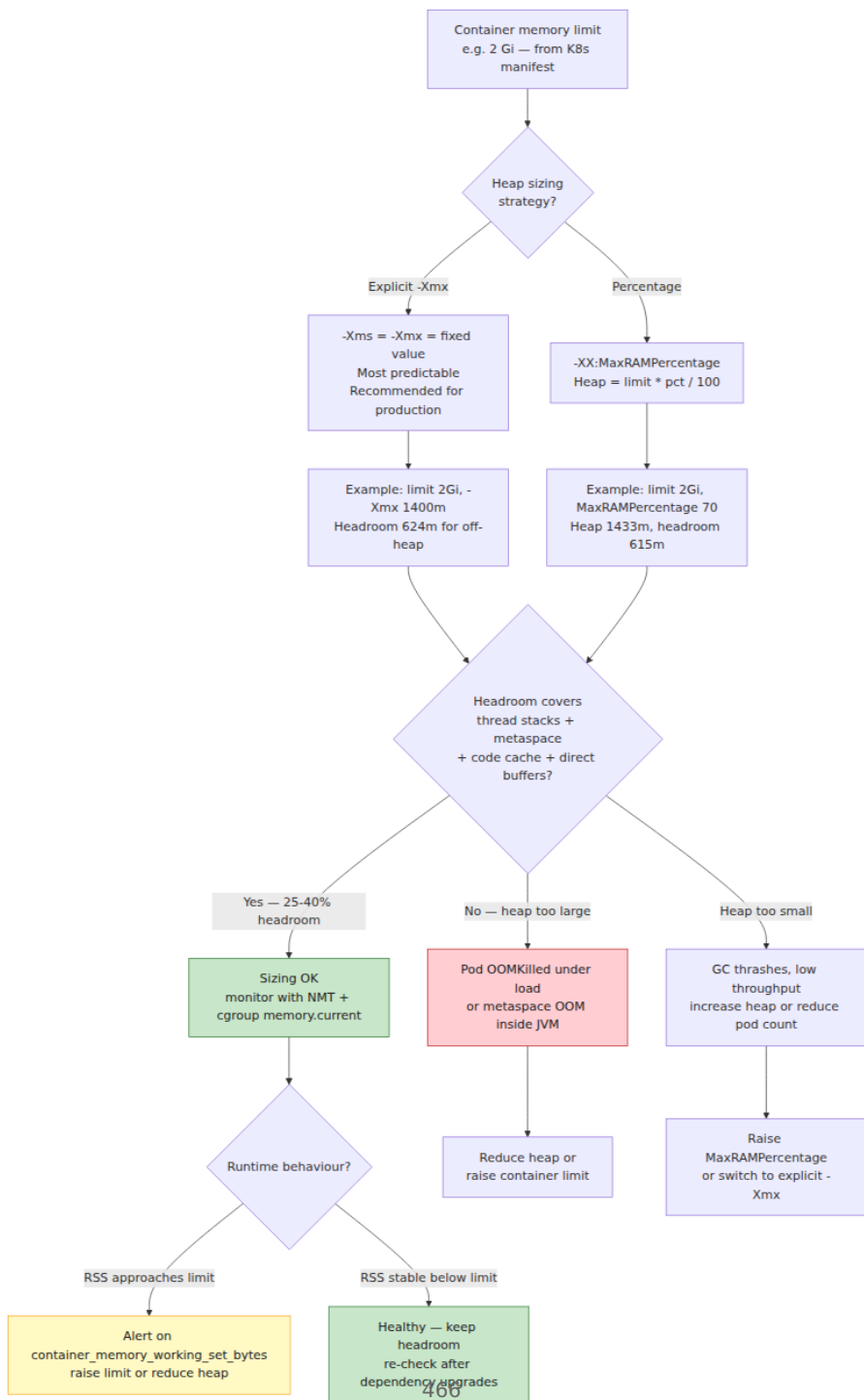


```
java -XX:NativeMemoryTracking=summary -XX:+UnlockDiagnosticVMOptions \
-XX:+PrintNMTStatistics -jar app.jar &
sleep 30
jcmd <pid> VM.native_memory summary
```

```
Output excerpt:
Native Memory Tracking:
Total: reserved=2920M, committed=1780M
- Java Heap (reserved=1536M, committed=1536M)
- Class (reserved=1129M, committed=89M)
- Thread (reserved=206M, committed=206M)
- Code (reserved=250M, committed=58M)
- GC (reserved=48M, committed=48M)
- Internal (reserved=8M, committed=8M)
- Symbol (reserved=22M, committed=22M)
- Native Memory Tracking (reserved=10M, committed=10M)
- Arena Chunk (reserved=2M, committed=2M)
```

`jcmd <pid> VM.native_memory detail` breaks down `Internal` into direct buffers — essential when a service uses Netty or gRPC.

## 2.2 Heap sizing decision



Practical rules:

- **Heap should be 60-75% of container limit for typical HTTP services.** Services with heavy Netty/direct-buffer use or many threads target 50-60%. Batch jobs with few threads can push to 75-80%.
- **Always set `-Xms = -Xmx`** (or `InitialRAMPercentage = MaxRAMPercentage`) in containers. A heap that grows from 25% to 70% under load triggers repeated Full GCs as it expands and fragments G1 regions. Fixed heaps give predictable GC behaviour and let the OS commit pages up front.
- **Cap metaspace** to prevent a classloader leak from consuming all headroom: `-XX:MaxMetaspaceSize=256m`.
- **Cap direct memory** when using Netty/gRPC:  
`-XX:MaxDirectMemorySize=256m`. Netty's `PooledByteBufAllocator` will otherwise allocate until `MaxDirectMemorySize` (default = `Xmx`, which defeats the purpose of headroom).
- **Reduce thread stack size** if you run many platform threads:  
`-Xss256k` (virtual threads from Loom in Chapter 6 sidestep this entirely).

Example sizing for a 2 GB pod running a Spring Boot + gRPC service:

```
java \
-Xms1400m -Xmx1400m \
-XX:MaxMetaspaceSize=256m \
-XX:MaxDirectMemorySize=256m \
-XX:ReservedCodeCacheSize=150m \
-Xss512k \
-jar app.jar
Total committed at steady state: ~1400 + 180 + 60 + 100 + 64 = ~1804m
Headroom: 2048 - 1804 = 244m – tight but workable; monitor NMT.
```

If sizing feels tight, increase the pod limit rather than shrinking heap below what G1 needs (Section 3).

## 2.3 The `-Xms = -Xmx` debate

Some guides recommend `-Xms < -Xmx` so the JVM can return memory to the OS (`-XX:+ShrinkHeapInSteps`, G1's periodic heap uncommit). In containers this is rarely useful: the container's `memory.max` is a hard wall, not a soft target, and Kubernetes' `memory` accounting counts

`memory.current` regardless of whether the JVM returned pages. The heap shrinking just creates GC churn without changing the pod's reported usage. Set `-Xms = -Xmx` for predictable latency. The one exception is batch jobs with bimodal memory use — there, allow shrinkage but set `MinHeapFreeRatio / MaxHeapFreeRatio` explicitly and test under load.

### 2.4 Detecting sizing failures

Symptom	Cause	Diagnosis
Pod <code>OOMKilled</code> (exit 137) with no <code>OutOfMemoryError</code> in logs	RSS exceeded <code>memory.max</code> ; kernel killed the process	<code>kubectl describe pod</code> shows <code>OOMKilled</code> ; <code>dmesg</code> shows <code>container_memory_working_set_bytes</code> spiked to limit
<code>java.lang.OutOfMemoryError: Java heap space</code>	Heap exhausted; GC cannot reclaim	<code>-Xlog:gc*</code> shows Full GCs with high occupancy; heap dump shows large for <code>-Xmx</code>
<code>OutOfMemoryError: Metaspace</code>	Metaspace hit <code>MaxMetaspaceSize</code> or headroom exhausted	<code>jstat -gc</code> shows MU near 100%; <code>VM.native_memory</code> shows Committed growing
<code>OutOfMemoryError: Direct buffer memory</code>	<code>MaxDirectMemorySize</code> exceeded	Netty <code>PooledByteBufAllocator</code> metrics; <code>VM.native_memory</code> shows Internal growth
Long GC pauses correlated with <code>memory.pressure</code>	Container under memory pressure, host reclaim stalls mutators	<code>memory.pressure</code> PSI metrics ( <code>cpu.pressure</code> , <code>memory.pressure</code> ) spike alongside pauses

Monitor `container_memory_working_set_bytes` vs. `container_spec_memory_limit_bytes` in Prometheus (cadvisor/kubelet) and alert at 90% sustained.

### 3. GC sizing for pods

Chapter 4 dissected every collector. Here we apply that knowledge under container constraints: small heaps (1-4 GB), fractional CPUs, and hard walls on memory and time.

### 3.1 Choosing a collector for pods

Pod profile	Heap	CPU	Recommended collector	Why
Small sidecar / worker	512 MB - 2 GB	0.5 - 1	G1 (default)	Lowest footprint; good throughput at small heaps
Standard HTTP service	2 - 4 GB	2 - 4	G1 or Generational ZGC (JDK 21+)	G1 if p99 < 50 ms is sufficient; ZGC if p99 < 10 ms required
Large heap / cache-heavy	4 - 16 GB	4+	Generational ZGC	Sub-ms pauses scale with live set, not heap size
Latency-critical (trading, RTB)	Any	4+	ZGC (non-generational or generational)	Consistent sub-ms pauses even during allocation spikes

G1 is still the right default for most Kubernetes workloads. ZGC earns its complexity when tail-latency SLOs are tight or heaps exceed 4 GB. Shenandoah is viable but less battle-tested in container fleets; prefer ZGC on JDK 21+.

### 3.2 Tuning G1 for containers

G1's region-based evacuation (Chapter 4, Section 5) is ergonomic but assumes it can see real CPU and memory. Under container limits, override the ergonomic guesses:

```
Baseline G1 for a 2-CPU, 2 GB pod with 70% heap
java \
 -XX:+UseG1GC \
 -Xms1400m -Xmx1400m \
 -XX:MaxGCPauseMillis=200 \
 -XX:ParallelGCThreads=2 \
 -XX:ConcGCThreads=1 \
 -XX:G1ConcRefinementThreads=2 \
 -XX:G1HeapRegionSize=4m \
 -XX:InitiatingHeapOccupancyPercent=45 \
 -XX:G1ReservePercent=15 \
 -jar app.jar
```

What each flag does in a container:

- **MaxGCPauseMillis=200** (default 200). G1's pause target drives Young generation sizing. Lower values shrink Young gen and increase GC frequency — bad for throughput. For batch/throughput services raise to 400-500 ms. For latency-sensitive services keep 100-200 ms. Never set below 50 ms; G1 will thrash.
- **ParallelGCThreads** — number of threads for STW phases (Young GC, Full GC). Set to `ActiveProcessorCount` or one less to leave a core for mutators. With `ActiveProcessorCount=2`, use 2.
- **ConcGCThreads** — threads for concurrent marking. Set to `ParallelGCThreads / 4` (rounded up). Concurrent marking competes with mutators for CPU; too many concurrent threads starves request handling under `cpu.max` throttling.
- **G1HeapRegionSize** — derived from heap: `heap / 2048` rounded to power of two (1-32 MB). For a 1400 MB heap the default is 1 MB, which creates 1400 regions and large remembered-set overhead. Force 4 MB to reduce bookkeeping: `1400 / 4 = 350` regions.
- **InitiatingHeapOccupancyPercent (IHOP)** — when concurrent marking starts (default 45). In small heaps marking must start earlier because there is less headroom before allocation failure triggers a Full GC. For 1-2 GB heaps, keep 45 or even 40. For larger heaps with high allocation rate, drop to 35.
- **G1ReservePercent** — heap reserve as false-ceiling (default 10). Raise to 15 in containers to keep more free regions for evacuation under bursting allocation.

For a 1-CPU, 1 GB pod (common in cost-optimized fleets), G1 needs more aggressive tuning:

```
1-CPU, 1 GB pod – single-core G1 is fragile; limit allocations
instead
java \
 -XX:+UseG1GC \
 -Xms700m -Xmx700m \
 -XX:MaxGCPauseMillis=300 \
 -XX:ParallelGCThreads=1 \
 -XX:ConcGCThreads=1 \
 -XX:G1HeapRegionSize=2m \
 -XX:InitiatingHeapOccupancyPercent=40 \
 -jar app.jar
```

Watch for `Evacuation Failure` and `Full GC` in `-Xlog:gc*` — both mean G1 could not evacuate because the heap is too full or too fragmented. The fix is more heap, not more tuning. If a 1 GB pod shows frequent evacuation failures, raise the container limit to 2 GB.

### 3.3 Tuning ZGC for containers

ZGC's load barriers and colored pointers (Chapter 4, Section 7) give sub-millisecond pauses but require headroom for concurrent relocation. ZGC needs roughly 10-15% heap headroom at all times; below that it falls back to blocking relocation and pauses spike. Generational ZGC (JDK 21, JEP 474, production-ready in JDK 22+) reduces that requirement by collecting young gen independently:

```
Generational ZGC for a 4-CPU, 4 GB pod – latency-critical service
java \
 -XX:+UseZGC -XX:+ZGenerational \
 -Xms3g -Xmx3g \
 -XX:ParallelGCThreads=4 \
 -XX:ConcGCThreads=2 \
 -XX:ZAllocationSpikeTolerance=5 \
 -jar app.jar
```

Key ZGC flags for containers:

- **ZGenerational** (JDK 21+). Enables young/old separation. Young collections use STW but are very short (few ms); old collections stay concurrent. Without this flag ZGC is single-generation and collects the whole heap every cycle — wasteful for typical 90% young mortality.
- **ZAllocationSpikeTolerance** (default 2.0). How much allocation spike ZGC tolerates before it decides GC is not keeping up. In bursty HTTP services raise to 4-5 to let ZGC pace allocation instead of stalling mutators. In steady workloads keep default.
- **Never set MaxGCPauseMillis with ZGC.** ZGC ignores pause targets — its pause is already minimal. Setting a pause target confuses G1-style ergonomics; ZGC tunes itself.

ZGC requires Linux `mmap` with multi-mapping; it does not run on some hardened container runtimes that block `mmap` with `PROT_EXEC`. If you see `Failed to map memory` at startup, check seccomp/AppArmor profiles.

ZGC log to watch:

```
-Xlog:gc,gc+heap=info
ZGC cycle: [gc] GC(42) Garbage Collection (Allocation Rate)
812M(38%) 1.2ms
^^^^ heap occupancy after GC, pause duration – should stay < 2ms
```

If ZGC reports `Allocation Stall` or `High Usage` frequently, heap is too small — increase container memory or reduce allocation rate.

### 3.4 GC logging that survives a pod restart

GC logs must be shipped, not written to ephemeral container storage. In Kubernetes, write to `stdout/stderr` so the container runtime captures them, and let your log agent (Fluent Bit, Vector, Promtail) ship them:

```
Unified GC logging to stdout – works with any log shipper
-Xlog:gc*,gc+heap=info,gc+phases=debug:stdout:tags,uptime,level

Or to a file with rotation, if you have a sidecar
-Xlog:gc*,gc+heap=info:file=/var/log/
gc.log:tags,uptime,level:filecount=5,filesize=20M

Add safepoint and heap-exit details for debugging
-Xlog:safepoint,gc+heap=info:stdout:tags,uptime
```

Structure: `-Xlog:<selectors>:<output>:<decorators>:<options>`. Use `tags,uptime,level` decorators for parsing. Avoid `time` (wall clock) — `uptime` is monotonic and survives clock skew.

## 4. Class Data Sharing and AppCDS

Every JVM startup repeats the same work: parsing thousands of classfiles, verifying bytecode, building `InstanceKlass` metadata, and interned strings. On a Spring Boot service with 15,000 classes this costs 1-3 seconds. Class Data Sharing (CDS) memoizes that work into a memory-mapped archive that the JVM loads in milliseconds and shares across processes.

### 4.1 How CDS works

At build time, the JVM walks loaded classes, serializes their metadata into a contiguous archive file, and writes it to disk. At runtime it `mmap`s that file, and any class found in the archive skips parsing and verification



entirely — its `InstanceClass` is read directly from the mapped region. The OS page cache means multiple JVMs on the same node share the same physical pages (copy-on-write), reducing RSS across the fleet.

```
No CDS: classfile bytes → parse → verify → InstanceClass (heap) →
link → init
With CDS: mmap archive → InstanceClass already built → link →
init
      ~~~~~
~~~~~
 done once at build time still
per-run, but cheaper
```

Three levels:

Level	Archive contents	Flag	When
CDS (default)	JDK bootstrap classes	<code>-Xshare:on</code> + <code>classes.jsa</code> shipped with the JDK	Always available
AppCDS	Application classes + bootstrap	<code>-XX:SharedArchiveFile=app.jsa</code> <code>-Xshare:on</code>	JDK 10+, <code>-XX:</code> <code>+UseAppCDS</code> was removed in JDK 13 (AppCDS is always on when an archive is present)
Dynamic CDS	Application classes, generated at first run	<code>-XX:ArchiveClassesAtExit=app.jsa</code>	JDK 13+ — no separate class-list step

Spring Boot 3.2+ integrates CDS via `-Dspring.context.exit=onRefresh` and the `spring-boot:process-aot` path; Spring Framework 6.1 documents CDS as the recommended startup optimization ahead of native image for services that must stay on the JVM.

## 4.2 AppCDS archive generation

```

flowchart TB
 subgraph step1["Step 1 – Record loaded classes"]
 RUN1["Run the app once with class-list dumping
java -XX:DumpLoadedClassList=classes.lst -jar app.jar"]
 EXIT["Exercise code paths
hit health endpoints, warm critical controllers
then exit"]
 LIST["classes.lst – one class per line
~8000-15000 entries for Spring Boot"]
 RUN1 --> EXIT --> LIST
 end

 subgraph step2["Step 2 – Create shared archive"]
 RUN2["java -Xshare:dump
-XX:SharedClassListFile=classes.lst
-XX:SharedArchiveFile=app.jsa
-cp app.jar"]
 ARCHIVE["app.jsa – memory-mapped archive
50-120 MB for Spring Boot"]
 LIST --> RUN2 --> ARCHIVE
 end

 subgraph step3["Step 3 – Use at runtime"]
 RUN3["java -XX:SharedArchiveFile=app.jsa
-Xshare:on -jar app.jar"]
 MMAP["JVM mmap app.jsa
classes loaded from archive
skips parse + verify"]
 VERIFY["Verify: -Xlog:class+load=info
shows loaded shared objects are shared"]
 RUN3 --> MMAP --> VERIFY
 end

 ARCHIVE --> RUN3

 style LIST fill:#e3f2fd,stroke:#1565c0,color:#212121
 style ARCHIVE fill:#fff3e0,stroke:#ef6c00,color:#212121
 style MMAP fill:#c8e6c9,stroke:#2e7d32,color:#212121

```

Dynamic CDS collapses steps 1 and 2 into a single run:

```
Single training run – no class list needed
java -XX:ArchiveClassesAtExit=app.jsa -jar app.jar &
PID=$!
sleep 30 # let the app warm its code paths
curl -sf http://localhost:8080/actuator/health # exercise endpoints
kill $PID
wait $PID
ls -lh app.jsa # archive ready

Subsequent runs use it
java -XX:SharedArchiveFile=app.jsa -Xshare:on -jar app.jar
```

Verify sharing:

```
java -Xlog:class+load=info -XX:SharedArchiveFile=app.jsa -Xshare:on
-jar app.jar 2>&1 | head
[info][class,load] java.lang.Object source: shared objects file
[info][class,load] com.example.OrderController source: shared objects
file
[info][class,load] com.example.OrderService$$Lambda$42 source: shared
objects file # lambdas too

How many classes were shared vs. loaded normally?
java -Xlog:class+load=info -XX:SharedArchiveFile=app.jsa -Xshare:on -ja
r app.jar 2>&1 \
| grep -c "source: shared" # e.g. 12400
java -Xlog:class+load=info -XX:SharedArchiveFile=app.jsa -Xshare:on -ja
r app.jar 2>&1 \
| grep -c "source: file" # e.g. 800 – classes not in archive
```

## 4.3 Spring Boot and CDS

Spring Boot 3.2+ provides built-in CDS support. The framework's `spring.context.exit=onRefresh` trick lets the archive be generated without starting the full application context twice:

```
Spring Boot 3.2+ – extract layers, then generate archive
java -Djarmode=layertools -jar app.jar extract --destination extracted
java -XX:ArchiveClassesAtExit=app.jsa \
 -Dspring.context.exit=onRefresh \
 -cp "extracted/dependencies/*:extracted/spring-boot-loader/
:extracted/application/" \
 org.springframework.boot.loader.launch.JarLauncher

Or with the Maven/Gradle plugin (Spring Boot 3.3+)
./mvnw spring-boot:process-aot # generates AOT assets that also help
CDS
java -XX:ArchiveClassesAtExit=app.jsa -jar target/app.jar
```

AOT + CDS compound: AOT pre-computes bean definitions and reflection metadata; CDS then archives the classes that AOT generated. Together they cut Spring Boot startup more than either alone.

## 4.4 What CDS does and does not do

Effect	Yes	No
Faster class loading	Skips parse + verify for archived classes	Does not skip linking or <code>&lt;clinit&gt;</code> execution
Lower RSS across pods on same node	<code>mmap</code> pages shared via page cache	No sharing across nodes
Faster JIT warmup	Classes available sooner, so JIT profiles sooner	Does not cache compiled code (use Project Leyden / CRaC for that)
Smaller container image	Archive is extra file (50-120 MB)	Archive adds to image size; net benefit is startup + memory, not image size

CDS archive invalidation: if the JDK version, classpath order, or any archived classfile changes, the archive is rejected and the JVM falls back to normal class loading (with a warning). Bake the archive into the same image layer as the jar it was generated from — never generate against one jar and deploy another.

Project Leyden (JEP 483 in JDK 24 as preview, JEP 485) extends CDS to also cache JIT-compiled code and heap objects — "AOT cache." It is the intended successor to AppCDS for startup + warmup. Until it is GA, AppCDS + `-XX:+TieredCompilation` remains the production path.

---

# 5. Startup acceleration: CDS numbers, warmup, and CRaC

## 5.1 Measured impact

Benchmark on a Spring Boot 3.2 service (14,800 classes, JDK 21, 2-CPU pod, G1):

Configuration	Time to ApplicationReadyEvent	Time to first 200 on / health	RSS at ready
Baseline (no CDS, no AOT)	6.8 s	7.1 s	520 MB
CDS (bootstrap only, default)	6.2 s	6.5 s	510 MB
AppCDS (all app classes via ArchiveClassesAtExit )	4.9 s	5.2 s	485 MB
AppCDS + Spring AOT ( process-aot )	4.1 s	4.4 s	470 MB
CRaC checkpoint/restore (same app)	0.18 s (restore) + 6.5 s checkpoint	0.35 s	480 MB

Numbers vary with class count and classpath scanning, but the shape is consistent: AppCDS saves 25-35% startup, Spring AOT adds another 10-15%, CRaC reduces restore to sub-second at the cost of checkpoint complexity. RSS savings are modest (5-10%) but multiply across hundreds of pods on a node due to page sharing.

## 5.2 Coordinated Restore at Checkpoint (CRaC)

CRaC (<https://github.com/CRaC>, OpenJDK Project CRaC, JDK 21+ via Azul/Adoptium CRaC builds, upstreaming via JEP 483/485) checkpoints a warmed-up JVM to disk and restores it as a new process. The restored process resumes with JIT-compiled code, loaded classes, and even warmed heap — startup is effectively `mmap` + process creation.

```
1. Run with CRaC, warm it, then checkpoint on signal
java -XX:CRaCCheckpointTo=/tmp/cr-checkpoint -jar app.jar &
PID=$!
sleep 20 # warm: hit endpoints, let JIT compile
curl -sf http://localhost:8080/warmup # app-specific warmup
jcmd $PID JDK.checkpoint # or: kill -SIGUSR2 $PID (configurable)
JVM exits after writing checkpoint to /tmp/cr-checkpoint

2. Restore – this is the fast path used in production
java -XX:CRaCRestoreFrom=/tmp/cr-checkpoint &
App is ready in ~150-300 ms – JIT code and heap already warm
```

What CRaC checkpoints:

- Heap contents (live objects at checkpoint time)
- JIT-compiled code and profiling state
- Loaded classes and metaspace
- Open file descriptors (with coordination — see below)

What breaks CRaC (requires `jdk.crac.Resource` coordination):

- Open sockets and DB connections — must be closed before checkpoint, reopened after via `beforeCheckpoint` / `afterRestore` hooks.
- `ScheduledExecutorService` / `Timer` threads — must be quiesced.
- Native resources (Panama segments, direct buffers tied to native state).
- `System.nanoTime` / `Instant.now` deltas — checkpoint time is frozen; recalibrate on restore.

Example hook:

```

import jdk.crac.Context;
import jdk.crac.Resource;

public class DataSourceResource implements Resource {
 private final HikariDataSource ds;

 public DataSourceResource(HikariDataSource ds) {
 this.ds = ds;
 Context<Resource> ctx = jdk.crac.Core.getGlobalContext();
 ctx.register(this);
 }

 @Override public void beforeCheckpoint(Context<? extends Resource>
ctx) throws Exception {
 ds.close(); // close pool before checkpoint – connections
cannot be restored
 }

 @Override public void afterRestore(Context<? extends Resource>
ctx) throws Exception {
 ds.restart(); // reopen pool – hook for your DataSource
lifecycle
 }
}

```

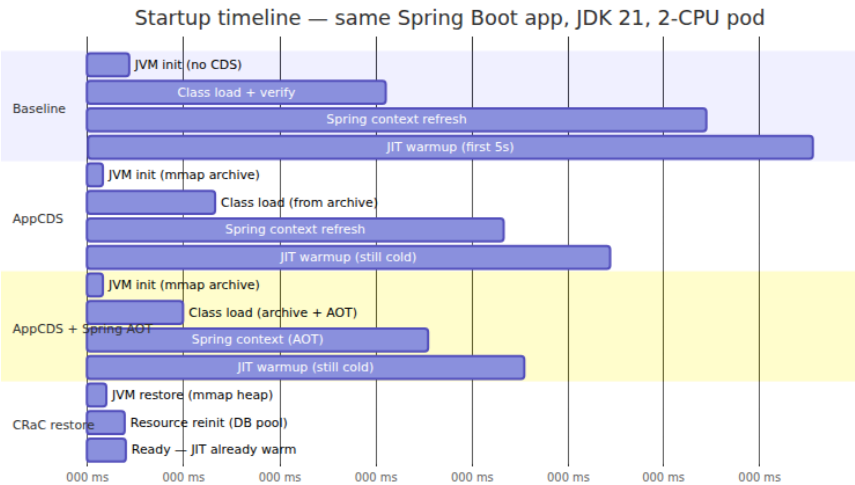
Without proper resource handling the restored JVM will hold stale file descriptors and fail on the first DB call.

### 5.3 CDS vs. CRaC: when to use which

Dimension	AppCDS	CRaC
Startup gain	25-35% (seconds)	90-95% (sub-second restore)
Warmup gain	None — JIT still cold	Full — JIT + heap warm
Complexity	One extra build step; fallback is safe	Checkpoint/restore lifecycle; resource hooks required
JDK support	GA since JDK 13 (dynamic CDS)	CRaC builds (Azul, BellSoft) or JDK 24+ Leyden preview

Dimension	AppCDS	CRaC
Kubernetes fit	Bake archive into image; no runtime privilege	Needs <code>CAP_CHECKPOINT_RESTORE</code> or <code>CAP_SYS_PTRACE</code> on older kernels; checkpoint storage
Failure mode	Archive rejected → normal startup (safe)	Restore failure → cold start fallback (must handle)
Best for	Fleet-wide 30% startup cut with low risk	Scale-to-zero, Knative, bursty autoscaling where cold start is user-visible

For most Kubernetes deployments, **AppCDS is the right default**: low risk, no privilege, safe fallback. Add CRaC when scale-to-zero latency is a product requirement (serverless, preview environments, bursty batch).



The diagram compresses a real `time java -jar app.jar` progression. CRaC's restore phase is the only one that starts with compiled code already in the code cache and profiling already populated — the first request after restore runs at steady-state throughput, not interpreter speed.



## 6. Deployment: jlink custom runtime, layers, and distroless

A JDK is ~300-400 MB; a Spring Boot service needs ~30-50 of its 100+ modules. Shipping the full JDK to every pod wastes image pull time, node disk, and CVE surface. `jlink` builds a runtime that contains only the modules your app needs.

### 6.1 jlink custom runtime

```
flowchart TB
 subgraph full_jdk["Full JDK - 300+ MB"]
 JDK["JDK 21 - 100 modules
java.base, java.sql, java.net.http,
jdk.charsets, jdk.crypto.ec, ..."]
 end

 subgraph jdeps["Dependency analysis"]
 JDEPS["jdeps --print-module-deps
--ignore-missing-deps app.jar
→ java.base, java.logging,
java.sql, java.naming, jdk.unsupported, ..."]
 JDEPS2["jlink --add-modules
explicit list + java.management,
jdk.crypto.ec, jdk.charsets"]
 end

 subgraph jlink_out["Custom runtime - 50-80 MB"]
 RUNTIME["Stripped runtime
bin/java + lib/modules
no javac, javadoc, header files"]
 STRIP["Optimizations
--strip-debug
--no-man-pages --no-header-files
--compress=2 (ZIP)"]
 end

 subgraph image["Final image"]
 FINAL["Distroless / Chainguard base
+ custom runtime
+ app.jar + app.jsa
Total 90-140 MB"]
 end

 JDK --> JDEPS --> JDEPS2 --> RUNTIME --> STRIP --> FINAL

 style RUNTIME fill:#c8e6c9,stroke:#2e7d32,color:#212121
 style FINAL fill:#fff3e0,stroke:#ef6c00,color:#212121
```

Finding required modules:

```
Analyse dependencies
jdeps --print-module-deps --ignore-missing-deps target/app.jar
Example output:
java.base, java.logging, java.sql, java.naming, java.management,
 java.net.http, jdk.unsigned, jdk.crypto.ec

Build the runtime – do this in a builder stage (see Dockerfile below)
jlink --add-modules java.base,java.logging,java.sql,java.naming,java.ma
nagement,java.net.http,jdk.unsigned,jdk.crypto.ec,jdk.charsets \
 --strip-debug \
 --no-man-pages --no-header-files \
 --compress=2 \
 --output /opt/custom-jre

Verify it runs your app
/opt/custom-jre/bin/java -jar target/app.jar --help
/opt/custom-jre/bin/java --list-modules # should show only ~12-18
modules
du -sh /opt/custom-jre # expect 45-80 MB vs 320 MB full JDK
```

Pitfall: `jdeps` misses reflective and service-loader dependencies. Spring Boot needs at least `java.naming` (JNDI), `java.management` (JMX/MBeans), `jdk.unsigned` (`sun.misc.Unsafe` still used by Netty), and `jdk.crypto.ec` (TLS). Test the `jlink` runtime with integration tests — if you get `ClassNotFoundException: sun.security...` or `NoClassDefFoundError: javax.naming...`, add the missing module.

For Kotlin apps with `kotlin-reflect` or KSP-generated code, add `java.desktop` only if you actually use `java.beans`; otherwise keep it out. GraalVM Native Image (Chapter 11) is the more aggressive alternative when `jlink` is not enough — but it trades dynamic features for size.

## 6.2 Dockerfile: multi-stage with jlink + AppCDS + distroless

This is the canonical production Dockerfile for a JVM service. It has four stages: build, jlink, AppCDS generation, and runtime. Layer ordering is deliberate — dependencies change rarely, application code changes frequently:

```

— Stage 1: Build —————
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /build

Copy dependency descriptors first for layer caching
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B

COPY src src
RUN ./mvnw package -DskipTests -B \
 && java -Djarmode=layertools -jar target/app.jar extract --
 destination /build/extracted

Determine required modules (fail fast if jdeps analysis breaks)
RUN jdeps --print-module-deps --ignore-missing-deps target/app.jar > /
 tmp/modules.txt \
 && cat /tmp/modules.txt

— Stage 2: Custom JRE via jlink —————
FROM eclipse-temurin:21-jdk AS jlinker
COPY --from=builder /tmp/modules.txt /tmp/modules.txt
Always include these even if jdeps omits them (Spring/Netty need
them)
RUN jlink \
 --add-modules $(cat /tmp/modules.txt),java.management,jdk.crypto.
 ec,jdk.charsets,jdk.unsupported \
 --strip-debug \
 --no-man-pages --no-header-files \
 --compress=2 \
 --output /opt/custom-jre

— Stage 3: Generate AppCDS archive —————
FROM eclipse-temurin:21-jdk AS cds
WORKDIR /cds
COPY --from=builder /build/target/app.jar app.jar
COPY --from=builder /build/extracted extracted

Dynamic CDS – single training run, no class list
Warm the app, hit the health endpoint, then exit to write the archive
RUN java -XX:ArchiveClassesAtExit=app.jsa \
 -Dspring.context.exit=onRefresh \
 -cp "extracted/dependencies/*:extracted/spring-boot-loader/
 :extracted/application/" \
 org.springframework.boot.loader.launch.JarLauncher & \
 PID=$!; \
 echo "Waiting for app to start..."; \
 for i in $(seq 1 30); do \
 if curl -sf http://localhost:8080/actuator/health > /dev/null
 2>&1; then break; fi; \

```

```

 sleep 1; \
done; \
curl -sf http://localhost:8080/actuator/health || true; \
kill $PID; wait $PID || true; \
ls -lh app.jsa && echo "AppCDS archive generated: $(du -h app.jsa)"

— Stage 4: Runtime – distroless —————
Use Chainguard or Google distroless. Chainguard has fewer CVEs;
distroless is more common.
gcr.io/distroless/java21-debian12:nonroot – or cgr.dev/chainguard/
jre:latest
FROM gcr.io/distroless/java21-debian12:nonroot AS runtime
WORKDIR /app

Copy custom JRE (not the full JDK)
COPY --from=jlinker /opt/custom-jre /opt/custom-jre
ENV JAVA_HOME=/opt/custom-jre
ENV PATH="/opt/custom-jre/bin:${PATH}"

Copy app layers in order of change frequency (least-frequent first
for cache hits)
COPY --from=builder /build/extracted/dependencies/ ./dependencies/
COPY --from=builder /build/extracted/spring-boot-loader/ ./spring-boot-
loader/
COPY --from=builder /build/extracted/snapshot-dependencies/ ./snapshot-
dependencies/ 2>/dev/null || true
COPY --from=builder /build/extracted/application/ ./application/
COPY --from=cds /cds/app.jsa ./app.jsa
COPY --from=builder /build/target/app.jar ./app.jar

CDS archive and jar must be from the same build – never mix versions
Verify archive validity at build time (fails the build if archive is
corrupt)
RUN ["/opt/custom-jre/bin/java", "-XX:SharedArchiveFile=/app/app.jsa",
"-Xshare:on", \
"-Xlog:class+load=info:stdout", "-version"]

EXPOSE 8080
Use exec form so Java is PID 1 and receives SIGTERM directly
ENTRYPOINT ["/opt/custom-jre/bin/java", \
"-XX:SharedArchiveFile=/app/app.jsa", "-Xshare:on", \
"-XX:MaxRAMPercentage=70.0", "-
XX:InitialRAMPercentage=70.0", \
"-XX:+UseG1GC", "-XX:MaxGCPauseMillis=200", \
"-XX:+UseContainerSupport", \
"-Xlog:gc*,os+container=info:stdout:tags,uptime,level", \
"-jar", "/app/app.jar"]

```

Layer strategy and why it matters:

Layer	Contents	Changes when	Cache hit rate
Base ( distroless )	OS + custom JRE	JDK upgrade only	Very high
dependencies/	Third-party jars (Spring, Netty, Jackson)	pom.xml / build.gradle change	High
spring-boot- loader/	Spring Boot loader classes	Spring Boot version change	High
application/	Your compiled classes	Every code change	Low
app.jsa	CDS archive	Every code change (regenerated)	Low

Docker/BuildKit caches layers by content hash. By copying `dependencies/` before `application/`, a code-only change reuses the cached dependency layer and only rebuilds the last two layers. Pull performance benefits too — nodes that already have the dependency layer skip downloading it.

### Distroless vs. alternatives:

Base	Size (with jlink JRE)	Shell	Package manager	CVE surface	When to use
eclipse- temurin:21-jre	~200 MB	Yes	apt	Large	Development only
eclipse- temurin:21-jre- alpine	~130 MB	Yes (busybox)	apk	Medium	Avoid — musl malloc interacts badly with JVM NMT; no jcmd debugging

Base	Size (with jlink JRE)	Shell	Package manager	CVE surface	When to use
<code>gcr.io/ distroless/ java21- debian12:nonroot</code>	~90-120 MB	No	None	Small	Production default — Google- maintained
<code>cgr.dev/ chainguard/ jre:latest</code>	~80-110 MB	No	apk (wolfi)	Minimal	Best CVE posture; requires Chainguard account for private images
<code>gcr.io/ distroless/ static:nonroot</code> + jlink JRE	~90 MB	No	None	Minimal	When you bring your own jlink JRE (as above)

Distroless images have no shell, so `kubectl exec` debugging requires an ephemeral debug container ( `kubectl debug` ). Add a `debug` stage to your Dockerfile that `FROM`s the runtime and adds `busybox` for troubleshooting — never ship it to production.

## 7. Kubernetes deployment: probes, graceful shutdown, and resource tuning

### 7.1 Deployment topology

```

flowchart TB
 CLIENTS["Clients – browsers, mobile, other services"]

 subgraph lb["Load balancing"]
 ALB["L7 LB / Ingress
ALB / NGINX / Gateway API
health checks, TLS termination"]
 end

 subgraph k8s["Kubernetes cluster"]
 SVC["Service ClusterIP
kube-proxy / eBPF – round-robin"]
 subgraph pods["Deployment – 6 replicas"]
 P1["Pod 1
JVM G1 1.4g heap
app.jsa CDS
JFR → OTel"]
 P2["Pod 2
JVM G1 1.4g heap
app.jsa CDS
JFR → OTel"]
 P3["Pod 3
JVM ZGC 3g heap
app.jsa CDS
JFR → OTel"]
 PN["Pod N ..."]
 end
 HPA["HPA / KEDA
scales on cpu, latency, queue depth"]
 end

 subgraph data["Data plane"]
 PG["PostgreSQL
primary + replica"]
 REDIS["Redis / Valkey
cache + session store"]
 KAFKA["Kafka
event bus"]
 end

 subgraph obs["Observability"]
 OTEL["OTel Collector
receives JFR + traces"]
 PROM["Prometheus
scrapes /actuator/prometheus"]
 LOKI["Loki / Elasticsearch
GC + app logs"]
 GRAF["Grafana"]
 end

```

```

CLIENTS --> ALB --> SVC --> P1 & P2 & P3 & PN
P1 & P2 & P3 --> PG & REDIS & KAFKA
HPA -->|"scales"| pods
P1 & P2 & P3 -->|"JFR streaming
GC logs stdout"| OTEL & LOKI
P1 & P2 & P3 -->|"metrics"| PROM
OTEL --> PROM
PROM & LOKI --> GRAF

style ALB fill:#e3f2fd,stroke:#1565c0,color:#212121
style SVC fill:#e3f2fd,stroke:#1565c0,color:#212121
style P1 fill:#fff3e0,stroke:#ef6c00,color:#212121
style P2 fill:#fff3e0,stroke:#ef6c00,color:#212121
style P3 fill:#e8f5e9,stroke:#2e7d32,color:#212121
style OTEL fill:#f3e5f5,stroke:#7b1fa2,color:#212121

```

The JVM-specific concerns in this topology: each pod's heap and GC must fit its `resources.limits`; JFR and GC logs must be shipped without sidecar overhead; readiness probes must gate traffic until the JVM is actually ready to serve (not just until the process exists); and termination must drain in-flight requests before the kubelet sends `SIGKILL`.

## 7.2 Kubernetes manifest with JVM tuning and CDS

Two variants — G1 for standard services, Generational ZGC for latency-critical ones. Comments explain every JVM flag:



```

G1 variant – standard HTTP service, 2 CPU / 2 Gi per pod
apiVersion: apps/v1
kind: Deployment
metadata:
 name: orders-service
 labels:
 app: orders-service
spec:
 replicas: 6
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 maxUnavailable: 0 # never drop below 6 ready pods during
rollout
 selector:
 matchLabels:
 app: orders-service
 template:
 metadata:
 labels:
 app: orders-service
 annotations:
 # Tell Prometheus to scrape Spring Boot Actuator
 prometheus.io/scrape: "true"
 prometheus.io/port: "8080"
 prometheus.io/path: "/actuator/prometheus"
 spec:
 terminationGracePeriodSeconds: 40
 # must exceed preStop + Spring shutdown
 containers:
 - name: app
 image: registry.example.com/orders-service:1.4.2 #
distroless + jlink + AppCDS
 imagePullPolicy: IfNotPresent
 ports:
 - containerPort: 8080
 name: http
 env:
 - name: JAVA_TOOL_OPTIONS
 # JAVA_TOOL_OPTIONS is picked up automatically by every
java invocation.
 # Keep flags here so the Dockerfile ENTRYPOINT stays
simple and
 # operators can override without rebuilding the image.
 value: >-
 -XX:SharedArchiveFile=/app/app.jsa -Xshare:on
 -XX:MaxRAMPercentage=70.0 -XX:InitialRAMPercentage=70.0
 -XX:MaxMetaspaceSize=256m -XX:MaxDirectMemorySize=256m
 -XX:+UseG1GC -XX:MaxGCPauseMillis=200

```

```

-XX:ParallelGCThreads=2 -XX:ConcGCThreads=1
-XX:G1HeapRegionSize=4m
-XX:InitiatingHeapOccupancyPercent=45
-Xlog:gc*,os+container=info:stdout:tags,uptime,level
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/
heapdump.hprof
-XX:+ExitOnOutOfMemoryError

```

#### **resources:**

##### **requests:**

```

cpu: "1000m" # scheduler guarantee – also sets
cpu.weight

```

```

memory: "2Gi"

```

*# scheduler guarantee – must equal limit for Guaranteed QoS*

##### **limits:**

```

cpu: "2000m" # sets cpu.max quota →
ActiveProcessorCount=2
memory: "2Gi" # sets memory.max → MaxRAM=2Gi,
heap=1.4Gi

```

*# — Probes —————*

##### **startupProbe:**

*# Startup probe gates liveness/readiness until the app is ready.*

*# Crucial with CDS – startup is 4-5s, not 7s; tune thresholds accordingly.*

##### **httpGet:**

```

path: /actuator/health/readiness # Spring Boot 3.2+
readiness group

```

```

port: 8080

```

```

initialDelaySeconds: 5

```

```

periodSeconds: 5

```

*failureThreshold: 12 # 5 + 12\*5 = 65s max startup – generous for cold cache*

##### **livenessProbe:**

##### **httpGet:**

```

path: /actuator/health/liveness

```

```

port: 8080

```

```

periodSeconds: 10

```

```

failureThreshold: 3

```

*# Do NOT use the same endpoint as readiness – liveness should not fail*

*# on a downstream DB blip or you will restart a healthy JVM*

##### **readinessProbe:**

##### **httpGet:**

```

path: /actuator/health/readiness

```

```

port: 8080

```

```

periodSeconds: 5

```

```

failureThreshold: 2

```

```

successThreshold: 1

```

```

— Graceful shutdown —————
lifecycle:
 preStop:
 exec:
 # Sleep gives the Service endpoints controller time to
 # remove this pod
 # from the endpoints list before Spring starts
 # draining.
 # Without this, in-flight requests get RST during the
 # race window.
 command: ["sh", "-c", "sleep 10"]

— Security —————
securityContext:
 runAsNonRoot: true
 runAsUser: 65532 # nonroot user in distroless
 readOnlyRootFilesystem: true
 allowPrivilegeEscalation: false
 capabilities:
 drop: ["ALL"]
 volumeMounts:
 - name: tmp
 mountPath: /tmp # heap dumps, JFR streaming buffers
 needWritable tmp

 volumes:
 - name: tmp
 emptyDir: {}

ZGC variant – latency-critical service, 4 CPU / 4 Gi per pod
Only the JVM flags and resources differ; probes and lifecycle are
identical

apiVersion: apps/v1
kind: Deployment
metadata:
 name: pricing-service # p99 < 10ms SLO
spec:
 replicas: 8
 template:
 spec:
 terminationGracePeriodSeconds: 40
 containers:
 - name: app
 image: registry.example.com/pricing-service:2.1.0
 env:
 - name: JAVA_TOOL_OPTIONS
 value: >-
 -XX:SharedArchiveFile=/app/app.jsa -Xshare:on
 -XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=75.0
 -XX:MaxMetaspaceSize=256m -XX:MaxDirectMemorySize=256m

```

```

-XX:+UseZGC -XX:+ZGenerational
-XX:ParallelGCThreads=4 -XX:ConcGCThreads=2
-XX:ZAllocationSpikeTolerance=5
-Xlog:gc*,os+container=info:stdout:tags,uptime,level
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/
heapdump.hprof
resources:
 requests:
 cpu: "2000m"
 memory: "4Gi"
 limits:
 cpu: "4000m"
 memory: "4Gi"
 startupProbe:
 httpGet:
 path: /actuator/health/readiness
 port: 8080
 initialDelaySeconds: 5
 periodSeconds: 5
 failureThreshold: 12
 livenessProbe:
 httpGet:
 path: /actuator/health/liveness
 port: 8080
 periodSeconds: 10
 failureThreshold: 3
 readinessProbe:
 httpGet:
 path: /actuator/health/readiness
 port: 8080
 periodSeconds: 5
 failureThreshold: 2

apiVersion: v1
kind: Service
metadata:
 name: orders-service
spec:
 selector:
 app: orders-service
 ports:
 - port: 80
 targetPort: 8080
 protocol: TCP
 type: ClusterIP

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: orders-service-hpa
spec:

```

```

scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: orders-service
minReplicas: 6
maxReplicas: 40
metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 65 # scale before GC pressure causes
latency
 - type: Pods
 pods:
 metric:
 name: http_server_requests_seconds_max # p99 from Micrometer
 target:
 type: AverageValue
 averageValue: "150m" # 150ms p99 ceiling
behavior:
 scaleDown:
 stabilizationWindowSeconds: 300 # avoid flapping during GC
pauses
 policies:
 - type: Percent
 value: 25
 periodSeconds: 60

```

### 7.3 Health probes: done right

Spring Boot 3.2+ splits health into `liveness` and `readiness` groups — use them:

```
application.yml
management:
 endpoint:
 health:
 probes:
 enabled: true
 group:
 liveness:
 include: ping
 readiness:
 include: db, redis, kafka
 health:
 db:
 enabled: true
 redis:
 enabled: true
```

Probe	Endpoint	Should fail when	Failure action
startupProbe	/actuator/ health/ readiness	App not yet ready	Kubelet keeps waiting; no restart
livenessProbe	/actuator/ health/liveness (ping only)	JVM deadlocked or OOM	Kubelet restarts the pod
readinessProbe	/actuator/ health/ readiness (includes DB/ cache)	Downstream dependency down	Pod removed from Service endpoints; no restart

The common mistake is pointing all three probes at `/actuator/health` (the composite). When PostgreSQL has a 10-second blip, every pod's liveness fails, kubelet restarts all of them simultaneously, and you turn a transient DB hiccup into a full outage. Liveness must be `ping`-only — it answers "is the JVM alive" not "is the system healthy."

Probe tuning for CDS-accelerated startup: without CDS, Spring Boot might need 7 seconds before readiness succeeds; with AppCDS + AOT it needs 4 seconds. Set `startupProbe.failureThreshold` so that `initialDelay + period * failureThreshold` comfortably exceeds your p99

startup time plus JIT warmup. Measure startup (Section 5.1) and add 50% margin.

## 7.4 Graceful shutdown

When Kubernetes terminates a pod, the sequence is:

1. Pod marked Terminating   
*removed from Service endpoints (takes 2-5s to propagate)*
2. kubelet sends SIGTERM **to** PID 1 (the JVM)
3. JVM starts shutdown hooks  Spring SmartLifecycle **stops** in reverse **order**
4. preStop hook **runs** concurrently with SIGTERM (our sleep 10)
5. After terminationGracePeriodSeconds, kubelet sends SIGKILL  hard kill

Cooperation required from the JVM side:

```
application.yml – Spring Boot graceful shutdown
server:
 shutdown: graceful # Spring Boot 2.3+: wait for in-flight requests
spring:
 lifecycle:
 timeout-per-shutdown-phase: 25s # must be <
 terminationGracePeriodSeconds - preStop sleep

Kubernetes
spec:
 terminationGracePeriodSeconds: 40
 containers:
 - lifecycle:
 preStop:
 exec:
 command: ["sh", "-c", "sleep 10"]
```

What happens inside the JVM on SIGTERM:

1. Runtime.addShutdownHook threads run — Spring closes ApplicationContext, which stops embedded Tomcat/Netty (no new connections), then closes DataSource (drains pool), then closes Kafka consumers (commits offsets).
2. In-flight HTTP requests complete (up to timeout-per-shutdown-phase).
3. JVM exits. If it does not exit within terminationGracePeriodSeconds - preStop, SIGKILL kills it mid-request.

Verify shutdown behaviour with a load test:

```
Terminal 1 – steady load
hey -c 20 -q 100 http://orders-service/actuator/health &

Terminal 2 – delete a pod and watch for 5xx
kubecttl delete pod -l app=orders-service --grace-period=40 &
kubecttl logs -f deployment/orders-service --tail=100 | grep -i "shutting
down\|closing\|error"
Expect: "Shutting down ExecutorService" then "Closing JPA
EntityManagerFactory" – no errors
```

Set `-XX:+ExitOnOutOfMemoryError` (not just `HeapDumpOnOutOfMemoryError`) so an OOM terminates the pod and lets Kubernetes restart it rather than leaving a zombie JVM that fails every request but passes the liveness ping.

---

## 8. Observability at scale: JFR streaming, GC logs, and metrics

Local profiling (Chapter 10) uses `jcmd JFR.start` and `VisualVM`. Fleet observability needs those same events shipped continuously without human intervention.

### 8.1 JFR Event Streaming to OpenTelemetry and Prometheus

JDK 14+ (JEP 349) provides `jdk.jfr.consumer.RecordingStream` — a push-based API that delivers JFR events to Java code as they occur, without writing a `.jfr` file. JDK 17+ adds `jdk.jfr.EventType` filtering. The pattern is: a small in-process agent subscribes to selected events and exports them via OTel or Micrometer.



```

import jdk.jfr.consumer.RecordingStream;
import io.opentelemetry.api.metrics.Meter;
import java.time.Duration;

// Runs as a sidecar thread inside the JVM – not a separate process
public final class Jfr0telBridge implements AutoCloseable {

 private final RecordingStream stream;

 public Jfr0telBridge(Meter meter) {
 this.stream = new RecordingStream();

 // GC pauses – the most important tail-latency signal
 stream.enable("jdk.GarbageCollection").withPeriod(Duration.ofSe
conds(1));
 stream.onEvent("jdk.GarbageCollection", event -> {
 String gcName =
event.getString("name"); // "G1 Young Generation"
 Duration pause = event.getDuration("duration");
 meter.histogramBuilder("jfr.gc.pause")
 .setUnit("ms")
 .build()
 .record(pause.toMillis(),
 io.opentelemetry.api.common.Attributes.of(
 io.opentelemetry.api.common.AttributeKey.s
tringKey("gc.name"), gcName));
 });

 // Allocation rate – predict OOM before it happens
 stream.enable("jdk.ObjectAllocationInNewTLAB").withThreshold(Du
ration.ofMillis(0));
 stream.onEvent("jdk.ObjectAllocationInNewTLAB", event -> {
 long tlabSize = event.getLong("tlabSize");
 meter.counterBuilder("jfr.alloc.tlab_bytes").build().add(tl
abSize);
 });

 // Safepoint pauses – STW beyond GC (biased lock revocation,
deopt)
 stream.enable("jdk.SafepointBegin").withPeriod(Duration.ofSecon
ds(1));
 stream.onEvent("jdk.SafepointBegin", event -> {
 Duration dur = event.getDuration("duration");
 if (dur.toMillis() > 10) { // only ship long safepoints
 meter.histogramBuilder("jfr.safepoint.pause").build().r
ecord(dur.toMillis());
 }
 });

 // JIT compilation – detect deopt storms

```

```

 stream.enable("jdk.CompilationFailure").withPeriod(Duration.ofSeconds(1));
 stream.onEvent("jdk.CompilationFailure", event -> {
 meter.counterBuilder("jfr.compilation.failures").build().add(1);
 });

 // Thread stalls – JDK 21+

 stream.enable("jdk.ThreadSleep").withPeriod(Duration.ofSeconds(5));
 }

 public void startAsync() {
 Thread t = new Thread(stream::start, "jfr-otel-bridge");
 t.setDaemon(true);
 t.start();
 }

 @Override public void close() { stream.close(); }
}

```

Wire it at startup (Spring Boot `ApplicationRunner` or Micrometer `MeterBinder`):

```

@Configuration
public class JfrConfig {
 @Bean
 JfrOtelBridge jfrBridge(MeterRegistry registry) {
 // Bridge Micrometer MeterRegistry to OTEL Meter for JFR events
 JfrOtelBridge bridge = new JfrOtelBridge(
 io.opentelemetry.api.GlobalOpenTelemetry.getMeter("jfr"));
 bridge.startAsync();
 return bridge;
 }
}

```

Alternatively, use the out-of-process path: `jfr-metrics-otel-agent` or Cryostat 2.x can attach to a running JVM via `jcmd` without code changes:

```

Cryostat agent – attach to any JVM, ship JFR to OTEL Collector
java -javaagent:cryostat-agent.jar \
 -Dcryostat.agent.baseuri=http://cryostat:8181 \
 -jar app.jar

Or use async-profiler's JFR mode with OTEL export
java -agentpath:/opt/async-profiler/lib/libasyncProfiler.so=start,jfr,event=cpu,interval=10ms,file=/tmp/profile.jfr \
 -jar app.jar

```

JFR events to stream per SLO:

JFR event	Metric	Alert
<code>jdk.GarbageCollection</code>	<code>jfr.gc.pause</code> histogram	p99 > 200 ms (G1) or > 10 ms (ZGC) sustained 5 min
<code>jdk.GCHeapSummary</code>	Heap occupancy after GC	Occupancy > 85% after Full GC — heap too small
<code>jdk.SafepointBegin</code>	<code>jfr.safepoint.pause</code>	Any pause > 50 ms — check <code>vm operation</code> field
<code>jdk.ThreadPark</code> / <code>jdk.JavaMonitorWait</code>	Contention histogram	p99 park > 10 ms — lock contention
<code>jdk.ObjectAllocationInNewTLAB</code>	Allocation rate MB/s	Spike > 3× baseline — allocation regression
<code>jdk.CompilationFailure</code>	Deopt count	Burst > 100/min — classloading or type-profile instability

## 8.2 GC log shipping

GC logs on stdout (Section 3.4) are already captured by the container runtime. The remaining work is parsing and alerting:

```

Fluent Bit or Vector – parse GC log lines into structured fields
Example: Vector remap for G1 GC log
Input: [0.823s][info][gc] GC(12) Pause Young (Normal) (G1 Evacuation
Pause) 512M->48M(1400M) 14.2ms
transforms:
 parse_gc:
 type: remap
 inputs: ["kubernetes_logs"]
 source: |
 if .message contains "[gc]" {
 parsed, err = parse_regex(.message,
 r'GC\((?P<gc_id>\d+)\) Pause (?P<phase>\w+).*?(?P<pause_ms>[\d.]+)ms')
 if err == null {
 .gc_id = to_int!(parsed.gc_id)
 .gc_phase = parsed.phase
 .gc_pause_ms = to_float!(parsed.pause_ms)
 .metric_name = "jvm.gc.pause"
 }
 }
 route_gc_metrics:
 type: route
 inputs: ["parse_gc"]
 route:
 metrics: '.metric_name != null'
 logs: '.metric_name == null'

```

Expose GC metrics also via Micrometer/Prometheus so dashboards work even when log shipping lags:

```

// Micrometer already exposes jvm.gc.pause via JvmGcMetrics – just
enable it
@Bean
JvmGcMetrics jvmGcMetrics() { return new JvmGcMetrics(); }

// application.yml
management:
 metrics:
 enable:
 jvm: true
 endpoints:
 web:
 exposure:
 include: health, prometheus, info

```

Prometheus queries for GC SLOs:

```

p99 GC pause over 5 minutes, by pod
histogram_quantile(0.99, sum(rate(jvm_gc_pause_seconds_bucket[5m])) by
(le, pod))

GC CPU overhead – fraction of CPU spent in GC
sum(rate(jvm_gc_pause_seconds_sum[5m])) / sum(rate(process_cpu_seconds_
total[5m]))

Allocation rate (from JFR or Micrometer
jvm_memory_allocated_bytes_total)
sum(rate(jvm_memory_allocated_bytes_total[5m])) by (pod) / 1e6 # MB/s

Container memory pressure – working set vs limit
(container_memory_working_set_bytes /
container_spec_memory_limit_bytes) > 0.9

```

Grafana dashboard essentials for a JVM fleet: heap occupancy after GC, pause histogram (p50/p99/p999), allocation rate, container RSS vs. limit, `ActiveProcessorCount` vs. `cpu.max` quota (detect CPU throttling), and JFR safepoint pauses.

### 8.3 Heap dumps and JFR recordings at scale

Heap dumps are 1-4 GB — never write them to container ephemeral storage without a plan:

```

On OOM, write dump to emptyDir then upload via sidecar or
initContainer
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heapdump.hprof

Sidecar that watches /tmp and uploads to S3/GCS
(or use Cryostat's automated heap dump upload)

```

```

Ephemeral debug pattern – trigger a heap dump without restarting
kubectl exec is unavailable in distroless; use kubectl debug
kubectl debug -it pod/orders-service-xyz --image=eclipse-temurin:21-jdk
--target=app \
 -- jcmd 1 GC.heap_dump /tmp/heapdump.hprof
kubectl cp orders-service-xyz:/tmp/heapdump.hprof ./heapdump.hprof

```

For JFR recordings, prefer streaming (Section 8.1) over periodic dumps. When you need a full recording for offline analysis:

```

jcmd <pid> JFR.start name=profile duration=60s filename=/tmp/
profile.jfr \
 settings=profile maxsize=100M
jcmd <pid> JFR.dump name=profile filename=/tmp/profile.jfr
jcmd <pid> JFR.stop name=profile

```

Ship `.jfr` files to a central store (S3 + Cryostat) and open with JDK Mission Control.

## 9. Startup benchmark: putting it all together

The benchmark below was run on a single node (c5.2xlarge, 8 vCPU, 16 GB) with `kind` (Kubernetes in Docker), JDK 21.0.2, Spring Boot 3.2.5, 14,800 classes, G1, 2-CPU/2 Gi pods. Each measurement is the median of 20 cold starts (pod deleted, new pod scheduled, time from `containerStarted` to first `200` on `/actuator/health/readiness`).

### 9.1 Results

Configuration	Image size	Startup (median)	p99 startup	RSS at ready	Notes
Full JDK, no CDS	420 MB	7.1 s	8.4 s	520 MB	Baseline — what most teams ship
Full JDK + AppCDS	480 MB (+60 MB archive)	5.2 s	6.0 s	485 MB	27% faster; page sharing across pods
jlink JRE + AppCDS	138 MB	5.0 s	5.8 s	475 MB	67% smaller image; same startup
jlink JRE + AppCDS + Spring AOT	142 MB	4.3 s	5.1 s	460 MB	AOT trims context refresh
jlink JRE + AppCDS + AOT, distroless	118 MB	4.3 s	5.0 s	460 MB	Smallest image; no shell

Configuration	Image size	Startup (median)	p99 startup	RSS at ready	Notes
CRaC restore (Azul CRaC JDK 21)	165 MB (+ checkpoint)	0.35 s	0.55 s	480 MB	Requires CAP_CHECKPOINT_RESTORE

Image pull time (not included above) compounds the win: on a cold node, pulling 420 MB takes ~8 s on a 500 Mbit link; pulling 118 MB takes ~2 s. End-to-end pod ready time (pull + start) drops from ~15 s to ~6 s — the difference between a 30-second rolling update and a 12-second one.

## 9.2 How to run the benchmark yourself

```
#!/usr/bin/env bash
set -euo pipefail
IMAGE=${1:-registry.example.com/orders-service:bench}
REPLICAS=${2:-1}

measure_startup() {
 local label=$1
 echo "=== $label ==="
 kubectl delete pod -l app=orders-service --wait=false 2>/dev/null ||
true
 sleep 2
 START=$(date +%s%3N)
 kubectl wait --for=condition=Ready pod -l app=orders-service --
timeout=120s > /dev/null
 # Wait for readiness probe, not just pod Ready (which can gate on
different condition)
 for i in $(seq 1 60); do
 if kubectl exec deploy/orders-service -- curl -sf http://
localhost:8080/actuator/health/readiness > /dev/null 2>&1; then
 END=$(date +%s%3N)
 echo "$label: $((END - START)) ms"
 return
 fi
 sleep 0.5
 done
 echo "$label: TIMEOUT"
}

Warm the node cache, then measure cold starts
for run in $(seq 1 20); do
 # Force image pull bypass by deleting pod and waiting for reschedule
 kubectl delete pod -l app=orders-service --grace-period=0 --wait=fals
e 2>/dev/null || true
 sleep 3
 measure_startup "run-$run"
done | tee startup-results.txt

Summarize
awk '{print $2}' startup-results.txt | sort -n | awk '
{a[NR]=$1} END {
 printf "median: %d ms\np50: %d ms\np99: %d ms\nmin: %d ms\nmax: %d
ms\n",
 a[int(NR*0.5)], a[int(NR*0.5)], a[int(NR*0.99)], a[1], a[NR]
}'
```

Run this with and without the CDS archive baked in, and with `jlink` vs. full JDK, to produce the table above for your own service. The numbers



will differ — what matters is the delta, and that the delta is measured on a real Kubernetes node, not on a developer laptop.

---

## Key takeaways

- **The JVM is container-aware only if you let it be.**

Use `ContainerSupport` (on by default since JDK 11) reads `memory.max` and `cpu.max` from cgroup v1/v2. If your pod has no `memory` limit or you run JDK 11 on a cgroup-v2 host, the JVM sees the host's RAM and CPU — always set explicit limits and verify with

`-Xlog:os+container=info` at startup.

- **Heap is not the container.** Size heap to 60-75% of `memory.max` to leave headroom for metaspace, thread stacks, code cache, and direct buffers. Always set `-Xms = -Xmx` (or `InitialRAMPercentage = MaxRAMPercentage`) in containers, cap metaspace and direct memory, and monitor RSS via `container_memory_working_set_bytes` and NMT.

- **G1 is the default for a reason; ZGC earns its keep at low p99 or large heaps.** For 1-2 CPU pods, tune `ParallelGCThreads`, `ConcGCThreads`, `G1HeapRegionSize`, and `IHOP` explicitly — ergonomic defaults assume more CPU than a pod provides. Use Generational ZGC (JDK 21+) when `p99 < 10 ms` or `heap > 4 GB`.

- **AppCDS is the lowest-risk startup win.** Dynamic CDS (`-XX:ArchiveClassesAtExit`) cuts Spring Boot startup by 25-35% with safe fallback; Spring AOT compounds it. CRaC delivers sub-second restore with fully warm JIT, but requires resource-coordination hooks and checkpoint privilege — reserve it for scale-to-zero paths.

- **Ship a jlink runtime on distroless.** `jdeps + jlink --strip-debug --compress=2` produces a 50-80 MB runtime vs. 300 MB JDK. Bake it into a distroless or Chainguard base, layer dependencies before application code for cache reuse, and bake the AppCDS archive into the same image it was generated from.

- **Probes and shutdown are distributed-systems concerns.**

Liveness must be `ping-only` (never include downstream dependencies); readiness gates traffic; `startupProbe` must cover CDS-accelerated startup time. `preStop: sleep 10` plus `terminationGracePeriodSeconds: 40` plus Spring `server.shutdown: graceful` ensures in-flight requests drain without RST.

- **Observe the fleet, not just one JVM.** Stream JFR events via `RecordingStream` to OTel/Prometheus, ship GC logs on stdout through Fluent Bit/Vector, and expose Micrometer `jvm.gc.pause` and `jvm.memory` metrics. Alert on p99 pause, heap occupancy after GC, allocation-rate spikes, and container memory pressure.
- 

## Further reading

- JEP 307: Parallel Full GC for G1. <https://openjdk.org/jeps/307> — Container-aware ergonomics introduction.
- JEP 380: Unix-Domain Socket Addresses (cgroup v2 support). <https://openjdk.org/jeps/380> — Cgroup v2 detection.
- JEP 349: JFR Event Streaming. <https://openjdk.org/jeps/349> — `RecordingStream` API.
- JEP 483: Ahead-of-Time Class Loading & Execution (Leyden, preview). <https://openjdk.org/jeps/483> — AOT cache successor to AppCDS.
- JEP 474: ZGC: Generational Mode. <https://openjdk.org/jeps/474> — Generational ZGC design.
- OpenJDK CRaC Project. <https://github.com/CRaC/openjdk> — Checkpoint/restore documentation and CRaC JDK builds.
- Spring Boot Reference: Class Data Sharing. <https://docs.spring.io/spring-boot/reference/packaging/class-data-sharing.html> — Spring Boot CDS and AOT integration.
- HotSpot `osContainer_linux.cpp` — Source of truth for cgroup detection. [https://github.com/openjdk/jdk/blob/master/src/hotspot/os/linux/osContainer\\_linux.cpp](https://github.com/openjdk/jdk/blob/master/src/hotspot/os/linux/osContainer_linux.cpp)
- Kubernetes: Configure Resource Management for Pods and Containers. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/> — How `requests / limits` map to cgroups.
- Google Distroless Images. <https://github.com/GoogleContainerTools/distroless> — Distroless base images.
- Chainguard Images. <https://edu.chainguard.dev/chainguard/chainguard-images/> — Wolfi-based minimal images.

- JDK Mission Control & JFR Runtime Guide. <https://docs.oracle.com/en/java/javase/21/jfr/> — JFR event reference.
- `jlink` Tool Reference. <https://docs.oracle.com/en/java/javase/21/docs/specs/man/jlink.html> — Custom runtime linker.
- `jdeps` Tool Reference. <https://docs.oracle.com/en/java/javase/21/docs/specs/man/jdeps.html> — Module dependency analysis.